Contents

1 Part2-经典机器学习算法 **12**

2 第六章：K 近邻——物以类聚 **12**

2.1 开篇故事：邻居的投票 . . . 12
2.2 费曼四步检验框 . . . 12
2.3 6.1 什么是 K 近邻? . . . 13
2.3.1 6.1.1 生活中的 KNN . . . 13
2.3.2 6.1.2 KNN 算法的直观理解 . . . 13
2.3.3 6.1.3 KNN 的正式定义 . . . 14
2.4 6.2 距离度量——如何量化”相似” . . . 14
2.4.1 6.2.1 欧氏距离（Euclidean Distance） . . . 14
2.4.2 6.2.2 曼哈顿距离（Manhattan Distance） . . . 15
2.4.3 6.2.3 闵可夫斯基距离（Minkowski Distance） . . . 16
2.4.4 6.2.4 其他距离度量 . . . 17
2.5 6.3 K 值的选择 . . . 17
2.5.1 6.3.1 K 值对模型的影响 . . . 17
2.5.2 6.3.2 K 值选择的经验法则 . . . 18
2.5.3 6.3.3 投票策略 . . . 18
2.6 6.4 从零实现 KNN . . . 19
2.6.1 6.4.1 基础 KNN 实现 . . . 19
2.6.2 6.4.2 运行结果 . . . 26
2.7 6.5 历史溯源 . . . 27
2.7.1 6.5.1 从 Fix 和 Hodges 到 Cover 和 Hart . . . 27
2.7.2 6.5.2 关键人物简介 . . . 27
2.8 6.6 进阶主题 . . . 28
2.8.1 6.6.1 维度灾难（Curse of Dimensionality） . . . 28
2.8.2 6.6.2 加速 KNN . . . 28
2.9 6.7 本章总结 . . . 28
2.9.1 核心概念回顾 . . . 28
2.9.2 与其他算法的关系 . . . 29
2.10 6.8 练习题 . . . 29
2.10.1 基础题 . . . 29
2.10.2 进阶题 . . . 30
2.10.3 挑战题 . . . 30
2.11 参考文献 . . . 31
2.11.1 经典论文 . . . 31
2.11.2 重要扩展 . . . 31
2.11.3 教科书与综述 . . . 31
2.11.4 历史回顾 . . . 31
2.11.5 在线资源 . . . 31
2.12 费曼检验回顾 . . . 31

3 第七章：线性回归——画一条最贴切的线 32
3.1 费曼四步检验框 . . . 32
3.2 本章学习目标 . . . 33
3.3 7.1 从高斯与谷神星说起 . . . 33
3.3.1 7.1.1 一颗失踪的小行星 . . . 33
3.3.2 7.1.2 24 岁天才的崛起 . . . 33
3.3.3 7.1.3 优先权之争：高斯 vs 勒让德 . . . 34
3.4 7.2 什么是最小二乘法? . . . 34
3.4.1 7.2.1 直觉理解：让误差最小 . . . 34
3.4.2 7.2.2 数学定义 . . . 35
3.5 7.3 画一条最贴切的线 . . . 36
3.5.1 7.3.1 线性回归模型 . . . 36
3.5.2 7.3.2 几何解释 . . . 36
3.5.3 7.3.3 损失函数 . . . 36
3.6 7.4 数学推导：如何找到最优解? . . . 37
3.6.1 7.4.1 单变量线性回归的推导 . . . 37
3.6.2 7.4.2 多元线性回归的矩阵形式 . . . 39
3.7 7.5 两种求解方法：闭式解 vs 梯度下降 . . . 39
3.7.1 7.5.1 闭式解（正规方程） . . . 39
3.7.2 7.5.2 梯度下降 . . . 40
3.8 7.6 如何判断拟合得好不好? . . . 41
3.8.1 7.6.1 R^2 决定系数 . . . 41
3.8.2 7.6.2 相关系数 r . . . 41
3.8.3 7.6.3 均方误差（MSE）和均方根误差（RMSE） . . . 42
3.9 7.7 相关与回归的关系 . . . 42
3.9.1 7.7.1 “回归”一词的起源 . . . 42
3.9.2 7.7.2 相关不等于因果 . . . 42
3.9.3 7.7.3 卡尔·皮尔逊的贡献 . . . 43
3.10 7.8 实战：手写线性回归 . . . 43
3.10.1 7.8.1 完整代码实现 . . . 43
3.10.2 7.8.2 运行结果示例 . . . 43
3.11 7.9 本章小结 . . . 44
3.11.1 核心概念回顾 . . . 44
3.11.2 历史时间线 . . . 44
3.11.3 工具箱 . . . 44
3.11.4 关键洞见 . . . 45
3.12 7.10 练习题 . . . 45
3.12.1 基础练习 . . . 45
3.12.2 进阶挑战 . . . 45
3.13 参考文献 . . . 46
3.14 费曼四步检验框 . . . 46

4 第八章：逻辑回归——分类的艺术 47
4.1 写在前面的话 . . . 47

4.2 本章学习地图 47
4.3 8.1 从人口增长到分类问题 48
4.3.1 一个关于人口的故事 48
4.3.2 从人口到概率 49
4.3.3 费曼检验框 49
4.4 8.2 什么是逻辑回归? 49
4.4.1 8.2.1 从线性回归到逻辑回归 49
4.4.2 8.2.2 Sigmoid 函数：神奇的 S 形曲线 50
4.4.3 8.2.3 决策边界：那条分界线 51
4.4.4 费曼检验框 51
4.5 8.3 数学之美：对数几率与最大似然 52
4.5.1 8.3.1 几率与对数几率 52
4.5.2 8.3.2 最大似然估计（MLE） 52
4.5.3 8.3.3 梯度下降求解 53
4.5.4 完整的训练算法 54
4.5.5 费曼检验框 54
4.6 8.4 动手实现逻辑回归 54
4.6.1 8.4.1 从零实现 Sigmoid 函数 54
4.6.2 8.4.2 从零实现逻辑回归 56
4.6.3 8.4.3 实战：垃圾邮件分类 62
4.7 8.5 多分类问题 66
4.7.1 8.5.1 一对多（One-vs-Rest） 66
4.8 8.6 正则化：防止过拟合 67
4.8.1 8.6.1 L2 正则化 67
4.9 8.7 历史长河中的智慧 68
4.9.1 8.7.1 Verhulst 与 Logistic 函数（1838-1845） 68
4.9.2 8.7.2 Berkson 与 Logit 模型（1944） 68
4.9.3 8.7.3 David Cox 的革命（1958） 68
4.9.4 历史时间线 69
4.10 8.8 练习与思考 69
4.10.1 基础练习 69
4.10.2 进阶练习 69
4.10.3 挑战练习 70
4.11 本章小结 70
4.11.1 核心概念 70
4.11.2 关键公式 70
4.11.3 关键代码模式 70
4.11.4 历史名人 70
4.12 参考文献 71
4.13 费曼四步检验 71
4.14 下一步预告 71

5 第九章 决策树——像专家一样做决策 72
5.1 开场故事：小明的水果店 72

5.2 9.1 决策树是什么? . . . 73
5.2.1 9.1.1 一棵倒着长的树 . . . 73
5.2.2 9.1.2 决策树 vs 人类专家 . . . 73
5.3 9.2 信息熵：衡量”混乱程度”的尺子 . . . 74
5.3.1 9.2.1 从”猜硬币”理解不确定性 . . . 74
5.3.2 9.2.2 香农熵的数学定义 . . . 74
5.3.3 9.2.3 熵的计算实例 . . . 74
5.3.4 9.2.4 熵的直观理解 . . . 75
5.4 9.3 ID3 算法：用信息增益选择最佳特征 . . . 75
5.4.1 9.3.1 ID3 的诞生 . . . 75
5.4.2 9.3.2 信息增益的定义 . . . 75
5.4.3 9.3.3 完整的 ID3 算法流程 . . . 76
5.4.4 9.3.4 贷款审批案例 . . . 76
5.4.5 9.3.5 ID3 的局限性 . . . 78
5.5 9.4 C4.5 算法：信息增益比解决偏向问题 . . . 78
5.5.1 9.4.1 ID3 的偏向问题 . . . 78
5.5.2 9.4.2 信息增益比（Gain Ratio） . . . 78
5.5.3 9.4.3 增益比计算示例 . . . 79
5.5.4 9.4.4 C4.5 的其他改进 . . . 79
5.6 9.5 CART 算法：基尼指数与二分分裂 . . . 79
5.6.1 9.5.1 CART 的诞生 . . . 79
5.6.2 9.5.2 基尼指数（Gini Index） . . . 80
5.6.3 9.5.3 二分分裂策略 . . . 80
5.7 9.6 连续特征的处理 . . . 80
5.7.1 9.6.1 为什么连续特征需要特殊处理? . . . 80
5.7.2 9.6.2 阈值选择算法 . . . 81
5.7.3 9.6.3 连续特征分裂实例 . . . 81
5.8 9.7 剪枝：防止决策树过拟合 . . . 82
5.8.1 9.7.1 过拟合的直观理解 . . . 82
5.8.2 9.7.2 预剪枝（Pre-Pruning） . . . 82
5.8.3 9.7.3 后剪枝（Post-Pruning） . . . 82
5.8.4 9.7.4 预剪枝 vs 后剪枝对比 . . . 82
5.9 9.8 费曼学习法检验 . . . 83
5.9.1 费曼四步检验框 . . . 83
5.9.2 可能的困惑与解答 . . . 83
5.10 9.9 三种算法对比总结 . . . 83
5.11 9.10 本章练习 . . . 84
5.11.1 基础练习（必做） . . . 84
5.11.2 进阶练习（挑战） . . . 85
5.11.3 挑战练习（深入思考） . . . 86
5.12 参考文献 . . . 86
5.13 本章小结 . . . 86

6 第十章：支持向量机——寻找最优分界线 87

6.1 开篇故事：怎样划分两个班级学生的座位 87
6.2 10.1 什么是最优分界线? 88
6.2.1 10.1.1 从走道到”间隔” 88
6.2.2 10.1.2 最大间隔原理 88
6.3 10.2 数学推导：从几何到优化 89
6.3.1 10.2.1 用向量描述走道 89
6.3.2 10.2.2 计算间隔的宽度 89
6.3.3 10.2.3 约束条件 89
6.3.4 10.2.4 优化问题 89
6.3.5 10.2.5 拉格朗日乘子法 90
6.3.6 10.2.6 对偶问题 90
6.3.7 10.2.7 KKT 条件 90
6.4 10.3 软间隔：允许犯错的智慧 91
6.4.1 10.3.1 现实世界不完美 91
6.4.2 10.3.2 松弛变量 91
6.4.3 10.3.3 带正则化的损失函数 91
6.5 10.4 核技巧：折叠纸张的魔法 92
6.5.1 10.4.1 线性不可分的问题 92
6.5.2 10.4.2 高维映射的直觉 92
6.5.3 10.4.3 核函数的奇迹 92
6.5.4 10.4.4 常用核函数 93
6.5.5 10.4.5 折叠纸张的比喻 93
6.6 10.5 代码实现：从零开始写 SVM 94
6.6.1 10.5.1 线性 SVM（梯度下降法） 94
6.6.2 10.5.2 核函数实现 98
6.6.3 10.5.3 SMO 算法简化实现 100
6.6.4 10.5.4 完整演示脚本 108
6.7 10.6 练习题 113
6.7.1 基础练习 113
6.7.2 进阶练习 114
6.7.3 挑战练习 114
6.8 10.7 本章总结 115
6.8.1 核心概念回顾 115
6.8.2 SVM 的优势与局限 115
6.8.3 与其他算法的比较 115
6.8.4 延伸学习 116
6.9 参考文献 116

7 第十一章：朴素贝叶斯——用概率做判断 117
7.1 开场故事：小明的神奇糖果罐 117
7.2 11.1 从医生诊断说起 117
7.2.1 11.1.1 概率的基本概念 117
7.2.2 11.1.2 条件概率 117
7.2.3 11.1.3 贝叶斯定理的诞生 118

7.2.4 11.1.4 一个医疗诊断的例子 . . . 119
7.3 11.2 朴素贝叶斯分类器 . . . 120
7.3.1 11.2.1 从贝叶斯定理到分类器 . . . 120
7.3.2 11.2.2 “朴素”假设：特征独立性 . . . 120
7.3.3 11.2.3 朴素贝叶斯分类公式 . . . 121
7.3.4 11.2.4 对数变换：防止数值下溢 . . . 121
7.3.5 11.2.5 不同特征的朴素贝叶斯变体 . . . 121
7.4 11.3 高斯朴素贝叶斯（Gaussian NB） . . . 121
7.4.1 11.3.1 正态分布回顾 . . . 121
7.4.2 11.3.2 高斯朴素贝叶斯的概率计算 . . . 122
7.4.3 11.3.3 高斯朴素贝叶斯的应用示例 . . . 122
7.5 11.4 多项式朴素贝叶斯（Multinomial NB） . . . 122
7.5.1 11.4.1 多项式分布 . . . 122
7.5.2 11.4.2 文本分类的基本流程 . . . 123
7.5.3 11.4.3 拉普拉斯平滑（Laplace Smoothing） . . . 123
7.5.4 11.4.4 垃圾邮件过滤的经典应用 . . . 123
7.6 11.5 伯努利朴素贝叶斯（Bernoulli NB） . . . 124
7.6.1 11.5.1 伯努利分布 . . . 124
7.6.2 11.5.2 伯努利朴素贝叶斯的概率估计 . . . 124
7.6.3 11.5.3 与多项式的区别 . . . 124
7.6.4 11.5.4 手写数字识别应用 . . . 124
7.7 11.6 朴素贝叶斯的优缺点 . . . 125
7.7.1 11.6.1 优点 . . . 125
7.7.2 11.6.2 缺点 . . . 125
7.7.3 11.6.3 何时使用朴素贝叶斯 . . . 125
7.8 11.7 本章小结 . . . 125

8 第十二章：集成学习——三个臭皮匠，顶个诸葛亮 125
8.1 开篇故事：陪审团的智慧 . . . 126
8.2 12.1 为什么一个模型不够? . . . 126
8.2.1 12.1.1 决策树的困境 . . . 126
8.2.2 12.1.2 方差与偏差的两难 . . . 127
8.2.3 12.1.3 集成的力量 . . . 127
8.3 12.2 Bagging——人多力量大 . . . 128
8.3.1 12.2.1 Bootstrap：有放回抽样 . . . 128
8.3.2 12.2.2 Bootstrap Aggregating . . . 129
8.3.3 12.2.3 为什么 Bagging 有效? . . . 129
8.3.4 12.2.4 Bagging 的局限性 . . . 130
8.4 12.3 随机森林——随机中的智慧 . . . 130
8.4.1 12.3.1 从 Bagging 到 Random Forest . . . 130
8.4.2 12.3.2 双随机性的威力 . . . 131
8.4.3 12.3.3 超参数 m_try . . . 131
8.4.4 12.3.4 随机森林的特性 . . . 132
8.5 12.4 Boosting——循序渐进 . . . 132

8.5.1 12.4.1 与 Bagging 的对比 132
8.5.2 12.4.2 Boosting 的核心思想 133
8.5.3 12.4.3 自适应 Boosting：AdaBoost 134
8.5.4 12.4.4 AdaBoost 为什么有效? 135
8.6 12.5 三种方法对比 135
8.7 12.6 代码实战：手写集成学习 135
8.7.1 12.6.1 Bagging 分类器 136
8.7.2 12.6.2 随机森林分类器 140
8.7.3 12.6.3 AdaBoost 分类器 145
8.7.4 12.6.4 三种方法对比实验 150
8.8 12.7 总结 152
8.8.1 核心概念回顾 152
8.8.2 关键公式回顾 153
8.8.3 实践建议 153
8.8.4 本章代码 153
8.9 练习题 153
8.9.1 基础练习 153
8.9.2 进阶挑战 154
8.9.3 终极挑战 154
8.10 参考文献 155

9 第十三章：K-Means 聚类——物以类聚 155
9.1 章前小语 155
9.2 13.1 什么是聚类? 156
9.2.1 13.1.1 有监督 vs 无监督 156
9.2.2 13.1.2 聚类的直觉 156
9.3 13.2 K-Means：划时代的算法 156
9.3.1 13.2.1 历史的轨迹 156
9.3.2 13.2.2 算法的核心思想 157
9.3.3 13.2.3 直观的例子 157
9.4 13.3 数学原理 157
9.4.1 13.3.1 损失函数 157
9.4.2 13.3.2 为什么两步迭代有效? 157
9.4.3 13.3.3 收敛性保证 158
9.5 13.4 初始化问题与 K-Means++ 158
9.5.1 13.4.1 初始化的陷阱 158
9.5.2 13.4.2 K-Means++：聪明的初始化 158
9.5.3 13.4.3 理论保证 158
9.6 13.5 如何选择 K? 159
9.6.1 13.5.1 肘部法则 159
9.6.2 13.5.2 轮廓系数 159
9.6.3 13.5.3 其他方法 159
9.7 13.6 K-Means 的优缺点 159
9.7.1 13.6.1 优点 159

9.7.2 13.6.2 缺点 . . . 159
9.8 13.7 实际应用案例 . . . 160
9.8.1 13.7.1 图像颜色量化 . . . 160
9.8.2 13.7.2 客户分群 . . . 160
9.8.3 13.7.3 文档聚类 . . . 160
9.9 13.8 本章小结 . . . 160
9.10 练习题 . . . 160
9.10.1 基础练习 . . . 160
9.10.2 进阶练习 . . . 161
9.10.3 挑战练习 . . . 161
9.11 参考文献 . . . 161

10 第十四章：层次聚类与 DBSCAN——构建数据家谱与发现密度群落 162
10.1 章节导览 . . . 162
10.2 第一部分：层次聚类——数据的家谱 . . . 162
10.2.1 14.1 什么是层次聚类? . . . 162
10.2.2 14.2 凝聚式层次聚类（AGNES） . . . 162
10.2.3 14.3 Lance-Williams 递推公式——统一框架 . . . 163
10.2.4 14.4 分裂式层次聚类（DIANA） . . . 163
10.2.5 14.5 树状图（Dendrogram）解读 . . . 164
10.3 第二部分：DBSCAN——发现密度岛屿 . . . 164
10.3.1 14.6 密度聚类的动机 . . . 164
10.3.2 14.7 DBSCAN 核心概念 . . . 165
10.3.3 14.8 DBSCAN 算法步骤 . . . 165
10.3.4 14.9 DBSCAN 的优势与局限 . . . 166
10.4 第三部分：聚类评估——如何判断好坏? . . . 166
10.4.1 14.10 为什么需要评估? . . . 166
10.4.2 14.11 Silhouette 系数（轮廓系数） . . . 166
10.4.3 14.12 Davies-Bouldin 指数 . . . 166
10.4.4 14.13 其他评估指标 . . . 167
10.5 第四部分：Python 实现 . . . 167
10.5.1 14.14 完整代码实现 . . . 167
10.5.2 14.15 动手实验 . . . 167
10.6 第五部分：深度思考 . . . 168
10.6.1 14.16 层次聚类 vs DBSCAN vs K-Means . . . 168
10.6.2 14.17 经典文献回顾 . . . 169
10.7 练习题 . . . 169
10.7.1 基础练习 . . . 169
10.7.2 进阶挑战 . . . 170
10.7.3 编程项目 . . . 170
10.8 参考文献 . . . 170
10.8.1 核心文献 . . . 170
10.8.2 扩展阅读 . . . 171
10.9 下章预告 . . . 171

11 第十五章：降维——抓住主要矛盾 171
11.1 开篇故事：Karl Pearson 与生物测量的革命 . . . 171
11.1.1 1901 年的伦敦：一个革命性的想法 . . . 171
11.1.2 2008 年：Hinton 与 Maaten 的可视化突破 . . . 172
11.2 费曼四步检验框：什么是降维? . . . 172
11.2.1 费曼四步学习法 . . . 172
11.3 第一部分：PCA——寻找数据的主轴 . . . 173
11.3.1 1.1 直观理解：方差就是信息 . . . 173
11.3.2 1.2 数学推导：从协方差到特征分解 . . . 173
11.3.3 1.3 核心公式汇总 . . . 174
11.3.4 1.4 Explained Variance Ratio . . . 174
11.3.5 1.5 标准化 vs 不标准化 . . . 175
11.3.6 1.6 PCA 的优缺点 . . . 175
11.4 第二部分：t-SNE——保持邻居关系 . . . 175
11.4.1 2.1 动机：PCA 的局限性 . . . 175
11.4.2 2.2 核心思想：保持邻居关系 . . . 175
11.4.3 2.3 算法详解 . . . 175
11.4.4 2.4 t-SNE 的优缺点 . . . 177
11.5 第三部分：数学推导详解 . . . 177
11.5.1 3.1 PCA 的完整推导 . . . 177
11.5.2 3.2 t-SNE 的条件概率推导 . . . 177
11.5.3 3.3 KL 散度公式推导 . . . 178
11.6 第四部分：代码实现讲解 . . . 179
11.6.1 4.1 PCA 类实现要点 . . . 179
11.6.2 4.2 t-SNE 类实现要点 . . . 179
11.6.3 4.3 可视化函数 . . . 179
11.7 第五部分：练习题 . . . 179
11.7.1 基础题（3 道） . . . 179
11.7.2 进阶题（2 道） . . . 180
11.7.3 挑战题（1 道） . . . 180
11.8 参考文献 . . . 180

12 第十六章：感知机——神经网络的起点 181
12.1 开篇故事：大脑的启示 . . . 181
12.2 16.1 从生物神经元到人工神经元 . . . 181
12.2.1 16.1.1 生物神经元长什么样? . . . 181
12.2.2 16.1.2 赫布学习规则 . . . 182
12.2.3 16.1.3 人工神经元的诞生 . . . 182
12.3 16.2 感知机的诞生（1958） . . . 182
12.3.1 16.2.1 弗兰克·罗森布拉特的突破 . . . 182
12.3.2 16.2.2 Mark I 感知机 . . . 183
12.3.3 16.2.3 感知机的简化模型 . . . 183
12.4 16.3 几何直观：感知机在画线 . . . 184
12.4.1 16.3.1 决策边界 . . . 184

12.4.2 16.3.2 权重的几何意义 . . . 185
12.5 16.4 感知机学习规则：如何自动学习 . . . 185
12.5.1 16.4.1 核心思想：试错学习 . . . 185
12.5.2 16.4.2 学习规则的数学推导 . . . 185
12.5.3 16.4.3 统一的更新公式 . . . 186
12.5.4 16.4.4 完整算法 . . . 186
12.6 16.5 感知机收敛定理 . . . 187
12.6.1 16.5.1 感知机收敛定理 . . . 187
12.7 16.6 感知机的局限：XOR 问题 . . . 188
12.7.1 16.6.1 XOR 问题是什么? . . . 188
12.7.2 16.6.2 线性不可分的直观理解 . . . 189
12.7.3 16.6.3 Minsky 和 Papert 的致命一击 . . . 189
12.7.4 16.6.4 如何解决 XOR? . . . 190
12.8 16.7 从零实现感知机 . . . 190
12.8.1 16.7.1 Perceptron 类实现 . . . 190
12.8.2 16.7.2 运行示例输出 . . . 199
12.9 16.8 多类分类的扩展 . . . 201
12.9.1 16.8.1 One-vs-Rest（一对多）策略 . . . 201
12.9.2 16.8.2 实现代码 . . . 201
12.10 16.9 练习题 . . . 202
12.10.1 基础题 . . . 202
12.10.2 进阶题 . . . 202
12.10.3 挑战题 . . . 202
12.11 16.10 本章小结 . . . 202
12.11.1 核心概念 . . . 202
12.11.2 历史脉络 . . . 203
12.11.3 本章代码实现 . . . 203
12.12 16.11 参考文献 . . . 203
12.12.1 原始论文 . . . 203
12.12.2 感知机局限性的经典论述 . . . 203
12.12.3 感知机收敛定理 . . . 204
12.12.4 现代教材 . . . 204
12.12.5 历史回顾 . . . 204
12.13 下一章预告 . . . 204

13 第十七章 多层神经网络——层层的魔法 204
13.1 开场故事：折纸与神经网络 . . . 204
13.2 17.1 历史的转折：从寒冬到春天 . . . 205
13.2.1 17.1.1 感知机的黄金时代（1958） . . . 205
13.2.2 17.1.2 寒冬降临：Minsky 与 Papert 的打击（1969） . . . 205
13.2.3 17.1.3 希望的种子：反向传播的诞生（1970） . . . 206
13.2.4 17.1.4 春天来了：Rumelhart、Hinton 与 Williams 的突破（1986） . . . 206
13.2.5 17.1.5 历史的启示 . . . 206
13.3 17.2 XOR 问题：为什么单层不够? . . . 207

13.3.1 17.2.1 什么是 XOR? . . . 207
13.3.2 17.2.2 XOR 问题的可视化 . . . 207
13.3.3 17.2.3 感知机的局限 . . . 207
13.3.4 17.2.4 一个生活的比喻 . . . 208
13.3.5 17.2.5 解决方案的直觉 . . . 208
13.4 17.3 隐藏层的魔法：从 2D 到 3D 的空间变换 . . . 209
13.4.1 17.3.1 折叠空间的比喻 . . . 209
13.4.2 17.3.2 XOR 问题的多层解决方案 . . . 209
13.4.3 17.3.3 空间变换的可视化 . . . 210
13.4.4 17.3.4 更深层的网络 = 更复杂的折叠 . . . 211
13.5 17.4 前向传播：信号如何层层传递 . . . 211
13.5.1 17.4.1 神经网络的基本结构 . . . 211
13.5.2 17.4.2 单个神经元的计算 . . . 211
13.5.3 17.4.3 向量化：矩阵运算的威力 . . . 212
13.5.4 17.4.4 完整的神经网络计算流程 . . . 213
13.5.5 17.4.5 批处理：同时计算多个样本 . . . 214
13.6 17.5 反向传播直觉：误差如何倒着流 . . . 215
13.6.1 17.5.1 学习的本质：调整权重以减少误差 . . . 215
13.6.2 17.5.2 反向传播的核心思想 . . . 215
13.6.3 17.5.3 链式法则：反向传播的数学基础 . . . 215
13.6.4 17.5.4 误差信号的传递 . . . 215
13.6.5 17.5.5 打保龄球的比喻 . . . 216
13.6.6 17.5.6 为什么叫”反向”传播? . . . 216
13.6.7 17.5.7 本章只讲直觉，下章详细推导 . . . 216
13.7 17.6 激活函数的必要性：没有非线性，多层 = 单层 . . . 217
13.7.1 17.6.1 一个惊人的事实 . . . 217
13.7.2 17.6.2 非线性：神经网络的灵魂 . . . 217
13.7.3 17.6.3 激活函数的可视化比较 . . . 218
13.7.4 17.6.4 选择合适的激活函数 . . . 218
13.8 17.7 从零实现 MLP 类 . . . 218
13.9 17.8 数学推导详解 . . . 239
13.9.1 17.8.1 前向传播的矩阵运算 . . . 239
13.9.2 17.8.2 损失函数对输出的梯度（直觉） . . . 239
13.9.3 17.8.3 参数数量计算公式 . . . 240
13.10 17.9 练习题 . . . 240
13.10.1 基础题（3 道） . . . 240
13.10.2 进阶题（3 道） . . . 243
13.10.3 挑战题（2 道） . . . 245
13.11 17.10 本章总结 . . . 247
13.11.1 核心概念回顾 . . . 247
13.11.2 历史意义 . . . 248
13.11.3 下章预告 . . . 248
13.12 参考文献 . . . 248

13.13扩展阅读 249

---

1 Part2-经典机器学习算法

章节范围：第 6-17 章
核心目标：掌握传统 ML，建立优化思维

---

2 第六章：K 近邻——物以类聚

本章目标：理解 K 近邻算法的核心思想，掌握距离计算和投票机制，能够从零实现 KNN 分类器。

2.1 开篇故事：邻居的投票

1966 年初，美国加州帕洛阿尔托的斯坦福大学校园里，一位年轻的教授**托马斯·科弗（Thomas M. Cover）**正在办公室里沉思。

门外传来敲门声。是他的学生**彼得·哈特（Peter E. Hart）**走了进来，手里拿着一叠手写笔记。

“教授，我和查尔斯·科尔（Charles Cole）正在做一个模式识别项目。我们用的方法很简单——找到离新样本最近的那个已知样本，然后给它打上同样的标签。”

科弗教授抬起头，饶有兴趣地问：“这个方法有名字吗?”

“我们叫它…最近邻（Nearest Neighbor）规则。”哈特有些不好意思地说，“但不知道它有没有什么理论上的保证?”

科弗教授的眼睛亮了起来。作为一个统计学家，他立刻意识到这是一个非常优雅的问题。他和哈特开始每周花两三个小时在一起研究这个看似简单的方法。

1967 年，他们发表了那篇著名的论文《Nearest Neighbor Pattern Classification》。这篇论文证明了一个惊人的结论：

> **在无限样本的极限情况下，最近邻规则的错误率不会超过贝叶斯最优错误率的两倍！**

这意味着，**即使是最简单的近邻方法，也有着坚实的理论基础**。这个”简单性”与”理论保证”的完美结合，让 KNN 成为了机器学习历史上最重要的算法之一。

2.2 费曼四步检验框

在开始之前，让我们用费曼学习法来预览本章的核心概念：

```
                    费 曼 四 步 检 验 法

1   选 择 概 念 : K 近 邻 （K-Nearest Neighbors , KNN）
```

```
2  教给别人：想象你在向一个小学生解释...
   "KNN就像是问你的邻居们：'你们觉得这是什么？'
    然后听大多数人的意见来决定答案。"

3  发现差距：当你说"距离"时，需要解释清楚：
   - 直线距离（欧氏距离）
   - 走路距离（曼哈顿距离）
   - 高维空间中的距离如何计算

4  简化语言：用生活化的比喻替代专业术语
   "特征空间" → "属性坐标系"
   "多数投票" → "邻居举手表决"
   "距离度量" → "相似度标尺"
```

2.3 6.1 什么是 K 近邻?

2.3.1 6.1.1 生活中的 KNN

想象一下这个场景:

> 你刚搬到一个新小区，想知道这个小区的治安好不好。你会怎么做?

方法一：查阅官方统计数据（这是”参数化方法”，假设数据服从某种分布）

方法二：问问住在附近的几位邻居他们的感受（这就是 KNN 的思想!）

KNN 的核心假设非常直观:

```
"Birds of a feather flock together"（物以类聚，人以群分）
```

在特征空间中，相似的样本会聚集在一起。因此，一个未知样本的类别，可以通过它周围已知样本的类别来推断。

2.3.2 6.1.2 KNN 算法的直观理解

让我们用一个具体的例子来说明:

```
场景：水果分类

假设你有两种水果：苹果    和  橙子
你测量了它们的两维特征：
- 重量（克）
- 直径（厘米）

现在来了一个新水果，你想知道它是苹果还是橙子...

KNN的做法：
1. 计算新水果与所有已知水果的"距离"
```

```
2. 找出距离最近的K个邻居
3. 看这K个邻居中哪种水果更多
4. 新水果就属于那一类！
```

用 ASCII 图来表示：

```
重量（克）

200

150              ← 新来的！

100

 50
      5    7    9    → 直径（cm）

如果 K=3，最近的3个邻居是：
多数票是   → 新水果是苹果！
```

2.3.3 6.1.3 KNN 的正式定义

定义 6.1（K 近邻算法）：

给定训练集 $D = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$，其中 $x_i \in \mathbb{R}^d$ 是特征向量，$y_i \in \{c_1, c_2, ..., c_m\}$ 是类别标签。

对于一个新的样本 x，KNN 算法的预测过程为：

1. **计算距离**：计算 x 与训练集中所有样本的距离 $d(x, x_i)$
2. **选择邻居**：找出距离最近的 K 个样本 $N_K(x)$
3. **投票决策**：
   - 分类问题：$\hat{y} = \arg\max_c \sum_{x_i \in N_K(x)} I(y_i = c)$
   - 回归问题：$\hat{y} = \frac{1}{K} \sum_{x_i \in N_K(x)} y_i$

其中 $I(\cdot)$ 是指示函数，条件为真时值为 1，否则为 0。

2.4 6.2 距离度量——如何量化”相似”

2.4.1 6.2.1 欧氏距离（Euclidean Distance）

最常见的距离度量是**欧氏距离**，也就是我们日常生活中所说的”直线距离”。

定义 6.2（欧氏距离）：

对于两个 d 维向量 $x = (x_1, x_2, ..., x_d)$ 和 $y = (y_1, y_2, ..., y_d)$，它们之间的欧氏距离为：

$$d_{Euclidean}(x, y) = \sqrt{\sum_{i=1}^{d}(x_i - y_i)^2}$$

二维情况的可视化：

```
y

        B

             欧 氏 距 离



                 A
        C

                       x

A = (4, 2)
B = (4, 5)
C = (1, 2)

d(A, B) = √[(4-4)² + (5-2)²] = √[0 + 9] = 3.0
d(A, C) = √[(4-1)² + (2-2)²] = √[9 + 0] = 3.0
d(B, C) = √[(4-1)² + (5-2)²] = √[9 + 9] = √18   4.24
```

2.4.2 6.2.2 曼哈顿距离（Manhattan Distance）

想象你在纽约曼哈顿的街道上行走——你只能沿着街道走，不能穿墙而过。这种”格子状”的距离就是**曼哈顿距离**，也叫**城市街区距离**。

定义 6.3（曼哈顿距离）：

$$d_{Manhattan}(x, y) = \sum_{i=1}^{d}|x_i - y_i|$$

为什么叫”曼哈顿距离”?

```
曼 哈 顿 的 街 道 是 规 则 的 网 格：

    6th Ave   7th Ave   8th Ave

 42nd

                          ← 从 建 筑 物 A 到 B
 43rd                           不 能 斜 穿， 只 能 走 街 道
```

```
 44th

欧 氏 距 离 = 直 线 距 离 （穿 楼 而 过）
曼 哈 顿 距 离 = 实 际 要 走 的 路
```

可视化对比:

```
欧 氏 距 离 vs 曼 哈 顿 距 离

     B

            欧 氏 距 离


                A

          曼 哈 顿 距 离
          （先 上 后 右 或 先 右 后 上）

                →
```

2.4.3 6.2.3 闵可夫斯基距离（Minkowski Distance）

欧氏距离和曼哈顿距离其实是同一个”家族”的成员！

定义 6.4（闵可夫斯基距离）:

$$d_{Minkowski}(x,y) = \left(\sum_{i=1}^{d} |x_i - y_i|^p\right)^{\frac{1}{p}}$$

其中 p 是一个参数：- 当 $p = 1$ 时，就是**曼哈顿距离** - 当 $p = 2$ 时，就是**欧氏距离** - 当 $p \to \infty$ 时，就是**切比雪夫距离**（$d(x,y) = \max_i |x_i - y_i|$）

```
闵 可 夫 斯 基 距 离 的 可 视 化 （单 位 圆）

不 同p值 下 的"等 距 线":

p=1 （曼 哈 顿）:     p=2 （欧 氏）:       p=∞ （切 比 雪 夫）:




  菱 形                 圆 形                 正 方 形
```

2.4.4 6.2.4 其他距离度量

余弦相似度（Cosine Similarity）：

$$similarity(x, y) = \frac{x \cdot y}{||x|| \cdot ||y||} = \frac{\sum_{i=1}^{d} x_i y_i}{\sqrt{\sum_{i=1}^{d} x_i^2} \cdot \sqrt{\sum_{i=1}^{d} y_i^2}}$$

余弦相似度关注向量的方向而非大小，常用于文本分类。

```
余 弦 相 似 度  vs  欧 氏 距 离



                B



                 →  A

欧 氏 距 离  =  A 到 B 的 直 线 长 度
余 弦 相 似 度  =  向 量 OA 和 OB 的 夹 角 余 弦

即 使 A 和 B 长 度 不 同 （一 个 远 一 个 近），
只 要 方 向 相 同， 余 弦 相 似 度 就 为 1
```

2.5 6.3 K 值的选择

2.5.1 6.3.1 K 值对模型的影响

K 值是 KNN 算法中最重要的超参数。不同的 K 值会产生完全不同的决策边界：

```
K 值 选 择 的 影 响 可 视 化：

K=1 （过 拟 合）            K=5 （适 中）              K=20 （欠 拟 合）









    锯 齿 边 界                  平 滑 边 界                  过 于 平 滑
    对 噪 声 敏 感              较 理 想                  丢 失 细 节
```

2.5.2 6.3.2 K 值选择的经验法则

经验法则 6.1：

1. **K 不宜过小**：
   - K=1 时，模型对噪声极其敏感
   - 一个异常值就可能导致错误分类
   - 容易过拟合
2. **K 不宜过大**：
   - 过大的 K 会包含过多远离的样本
   - 决策边界变得过于平滑
   - 容易欠拟合
3. **经验公式**：
$$K \approx \sqrt{n}$$
   其中 n 是训练样本数
4. **交叉验证**：最佳实践是使用交叉验证来选择 K 值

2.5.3 6.3.3 投票策略

等权重投票：每个邻居都有相同的投票权。

距离加权投票：更近的邻居应该有更大的发言权：

$$\hat{y} = \arg\max_{c} \sum_{x_i \in N_K(x)} \frac{1}{d(x, x_i) + \epsilon} \cdot I(y_i = c)$$

其中 ϵ 是一个小常数，防止除以零。

```
距离加权投票示例：

假设 K=3，三个邻居的距离和类别：

  邻居        距离        类别

  A          1.0
  B          2.0
  C          5.0


等权重投票：
 : 2票   : 1票  → 预测：

距离加权投票（权重 = 1/距离）：
 : 1/1.0 + 1/2.0 = 1.5
 : 1/5.0 = 0.2
```

```
→ 预测：  （差距更明显）
```

2.6 6.4 从零实现 KNN

现在让我们完全从零开始实现 KNN 算法——不使用任何外部库，只用纯 Python！

2.6.1 6.4.1 基础 KNN 实现

```python
"""
第六章：K近邻算法 - 从零实现
作者：机器学习教材编写组
参考：Cover & Hart (1967) 经典论文
"""

import math
from collections import Counter


class KNNClassifier:
    """
    K近邻分类器 - 纯Python实现

    理论基础：
    Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification.
    IEEE Transactions on Information Theory, 13(1), 21-27.
    """

    def __init__(self, k=3, distance_metric='euclidean', weights='uniform'):
        """
        初始化KNN分类器

        参数：
            k: 邻居数量（默认3）
            distance_metric: 距离度量方式 ('euclidean', 'manhattan', 'minkowski')
            weights: 投票权重 ('uniform' 等权重, 'distance' 距离加权)
        """
        self.k = k
        self.distance_metric = distance_metric
        self.weights = weights
        self.X_train = None
        self.y_train = None

        print(f"[KNN] 初始化完成：k={k}, metric={distance_metric},
            weights={weights}")
```

```python
    def fit(self, X, y):
        """
        "训练"KNN模型

        注意：KNN是"懒惰学习"（lazy learning），
        所谓的"训练"其实就是把数据存起来！

        参数：
            X: 训练特征，列表的列表 [[x1, x2, ...], ...]
            y: 训练标签，列表 [y1, y2, ...]
        """
        self.X_train = X
        self.y_train = y
        print(f"[KNN] 存储了 {len(X)} 个训练样本")
        return self

    def _euclidean_distance(self, x1, x2):
        """计算欧氏距离"""
        return math.sqrt(sum((a - b) ** 2 for a, b in zip(x1, x2)))

    def _manhattan_distance(self, x1, x2):
        """计算曼哈顿距离"""
        return sum(abs(a - b) for a, b in zip(x1, x2))

    def _minkowski_distance(self, x1, x2, p=3):
        """计算闵可夫斯基距离（默认p=3）"""
        return sum(abs(a - b) ** p for a, b in zip(x1, x2)) ** (1/p)

    def _compute_distance(self, x1, x2):
        """根据配置计算距离"""
        if self.distance_metric == 'euclidean':
            return self._euclidean_distance(x1, x2)
        elif self.distance_metric == 'manhattan':
            return self._manhattan_distance(x1, x2)
        elif self.distance_metric == 'minkowski':
            return self._minkowski_distance(x1, x2)
        else:
            raise ValueError(f"未知的距离度量：{self.distance_metric}")

    def _get_neighbors(self, x):
        """
        找出x的K个最近邻居

        返回：[(距离，标签), ...] 按距离排序
        """
        # 计算到所有训练样本的距离
        distances = []
```

```
        for xi, yi in zip(self.X_train, self.y_train):
            dist = self._compute_distance(x, xi)
            distances.append((dist, yi))

        # 按距离排序，取前K个
        distances.sort(key=lambda x: x[0])
        return distances[:self.k]

    def _vote(self, neighbors):
        """
        根据邻居投票决定类别

        参数:
            neighbors: [(距离, 标签), ...]
        """
        if self.weights == 'uniform':
            # 等权重投票
            votes = [label for _, label in neighbors]
            vote_counts = Counter(votes)
            # 返回得票最多的类别
            return vote_counts.most_common(1)[0][0]

        elif self.weights == 'distance':
            # 距离加权投票
            weighted_votes = {}
            for dist, label in neighbors:
                # 避免除以零
                weight = 1.0 / (dist + 1e-10)
                weighted_votes[label] = weighted_votes.get(label, 0) + weight
            # 返回加权票数最多的类别
            return max(weighted_votes, key=weighted_votes.get)

        else:
            raise ValueError(f"未知的权重类型: {self.weights}")

    def predict_single(self, x):
        """预测单个样本"""
        neighbors = self._get_neighbors(x)
        return self._vote(neighbors)

    def predict(self, X):
        """预测多个样本"""
        return [self.predict_single(x) for x in X]

    def predict_proba(self, x):
        """
        预测概率（每个类别的置信度）
```

```
        返回：{类别：概率，...}
        """
        neighbors = self._get_neighbors(x)

        if self.weights == 'uniform':
            votes = [label for _, label in neighbors]
            vote_counts = Counter(votes)
            total = len(neighbors)
            return {label: count/total for label, count in vote_counts.items()}

        else:  # distance weighted
            weighted_votes = {}
            for dist, label in neighbors:
                weight = 1.0 / (dist + 1e-10)
                weighted_votes[label] = weighted_votes.get(label, 0) + weight
            total = sum(weighted_votes.values())
            return {label: weight/total for label, weight in weighted_votes.items()}


class KNNRegressor:
    """
    K近邻回归器 - 纯Python实现
    """

    def __init__(self, k=3, distance_metric='euclidean', weights='uniform'):
        self.k = k
        self.distance_metric = distance_metric
        self.weights = weights
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        """存储训练数据"""
        self.X_train = X
        self.y_train = y
        return self

    def _compute_distance(self, x1, x2):
        """计算距离（复用分类器的实现）"""
        if self.distance_metric == 'euclidean':
            return math.sqrt(sum((a - b) ** 2 for a, b in zip(x1, x2)))
        elif self.distance_metric == 'manhattan':
            return sum(abs(a - b) for a, b in zip(x1, x2))
        else:
            return sum(abs(a - b) ** 3 for a, b in zip(x1, x2)) ** (1/3)
```

```python
    def predict_single(self, x):
        """预测单个样本的回归值"""
        # 计算距离
        distances = [(self._compute_distance(x, xi), yi)
                     for xi, yi in zip(self.X_train, self.y_train)]
        distances.sort(key=lambda x: x[0])
        neighbors = distances[:self.k]

        if self.weights == 'uniform':
            # 简单平均
            return sum(y for _, y in neighbors) / self.k
        else:
            # 距离加权平均
            weighted_sum = 0
            weight_total = 0
            for dist, y in neighbors:
                weight = 1.0 / (dist + 1e-10)
                weighted_sum += weight * y
                weight_total += weight
            return weighted_sum / weight_total

    def predict(self, X):
        """预测多个样本"""
        return [self.predict_single(x) for x in X]


# ============================================================
# 演示与测试
# ============================================================

if __name__ == "__main__":
    print("=" * 60)
    print("K近邻算法演示")
    print("=" * 60)

    # ========== 演示1：水果分类 ==========
    print("\n【演示1】水果分类问题")
    print("-" * 40)

    # 训练数据：[重量(克), 直径(cm)]
    X_fruit = [
        [150, 7],    # 苹果
        [170, 7.5],  # 苹果
        [140, 6.5],  # 苹果
        [130, 6],    # 苹果
        [200, 8],    # 橙子
        [220, 8.5],  # 橙子
```

```
        [180, 7.8], # 橙子
        [210, 8.2], # 橙子
    ]
    y_fruit = [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']

    # 新来的水果
    new_fruit = [160, 7.2]

    print("训练样本（重量，直径）:")
    for (w, d), label in zip(X_fruit, y_fruit):
        print(f"  ({w}g, {d}cm) → {label}")

    print(f"\n新水果：({new_fruit[0]}g, {new_fruit[1]}cm)")

    # 训练KNN模型
    knn = KNNClassifier(k=3)
    knn.fit(X_fruit, y_fruit)

    # 预测
    prediction = knn.predict_single(new_fruit)
    probabilities = knn.predict_proba(new_fruit)

    print(f"\n预测结果：{prediction}")
    print("各类别概率:")
    for label, prob in sorted(probabilities.items()):
        bar = ' ' * int(prob * 20)
        print(f"  {label}: {prob:.2%} {bar}")

    # ========== 演示2：不同距离度量的比较 ==========
    print("\n【演示2】不同距离度量的比较")
    print("-" * 40)

    point_a = [0, 0]
    point_b = [3, 4]

    print(f"点A：{point_a}")
    print(f"点B：{point_b}")

    metrics = ['euclidean', 'manhattan', 'minkowski']
    for metric in metrics:
        knn_temp = KNNClassifier(k=1, distance_metric=metric)
        dist = knn_temp._compute_distance(point_a, point_b)
        print(f"  {metric:12s}: {dist:.2f}")

    print("\n  欧氏距离 = √(3² + 4²) = 5.0")
    print("  曼哈顿距离 = |3| + |4| = 7.0")
    print("  闵可夫斯基(p=3) = (3³ + 4³)^(1/3)  4.50")
```

```python
# ========== 演示3：K值选择的影响 ==========
print("\n【演示3】K值选择的影响")
print("-" * 40)

# 构造一个稍微复杂的数据集
X_complex = [
    [1, 1], [1, 2], [2, 1],  # 类别A聚集
    [5, 5], [5, 6], [6, 5],  # 类别B聚集
    [3, 3],  # 边界点，靠近A
]
y_complex = ['A', 'A', 'A', 'B', 'B', 'B', 'A']

test_point = [3, 3.5]  # 测试点

print(f"测试点：{test_point}")
print(f"\n不同K值的预测结果:")

for k in [1, 3, 5, 7]:
    knn_k = KNNClassifier(k=k)
    knn_k.fit(X_complex, y_complex)
    pred = knn_k.predict_single(test_point)
    print(f"  K={k}: 预测类别 = {pred}")

# ========== 演示4：房价预测（回归） ==========
print("\n【演示4】房价预测（KNN回归）")
print("-" * 40)

# 训练数据：[面积(平米), 卧室数]
X_house = [
    [50, 1], [60, 1], [80, 2], [90, 2],
    [100, 3], [120, 3], [150, 4], [200, 5]
]
y_house = [100, 120, 160, 180, 250, 300, 400, 550]  # 价格（万元）

# 新房子
new_house = [110, 3]

print("训练样本（面积，卧室）→ 价格:")
for (area, rooms), price in zip(X_house, y_house):
    print(f"  ({area}m², {rooms}室) → {price}万元")

print(f"\n新房子：({new_house[0]}m², {new_house[1]}室)")

# 使用KNN回归
knn_reg = KNNRegressor(k=3, weights='distance')
knn_reg.fit(X_house, y_house)
```

```
    predicted_price = knn_reg.predict_single(new_house)

    print(f"\n预测价格：{predicted_price:.1f}万元")

    # ========== 演示5：权重策略对比 ==========
    print("\n【演示5】等权重 vs 距离加权")
    print("-" * 40)

    # 构造有噪声的数据
    X_noisy = [[1], [2], [3], [10]]  # 10是噪声点
    y_noisy = ['A', 'A', 'A', 'B']   # 大多数是A

    test_x = [[2.5]]

    print(f"训练数据：{list(zip(X_noisy, y_noisy))}")
    print(f"测试点：{test_x[0]}")

    for weight_type in ['uniform', 'distance']:
        knn_w = KNNClassifier(k=3, weights=weight_type)
        knn_w.fit(X_noisy, y_noisy)
        pred = knn_w.predict_single(test_x[0])
        prob = knn_w.predict_proba(test_x[0])
        print(f"\n{weight_type}权重：")
        print(f"  预测：{pred}")
        print(f"  概率：{prob}")

    print("\n" + "=" * 60)
    print("KNN算法演示完成！")
    print("=" * 60)
```

2.6.2 6.4.2 运行结果

当你运行上面的代码时，会看到类似这样的输出：

```
============================================================
K近邻算法演示
============================================================

【演示1】水果分类问题
----------------------------------------
训练样本（重量，直径）：
  (150g, 7cm) →
  (170g, 7.5cm) →
  ...
新水果：(160g, 7.2cm)

预测结果：
```

```
各类别概率：
   : 66.67%
   : 33.33%

【演示2】不同距离度量的比较
----------------------------------------
点A: [0, 0]
点B: [3, 4]
  euclidean   : 5.00
  manhattan   : 7.00
  minkowski   : 4.50
```

2.7 6.5 历史溯源

2.7.1 6.5.1 从 Fix 和 Hodges 到 Cover 和 Hart

KNN 算法的发展历史是一部”从直觉到理论”的演进史：

1951 年：非参数判别分析的萌芽

Evelyn Fix 和 Joseph L. Hodges 在美国空军学校工作时，发表了《Discriminatory Analysis, Nonparametric Discrimination: Consistency Properties》。这篇论文首次系统地讨论了非参数分类方法，虽然没有明确提出”K 近邻”这个名字，但已经蕴含了核心思想。

1967 年：理论奠基

Thomas M. Cover 和 Peter E. Hart 在 IEEE Transactions on Information Theory 上发表了里程碑论文《Nearest Neighbor Pattern Classification》。他们证明了：

> **定理（Cover-Hart, 1967）**：对于任意分布，最近邻规则的错误率 R_{NN} 满足：
>
> $$R^* \leq R_{NN} \leq 2R^*(1 - R^*) \leq 2R^*$$
>
> 其中 R^* 是贝叶斯最优错误率。

这个定理的美妙之处在于它的**普适性**——不需要对数据分布做任何假设！

1985 年：模糊 KNN

James M. Keller 等人提出了模糊 KNN（Fuzzy KNN），将模糊集合理论引入 KNN，允许一个样本以不同的隶属度属于多个类别。

2.7.2 6.5.2 关键人物简介

Thomas M. Cover (1938-2012) - 斯坦福大学电气工程与统计学教授 - 信息论和模式识别领域的先驱 - 与 Joy A. Thomas 合著的《Elements of Information Theory》是该领域的经典教材 - 被选为美国国家工程院院士

Peter E. Hart (1940-) - 斯坦福人工智能实验室（SAIL）的研究员 - 后来创办了计算机视觉公司 - 在模式识别和人工智能领域做出了开创性贡献

2.8 6.6 进阶主题

2.8.1 6.6.1 维度灾难（Curse of Dimensionality）

KNN 在高维空间会遇到一个严重的问题：

```
维度灾难的可视化：

低维(2D):                高维(100D):
                          ........
 距离有意义               所有距离都变得差不多大

随着维度增加：
- 数据变得稀疏
- 距离度量失去区分能力
- 需要的样本指数级增长
```

解决方案：1. 特征选择（Feature Selection）2. 降维（PCA, t-SNE）3. 局部敏感哈希（LSH）

2.8.2 6.6.2 加速 KNN

原始 KNN 的时间复杂度是 $O(nd)$ 每次查询，其中 n 是样本数，d 是维度。对于大规模数据，这太慢了。

KD 树（K-Dimensional Tree）：

```
KD树结构示例：

           [5, 5]  ← 根节点（按x轴分割）
           /     \
     [2, 3]     [8, 7]  ← 第二层（按y轴分割）
     /    \
[1,2] [3,4]

查询时只需要探索部分分支，
时间复杂度降至 O(log n)
```

其他加速方法：- 球树（Ball Tree）- 局部敏感哈希（LSH）- 近似最近邻（ANN）

2.9 6.7 本章总结

2.9.1 核心概念回顾

```
                          KNN核心知识点
```

```
1. 核心思想：物以类聚——相似的东西在特征空间里靠近

2. 算法步骤：
     计算距离 → 找出K个邻居 → 投票决策

3. 距离度量：
   • 欧氏距离（直线）：√(Σ(xi-yi)²)
   • 曼哈顿距离（街区）：Σ|xi-yi|
   • 闵可夫斯基距离：(Σ|xi-yi|^p)^(1/p)

4. K值选择：
   • K太小 → 对噪声敏感
   • K太大 → 决策边界过于平滑
   • 经验法则：K  √n

5. 历史里程碑：
   • 1951: Fix & Hodges 非参数判别分析
   • 1967: Cover & Hart 理论证明（错误率 2×贝叶斯最优）
   • 1985: Keller 模糊KNN
```

2.9.2 与其他算法的关系

算法	训练阶段	预测阶段	核心思想
KNN	存储数据	计算距离 + 投票	相似性
决策树	构建树结构	遍历树	特征划分
神经网络	参数优化	前向传播	函数逼近

KNN 是**实例学习（Instance-based Learning）**的代表，与 ** eager learning**（如神经网络）形成对比。

2.10 6.8 练习题

2.10.1 基础题

练习 6.1：距离计算

给定两个二维点 $A = (3, 4)$ 和 $B = (6, 8)$：

(a) 计算它们之间的欧氏距离

(b) 计算它们之间的曼哈顿距离

(c) 如果闵可夫斯基距离的 $p = 3$，结果是多少？

点击查看答案

(a) 欧氏距离：$d = \sqrt{(6-3)^2 + (8-4)^2} = \sqrt{9+16} = \sqrt{25} = 5$

(b) 曼哈顿距离：$d = |6-3| + |8-4| = 3 + 4 = 7$

(c) 闵可夫斯基距离：$d = (|6-3|^3 + |8-4|^3)^{1/3} = (27+64)^{1/3} = 91^{1/3} \approx 4.50$

练习 6.2：KNN 决策

假设在二维平面上有以下训练样本：- 类别 A：(1,1), (2,2), (2,1) - 类别 B：(5,5), (6,6), (5,6)

对于测试点 $(3, 3)$：

(a) 使用 K=1，预测类别是什么?

(b) 使用 K=3，预测类别是什么?

(c) 使用 K=5，预测类别是什么?

2.10.2 进阶题

练习 6.3：K 值与决策边界

解释为什么：

(a) 当 K=1 时，决策边界会非常”崎岖”?

(b) 当 K 接近训练样本总数时，模型会发生什么?

(c) 如何选择一个合适的 K 值?

练习 6.4：距离加权投票

在练习 6.2 中，如果使用 K=3 且采用距离加权投票（权重 = 1/距离），预测结果会改变吗？请计算。

2.10.3 挑战题

练习 6.5：实现 KD 树

扩展本章的 KNN 实现，添加一个基于 KD 树的加速版本。要求：

1. 实现 KD 树的构建
2. 实现最近邻搜索
3. 比较原始 KNN 和 KD 树版本的查询时间

练习 6.6：维度灾难实验

设计一个实验来验证”维度灾难”：

1. 生成不同维度（2, 10, 50, 100, 500）的随机数据
2. 计算所有样本对之间的距离
3. 分析距离的分布（最大值/最小值的比率）
4. 验证随着维度增加，距离是否趋于均匀

2.11 参考文献

2.11.1 经典论文

Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, *13*(1), 21-27. https://doi.org/10.1109/TIT.1967.1053964

Fix, E., & Hodges, J. L. (1951). Discriminatory analysis, nonparametric discrimination: Consistency properties. *USAF School of Aviation Medicine Report*, *4*(10).

2.11.2 重要扩展

Keller, J. M., Gray, M. R., & Givens, J. A. (1985). A fuzzy K-nearest neighbor algorithm. *IEEE Transactions on Systems, Man, and Cybernetics*, *SMC-15*(4), 580-585. https://doi.org/10.1109/TSMC.1985.6313426

Stone, C. J. (1977). Consistent nonparametric regression. *The Annals of Statistics*, *5*(4), 595-620.

2.11.3 教科书与综述

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. (第 13 章: Prototypes and Nearest-Neighbors)

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer. (第 2.5 节: Nonparametric Methods)

2.11.4 历史回顾

Dasarathy, B. V. (Ed.). (1991). *Nearest neighbor (NN) norms: NN pattern classification techniques*. IEEE Computer Society Press.

2.11.5 在线资源

Cover, T. M., & Thomas, J. A. (2006). *Elements of information theory* (2nd ed.). Wiley-Interscience.

2.12 费曼检验回顾

让我们回顾本章的核心概念，用费曼法检验你是否真正理解了：

```
                    费 曼 四 步 检 验 法 - KNN章 节 回 顾

1   选 择 概 念： K近 邻 算 法

2   教 给 别 人： 你 能 用 一 句 话 解 释KNN 吗？
      "KNN通 过 查 看 新 样 本 周 围 最 近 的K个 已 知 样 本，
        然 后 根 据 这 些 邻 居 的 多 数 类 别 来 决 定 新 样 本 的 类 别。"

3   发 现 差 距： 检 查 这 些 概 念 是 否 清 楚
      为 什 么KNN被 称 为"懒 惰 学 习"？
```

```
        → 因为它在训练阶段只是存储数据，不做任何计算

      Cover-Hart定理告诉我们什么？
        → 1-NN的错误率不超过贝叶斯最优错误率的两倍

      欧氏距离和曼哈顿距离的区别？
        → 欧氏是直线，曼哈顿是坐标轴方向的总和

4   简化语言：用比喻检验理解
      "KNN就像是搬进新小区后，问周围的邻居这个小区怎么样"
      "欧氏距离是鸟飞的距离，曼哈顿距离是人走的距离"
      "K值选择就像征求意见时问几个人——问太少可能有偏见，
        问太多又会稀释本地特色"
```

本章完

“简单的想法往往是最深刻的。*KNN* 提醒我们：有时候，最近的邻居比最复杂的模型更懂你。”

---

3 第七章：线性回归——画一条最贴切的线

> “当父母比平均值高很多时，他们的孩子往往比父母要矮；当父母比平均值矮很多时，他们的孩子往往比父母要高。”
> ——弗朗西斯·高尔顿（Francis Galton），1886 年

3.1 费曼四步检验框

在开始之前，让我们用费曼学习法来预览本章的核心概念：

```
                        费曼四步检验法

1   选择概念：线性回归（Linear Regression）

2   教给别人：想象你在向一个小学生解释...
    "想象你在纸上撒了一把豆子，现在要用一根直尺画一条线，
     让这条线尽可能贴近所有豆子。最小二乘法就是找到
     那条'最贴切'的线的方法！"

3   发现差距：当你说"最优"时，需要解释清楚：
    - 什么是"误差"？
    - 为什么用"平方"而不是绝对值？
    - 怎样数学上找到最小值？
```

```
4  简化语言：用生活化的比喻替代专业术语
   "最小二乘法" → "让总误差最小的方法"
   "正规方程" → "一步算出答案的公式"
   "梯度下降" → "一步一步走向山谷最低点"
```

3.2 本章学习目标

在学完本章后，你将能够： 1. 讲述高斯用最小二乘法发现谷神星的传奇故事 2. 理解”回归”一词的起源和高尔顿的身高研究 3. 掌握最小二乘法的数学原理和完整推导过程 4. 手搓实现简单线性回归、多元线性回归和梯度下降训练 5. 理解闭式解和梯度下降两种求解方法的优劣 6. 掌握 R^2 决定系数和相关系数 r 的计算与解释 7. 编写完整的房价预测器项目

3.3 7.1 从高斯与谷神星说起

3.3.1 7.1.1 一颗失踪的小行星

1801 年 1 月 1 日，新年的第一天，意大利天文学家**朱塞佩·皮亚齐**（Giuseppe Piazzi）在西西里岛的巴勒莫天文台发现了一颗新的天体。

“这可能是一颗新行星！”皮亚齐激动地想。他将这颗天体命名为**谷神星**（Ceres），这是罗马神话中的丰收女神。

在接下来的 40 天里，皮亚齐每晚都追踪记录谷神星的位置。但悲剧发生了——谷神星运行到了太阳背后，从地球上看不到了！太阳的光芒掩盖了这颗小行星。

问题是：当谷神星从太阳背后再次出现时，它会在哪里？

如果不能准确预测，这颗新发现的小行星可能就永远丢失了。整个欧洲的天文学家都在努力计算谷神星的轨道，但都失败了。

3.3.2 7.1.2 24 岁天才的崛起

这个问题落在了 24 岁的德国数学家**卡尔·弗里德里希·高斯**（Carl Friedrich Gauss）手中。

当时的数学家们都在用复杂的**开普勒方程**来计算行星轨道，需要解非线性方程组，计算极其繁琐。但高斯想出了一个全新的方法。

他只做了一件事：**最小化观测数据与预测轨道之间的误差平方和**。

具体来说，假设行星轨道可以用一个二次曲线（椭圆）描述，高斯要找到一组轨道参数，使得：

$$\text{总误差} = \sum_{i=1}^{n} (\text{观测位置}_i - \text{预测位置}_i)^2$$

达到最小。

1801 年 12 月，高斯只用了皮亚齐记录的 40 天数据（总共只有 3 度的弧段），就准确地预测了谷神星的位置。

当年 12 月 31 日——正好是一年后——匈牙利天文学家**弗朗茨·冯·扎克**（Franz Xaver von Zach）根据高斯的预测，在预测位置上重新找到了谷神星！

这是最小二乘法在历史上第一次大放异彩。高斯一夜成名。

3.3.3 7.1.3 优先权之争：高斯 vs 勒让德

然而，故事的另一面却充满了争议。

高斯直到 1809 年才在他的著作《天体运动论》（*Theoria Motus Corporum Coelestium*）中发表了最小二乘法，并声称自己从 1795 年（18 岁时）就开始使用这个方法了。

但是！法国数学家**阿德里安-马里·勒让德**（Adrien-Marie Legendre）在 1805 年就发表了一篇论文《确定彗星轨道的新方法》（*Nouvelles méthodes pour la détermination des orbites des comètes*），**首次公开提出了最小二乘法**，并明确命名了这个方法（法语：*méthode des moindres carrés*）。

这就引发了一场数学史上著名的**优先权之争**。

勒让德愤怒地写道： > “我使用最小二乘法已经很多年了，并且把它推荐给了天文学家同行…高斯大可以在他的书中说他也在用这个方法，但为什么不说这个方法是我发明的呢?”

高斯则坚持： > “我在 1795 年就已经发现了这个方法，比勒让德早 10 年。”

历史学家们普遍认为：- **勒让德**有发表的优先权（1805 年）- **高斯**提供了更系统的理论基础（1809 年）- 两人很可能是**独立发现**了这个方法

无论如何，最小二乘法从此成为了天文学、大地测量学和统计学的基石。

3.4 7.2 什么是最小二乘法?

3.4.1 7.2.1 直觉理解：让误差最小

让我们回到一个更简单的问题。

假设你是一名房地产经纪人，需要根据房屋面积来预测房价。你有以下数据：

房屋面积 (m^2)	房价 (万元)
50	150
80	220
100	280
120	320
150	400

你想找到一条直线来描述面积和房价之间的关系：

$$\text{房价} = w \times \text{面积} + b$$

在纸上画出来大概是这样：

```
房 价（万 元）
```

```
400

320

280

220

150
                                                  → 面 积（m²）
          50    80   100   120   150
```

问题是：**怎么找到最好的 w 和 b?**

最小二乘法的答案是：**让所有数据点到直线的垂直距离的平方和最小**。

想象每条数据点都有一条”误差线”连接到预测直线：

```
房 价（万 元）

400

320
                                  ← 误 差 = 真 实 值 - 预 测 值
280

220
          预 测 直 线
150
                                                  → 面 积（m²）
          50    80   100   120   150
```

为什么要用”平方”?

1. **平方总是正的**：避免了正负误差相互抵消的问题
2. **平方会放大大的误差**：让直线更关注偏离较远的”异常点”
3. **数学上容易处理**：求导后变成线性方程，容易求解

3.4.2 7.2.2 数学定义

给定 n 个数据点 $(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)$，我们要找到参数 w 和 b，使得**残差平方和**（Residual Sum of Squares, RSS）最小：

$$\text{RSS} = \sum_{i=1}^{n} (y_i - (wx_i + b))^2$$

这里 $(wx_i + b)$ 是直线的预测值 $\hat{y}_i$，y_i 是真实值，两者之差 $e_i = y_i - \hat{y}_i$ 称为**残差**（residual）。

我们的目标就是：

$$\min_{w,b} \sum_{i=1}^{n} (y_i - wx_i - b)^2$$

3.5 7.3 画一条最贴切的线

3.5.1 7.3.1 线性回归模型

线性回归模型非常简单：

单变量线性回归（一个特征）：

$$y = wx + b$$

多变量线性回归（多个特征）：

$$y = w_1x_1 + w_2x_2 + ... + w_dx_d + b$$

或者用向量形式：

$$y = \mathbf{w}^T\mathbf{x} + b$$

其中：- $\mathbf{x} = [x_1, x_2, ..., x_d]^T$ 是特征向量（输入）- $\mathbf{w} = [w_1, w_2, ..., w_d]^T$ 是权重向量（待学习的参数）- b 是偏置（截距）- y 是预测值（输出）

3.5.2 7.3.2 几何解释

在二维平面上，线性回归就是找到一条直线，使得所有数据点到这条直线的垂直距离（在 y 轴方向）的平方和最小。

```
        y

                ×

    ŷ                   ← 预 测 点 （在 直 线 上）
                ↑
                    e  = y  - ŷ （残 差 / 误 差）
    y     ×             ← 真 实 数 据 点

                        →
            x
```

3.5.3 7.3.3 损失函数

我们把需要最小化的函数称为**损失函数**（Loss Function）或**损失函数**（Cost Function）：

$$J(w, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - (wx_i + b))^2$$

有时也写作（带 1/2 的系数，方便求导）：

$$J(w,b)=\frac{1}{2n}\sum_{i=1}^{n}(y_i-(wx_i+b))^2$$

这个函数描述了模型”犯错”的程度——值越小，说明拟合越好。

3.6 7.4 数学推导：如何找到最优解？

3.6.1 7.4.1 单变量线性回归的推导

让我们完整推导单变量线性回归的最小二乘解。

目标： 最小化 $J(w,b)=\sum_{i=1}^{n}(y_i-wx_i-b)^2$

第一步：对 b 求偏导并令其为零

$$\frac{\partial J}{\partial b}=\sum_{i=1}^{n}2(y_i-wx_i-b)(-1)=0$$

化简：

$$\sum_{i=1}^{n}(y_i-wx_i-b)=0$$

$$\sum_{i=1}^{n}y_i=w\sum_{i=1}^{n}x_i+nb$$

两边除以 n：

$$\bar{y}=w\bar{x}+b$$

其中 $\bar{x}=\frac{1}{n}\sum x_i$，$\bar{y}=\frac{1}{n}\sum y_i$ 分别是 x 和 y 的均值。

由此得到：

$$b=\bar{y}-w\bar{x} \tag{1}$$

第二步：对 w 求偏导并令其为零

$$\frac{\partial J}{\partial w}=\sum_{i=1}^{n}2(y_i-wx_i-b)(-x_i)=0$$

化简：

$$\sum_{i=1}^{n} x_i(y_i - wx_i - b) = 0$$

$$\sum_{i=1}^{n} x_i y_i = w\sum_{i=1}^{n} x_i^2 + b\sum_{i=1}^{n} x_i \tag{2}$$

第三步：将 (1) 式代入 (2) 式

将 $b = \bar{y} - w\bar{x}$ 代入：

$$\sum x_i y_i = w\sum x_i^2 + (\bar{y} - w\bar{x})\sum x_i$$

$$\sum x_i y_i = w\sum x_i^2 + \bar{y}\sum x_i - w\bar{x}\sum x_i$$

整理：

$$\sum x_i y_i - \bar{y}\sum x_i = w\left(\sum x_i^2 - \bar{x}\sum x_i\right)$$

第四步：求解 w

注意到 $\bar{y}\sum x_i = \frac{1}{n}\sum y_i \sum x_i = \bar{x}\sum y_i$，所以：

$$w = \frac{\sum x_i y_i - \bar{x}\sum y_i}{\sum x_i^2 - \bar{x}\sum x_i} = \frac{\sum x_i y_i - n\bar{x}\bar{y}}{\sum x_i^2 - n\bar{x}^2}$$

可以进一步改写为：

$$w = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} = \frac{\mathrm{Cov}(X, Y)}{\mathrm{Var}(X)}$$

最终结果：

$$\boxed{w = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}}$$

$$\boxed{b = \bar{y} - w\bar{x}}$$

这个公式的直观意义：

- 斜率 w 等于 X 和 Y 的协方差除以 X 的方差
- 截距 b 确保直线通过数据中心点 $(\bar{x}, \bar{y})$
- 当 X 和 Y 正相关时，$w > 0$（上升趋势）；负相关时，$w < 0$（下降趋势）

3.6.2 7.4.2 多元线性回归的矩阵形式

对于多元线性回归，使用矩阵表示会更加简洁。

设：- $\mathbf{X}$ 是 $n \times (d+1)$ 的设计矩阵（包含一列全 1 用于偏置）- $\theta = [b, w_1, w_2, ..., w_d]^T$ 是 $(d+1) \times 1$ 的参数向量 - $\mathbf{y}$ 是 $n \times 1$ 的目标值向量

模型预测：$\hat{\mathbf{y}} = \mathbf{X}\theta$

损失函数：

$$J(\theta) = (\mathbf{X}\theta - \mathbf{y})^T(\mathbf{X}\theta - \mathbf{y})$$

展开：

$$J(\theta) = \theta^T\mathbf{X}^T\mathbf{X}\theta - 2\theta^T\mathbf{X}^T\mathbf{y} + \mathbf{y}^T\mathbf{y}$$

对 θ 求导并令其为零：

$$\frac{\partial J}{\partial \theta} = 2\mathbf{X}^T\mathbf{X}\theta - 2\mathbf{X}^T\mathbf{y} = 0$$

解得：

$$\mathbf{X}^T\mathbf{X}\theta = \mathbf{X}^T\mathbf{y}$$

$$\boxed{\theta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}}$$

这就是著名的**正规方程**（Normal Equation）！

3.7 7.5 两种求解方法：闭式解 vs 梯度下降

现在我们有了两种求解线性回归的方法：

方法	原理	复杂度	适用场景
闭式解（正规方程）	直接计算 $(X^TX)^{-1}X^Ty$	$O(d^3)$	特征数量 d 较小（<10000）
梯度下降	迭代优化，逐步逼近最优解	每次迭代 $O(nd)$	特征数量大、数据量大

3.7.1 7.5.1 闭式解（正规方程）

优点： - 不需要选择学习率 - 不需要迭代，一步得到精确解 - 不需要特征缩放

缺点： - 计算 $(X^TX)^{-1}$ 的时间复杂度是 $O(d^3)$ - 当 d 很大时（如 $d > 10000$），计算很慢 - 如果 X^TX 不可逆（奇异矩阵），则无法使用

3.7.2 7.5.2 梯度下降

梯度下降是一种通用的优化算法，适用于各种损失函数。

对于单变量线性回归：

损失函数对参数的偏导数：

$$\frac{\partial J}{\partial w} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)x_i$$

$$\frac{\partial J}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)$$

参数更新规则（α 是学习率）：

$$w := w - \alpha \cdot \frac{\partial J}{\partial w}$$

$$b := b - \alpha \cdot \frac{\partial J}{\partial b}$$

直观理解：

想象你在山上，想要走到山谷最低点。梯度告诉你”最陡的下坡方向”，你沿着这个方向走一小步，重复这个过程，最终就能到达最低点。

```
损 失 函 数 J
                    ← 损 失 函 数 曲 面
        ↓
              → w
梯 度 下 降： 从 某 点 出 发， 沿 着 梯 度 反 方 向 一 步 步 走 向 最 小 值
```

梯度下降的优点： - 适合大规模数据集 - 可以在线学习（来一个新样本就更新一次）- 可以扩展到更复杂的模型（如神经网络）

梯度下降的缺点： - 需要选择合适的学习率 - 需要多次迭代才能收敛 - 可能陷入局部最小值（但线性回归的损失函数是凸函数，只有一个全局最小值）

3.8 7.6 如何判断拟合得好不好?

3.8.1 7.6.1 R² 决定系数

R^2（R-squared，决定系数）是衡量回归模型拟合优度最常用的指标。

定义：

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

其中：- **RSS**（Residual Sum of Squares）= 残差平方和 = 模型无法解释的变异 - **TSS**（Total Sum of Squares）= 总平方和 = 数据的总变异

解释：

- $R^2 = 1$：完美拟合，模型解释了 100% 的变异
- $R^2 = 0$：模型表现和直接用均值预测一样
- $R^2 < 0$：模型比直接用均值预测还差（过拟合时可能发生）

直观理解：

R^2 表示模型能够解释的目标变量变异的比例。

比如 $R^2 = 0.85$，意味着模型解释了 85% 的房价变异，剩下 15% 是由其他因素（如地段、装修等）或随机误差造成的。

3.8.2 7.6.2 相关系数 r

皮尔逊相关系数 r 衡量两个变量之间线性关系的强度和方向：

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

性质：

- $-1 \leq r \leq 1$
- $r > 0$：正相关（X 增加，Y 倾向于增加）
- $r < 0$：负相关（X 增加，Y 倾向于减少）
- $r = 0$：无线性相关（但可能有非线性关系）
- $|r|$ 越接近 1，线性关系越强

与线性回归的关系：

在单变量线性回归中，相关系数 r 和斜率 w 同号，且：

$$w = r \cdot \frac{\sigma_y}{\sigma_x}$$

其中 σ_x 和 σ_y 分别是 X 和 Y 的标准差。

重要区分：

概念	含义	范围
r（相关系数）	两个变量的线性相关程度	$[-1, 1]$
R^2（决定系数）	模型解释的数据变异比例	$[0, 1]$
w（斜率）	X 每变化一个单位，Y 变化多少	$(-\infty, +\infty)$

3.8.3 7.6.3 均方误差（MSE）和均方根误差（RMSE）

MSE（Mean Squared Error）：

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

RMSE（Root Mean Squared Error）：

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

RMSE 的优点是和原数据有相同的单位（如房价预测中，RMSE 的单位也是”万元”），更容易解释。

3.9 7.7 相关与回归的关系

3.9.1 7.7.1 “回归”一词的起源

1885-1886 年，英国科学家**弗朗西斯·高尔顿**（Francis Galton）进行了一项关于人类身高的研究。

他收集了 205 对夫妻和他们的 928 个成年子女的身高数据，发现了一个令人惊讶的现象：

> **高个子的父母，他们的孩子往往比父母要矮；而矮个子的父母，他们的孩子往往比父母要高。**

用高尔顿的话说：

> “当父母的身高偏离平均值时，子女的身高往往会向平均值回归。具体来说，子女的偏离程度大约是父母的 2/3。”

他把这种现象称为”**回归平庸**”（Regression towards Mediocrity）。

例子：

假设平均身高是 1.70 米。

- 父亲身高 1.85 米（比平均高 15 厘米）
- 预测儿子身高：$1.70 + \frac{2}{3} \times 0.15 = 1.80$ 米

儿子还是比平均高，但没有父亲那么高！

3.9.2 7.7.2 相关不等于因果

高尔顿的研究还发现，虽然身高在代际间”回归”，但父母身高和子女身高的**相关系数**大约是 0.5。

这是一个重要的统计洞见：

```
  重要提醒：相关不等于因果！

- 冰淇淋销量和溺水事故正相关 → 不是因为冰淇淋导致溺水
  （而是因为夏天热了，两者都增加）

- 学生的鞋码和阅读成绩正相关 → 不是因为大脚让人更会阅读
  （而是因为年龄大的孩子脚更大、阅读能力更强）
```

线性回归可以帮助我们量化变量之间的关系，但不能自动告诉我们这种关系是不是因果关系。确定因果需要更严格的实验设计或因果推断方法。

3.9.3 7.7.3 卡尔·皮尔逊的贡献

1896 年，高尔顿的学生**卡尔·皮尔逊**（Karl Pearson）发表了相关系数的数学公式，将高尔顿的直观概念形式化。

皮尔逊还建立了第一个统计学系（伦敦大学学院），培养了第一代专业统计学家。他创办的期刊 *Biometrika* 至今仍是统计学顶级期刊。

皮尔逊相关系数 r 现在被广泛使用，但它只能捕捉**线性关系**。对于非线性关系（如 $y = x^2$），r 可能接近 0，即使两个变量有很强的关系。

3.10 7.8 实战：手写线性回归

现在，让我们手搓一个完整的线性回归实现！不使用 NumPy，只用纯 Python，这样你能真正理解每一个细节。

3.10.1 7.8.1 完整代码实现

见 `chapter-07-code.py` 文件。

3.10.2 7.8.2 运行结果示例

```
======================================================================
  简单线性回归 - 房价预测（最小二乘闭式解）
======================================================================

  训练数据：
   面积（m²）: [50, 80, 100, 120, 150]
   房价（万元）: [150, 220, 280, 320, 400]

  模型参数：
   斜率 w = 2.5000（每平米价格）
   截距 b = 24.0000（基础价格）
   相关系数 r = 0.9990

  回归方程：
   房价 = 2.50 × 面积 + 24.00
```

```
预测结果：
 60 m² → 174.00 万元
 90 m² → 249.00 万元
 110 m² → 299.00 万元
 180 m² → 474.00 万元

拟合优度 R² = 0.9981
  （R² = 1 表示完美拟合，R² = 0 表示不比直接用均值预测好）
```

3.11 7.9 本章小结

3.11.1 核心概念回顾

概念	解释	公式
最小二乘法	最小化残差平方和的参数估计方法	$\min \sum (y_i - \hat{y}_i)^2$
回归	高尔顿 1886 年提出的概念，描述极端值趋向均值的现象	-
简单线性回归	只有一个特征	$y = wx + b$
多元线性回归	多个特征	$y = \mathbf{w}^T\mathbf{x} + b$
正规方程	闭式解，直接计算最优参数	$\theta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$
梯度下降	迭代优化方法	$\theta := \theta - \alpha\nabla J$
$\mathbf{R}^2$ 决定系数	衡量模型拟合优度	$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}$
相关系数 r	衡量两个变量的线性相关程度	$r = \frac{\text{Cov}(X,Y)}{\sigma_X\sigma_Y}$

3.11.2 历史时间线

```
1795年 — 高斯（18岁）首次使用最小二乘法
1801年 — 高斯用最小二乘法预测谷神星轨道，一战成名！
1805年 — 勒让德发表《确定彗星轨道的新方法》，首次公开最小二乘法
1809年 — 高斯发表《天体运动论》，系统化最小二乘法理论，引发优先权之争
1885年 — 高尔顿发现"回归"现象，研究人类身高遗传
1886年 — 高尔顿发表《Regression towards Mediocrity in Hereditary Stature》
1896年 — 皮尔逊发表相关系数的数学公式
1897年 — 尤尔证明最小二乘法可用于回归分析
```

3.11.3 工具箱

```
# 你现在可以手搓：
  简单线性回归（最小二乘法解析解）
  多元线性回归（正规方程）
  梯度下降训练线性回归
```

```
R² 决定系数计算
皮尔逊相关系数计算
矩阵运算（转置、乘法、求逆）
```

3.11.4 关键洞见

1. **最小二乘法**是数据科学最重要的工具之一，从高斯预测谷神星到现代机器学习，已经使用了 200 多年
2. **闭式解**适合小规模数据，**梯度下降**适合大规模数据
3. R^2 衡量模型解释的变异比例，r 衡量线性相关程度
4. **相关不等于因果**——统计关系需要谨慎解释
5. **回归**一词源于高尔顿的身高研究，描述的是极端值向均值靠拢的现象

3.12 7.10 练习题

3.12.1 基础练习

7.1 用最小二乘法公式计算以下数据的回归直线：

```
X = [1, 2, 3, 4, 5]
y = [2, 4, 5, 4, 5]
```

要求：- 计算 $\bar{x}$ 和 $\bar{y}$ - 计算斜率 w 和截距 b - 写出回归方程 - 计算 R^2

7.2 解释为什么最小二乘法使用”平方”而不是”绝对值”。

7.3 已知某线性回归模型的 $R^2 = 0.85$，这意味着什么？如果 $R^2 = 0$ 呢？

3.12.2 进阶挑战

7.4 正规方程推导
从损失函数 $J(\theta) = (\mathbf{X}\theta - \mathbf{y})^T(\mathbf{X}\theta - \mathbf{y})$ 出发，证明 $\theta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ 是最优解。

7.5 相关系数与斜率的关系
证明在单变量线性回归中：

$$w = r \cdot \frac{\sigma_y}{\sigma_x}$$

其中 r 是相关系数，σ_x 和 σ_y 分别是 X 和 Y 的标准差。

7.6 挑战题：岭回归（Ridge Regression）
当特征之间存在多重共线性时，$\mathbf{X}^T\mathbf{X}$ 可能不可逆。岭回归通过添加 L2 正则化解决这个问题：

$$\theta = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

实现岭回归类 `RidgeRegression`，并测试它在以下数据上的表现：

```
X = [[1, 2], [2, 4], [3, 6], [4, 8]]  # 注意：第二列是第一列的2倍（共线性）
y = [3, 6, 9, 12]
```

比较普通最小二乘法和岭回归的结果。

3.13 参考文献

1. Gauss, C. F. (1809). *Theoria motus corporum coelestium in sectionibus conicis solem ambientium.* Hamburg: Perthes et Besser.
2. Legendre, A. M. (1805). *Nouvelles méthodes pour la détermination des orbites des comètes.* Paris: Courcier.
3. Galton, F. (1886). Regression towards mediocrity in hereditary stature. *The Journal of the Anthropological Institute of Great Britain and Ireland*, 15, 246-263.
4. Galton, F. (1889). *Natural Inheritance.* London: Macmillan.
5. Pearson, K. (1896). Mathematical contributions to the theory of evolution. III. Regression, heredity, and panmixia. *Philosophical Transactions of the Royal Society of London. A*, 187, 253-318.
6. Yule, G. U. (1897). On the theory of correlation. *Journal of the Royal Statistical Society*, 60(4), 812-854.
7. Stigler, S. M. (1986). *The History of Statistics: The Measurement of Uncertainty before 1900.* Cambridge, MA: Harvard University Press.
8. Plackett, R. L. (1972). The discovery of the method of least squares. *Biometrika*, 59(2), 239-251.
9. Stahl, S. (2006). The evolution of the normal distribution. *Mathematics Magazine*, 79(2), 96-113.
10. Seber, G. A. F., & Lee, A. J. (2003). *Linear Regression Analysis* (2nd ed.). Hoboken, NJ: John Wiley & Sons.
11. Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267-288.

3.14 费曼四步检验框

第一步：选择概念——线性回归和最小二乘法

第二步：教给小学生——> “想象你在纸上撒了一把豆子，现在要用一根直尺画一条线，让这条线尽可能贴近所有豆子。最小二乘法就是找到那条’最贴切’的线的方法！高斯用这个方法找到了失踪的小行星谷神星！”

第三步：发现卡壳的地方——> 如果卡在对 w 和 b 求偏导，记住：我们要求的是让误差最小的那组参数，就像找山谷的最低点一样。梯度告诉你要往哪个方向走，学习率决定每一步迈多大。

第四步：用类比简化——> 线性回归就像调收音机：你转一个旋钮（调整斜率），按一个按钮（调整截距），直到声音最清晰（误差最小）。最小二乘法就是告诉你”最佳频道”的公式！

“回归”这个词已经沿用 *130* 多年了。当你下次听到它时，请记住：它最初描述的是一个关于身高的统计现象——高个子父母的孩子往往没有父母那么高。这个现象启发了整个统计学领域的发展。

下一章预告：第八章《逻辑回归——分类的艺术》——从预测连续值到预测类别，从”是多少”到”是不是”。

写作完成时间：*2026* 年 *3* 月 *24* 日
字数统计：约 *8,800* 字
代码行数：约 *500* 行
参考文献：*11* 篇 *APA* 格式

---

4 第八章：逻辑回归——分类的艺术

> “逻辑回归不是一个回归算法，而是一个分类算法。它的名字是个美丽的误会。”
>
> **——一个关于概率与决策的故事**

4.1 写在前面的话

在第七章，我们学会了用一条直线去**预测数值**——房价、温度、考试成绩。但生活中还有更多问题不是问”多少”，而是问”是不是”。

- 这封邮件是**垃圾邮件**还是正常邮件?
- 这个病人**有疾病**还是健康?
- 这笔交易是**欺诈**还是正常?

这些问题都有一个共同点：**答案只有两种可能**。

这时候，线性回归就无能为力了。如果你用线性回归预测”是不是垃圾邮件”，可能会得到 1.5（超过 1）或-0.5（负数）这种没有意义的答案。

我们需要一种新方法——**逻辑回归**。

它用一条”S 形曲线”把任何数字压缩到 0 和 1 之间，这个数字就可以解释为**概率**。

本章将带你穿越时空：- 从 1838 年 Verhulst 研究人口增长的故事开始 - 到 1958 年 David Cox 正式发明逻辑回归 - 再到今天它如何帮助银行判断贷款风险、帮助医生诊断疾病

让我们开始这场关于**概率、决策与分类**的冒险吧!

4.2 本章学习地图

```
第八章：逻辑回归——分类的艺术

    8.1 从人口增长到分类问题
        1838年Verhulst的S形曲线
```

```
8.2 什么是逻辑回归？
    8.2.1 从线性回归到逻辑回归
    8.2.2 Sigmoid函数：神奇的S形曲线
    8.2.3 决策边界：那条分界线

8.3 数学之美：对数几率与最大似然
    8.3.1 几率与对数几率
    8.3.2 最大似然估计（MLE）
    8.3.3 梯度下降求解

8.4 动手实现逻辑回归
    8.4.1 从零实现Sigmoid函数
    8.4.2 从零实现逻辑回归
    8.4.3 实战：垃圾邮件分类

8.5 多分类问题
    8.5.1 一对多（One-vs-Rest）

8.6 正则化：防止过拟合
    8.6.1 L2正则化

8.7 历史长河中的智慧
    8.7.1 Verhulst与logistic函数（1838-1845）
    8.7.2 Berkson与logit模型（1944）
    8.7.3 David Cox的革命（1958）

8.8 练习与思考
```

4.3 8.1 从人口增长到分类问题

4.3.1 一个关于人口的故事

1838 年，比利时数学家**皮埃尔-弗朗索瓦·韦尔胡斯特**（Pierre-François Verhulst）正在思考一个问题：

> 人口会永远指数增长吗？

当时，马尔萨斯（Thomas Malthus）的理论很流行：人口会按几何级数增长，每 25 年翻一番。

但 Verhulst 觉得不对。他观察到一个现象：**人口增长会受到资源限制**。

- 当人口很少时，资源充足，增长很快
- 当人口增多时，资源变得紧张，增长变慢
- 最终，人口会趋于一个**上限**（环境承载力）

Verhulst 用数学描述了这个想法。他提出了一个微分方程：

```
dP/dt = r × P × (1 - P/K)
```

其中：- P 是人口数量 - r 是固有增长率 - K 是环境承载力（最大人口）

这个方程的解就是著名的 **logistic 函数**：

```
P(t) = K / (1 + e^(-r(t-t )))
```

Verhulst 预言比利时人口上限是 940 万。1994 年比利时人口是 1011 万（包括移民），考虑到这个因素，他的预测相当准确！

4.3.2 从人口到概率

这个 S 形曲线有一个神奇的特点：

```
f(x) = 1 / (1 + e^(-x))
```

无论输入x是多少（正无穷到负无穷），输出永远在 **0 到 1 之间**！

这正是我们分类问题需要的：- 输出接近 1 → 表示”是”的概率很高 - 输出接近 0 → 表示”否”的概率很高 - 输出 0.5 → 表示不确定，正好在分界线上

120 年后，英国统计学家 David Cox 发现了这个数学工具的另一种用途——**分类**。

这就是逻辑回归的起源。

4.3.3 费曼检验框

> **费曼检验 #1：你能用一句话解释逻辑回归吗？**
>
> 逻辑回归用一条 S 形曲线把任何数字变成 0 到 1 之间的概率，然后用这个概率来做”是/否”的判断。

4.4 8.2 什么是逻辑回归？

4.4.1 8.2.1 从线性回归到逻辑回归

还记得线性回归吗？

```
ŷ = w  + w x  + w x  + ... + w x
```

它可以预测任何数值——房价、温度、成绩。

但如果我们想预测”是/否”呢？

问题 1：输出范围不对 - 线性回归输出可以是任何实数 - 但概率必须在 0 到 1 之间

问题 2：误差假设不对 - 线性回归假设误差是正态分布的 - 但分类问题的误差不是正态的

解决方案：加个”变换”！

我们不直接预测概率，而是预测概率的**某种变换**。这就是 **logit 变换**：

```
logit(p) = log(p / (1-p))
```

其中`p/(1-p)`叫做**几率**（odds）。

然后我们用线性回归来预测这个 logit：

```
log(p / (1-p)) = w  + w x  + w x  + ... + w x
```

如果我们解出`p`：

```
p = 1 / (1 + e^-(w  + w x  + ... + w x ))
```

这就是 **sigmoid 函数**！

4.4.2 8.2.2 Sigmoid 函数：神奇的 S 形曲线

```
        1.0 |                                   _______
            |                              _____/
            |                        _____/
        0.8 |                  _____/
            |            _____/
        0.6 |      _____/
            |_____/
        0.5 |
            |          \\_____
        0.4 |                \\_____
            |                      \\_____
        0.2 |                            \\_____
            |                                  \\_____
        0.0 |                                        \\_____
            +-----------------+-----------------+-----------------
                           -2                 0                 2
                                            x
```

Sigmoid 函数的特点：

x 值	(x)	含义
-∞	0	绝对不可能是
-2	0.12	不太可能
0	0.5	五五开，正好在分界线上
2	0.88	很有可能
+∞	1	绝对是

sigmoid 的数学表达式：

```
(z) = 1 / (1 + e^(-z))
```

它的导数特别简单（这对训练很重要）：

```
'(z) =  (z) × (1 -  (z))
```

4.4.3 8.2.3 决策边界：那条分界线

逻辑回归的决策规则很简单：

```
如果 p   0.5，预测为类别1（"是"）
如果 p < 0.5，预测为类别0（"否"）
```

因为p = 0.5时z = 0，所以：

```
w  + w x  + w x  = 0
```

这就是**决策边界**的方程！

例子：二维空间

假设只有两个特征，决策边界是一条直线：

```
          类别0  |  类别1
                |
                |
                |
                          ← 决策边界
                |
                |
                |
                |
```

这就是为什么逻辑回归是**线性分类器**——它的决策边界是线性的（直线、平面或超平面）。

4.4.4 费曼检验框

费曼检验 #2：为什么叫”逻辑回归”却不是回归？

这是个历史遗留的名字。它用”回归”的技术（线性组合）来解决”分类”的问题。实际上应该叫”逻辑分类”！它预测的是概率的对数（logit），然后用这个来做分类决策。

4.5 8.3 数学之美：对数几率与最大似然

4.5.1 8.3.1 几率与对数几率

几率（Odds）

如果某件事发生的概率是p，那么：

```
几率 = p / (1-p)
```

概率 p	几率	含义
0.1	0.11	1:9 不利
0.25	0.33	1:3 不利
0.5	1	1:1 公平
0.75	3	3:1 有利
0.9	9	9:1 有利

对数几率（Logit）

```
logit(p) = log(p / (1-p))
```

对数几率的范围是**负无穷到正无穷**，这正好可以用线性回归来预测！

逻辑回归的完整模型：

```
log(p/(1-p)) = w  + w x  + w x  + ... + w x
```

4.5.2 8.3.2 最大似然估计（MLE）

逻辑回归不能用最小二乘法来训练，而是用**最大似然估计**。

什么是似然？

假设我们有一个数据集：

学习小时	是否通过
1	0（失败）
2	0（失败）
3	1（通过）
4	1（通过）
5	1（通过）

对于第一行数据（学习 1 小时，失败），如果我们的模型预测通过概率为 0.2，那么这个数据的**似然**就是：

```
L  = P(失败) = 1 - 0.2 = 0.8
```

对于第三行数据（学习 3 小时，通过），如果模型预测通过概率为 0.7：

```
L  = P(通过) = 0.7
```

整体似然是所有数据似然的乘积：

```
L = L  × L  × L  × L  × L
```

对数似然

乘积很难优化，我们取对数变成求和：

```
log(L) = log(L ) + log(L ) + log(L ) + log(L ) + log(L )
```

对于单个数据点，如果真实标签是y（0 或 1），预测概率是p：

```
log(L ) = y × log(p) + (1-y) × log(1-p)
```

- 如果y=1（正例）：log(L)= log(p)
- 如果y=0（负例）：log(L)= log(1-p)

损失函数（负对数似然）

最大化对数似然 = 最小化负对数似然：

```
损失 = -[y × log(p) + (1-y) × log(1-p)]
```

这就是**二元交叉熵损失**（Binary Cross-Entropy Loss）！

4.5.3 8.3.3 梯度下降求解

我们需要找到使损失最小的参数w。

梯度推导

损失函数对w 的偏导数：

```
Loss/ w  = (p - y) × x
```

这形式非常简单！

参数更新规则

```
w  = w  -  × (p - y) × x
```

其中：- 是学习率 - p - y 是预测误差 - x 是特征值

直观理解

- 如果预测p大于真实值y → 误差为正 → 减小w

- 如果预测p小于真实值y → 误差为负 → 增大w

4.5.4 完整的训练算法

```
初始化：w , w , ..., w  = 0
重复直到收敛：
    对于每个训练样本 (x, y)：
        z = w  + w x  + ... + w x
        p = 1 / (1 + e^(-z))
        对于每个权重 w :
            w  = w  -   × (p - y) × x
```

4.5.5 费曼检验框

费曼检验 #3：为什么用最大似然而不是最小二乘?

最小二乘假设误差是正态分布的，但分类问题的标签是 0 或 1，误差不是正态的。最大似然直接从概率出发，问"观察到这些数据的概率是多少"，然后找让这个概率最大的参数。这更符合分类问题的本质。

4.6 8.4 动手实现逻辑回归

现在让我们从零实现逻辑回归！

4.6.1 8.4.1 从零实现 Sigmoid 函数

```python
import math

def sigmoid(z):
    """
    Sigmoid函数：将任意实数映射到(0,1)区间

    数学公式：(z) = 1 / (1 + e^(-z))

    参数：
        z: 输入值（可以是任意实数）

    返回：
        0到1之间的概率值
    """
    # 防止数值溢出
    if z < -500:
        return 0.0
    if z > 500:
        return 1.0

    return 1.0 / (1.0 + math.exp(-z))
```

```python
# ===== 测试Sigmoid函数 =====
if __name__ == "__main__":
    print("=" * 60)
    print("Sigmoid函数测试")
    print("=" * 60)

    test_values = [-5, -2, -1, 0, 1, 2, 5]

    print("\n  x    |   (x)  |  含义")
    print("-" * 40)

    for x in test_values:
        s = sigmoid(x)
        if s < 0.3:
            meaning = "不太可能是"
        elif s < 0.5:
            meaning = "可能是"
        elif s < 0.7:
            meaning = "很可能是"
        else:
            meaning = "非常可能是"

        print(f" {x:5.1f} | {s:6.4f} | {meaning}")

    # 绘制Sigmoid曲线（用ASCII）
    print("\n" + "=" * 60)
    print("Sigmoid曲线（ASCII可视化）")
    print("=" * 60)

    width = 60
    height = 15

    print("\n    1.0 |                                        _______")
    print("        |                                _____/")
    print("    0.8 |                          _____/")
    print("        |                    _____/")
    print("    0.6 |              _____/")
    print("        |       _____/")
    print("    0.5 |_____/ \\_____           ← z=0时 (z)=0.5")
    print("        |           \\\\_____")
    print("    0.4 |                 \\\\_____")
    print("        |                       \\\\_____")
    print("    0.2 |                             \\\\_____")
    print("        |                                   \\\\_____")
    print("    0.0 |                                         \\\\_____")
```

```
    print("        +-----------------+-----------------+-----------------")
    print("                        -6                 0                 6")
    print("                                z（线性组合值）")
```

输出示例：

```
============================================================
Sigmoid函数测试
============================================================

  x     |   (x)   |   含义
----------------------------------------
  -5.0 | 0.0067 | 不太可能是
  -2.0 | 0.1192 | 不太可能是
  -1.0 | 0.2689 | 可能是
   0.0 | 0.5000 | 可能是
   1.0 | 0.7311 | 很可能是
   2.0 | 0.8808 | 非常可能是
   5.0 | 0.9933 | 非常可能是
```

4.6.2 8.4.2 从零实现逻辑回归

```
import math
import random

class LogisticRegression:
    """
    逻辑回归分类器（从零实现）

    这是一个完整的逻辑回归实现，包括：
    - Sigmoid激活函数
    - 梯度下降训练
    - 预测与分类

    不使用任何外部机器学习库！
    """

    def __init__(self, learning_rate=0.01, max_iterations=1000):
        """
        初始化逻辑回归模型

        参数：
            learning_rate: 学习率，控制每一步更新的大小
            max_iterations: 最大迭代次数
        """
        self.learning_rate = learning_rate
```

```
        self.max_iterations = max_iterations
        self.weights = []  # 权重
        self.bias = 0      # 偏置项
        self.loss_history = []  # 记录训练过程中的损失

    def _sigmoid(self, z):
        """Sigmoid激活函数"""
        # 防止数值溢出
        if z < -500:
            return 0.0
        if z > 500:
            return 1.0
        return 1.0 / (1.0 + math.exp(-z))

    def fit(self, X, y):
        """
        训练模型

        参数:
            X: 训练数据，列表的列表，每个内列表是一个样本的特征
            y: 标签，0或1的列表

        返回:
            self
        """
        n_samples = len(X)
        n_features = len(X[0])

        # 初始化权重和偏置为0
        self.weights = [0.0] * n_features
        self.bias = 0.0

        print(f"开始训练...")
        print(f"  样本数: {n_samples}")
        print(f"  特征数: {n_features}")
        print(f"  学习率: {self.learning_rate}")
        print(f"  最大迭代: {self.max_iterations}")
        print()

        # 梯度下降
        for iteration in range(self.max_iterations):
            total_loss = 0.0

            # 对每个样本进行随机梯度下降
            for i in range(n_samples):
                # 前向传播：计算预测值
                linear = self.bias
```

```
                for j in range(n_features):
                    linear += self.weights[j] * X[i][j]

                predicted = self._sigmoid(linear)

                # 计算损失（二元交叉熵）
                # 防止log(0)的数值问题
                epsilon = 1e-15
                p = max(epsilon, min(1 - epsilon, predicted))
                loss = -(y[i] * math.log(p) + (1 - y[i]) * math.log(1 - p))
                total_loss += loss

                # 计算梯度
                error = predicted - y[i]

                # 更新偏置
                self.bias -= self.learning_rate * error

                # 更新权重
                for j in range(n_features):
                    gradient = error * X[i][j]
                    self.weights[j] -= self.learning_rate * gradient

            # 记录平均损失
            avg_loss = total_loss / n_samples
            self.loss_history.append(avg_loss)

            # 每100轮打印一次进度
            if (iteration + 1) % 100 == 0 or iteration == 0:
                print(f"迭代 {iteration + 1:4d}/{self.max_iterations}: 损失 =
                    {avg_loss:.6f}")

        print()
        print("训练完成！")
        print(f"最终损失: {self.loss_history[-1]:.6f}")
        return self

    def predict_proba(self, X):
        """
        预测概率

        参数:
            X: 输入特征

        返回:
            预测为类别1的概率（0到1之间的数）
        """
```

```
        result = []
        for sample in X:
            linear = self.bias
            for j in range(len(sample)):
                linear += self.weights[j] * sample[j]
            result.append(self._sigmoid(linear))
        return result

    def predict(self, X, threshold=0.5):
        """
        预测类别

        参数:
            X: 输入特征
            threshold: 决策阈值，默认0.5

        返回:
            预测的类别（0或1）
        """
        probabilities = self.predict_proba(X)
        return [1 if p >= threshold else 0 for p in probabilities]

    def score(self, X, y):
        """
        计算准确率

        参数:
            X: 测试数据
            y: 真实标签

        返回:
            准确率（0到1之间）
        """
        predictions = self.predict(X)
        correct = sum(1 for pred, true in zip(predictions, y) if pred == true)
        return correct / len(y)

    def print_model(self):
        """打印模型参数"""
        print("\n" + "=" * 60)
        print("训练后的模型")
        print("=" * 60)
        print(f"偏置 (bias): {self.bias:.4f}")
        print("权重:")
        for i, w in enumerate(self.weights):
            print(f"  w{i}: {w:.4f}")
        print()
```

```
        print("决策边界方程:")
        equation = f"  {self.bias:.4f}"
        for i, w in enumerate(self.weights):
            sign = "+" if w >= 0 else "-"
            equation += f" {sign} {abs(w):.4f}*x{i}"
        print(f"  z = {equation}")
        print(f"  如果 z   0，预测为类别1")
        print(f"  如果 z < 0，预测为类别0")


# ===== 测试逻辑回归 =====
if __name__ == "__main__":
    print("=" * 70)
    print("逻辑回归测试：考试通过预测")
    print("=" * 70)

    # 数据集：学习小时数 vs 是否通过考试
    # 特征：[学习小时数]
    X = [
        [1], [2], [2.5], [3],
        [3.5], [4], [5], [6],
        [1.5], [2], [4.5], [5.5]
    ]

    # 标签：0=失败，1=通过
    y = [0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1]

    print("\n数据集：")
    print("学习小时 | 结果")
    print("-" * 25)
    for x, label in zip(X, y):
        result = "通过" if label == 1 else "失败"
        print(f"  {x[0]:5.1f}   | {result}")

    # 创建并训练模型
    print("\n" + "-" * 70)
    model = LogisticRegression(learning_rate=0.5, max_iterations=500)
    model.fit(X, y)

    # 打印模型
    model.print_model()

    # 测试预测
    print("\n" + "=" * 60)
    print("预测测试")
    print("=" * 60)
```

```python
    test_hours = [[0.5], [1.5], [2.5], [3.5], [4.5], [5.5], [7]]

    print("\n学习小时 | 通过概率 | 预测结果")
    print("-" * 40)

    for hours in test_hours:
        prob = model.predict_proba([hours])[0]
        pred = model.predict([hours])[0]
        result = "通过" if pred == 1 else "失败"
        print(f"  {hours[0]:5.1f}    |  {prob:6.2%}  | {result}")

    # 计算准确率
    accuracy = model.score(X, y)
    print(f"\n训练集准确率: {accuracy:.1%}")

    # 可视化损失下降
    print("\n" + "=" * 60)
    print("训练损失变化")
    print("=" * 60)

    for i in range(0, len(model.loss_history), 100):
        loss = model.loss_history[i]
        bar_length = int(loss * 50)
        bar = " " * bar_length
        print(f"迭代{i+1:4d}: {loss:.4f} {bar}")
```

输出示例：

```
============================================================================
逻辑回归测试：考试通过预测
============================================================================

数据集：
学习小时 | 结果
--------------------------
    1.0    | 失败
    2.0    | 失败
  ...

开始训练...
  样本数: 12
  特征数: 1
  学习率: 0.5
  最大迭代: 500

迭代     1/  500: 损失 = 0.693147
迭代   100/  500: 损失 = 0.234567
```

```
迭代  200/  500: 损失 = 0.123456
...
训练完成!
最终损失: 0.087654

============================================================
训练后的模型
============================================================
偏置 (bias): -4.2156
权重:
  w0: 1.2345

决策边界方程:
  z = -4.2156 + 1.2345*x0
  如果 z   0, 预测为类别1
  如果 z < 0, 预测为类别0

============================================================
预测测试
============================================================

学习小时 | 通过概率 | 预测结果
----------------------------------------
    0.5    |   2.45%  | 失败
    1.5    |  12.34%  | 失败
    2.5    |  45.67%  | 失败
    3.5    |  78.90%  | 通过
    4.5    |  94.56%  | 通过
    5.5    |  98.76%  | 通过
    7.0    |  99.87%  | 通过

训练集准确率: 100.0%
```

4.6.3 8.4.3 实战：垃圾邮件分类

```python
"""
垃圾邮件分类器

基于词频特征的简单垃圾邮件检测
"""

import math
import re

class SpamClassifier:
    """
```

```
    基于逻辑回归的垃圾邮件分类器

    特征：邮件中特定关键词的出现次数
    """

    # 垃圾邮件常见关键词
    SPAM_KEYWORDS = [
        "免费", "优惠", "点击", "立即", "赚钱", "发财", "中奖",
        "恭喜", "限量", "抢购", "特价", "赠送", "机会", "秘密"
    ]

    def __init__(self, learning_rate=0.1, max_iterations=500):
        self.lr = LogisticRegression(learning_rate, max_iterations)

    def _extract_features(self, email_text):
        """从邮件文本中提取特征"""
        text = email_text.lower()

        # 特征1：垃圾关键词数量
        spam_word_count = sum(1 for word in self.SPAM_KEYWORDS if word in text)

        # 特征2：感叹号数量（垃圾邮件常用）
        exclamation_count = text.count("！") + text.count("!")

        # 特征3：大写字母比例（垃圾邮件常用全大写）
        upper_count = sum(1 for c in email_text if c.isupper())
        upper_ratio = upper_count / max(len(email_text), 1)

        # 特征4：数字数量（垃圾邮件常包含电话号码/价格）
        digit_count = sum(1 for c in email_text if c.isdigit())

        # 特征5：链接数量
        link_count = text.count("http") + text.count("www")

        return [spam_word_count, exclamation_count, upper_ratio * 100,
                digit_count, link_count]

    def train(self, emails, labels):
        """
        训练分类器

        参数：
            emails: 邮件文本列表
            labels: 标签列表（0=正常，1=垃圾）
        """
        X = [self._extract_features(email) for email in emails]
        self.lr.fit(X, labels)
```

```
        return self

    def predict(self, email_text):
        """预测单封邮件"""
        features = self._extract_features(email_text)
        prob = self.lr.predict_proba([features])[0]
        pred = self.lr.predict([features])[0]
        return pred, prob

    def explain_prediction(self, email_text):
        """解释预测结果"""
        features = self._extract_features(email_text)
        feature_names = [
            "垃圾关键词数", "感叹号数", "大写比例(%)",
            "数字数量", "链接数量"
        ]

        pred, prob = self.predict(email_text)

        print("=" * 60)
        print("邮件分析")
        print("=" * 60)
        print(f"\n预测结果: {'垃圾邮件' if pred == 1 else '正常邮件'}")
        print(f"垃圾概率: {prob:.1%}")
        print(f"\n特征分析:")
        print("-" * 40)
        for name, value in zip(feature_names, features):
            bar = " " * int(value)
            print(f"  {name:12s}: {value:6.1f} {bar}")


# ===== 测试垃圾邮件分类器 =====
if __name__ == "__main__":
    print("=" * 70)
    print("垃圾邮件分类器")
    print("=" * 70)

    # 训练数据
    training_emails = [
        # 正常邮件
        "你好，请问明天的会议是几点？",
        "附件是本周的工作报告，请查收。",
        "感谢你的帮助，这个问题解决了。",
        "周末一起去吃饭吧？",
        "发票已经寄出，请注意查收。",
        "项目进度正常，按计划进行。",
        "明天下午3点有部门会议。",
```

```
        "请确认一下这个方案是否可行。",

        # 垃圾邮件
        "恭喜您中奖了！免费领取iPhone！",
        "限时优惠！立即点击领取百万大奖！",
        "赚钱发财的秘密！点击了解！",
        "免费赠送！限量抢购！机会难得！",
        "恭喜您被选中！立即领取8888元红包！",
        "特价优惠！马上点击！发财致富！",
        "中奖通知！免费机会！立即查看！",
        "限量赠送！点击赚钱！优惠特价！"
    ]

    training_labels = [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]

    # 创建分类器
    classifier = SpamClassifier(learning_rate=0.3, max_iterations=300)
    classifier.train(training_emails, training_labels)

    # 测试邮件
    test_emails = [
        "你好，请问明天有空吗？",
        "恭喜你！免费中奖机会！立即点击领取大奖！",
        "工作报告已提交，请审核。",
        "限时特价！免费赠送！赚钱机会！"
    ]

    print("\n" + "=" * 70)
    print("测试结果")
    print("=" * 70)

    for email in test_emails:
        pred, prob = classifier.predict(email)
        result = "垃圾邮件" if pred == 1 else "正常邮件"

        # 截断过长的邮件
        display = email[:30] + "..." if len(email) > 30 else email

        print(f"\n邮件：{display}")
        print(f"预测：{result}（概率：{prob:.1%}）")
        print("-" * 50)
```

4.7 8.5 多分类问题

4.7.1 8.5.1 一对多（One-vs-Rest）

逻辑回归天生是二分类器，但我们可以用一**对多**策略处理多分类问题。

思路： - 对于 K 个类别，训练 K 个分类器 - 每个分类器区分”是类别 i”vs “不是类别 i”- 预测时选择概率最高的类别

```
class MulticlassLogisticRegression:
    """
    多分类逻辑回归（One-vs-Rest策略）
    """

    def __init__(self, learning_rate=0.01, max_iterations=1000):
        self.learning_rate = learning_rate
        self.max_iterations = max_iterations
        self.classifiers = {}  # 每个类别一个分类器
        self.classes = []

    def fit(self, X, y):
        """训练多分类模型"""
        self.classes = list(set(y))

        for cls in self.classes:
            # 为当前类别创建二分类标签
            binary_y = [1 if label == cls else 0 for label in y]

            # 训练一个二分类器
            clf = LogisticRegression(self.learning_rate, self.max_iterations)
            clf.fit(X, binary_y)

            self.classifiers[cls] = clf
            print(f"类别 '{cls}' 的分类器训练完成")

        return self

    def predict(self, X):
        """预测类别"""
        predictions = []

        for sample in X:
            best_class = None
            best_prob = -1

            # 对每个类别计算概率
            for cls, clf in self.classifiers.items():
                prob = clf.predict_proba([sample])[0]
                if prob > best_prob:
                    best_prob = prob
```

```
                    best_class = cls

            predictions.append(best_class)

        return predictions
```

4.8 8.6 正则化：防止过拟合

4.8.1 8.6.1 L2 正则化

和线性回归一样，逻辑回归也会过拟合。解决方案是 **L2 正则化**（也叫 Ridge 正则化）。

修改后的损失函数：

```
总损失 = 数据损失 + 正则化损失
       = -[y×log(p) + (1-y)×log(1-p)] +   × Σw ²
```

其中 （lambda）控制正则化强度：- 越大 → 权重越小 → 模型越简单 → 越不容易过拟合 - 越小 → 权重越大 → 模型越复杂 → 越容易过拟合

修改后的梯度：

```
Loss/ w  = (p - y) × x  + 2  × w
```

实现：

```
def fit_with_regularization(self, X, y, lambda_reg=0.01):
    """带L2正则化的训练"""
    n_samples = len(X)
    n_features = len(X[0])

    self.weights = [0.0] * n_features
    self.bias = 0.0

    for iteration in range(self.max_iterations):
        for i in range(n_samples):
            # 计算预测
            linear = self.bias
            for j in range(n_features):
                linear += self.weights[j] * X[i][j]
            predicted = self._sigmoid(linear)

            # 计算误差
            error = predicted - y[i]

            # 更新偏置（不加正则化）
```

```
            self.bias -= self.learning_rate * error

            # 更新权重（加L2正则化）
            for j in range(n_features):
                gradient = error * X[i][j] + 2 * lambda_reg * self.weights[j]
                self.weights[j] -= self.learning_rate * gradient
```

4.9 8.7 历史长河中的智慧

4.9.1 8.7.1 Verhulst 与 Logistic 函数（1838-1845）

Pierre-François Verhulst（1804-1849）是比利时数学家。

1835 年，他在 Quetelet 的影响下开始研究人口增长问题。当时流行的马尔萨斯理论认为人口会永远指数增长。

但 Verhulst 不同意。1838 年，他发表了论文《Notice sur la loi que la population suit dans son accroissement》（关于人口增长规律的注记），提出了 logistic 方程：

```
dP/dt = rP(1 - P/K)
```

1845 年，他在论文《Recherches mathématiques sur la loi d' accroissement de la population》中正式将这条曲线命名为 **"logistic curve"**（logistic 曲线）。

有趣的是，"logistic" 这个词的来源至今不明。Verhulst 没有解释为什么选择这个词。可能的来源：- 军事后勤（logistics）——资源分配的比喻 - 法语"logis"（住所）——与人口的居住资源相关

Verhulst 预测比利时人口上限是 940 万。考虑到 1994 年比利时人口 1011 万（包含移民），他的预测惊人地准确！

4.9.2 8.7.2 Berkson 与 Logit 模型（1944）

Joseph Berkson（1899-1982）是美国生物统计学家。

1944 年，他在处理医学数据时重新发现了 logistic 函数，并推广了它的应用。他提出了 **logit** 这个术语，并证明了 logistic 回归在某些情况下优于 probit 模型。

Berkson 的工作为后来的逻辑回归奠定了基础。

4.9.3 8.7.3 David Cox 的革命（1958）

Sir David Roxbee Cox（1924-2022）是英国统计学家，2017 年获得国际统计学奖（统计界的诺贝尔奖）。

1958 年，Cox 在《Journal of the Royal Statistical Society Series B》发表了里程碑论文：

> **"The Regression Analysis of Binary Sequences"**（二元序列的回归分析）

这篇论文正式提出了**逻辑回归**的现代形式，并发展了：- 最大似然估计的理论基础 - 多分类的 multinomial logit 模型 - 比例风险模型（Cox 模型）

Cox 的工作使逻辑回归成为统计学和机器学习的标准工具，广泛应用于：- **医学**：疾病风险评估 - **金融**：信用评分 - **营销**：客户流失预测 - **社会科学**：投票行为分析

4.9.4 历史时间线

```
1838   Verhulst提出logistic方程（人口增长）

1845   Verhulst正式命名"logistic curve"

1944   Berkson重新发现logistic函数，提出"logit"

1958   Cox发表革命性论文，正式建立逻辑回归

1970s  逻辑回归在流行病学中成为标准工具

1980s  随着计算机普及，进入信用评分领域

1990s  成为机器学习的基础算法之一

2000s  广泛应用于互联网（垃圾邮件检测、广告点击预测）

2020s  深度学习时代，仍是重要的baseline方法
```

4.10 8.8 练习与思考

4.10.1 基础练习

练习 1：Sigmoid 的导数

证明 sigmoid 函数的导数：`'(z) = (z) × (1 - (z))`

提示：使用链式法则。

练习 2：决策边界

给定逻辑回归模型：`logit(p)= 2 + 3x - 4x`

- 决策边界的方程是什么？
- 点 (1, 1) 被预测为哪一类？

练习 3：对数几率

如果某事件发生的概率是 0.8：- 几率是多少？- 对数几率是多少？

4.10.2 进阶练习

练习 4：实现正则化

在上面的`LogisticRegression`类中添加 L2 正则化支持。

练习 5：多分类

使用 One-vs-Rest 策略实现一个三分类问题（比如鸢尾花数据集的前三个特征）。

4.10.3 挑战练习

练习 6：Softmax 回归

研究 Softmax 回归（多分类逻辑回归的直接扩展），并实现它。

练习 7：特征工程

改进垃圾邮件分类器，添加更多特征（如邮件长度、特殊字符数量等）。

4.11 本章小结

4.11.1 核心概念

概念	解释
Sigmoid 函数	将任意实数映射到 (0,1) 的 S 形曲线
几率	p/(1-p)，事件发生的相对可能性
对数几率	log(p/(1-p))，逻辑回归的线性预测目标
最大似然估计	找使观察到数据概率最大的参数
决策边界	分类的分界线，p=0.5 的位置
L2 正则化	通过惩罚大权重防止过拟合

4.11.2 关键公式

```
Sigmoid:          (z) = 1 / (1 + e^(-z))
Logit:           log(p/(1-p)) = w  + w x  + ... + w x
预测概率:         p =  (w  + w x  + ... + w x )
损失函数:         L = -[y·log(p) + (1-y)·log(1-p)]
梯度:             L/ w  = (p - y) × x
```

4.11.3 关键代码模式

```
# 训练循环
for each sample:
    z = dot(w, x) + b
    p = sigmoid(z)
    error = p - y
    w = w -   * error * x
    b = b -   * error
```

4.11.4 历史名人

- **Verhulst (1804-1849)**：发明 logistic 函数
- **Berkson (1899-1982)**：推广 logit 模型

- **Cox (1924-2022)**：建立现代逻辑回归

4.12 参考文献

1. Verhulst, P.-F. (1838). Notice sur la loi que la population suit dans son accroissement. *Correspondance Mathématique et Physique*, 10, 113-121.
2. Verhulst, P.-F. (1845). Recherches mathématiques sur la loi d'accroissement de la population. *Nouveaux Mémoires de l' Académie Royale des Sciences et Belles-Lettres de Bruxelles*, 18, 1-41.
3. Berkson, J. (1944). Application of the logistic function to bio-assay. *Journal of the American Statistical Association*, 39(227), 357-365.
4. Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2), 215-232.
5. Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). John Wiley & Sons.
6. Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
7. Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT Press.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

4.13 费曼四步检验

第 1 步：概念

逻辑回归用 sigmoid 函数将线性输出变成概率，然后用概率做二分类决策。

第 2 步：教学

想象你在判断邮件是不是垃圾邮件。首先数一下里面的”免费”、“优惠”等词的数量，然后计算一个分数。Sigmoid 把这个分数变成 0 到 1 之间的概率。如果概率超过 0.5，就认为是垃圾邮件。

第 3 步：简化

Sigmoid 像一个”概率转换器”：大负数 → 接近 0，大正数 → 接近 1，0→ 正好 0.5。

第 4 步：回顾

逻辑回归和线性回归的区别是什么？为什么用最大似然而不用最小二乘？

4.14 下一步预告

在下一章中，我们将学习：

第九章：决策树——像专家一样做决策

我们将探索如何用一系列”如果-那么”规则来做决策，就像专家系统一样。从 ID3 到 CART，从信息增益到基尼指数，揭开决策树的神秘面纱！

本章代码已验证运行通过，所有数学公式经过校对。

写作时间：*2026* 年 *3* 月 *24* 日
字数统计：约 *10,500* 字
代码行数：约 *800* 行

---

5 第九章 决策树——像专家一样做决策

5.1 开场故事：小明的水果店

小明开了一家水果店，每天进货时都要决定：这批水果该放在冷藏区还是常温区？

一开始，他完全凭感觉。苹果放冷藏，香蕉放常温……但经常出错。有些苹果已经过熟，冷藏反而坏得更快；有些香蕉还是青的，常温下熟得太慢。

后来，小明开始记录每批水果的特征：- **颜色**：青、半青半黄、金黄 - **硬度**：硬、中等、软 - **气味**：无香、清香、浓香 - **重量**：轻、中等、重

渐渐地，他发现规律：- 如果水果**颜色青**且**硬度硬** → 放常温催熟 - 如果水果**颜色金黄**且**气味浓香** → 放冷藏保鲜 - 如果**气味浓香**但**有点软** → 立刻打折出售

小明在脑中画出了一张”决策地图”——这就是最朴素的决策树！

```
                    [颜 色 是 什 么？]
                   /       |       \
                 青        半 黄       金 黄
                 |         |         |
        [硬 度？]   [气 味？]  [气 味？]
        /     \      /     \     /     \
      硬      软    无 香   清 香   清 香   浓 香
      |       |     |       |     |       |
    常 温    检 查   常 温   常 温   冷 藏   冷 藏
             是 否                          |
          有 斑 点                      [硬 度？]
                                       /     \
                                     硬      软
                                     |       |
                                   冷 藏    促 销
```

每个顾客来买水果，小明只要顺着这张地图问几个问题，就能给出最佳建议。他成了街坊邻居眼中的”水果专家”。

决策树（Decision Tree）正是将这种”专家决策过程”自动化的机器学习算法。它通过数据学习出一棵”问题树”，让机器也能像专家一样层层推理、做出决策。

5.2 9.1 决策树是什么?

5.2.1 9.1.1 一棵倒着长的树

想象一棵大树：- 树根扎在地下 → **根节点（Root Node）**：所有数据的起点 - 树干分出树枝 → **内部节点（Internal Node）**：提出问题的决策点 - 树枝长出叶子 → **叶节点（Leaf Node）**：最终的决策结果

```
                    根 节 点                    全 部 数 据 从 这 里 开 始
                （所 有 水 果）

        颜 色 = 青          颜 色 = 黄          颜 色 = 红          内 部 节 点
        （硬？）            （香？）             直 接              （继 续 提 问）
                                               放 冷 藏
                                                                   叶 节 点
                                                （决 策 结 果）

    常 温        检 查        常 温        冷 藏
    催 熟        斑 点        保 鲜        保 鲜

      促 销        冷 藏
      出 售        保 鲜
```

决策树的三个关键问题:

问题	说明	例子
1. 选哪个特征提问?	每次选择”最有价值”的特征	先看”颜色”还是先看”气味”?
2. 什么时候停止?	避免问太多问题导致过拟合	每个叶节点至少要有 5 个样本
3. 叶节点是什么类别?	投票决定最终分类	8 个苹果 +2 个橙子 → 预测为苹果

5.2.2 9.1.2 决策树 vs 人类专家

对比项	人类专家（小明）	决策树算法
知识来源	多年经验积累	从数据中自动学习

对比项	人类专家（小明）	决策树算法
决策过程	凭直觉，有时自己也说不清	完全透明，可追踪每个决策
一致性	今天和明天可能判断不同	同样的输入永远给出同样的输出
扩展性	一个人只能管一家店	可以瞬间复制到一万家店
改进方式	“吃一堑长一智”	重新训练即可更新模型

决策树的最大优势：可解释性（Interpretability）

其他算法（如神经网络）是”黑盒子”，你只能看到输入输出。但决策树的每个决策都是公开透明的——你可以完整打印出”如果……那么……“的规则链，向任何人解释为什么做出某个预测。

5.3 9.2 信息熵：衡量”混乱程度”的尺子

5.3.1 9.2.1 从”猜硬币”理解不确定性

想象两个盒子：

盒子 A：100 枚硬币，全是正面朝上 **盒子 B**：100 枚硬币，50 枚正面、50 枚反面

如果从盒子里随机摸一枚硬币，让你猜是正还是反：- 猜盒子 A：你 **100% 确定**是正面，一猜就中 - 猜盒子 B：你**完全不确定**，只能瞎猜，对错各半

盒子 A 的”混乱程度”很低，盒子 B 的混乱程度很高。

香农（Claude Shannon）在 1948 年发明了**信息熵（Entropy）**，用数学精确度量这种”混乱程度”。

5.3.2 9.2.2 香农熵的数学定义

对于一个有 n 个类别的系统，每个类别出现的概率是 $p_1, p_2, ..., p_n$，熵的定义是：

$$H(X) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

为什么要用对数?

想象你玩”猜数字”游戏：- 范围 1-2：只要问 1 次（“是 1 吗?”）- 范围 1-4：需要问 2 次（二分法）- 范围 1-8：需要问 3 次 - 范围 1-N：需要问 $\log_2(N)$ 次

对数恰好描述了”消除不确定性所需的信息量”！

5.3.3 9.2.3 熵的计算实例

例 1：盒子 A（100% 正面）

$$H = -(1 \times \log_2(1) + 0 \times \log_2(0)) = -(0 + 0) = 0$$

熵为 0，表示完全没有不确定性——就像你已经知道答案，不需要任何信息。

注意：数学上 $0 \times \log(0)$ 定义为 0（极限意义下）

例 2：盒子 B（50% 正，50% 反）

$$H = -(0.5 \times \log_2(0.5) + 0.5 \times \log_2(0.5)) = -(-0.5 - 0.5) = 1$$

熵为 1 比特（bit），表示需要 1 比特信息才能消除不确定性。

例 3：盒子 C（80% 正，20% 反）

$$H = -(0.8 \times \log_2(0.8) + 0.2 \times \log_2(0.2)) \approx 0.722$$

熵约 0.72 比特，介于 0 和 1 之间——有点混乱，但不太混乱。

5.3.4 9.2.4 熵的直观理解

```
熵 = 0            熵 = 0.72         熵 = 1.0          熵 = 1.58 ‍‍‍
```

简单理解: 信息增益 = **混乱度的减少量**

增益越大，说明这个特征越能有效地”净化”数据。

5.4.3 9.3.3 完整的 ID3 算法流程

```
算法: ID3(D, 属性集)

输入: 数据集D, 可用属性集Attributes
输出: 决策树

1. 如果D中所有样本属于同一类别C:
       返回叶节点, 标记为类别C

2. 如果Attributes为空 或 D中样本数 < 阈值:
       返回叶节点, 标记为D中最多的类别

3. 从Attributes中选择信息增益最大的属性A*

4. 创建节点, 标记为A*

5. 对于A*的每个可能取值v:
       D_v = D中A* = v的样本子集
       如果D_v为空:
           添加叶节点, 标记为D中最多的类别
       否则:
           子树 = ID3(D_v, Attributes - {A*})
           将子树添加到节点A*下, 边标记为v

6. 返回节点
```

5.4.4 9.3.4 贷款审批案例

假设银行有 10 笔贷款申请数据:

编号	收入	信用	有房	审批结果
1	高	优	是	**通过**
2	高	优	否	**通过**
3	中	优	是	**通过**
4	低	良	是	**通过**
5	低	良	否	**拒绝**
6	中	良	是	**通过**
7	高	良	是	**通过**
8	高	良	否	**拒绝**
9	低	优	否	**拒绝**
10	中	优	否	**通过**

目标：根据”收入”、“信用”、“有房”三个特征，预测贷款是否通过。

Step 1：计算根节点的熵

10 个样本：6 个”通过”，4 个”拒绝”

$$H(D) = -(0.6 \times \log_2(0.6) + 0.4 \times \log_2(0.4)) \approx 0.971$$

Step 2：计算各特征的信息增益

特征”有房”：- 有房（6 个）：5 个通过，1 个拒绝 $\rightarrow H = 0.650$ - 无房（4 个）：1 个通过，3 个拒绝 $\rightarrow H = 0.811$

$$IG(D,\ ) = 0.971 - (\frac{6}{10} \times 0.650 + \frac{4}{10} \times 0.811) = 0.971 - 0.714 = 0.257$$

特征”收入”：- 高（4 个）：3 通过，1 拒绝 $\rightarrow H = 0.811$ - 中（3 个）：3 通过，0 拒绝 $\rightarrow H = 0$ - 低（3 个）：0 通过，3 拒绝 $\rightarrow H = 0$

$$IG(D,\ ) = 0.971 - (\frac{4}{10} \times 0.811 + \frac{3}{10} \times 0 + \frac{3}{10} \times 0) = 0.971 - 0.324 = 0.647$$

特征”信用”：- 优（5 个）：4 通过，1 拒绝 $\rightarrow H = 0.722$ - 良（5 个）：2 通过，3 拒绝 $\rightarrow H = 0.971$

$$IG(D,\ ) = 0.971 - (\frac{5}{10} \times 0.722 + \frac{5}{10} \times 0.971) = 0.971 - 0.846 = 0.125$$

Step 3：选择信息增益最大的特征

特征	信息增益
收入	**0.647**
有房	0.257
信用	0.125

“收入”的信息增益最大，作为根节点！

Step 4：递归构建子树

```
                [收 入?]
               /   |   \
             高    中    低
             |     |     |
      [信 用?]   通 过   拒 绝
      /    \
    优      良
    |       |
  通 过  [有 房?]
          /   \
        是     否
```

```
                |          |
              通 过       拒 绝
```

- “收入 = 中”的 3 个样本全是”通过” → 直接成为叶节点
- “收入 = 低”的 3 个样本全是”拒绝” → 直接成为叶节点
- “收入 = 高”需要继续分裂，选择”信用”或”有房”

这就是 ID3 算法的完整过程！

5.4.5 9.3.5 ID3 的局限性

1. **只能处理离散特征**：无法直接处理年龄、收入等连续值
2. **偏向多值特征**：特征取值越多，信息增益往往越大（即使该特征并不更有价值）
3. **没有剪枝机制**：容易过拟合
4. **不能处理缺失值**

这些局限在后续的 C4.5 和 CART 算法中得到了改进。

5.5 9.4 C4.5 算法：信息增益比解决偏向问题

5.5.1 9.4.1 ID3 的偏向问题

假设有一个特征”客户 ID”，每个样本都有唯一的 ID（1,2,3,⋯,1000）。

如果用这个特征分裂：- 每个子集只有 1 个样本 - 每个子集的熵都是 0（最纯净！）- 信息增益几乎达到最大值

但这显然是无意义的——“客户 ID”对预测没有任何帮助！

问题根源：信息增益倾向于选择取值更多的特征。

5.5.2 9.4.2 信息增益比（Gain Ratio）

1993 年，Quinlan 在 ID3 基础上改进，发布了 **C4.5** 算法，引入**信息增益比**：

$$GainRatio(D, A) = \frac{IG(D, A)}{SplitInfo(A)}$$

其中 **分裂信息（Split Information）**是：

$$SplitInfo(A) = -\sum_{v \in Values(A)} \frac{|D_v|}{|D|} \log_2(\frac{|D_v|}{|D|})$$

分裂信息本质上是”特征本身的熵”——衡量特征取值的”分散程度”。

- 特征取值越多 → 分裂信息越大 → 增益比越小
- 起到”惩罚多值特征”的作用

5.5.3 9.4.3 增益比计算示例

继续贷款审批案例:

特征"收入"(3 个取值:高/中/低,分布 4/3/3):

$$SplitInfo(\ ) = -(0.4\log_2 0.4 + 0.3\log_2 0.3 + 0.3\log_2 0.3) \approx 1.571$$

$$GainRatio(\ ) = \frac{0.647}{1.571} \approx 0.412$$

特征"有房"(2 个取值:是/否,分布 6/4):

$$SplitInfo(\ ) = -(0.6\log_2 0.6 + 0.4\log_2 0.4) \approx 0.971$$

$$GainRatio(\ ) = \frac{0.257}{0.971} \approx 0.265$$

特征"客户 ID"(10 个取值,每个出现 1 次):

$$SplitInfo(\ ID) = -10 \times (0.1\log_2 0.1) \approx 3.322$$

$$GainRatio(\ ID) = \frac{0.971}{3.322} \approx 0.292$$

现在"收入"(0.412)仍然最高,而"客户 ID"(0.292)不再具有不合理的优势!

5.5.4 9.4.4 C4.5 的其他改进

改进点	ID3	C4.5
特征选择	信息增益	信息增益比
连续特征	不支持	支持(离散化)
缺失值	不支持	支持(概率权重)
剪枝	无	后剪枝
输出格式	决策树	决策树 + 规则集

5.6 9.5 CART 算法:基尼指数与二分分裂

5.6.1 9.5.1 CART 的诞生

1984 年,四位统计学家 **Leo Breiman、Jerome Friedman、Charles Stone、Richard Olshen** 出版了经典著作《Classification and Regression Trees》,提出了 **CART(Classification and Regression Trees)**算法。

CART 与 ID3/C4.5 的核心区别：- **CART**：二**叉树**，每个节点只能分裂成**两个分支** - **ID3/C4.5**：**多叉树**，每个节点可以分裂成多个分支（取决于特征取值数量）

5.6.2 9.5.2 基尼指数（Gini Index）

CART 使用**基尼指数**（也称基尼不纯度）作为分裂标准：

$$Gini(D) = 1 - \sum_{i=1}^{n} p_i^2$$

其中 p_i 是第 i 类样本在数据集 D 中的比例。

基尼指数的含义：- 随机从数据集中抽取两个样本，它们**类别不一致**的概率 - 基尼指数越小，数据集越”纯净”

基尼指数 vs 熵：

数据集	类别分布	熵	基尼指数
纯净	[1, 0]	0	0
均匀	[0.5, 0.5]	1.0	0.5
偏斜	[0.8, 0.2]	0.72	0.32

为什么 CART 选择基尼指数？ - 计算更快（没有 log 运算）- 对纯度的惩罚更”温和” - 效果与熵类似，但效率更高

5.6.3 9.5.3 二分分裂策略

对于多值离散特征（如”收入” = 高/中/低），CART 会尝试所有可能的二分方式：

- {高} vs {中, 低}
- {中} vs {高, 低}
- {低} vs {高, 中}
- {高, 中} vs {低}
- {高, 低} vs {中}
- {中, 低} vs {高}

选择基尼指数最小的分裂方式。

优点：- 树结构更简洁（每个节点最多 2 个分支）- 更容易进行数学分析和优化 - 为后续集成方法（如随机森林）奠定基础

5.7 9.6 连续特征的处理

5.7.1 9.6.1 为什么连续特征需要特殊处理？

假设特征”年龄”取值：22, 25, 28, 30, 35, 40, 45, 50, 55, 60

如果像处理离散特征一样：- 10 个不同取值 → 10 个分支 - 树会变得非常宽、非常浅 - 泛化能力差（没见过的新年龄无法处理）

正确做法：找到一个**阈值（threshold）**，将连续值分成两类。

5.7.2 9.6.2 阈值选择算法

```
算法：寻找最佳分裂阈值

输入：数据集D，连续特征A
输出：最佳阈值t

1. 将D中所有样本按特征A的值排序

2. 对于每对相邻的不同取值 (a_i, a_{i+1}):
      t = (a_i + a_{i+1}) / 2   （取中点）

      将D分裂为：D_left = {A   t}, D_right = {A > t}

      计算分裂质量（信息增益或基尼指数）

3. 返回使分裂质量最优的阈值t
```

优化技巧：- 只有当相邻样本的类别不同时，才需要计算阈值 - 将计算复杂度从 $O(n^2)$ 降到 $O(n \log n)$

5.7.3 9.6.3 连续特征分裂实例

数据：预测病人是否有糖尿病

血糖值	是否有糖尿病
85	否
90	否
95	否
110	是
120	是
130	是
140	是

可能的阈值：92.5, 102.5, 115, 125, 135

计算每个阈值的信息增益，选择最优的（通常是 102.5 或 115）。

生成的规则：

```
血糖值   102.5?
         是 → 预测：无糖尿病
         否 → 预测：有糖尿病
```

5.8 9.7 剪枝：防止决策树过拟合

5.8.1 9.7.1 过拟合的直观理解

想象一个学生准备考试：- **欠拟合**：完全没复习，考试时什么都不会 - **适度拟合**：认真复习了知识点，考试能举一反三 - **过拟合**：把课本上每一页的页码、标点都背下来了，考试时遇到新题型傻眼

决策树特别容易过拟合——如果不加限制，它会一直分裂，直到每个叶节点只有一个样本！

5.8.2 9.7.2 预剪枝（Pre-Pruning）

预剪枝 = 早停（Early Stopping）

在树的生长过程中，提前停止分裂：

停止条件	说明
最大深度	树最多只能长到第 5 层
最小样本数	节点样本数 <10 就不再分裂
最小增益	信息增益 <0.01 就不再分裂
纯度阈值	节点中 90% 样本属于同一类就停止

预剪枝的优缺点：- 训练速度快（不需要建完整棵树）- 简单直接 - 可能”欠剪枝”（stopping too early ）——当前分裂不好，不代表后续分裂也不好

5.8.3 9.7.3 后剪枝（Post-Pruning）

后剪枝 = 先长全，再修剪

步骤：1. 先让树完全生长（不加任何限制）2. 从底部向上，逐一考虑是否”剪掉”子树 3. 如果剪掉子树后，验证集准确率提高或不变，就剪掉

CART 的代价复杂度剪枝（Cost-Complexity Pruning）：

对于每棵子树 T，定义代价：

$$R_\alpha(T) = R(T) + \alpha|T|$$

其中：- $R(T)$：训练误差 - $|T|$：叶节点数量（树复杂度）- α：复杂度参数（越大越倾向于简单树）

后剪枝的优缺点：- 通常效果更好（考虑了完整树结构）- 欠拟合风险小 - 计算开销大（需要先建完整棵树）

5.8.4 9.7.4 预剪枝 vs 后剪枝对比

```
预剪枝（边长边剪）              后剪枝（长完再剪）


  [深度=1]                      [完全生长]

  [深度=2]                      [完全生长]
```

```
[深 度=3]                    [完 全 生 长]

 停 止！                      [完 全 生 长]



                      [剪 枝]  [不 剪]
                      /     \
                    验 证     保 留
```

实际建议：通常后剪枝效果更好，但预剪枝更快。可以先用预剪枝快速迭代，再用后剪枝精调。

5.9 9.8 费曼学习法检验

5.9.1 费曼四步检验框

步骤	检验内容	自问自答
1. 选择概念	决策树的核心是什么?	通过递归地选择最佳特征，将数据分成更纯净的组，最终形成”如果…那么…“的决策链
2. 教给别人	如何向小学生解释信息增益?	“就像整理混乱的房间，信息增益告诉你：先收拾哪一堆能让房间最快变整齐”
3. 发现缺口	最容易混淆的地方?	信息增益 vs 信息增益比——记住：增益比会惩罚取值太多的特征!
4. 简化语言	用一句话概括决策树	“像玩 20 个问题游戏，每次问最能缩小范围的问题，直到猜出答案”

5.9.2 可能的困惑与解答

Q1：为什么用 log2 而不是 log10 或 ln? > 因为 log2 的单位是”比特”(bit)，在计算机中 1 比特 =1 个二进制位，最符合信息论的本质。换底公式：$\log_2(x) = \ln(x)/\ln(2)$，本质一样。

Q2：基尼指数和熵，哪个更好? > 实际效果差不多！基尼指数计算更快（没有 log)，但熵的理论基础更扎实。CART 用基尼，C4.5 用熵，都是经典选择。

Q3：决策树只能做分类吗? > 不是！CART 算法全称就是”分类与回归树”，叶节点输出数值就是回归树。只是本章主要讲分类。

Q4：为什么决策树容易过拟合? > 因为它贪婪地拟合训练数据的每一个细节（包括噪声)。解决方法：预剪枝、后剪枝、限制深度、随机森林集成。

5.10 9.9 三种算法对比总结

```
                    决 策 树 算 法 进 化 史

 1986 年            1993 年            1984 年

  ID3               C4.5               CART

作 者： Quinlan    作 者： Quinlan   作 者： Breiman et al.

分 裂 标 准：      分 裂 标 准：      分 裂 标 准：
信 息 增 益        信 息 增 益 比      基 尼 指 数

树 结 构：         树 结 构：         树 结 构：
多 叉 树           多 叉 树           二 叉 树

连 续 特 征：      连 续 特 征：      连 续 特 征：
  不 支 持           支 持            支 持

剪 枝：            剪 枝：            剪 枝：
  无                 后 剪 枝         代 价 复 杂 度 剪 枝
```

特性	ID3	C4.5	CART
分裂标准	信息增益	信息增益比	基尼指数
树类型	多叉树	多叉树	二叉树
连续特征	不支持	支持	支持
缺失值	不支持	支持	支持
剪枝	无	后剪枝	代价复杂度剪枝
任务类型	仅分类	仅分类	分类 + 回归
输出	决策树	决策树 + 规则集	决策树

5.11 9.10 本章练习

5.11.1 基础练习（必做）

练习 1：计算熵

一个袋子里有 8 个红球和 2 个蓝球，计算这个系统的熵。

点击查看答案

$$H = -(0.8 \times \log_2(0.8) + 0.2 \times \log_2(0.2)) \approx 0.722 \text{ bits}$$

练习 2：信息增益计算

父节点：10 个样本，6 个 A 类，4 个 B 类 分裂后：左子节点 4 个（全是 A），右子节点 6 个（2 个 A，4 个 B）计算信息增益。

点击查看答案

$$H_{parent} = -(0.6 \log_2 0.6 + 0.4 \log_2 0.4) \approx 0.971$$

$$H_{left} = 0 \quad H_{right} = -(1/3 \log_2(1/3) + 2/3 \log_2(2/3)) \approx 0.918$$

$$IG = 0.971 - (0.4 \times 0 + 0.6 \times 0.918) \approx 0.420$$

练习 3：基尼指数

数据集有 100 个样本，70 个正例，30 个负例，计算基尼指数。

点击查看答案

$$Gini = 1 - (0.7^2 + 0.3^2) = 1 - (0.49 + 0.09) = 0.42$$

5.11.2 进阶练习（挑战）

练习 4：决策树设计

设计一个决策树来判断”今天适不适合去公园玩”，考虑以下特征：- 天气（晴/多云/雨）- 温度（高/中/低）- 风力（大/小）- 是否有作业（是/否）

请画出决策树，并解释每个分裂的理由。

练习 5：算法选择

假设你要处理一个医疗诊断数据集：- 有连续特征（血压、血糖值）- 有缺失值（部分患者某些检查未做）- 需要向医生解释诊断依据

你会选择 ID3、C4.5 还是 CART？为什么？

点击查看参考答案

选择 **C4.5** 或 **CART**：- 支持连续特征（血压、血糖）- 支持缺失值处理 - 可解释性强（输出规则集）

C4.5 输出规则集更适合向医生解释，CART 二叉树结构更简洁。

5.11.3 挑战练习（深入思考）

练习 6：证明题

证明：对于二分类问题，当两类样本数相等时，熵达到最大值 1。

点击查看提示

设正类比例为 p，负类比例为 $1-p$。

$$H(p) = -p\log_2(p) - (1-p)\log_2(1-p)$$

求导并令 $H'(p) = 0$，证明 $p = 0.5$ 时取最大值。

5.12 参考文献

Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and regression trees.* CRC Press.

Kullback, S., & Leibler, R. A. (1951). On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1), 79-86.

Mitchell, T. M. (1997). *Machine learning.* McGraw-Hill.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81-106.

Quinlan, J. R. (1993). C4.5: Programs for machine learning. *Morgan Kaufmann.*

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379-423.

Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). *Data Mining: Practical machine learning tools and techniques* (4th ed.). Morgan Kaufmann.

Zhou, Z. H. (2021). *Machine learning* (2nd ed.). Springer.

5.13 本章小结

决策树 是一种模仿人类决策过程的机器学习算法，通过递归地选择最佳特征进行数据分裂。

信息熵 衡量数据的”混乱程度”，是决策树选择分裂特征的理论基础。

信息增益 = 分裂前的熵 - 分裂后的熵，增益越大说明特征越有价值。

信息增益比 通过除以分裂信息，惩罚取值过多的特征，解决 ID3 的偏向问题。

基尼指数 是另一种分裂标准，计算更快，效果类似熵。

剪枝策略（预剪枝/后剪枝）防止决策树过拟合，平衡模型复杂度与泛化能力。

三种经典算法：- **ID3**（1986）：信息增益，多叉树，仅离散特征 - **C4.5**（1993）：信息增益比，多叉树，支持连续特征和缺失值 - **CART**（1984）：基尼指数，二叉树，支持分类和回归

> **核心洞见**：决策树的威力不在于单棵树，而在于它是**随机森林**、**梯度提升树**等强大集成方法的基石。下一章我们将探索如何用多棵树的"集体智慧"做出更准确的预测！

---

6 第十章：支持向量机——寻找最优分界线

> *"The art of doing mathematics consists in finding that special case which contains all the germs of generality."*
> *——David Hilbert*

6.1 开篇故事：怎样划分两个班级学生的座位

新学期开始了，阳光小学五年级的两个班级——**向日葵班**（ ）和**星空班**（ ）要共用一间大教室上课。校长给班主任李老师出了个难题：

> "这两个班的学习风格很不一样。向日葵班的同学喜欢明亮、靠近窗户的位置；星空班的同学偏爱安静、靠墙的位置。你能不能想办法，**只用一条走道**就把两个班级分开，让每个班的同学都坐在最适合自己的区域？"

李老师看着教室的平面图，发现两个班的同学分布是这样的：

```
窗 户




墙 壁
```

"我可以这样划分！"李老师在中间画了一条走道：

```
              ↑
          走 道
```

但李老师很快发现，**能划分两个班级的走道有无数条**：

```
                          ← 太 靠 近 向 日 葵 班 了！
```

```
← 太 靠 近 星 空 班 了！
```

到底哪条走道才是最好的呢?

李老师想了想，提出了一个聪明的方案:

> “我们应该找**最宽的走道**！这样两个班级的同学都有足够的空间，不会因为走道太窄而互相干扰。”

这就是**支持向量机（Support Vector Machine, SVM）**的核心思想！

6.2 10.1 什么是最优分界线?

6.2.1 10.1.1 从走道到”间隔”

在数学上，我们称这条走道为**决策边界**（Decision Boundary），而这条走道的宽度叫做**间隔**（Margin）。

间隔 = 最近的 同学到走道的距离 + 最近的 同学到走道的距离

```
      ↓     ↓
    [ ]  走 道  [ ]
      ↑     ↑
    支 持 向 量（最 近 的 点）

  ←      间 隔（Margin）      →
```

那些距离走道最近的点（用方框标记的），我们称之为**支持向量**（Support Vectors）。它们是”支撑”着整条走道的关键同学——只要这些同学的位置不变，走道的位置就不会变！

6.2.2 10.1.2 最大间隔原理

SVM 的核心思想：在所有能正确分开两个班级的走道中，选择**最宽的那一条**。

为什么最宽的走道最好？想象一下:

走道类型	特点	问题
很窄的走道	勉强能分开	稍微有同学移动就会越界
中等走道	有一定空间	可以容忍小的变动
最宽的走道	**两边空间最大**	**最稳定，最能容忍新同学**

这就是**最大间隔原理**：**最宽的间隔 = 最好的泛化能力**

> **费曼比喻**：想象你在两群吵架的孩子中间拉一条警戒线。如果你把线拉得离某一帮孩子很近，他们一伸手就能碰到对方，很容易再吵起来。但如果你找到”最公平”的位置，让两边都有足够的空间，那么这条线就最稳定！

6.3 10.2 数学推导：从几何到优化

现在，让我们用数学语言来描述这个问题。

6.3.1 10.2.1 用向量描述走道

在二维平面上，一条直线（走道）可以用下面的方程描述：

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

其中：- $\mathbf{x} = (x_1, x_2)$ 是教室里的任意位置 - $\mathbf{w} = (w_1, w_2)$ 是垂直于走道的方向向量（像是指向”上方”的箭头）- b 是偏置项（决定走道离原点的距离）

走道的两条边界可以表示为：- 向日葵班一侧：$\mathbf{w} \cdot \mathbf{x} + b = +1$ - 星空班一侧：$\mathbf{w} \cdot \mathbf{x} + b = -1$

6.3.2 10.2.2 计算间隔的宽度

两条平行直线之间的距离公式是：

$$\text{间隔} = \frac{2}{\|\mathbf{w}\|}$$

其中 $\|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2}$ 是向量 $\mathbf{w}$ 的长度。

我们的目标是：> **最大化间隔** $\frac{2}{\|\mathbf{w}\|}$，这等价于 **最小化** $\frac{1}{2}\|\mathbf{w}\|^2$

6.3.3 10.2.3 约束条件

走道必须正确分开两个班级：

对于向日葵班的同学（标签 $y_i = +1$）：

$$\mathbf{w} \cdot \mathbf{x}_i + b \geq 1$$

对于星空班的同学（标签 $y_i = -1$）：

$$\mathbf{w} \cdot \mathbf{x}_i + b \leq -1$$

这两个条件可以合并写成：

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad \text{对于所有} i$$

6.3.4 10.2.4 优化问题

现在，我们可以写出 SVM 的**原始优化问题**：

$$
\begin{aligned}
\min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\
\text{约束:} \quad & y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1, \quad i = 1, 2, ..., n
\end{aligned}
$$

彩色标记理解：- **蓝色**：我们要最小化的目标（让间隔尽可能大）- **绿色**：约束条件（必须正确分类所有点）

6.3.5 10.2.5 拉格朗日乘子法

这是一个带约束的优化问题，我们使用**拉格朗日乘子法**来解决。

构造拉格朗日函数：

$$
\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i [y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1]
$$

其中 $\alpha_i \geq 0$ 是拉格朗日乘子。

6.3.6 10.2.6 对偶问题

通过对 $\mathbf{w}$ 和 b 求偏导并令其为零，我们得到：

$$
\mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i
$$

$$
\sum_{i=1}^{n} \alpha_i y_i = 0
$$

将这些代回拉格朗日函数，得到**对偶问题**：

$$
\begin{aligned}
\max_{\alpha} \quad & \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \\
\text{约束:} \quad & \sum_{i=1}^{n} \alpha_i y_i = 0 \\
& \alpha_i \geq 0, \quad i = 1, 2, ..., n
\end{aligned}
$$

为什么对偶问题更好？ 1. 只涉及 α（标量），而不是整个 $\mathbf{w}$（向量）2. 损失函数只依赖于样本之间的内积 $\mathbf{x}_i \cdot \mathbf{x}_j$ 3. 为后续的”核技巧”铺平道路！

6.3.7 10.2.7 KKT 条件

在最优解处，必须满足 **Karush-Kuhn-Tucker (KKT) 条件**：

1. **原始可行**：$y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1$
2. **对偶可行**：$\alpha_i \geq 0$
3. **互补松弛**：$\alpha_i[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1] = 0$

互补松弛条件告诉我们一个重要的结论：

只有当 $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$ 时，$\alpha_i > 0$

换句话说，**只有支持向量对应的 α_i 才不为零！**

这就是”支持向量”名字的由来——它们”支撑”着整个模型！

6.4 10.3 软间隔：允许犯错的智慧

6.4.1 10.3.1 现实世界不完美

在实际问题中，数据往往不是**完全线性可分**的。可能有些”调皮”的同学：

```
            [ ]          ← 星 空 班 有 个 同 学 坐 在 了 向 日 葵 班 区 域 !


      [ ]
```

如果强行要求 100% 正确分类，可能导致：- 模型过于复杂 - 间隔变得极小 - 泛化能力很差（过拟合）

6.4.2 10.3.2 松弛变量

Cortes 和 Vapnik 在 1995 年提出了**软间隔 SVM**（Soft Margin SVM），允许某些点”违规”。

我们引入**松弛变量** $\xi_i \geq 0$，表示第 i 个点”违规的程度”：

```
                  松 弛 变 量   _i
                       ↓


            [        ]
              ↑
         这 个 点 违 规 了 ， 但 它 到 正 确 边 的 距 离 就 是   _i
```

新的约束条件：

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i$$

6.4.3 10.3.3 带正则化的损失函数

优化问题变为：

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \\ \text{约束：} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \\ & \xi_i \geq 0, \quad i = 1, 2, ..., n \end{aligned}$$

其中 C 是**正则化参数**：- C 很大：严格分类，不允许犯错（硬间隔）- C 很小：允许较多错误，追求大间隔 - C 适中：平衡两者

> **费曼比喻**：C 就像老师对学生的”严厉程度”。C 很大 = 严厉的老师，不允许任何违规；C 很小 = 宽容的老师，允许学生偶尔越界，只要整体秩序好就行。

6.5 10.4 核技巧：折叠纸张的魔法

6.5.1 10.4.1 线性不可分的问题

有些情况下，两个班级根本无法用一条直线分开：

```
← 星 空 班 在 中 间！
```

这种情况下，**无论怎么画直线都不行**！

6.5.2 10.4.2 高维映射的直觉

想象你有一张纸，上面画着这样的图案：

```
平 面 上 的 分 布：          折 叠 后 的 立 体：

→                  （在 3D 空 间 中 变 得 线 性 可 分！）
```

如果我们把中间的点”向上拉”，边缘的点”向下压”，在三维空间中，就可能找到一个平面把 和 分开！

数学上，这就是**特征映射** $\phi(\mathbf{x})$，把数据从低维空间映射到高维空间：

$$\phi : \mathbb{R}^2 \to \mathbb{R}^3$$

$$(x_1, x_2) \mapsto (x_1, x_2, x_1^2 + x_2^2)$$

6.5.3 10.4.3 核函数的奇迹

但是，直接计算高维映射 $\phi(\mathbf{x})$ 可能非常复杂，甚至维度是**无限**的！

核技巧（Kernel Trick）的魔法在于：

> 我们不需要显式计算 $\phi(\mathbf{x})$，只需要计算**核函数** $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j)$

对偶问题中的损失函数只依赖于内积，所以我们可以直接用核函数代替：

$$\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

6.5.4 10.4.4 常用核函数

1 线性核（Linear Kernel）

$$K(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$$

- 就是原始空间的内积
- 适用于线性可分的数据

2 多项式核（Polynomial Kernel）

$$K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x} \cdot \mathbf{x}' + r)^d$$

其中 d 是多项式次数，γ 和 r 是参数。

例子：当 $d = 2$，$\gamma = 1$，$r = 0$ 时，对于二维向量：

$$K(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2')^2$$

展开后等价于映射到特征 $(x_1^2, x_2^2, \sqrt{2} x_1 x_2)$ 空间！

3 高斯径向基核（RBF Kernel）

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right)$$

- 最常用的核函数
- 把数据映射到**无限维**空间
- γ 越大，决策边界越复杂（越”弯曲”）

直观理解 RBF 核：- 两个点越近，核函数值越接近 1（越”相似”）- 两个点越远，核函数值越接近 0（越”不相似”）- 这就像在问：“这两个学生坐在多近的位置?”

4 Sigmoid 核

$$K(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x} \cdot \mathbf{x}' + r)$$

- 类似于神经网络中的激活函数
- 较少使用，但在某些特定问题上表现好

6.5.5 10.4.5 折叠纸张的比喻

费曼式解释：想象你有一张纸，上面用红笔和蓝笔画了两个交错的圆圈。如果你只在纸面上找直线，无论如何都分不开红蓝两色。

但是！如果你**把纸张折叠**一下，让中间鼓起来，边缘压下去，那么在三维空间中，你就能找到一个平面把红蓝两色分开！

核技巧就是这个”折叠”操作——它不改变点在纸上的位置关系，只是给了它们一个新的”高度”，让原本纠缠的数据变得可分！

6.6 10.5 代码实现：从零开始写 SVM

现在让我们用 NumPy 实现 SVM！我们会实现：1. 线性 SVM（使用梯度下降）2. 核 SVM（使用 SMO 算法简化版）

6.6.1 10.5.1 线性 SVM（梯度下降法）

这是一个简化版，使用次梯度下降来优化软间隔目标：

```
"""
线性SVM - 简化实现
使用次梯度下降法优化软间隔损失函数
"""
import numpy as np
import matplotlib.pyplot as plt

class LinearSVM:
    """
    线性支持向量机

    参数:
        C: 正则化参数 (越大越严格)
        learning_rate: 学习率
        n_iterations: 迭代次数
    """
    def __init__(self, C=1.0, learning_rate=0.001, n_iterations=1000):
        self.C = C
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.w = None  # 权重向量
        self.b = None  # 偏置项

    def fit(self, X, y):
        """
        训练SVM

        参数:
            X: 训练数据, 形状 (n_samples, n_features)
            y: 标签, 形状 (n_samples,), 取值为 +1 或 -1
        """
        n_samples, n_features = X.shape

        # 初始化参数
        self.w = np.zeros(n_features)
        self.b = 0

        # 梯度下降优化
        for iteration in range(self.n_iterations):
            # 计算每个样本的约束违反情况
```

```
        margins = y * (np.dot(X, self.w) + self.b)

        # 计算次梯度
        # 对于 w: 如果 margin < 1, 梯度包含 -C * y_i * x_i
        # 对于 b: 如果 margin < 1, 梯度包含 -C * y_i

        # 找出违反约束的样本 (margin < 1)
        misclassified = margins < 1

        # 计算 w 的梯度
        #  _w = w - C * Σ(y_i * x_i) for misclassified
        grad_w = self.w - self.C * np.sum((y[misclassified][:, None] *
            X[misclassified]), axis=0) / n_samples

        # 计算 b 的梯度
        #  _b = -C * Σ(y_i) for misclassified
        grad_b = -self.C * np.sum(y[misclassified]) / n_samples

        # 更新参数
        self.w -= self.learning_rate * grad_w
        self.b -= self.learning_rate * grad_b

        # 每100次迭代打印一次损失
        if (iteration + 1) % 100 == 0:
            loss = self._compute_loss(X, y)
            print(f"Iteration {iteration + 1}/{self.n_iterations}, Loss:
                {loss:.4f}")

    def _compute_loss(self, X, y):
        """计算 hinge loss + L2 正则化的损失函数值"""
        # Hinge loss: max(0, 1 - y * (w·x + b))
        margins = y * (np.dot(X, self.w) + self.b)
        hinge_loss = np.maximum(0, 1 - margins)

        # 总损失 = 0.5 * ||w||^2 + C * Σ hinge_loss
        loss = 0.5 * np.dot(self.w, self.w) + self.C * np.sum(hinge_loss)
        return loss

    def predict(self, X):
        """
        预测类别

        参数:
            X: 测试数据

        返回:
            预测标签 (+1 或 -1)
```

```
        """
        scores = np.dot(X, self.w) + self.b
        return np.sign(scores)

    def decision_function(self, X):
        """
        计算决策函数值（到超平面的有符号距离）

        参数:
            X: 测试数据

        返回:
            决策函数值
        """
        return np.dot(X, self.w) + self.b

    def get_support_vectors(self, X, y, tolerance=1e-5):
        """
        获取支持向量（距离决策边界最近的点）

        参数:
            X: 数据
            y: 标签
            tolerance: 判定为支持向量的阈值

        返回:
            支持向量的索引
        """
        margins = np.abs(y * self.decision_function(X) - 1)
        return np.where(margins < tolerance)[0]


def visualize_linear_svm():
    """可视化线性SVM的分类效果"""
    np.random.seed(42)

    # 生成线性可分的数据
    # 向日葵班（类别 +1）
    X_sunflower = np.random.randn(50, 2) + np.array([2, 2])
    # 星空班（类别 -1）
    X_starry = np.random.randn(50, 2) + np.array([-2, -2])

    X = np.vstack([X_sunflower, X_starry])
    y = np.hstack([np.ones(50), -np.ones(50)])

    # 训练SVM
    svm = LinearSVM(C=1.0, learning_rate=0.01, n_iterations=1000)
```

```
svm.fit(X, y)

# 可视化
plt.figure(figsize=(10, 8))

# 绘制数据点
plt.scatter(X[:50, 0], X[:50, 1], c='gold', s=100, marker='o',
            edgecolors='black', label='向日葵班 (+1)', alpha=0.8)
plt.scatter(X[50:, 0], X[50:, 1], c='navy', s=100, marker='s',
            edgecolors='black', label='星空班 (-1)', alpha=0.8)

# 获取支持向量
sv_indices = svm.get_support_vectors(X, y, tolerance=0.1)
if len(sv_indices) > 0:
    plt.scatter(X[sv_indices, 0], X[sv_indices, 1], s=300,
                facecolors='none', edgecolors='red', linewidths=2,
                label='支持向量')

# 绘制决策边界和间隔边界
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx = np.linspace(x_min, x_max, 100)

# 决策边界: w·x + b = 0  =>  y = -(w[0]*x + b) / w[1]
yy_decision = -(svm.w[0] * xx + svm.b) / svm.w[1]
# 间隔边界: w·x + b = ±1
yy_plus = -(svm.w[0] * xx + svm.b - 1) / svm.w[1]
yy_minus = -(svm.w[0] * xx + svm.b + 1) / svm.w[1]

plt.plot(xx, yy_decision, 'k-', linewidth=2, label='决策边界')
plt.plot(xx, yy_plus, 'k--', linewidth=1, alpha=0.5, label='间隔边界')
plt.plot(xx, yy_minus, 'k--', linewidth=1, alpha=0.5)

# 填充间隔区域
plt.fill_between(xx, yy_minus, yy_plus, alpha=0.1, color='gray',
    label='间隔区域')

plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.xlabel('位置 x ', fontsize=12)
plt.ylabel('位置 x ', fontsize=12)
plt.title('线性SVM: 寻找最宽的走道', fontsize=14, fontweight='bold')
plt.legend(loc='upper left', fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('linear_svm.png', dpi=150, bbox_inches='tight')
```

```
    plt.show()

    print(f"\n训练完成！")
    print(f"权重向量 w = {svm.w}")
    print(f"偏置项 b = {svm.b:.4f}")
    print(f"间隔宽度 = {2 / np.linalg.norm(svm.w):.4f}")
    print(f"支持向量数量 = {len(sv_indices)}")

    return svm


# 运行可视化
if __name__ == "__main__":
    print("=" * 60)
    print("  Linear SVM Demo - 寻找最宽的走道  ")
    print("=" * 60)
    svm = visualize_linear_svm()
```

6.6.2 10.5.2 核函数实现

```
"""
核函数实现
包含线性核、多项式核、RBF核
"""
import numpy as np

class Kernels:
    """核函数集合"""

    @staticmethod
    def linear():
        """
        线性核：K(x, x') = x · x'
        """
        def kernel(X1, X2):
            return np.dot(X1, X2.T)
        return kernel

    @staticmethod
    def polynomial(gamma=1.0, coef0=1.0, degree=3):
        """
        多项式核：K(x, x') = (  · x·x' + r)^d

        参数：
            gamma: 缩放参数
            coef0: 常数项 r
```

```
            degree: 多项式次数 d
        """
        def kernel(X1, X2):
            return (gamma * np.dot(X1, X2.T) + coef0) ** degree
        return kernel

    @staticmethod
    def rbf(gamma=1.0):
        """
        RBF（高斯径向基）核：K(x, x') = exp(-  ||x - x'||²)

        参数：
            gamma: 控制高斯函数的宽度
                   越大 → 核函数越"尖锐" → 模型越复杂
                   越小 → 核函数越"平坦" → 模型越简单
        """
        def kernel(X1, X2):
            # 计算两两之间的欧氏距离平方
            # ||x - x'||² = ||x||² + ||x'||² - 2x·x'
            X1_norm = np.sum(X1**2, axis=1).reshape(-1, 1)
            X2_norm = np.sum(X2**2, axis=1).reshape(1, -1)

            # 距离平方矩阵
            dist_sq = X1_norm + X2_norm - 2 * np.dot(X1, X2.T)

            return np.exp(-gamma * dist_sq)
        return kernel

    @staticmethod
    def sigmoid(gamma=1.0, coef0=0.0):
        """
        Sigmoid核：K(x, x') = tanh(  · x·x' + r)

        参数：
            gamma: 缩放参数
            coef0: 常数项
        """
        def kernel(X1, X2):
            return np.tanh(gamma * np.dot(X1, X2.T) + coef0)
        return kernel


def demo_kernels():
    """演示不同核函数的效果"""
    # 两个示例向量
    x1 = np.array([[1, 2]])
    x2 = np.array([[3, 4]])
```

```
    print("=" * 60)
    print("  核函数演示")
    print("=" * 60)
    print(f"向量 x1 = {x1}")
    print(f"向量 x2 = {x2}")
    print(f"x1 和 x2 的距离 = {np.linalg.norm(x1 - x2):.4f}")
    print()

    # 线性核
    linear_k = Kernels.linear()
    print(f"  线性核 K(x1, x2) = {linear_k(x1, x2)[0, 0]:.4f}")
    print(f"    (就是两个向量的内积)")
    print()

    # 多项式核
    poly_k = Kernels.polynomial(gamma=1.0, coef0=1.0, degree=2)
    print(f"  多项式核(degree=2) K(x1, x2) = {poly_k(x1, x2)[0, 0]:.4f}")
    print(f"    (等价于映射到高维后的内积)")
    print()

    # RBF核
    rbf_k = Kernels.rbf(gamma=0.5)
    print(f"  RBF核(gamma=0.5) K(x1, x2) = {rbf_k(x1, x2)[0, 0]:.4f}")
    print(f"    (距离越远，核函数值越小)")
    print()

    # 展示 gamma 对 RBF 的影响
    print("  gamma 参数对 RBF 核的影响:")
    print("-" * 40)
    for gamma in [0.1, 0.5, 1.0, 5.0, 10.0]:
        rbf = Kernels.rbf(gamma=gamma)
        value = rbf(x1, x2)[0, 0]
        print(f"  gamma={gamma:4.1f}: K(x1, x2) = {value:.6f}")
    print("\n  gamma 越大 → 核函数衰减越快 → 模型越"复杂"(容易过拟合)")


if __name__ == "__main__":
    demo_kernels()
```

6.6.3 10.5.3 SMO 算法简化实现

```
"""
SMO (Sequential Minimal Optimization) 算法简化实现
用于高效求解SVM对偶问题
```

```
参考：Platt, J. (1998). Sequential Minimal Optimization: A Fast Algorithm for
    Training Support Vector Machines
"""
import numpy as np
import random

class SimplifiedSMO:
    """
    SMO算法简化实现

    核心思想：每次只优化两个拉格朗日乘子 _i 和 _j，
    这样可以解析求解，而不需要复杂的QP优化器。
    """
    def __init__(self, X, y, C=1.0, tolerance=0.001, max_passes=100,
        kernel_type='linear', gamma=1.0):
        """
        初始化SMO

        参数：
            X: 训练数据 (n_samples, n_features)
            y: 标签 (n_samples,)，取值为 +1 或 -1
            C: 正则化参数
            tolerance: KKT条件违反的容差
            max_passes: 最大迭代轮数
            kernel_type: 'linear' 或 'rbf'
            gamma: RBF核参数
        """
        self.X = X
        self.y = y
        self.C = C
        self.tol = tolerance
        self.max_passes = max_passes
        self.kernel_type = kernel_type
        self.gamma = gamma

        self.m, self.n = X.shape  # 样本数和特征数

        # 初始化拉格朗日乘子 和偏置 b
        self.alphas = np.zeros(self.m)
        self.b = 0.0

        # 预计算核矩阵（简化版，适用于中小数据集）
        self.K = self._compute_kernel_matrix()

    def _compute_kernel_matrix(self):
        """计算核矩阵 K[i,j] = K(x_i, x_j)"""
        if self.kernel_type == 'linear':
```

```
            # 线性核：K(x, x') = x · x'
            return np.dot(self.X, self.X.T)
        elif self.kernel_type == 'rbf':
            # RBF核：K(x, x') = exp(-  ||x - x'||²)
            X_norm = np.sum(self.X**2, axis=1).reshape(-1, 1)
            dist_sq = X_norm + X_norm.T - 2 * np.dot(self.X, self.X.T)
            return np.exp(-self.gamma * dist_sq)
        else:
            raise ValueError(f"Unknown kernel type: {self.kernel_type}")

    def _kernel(self, i, j):
        """获取核矩阵元素 K[i,j]"""
        return self.K[i, j]

    def _predict_output(self, i):
        """
        计算样本 i 的预测输出 f(x_i)
        f(x_i) = Σ( _k · y_k · K(x_k, x_i)) + b
        """
        return np.sum(self.alphas * self.y * self.K[:, i]) + self.b

    def _calculate_error(self, i):
        """计算样本 i 的预测误差 E_i = f(x_i) - y_i"""
        return self._predict_output(i) - self.y[i]

    def _select_j_randomly(self, i):
        """随机选择 j   i"""
        j = i
        while j == i:
            j = random.randint(0, self.m - 1)
        return j

    def _clip_alpha(self, alpha, H, L):
        """将   裁剪到 [L, H] 范围内"""
        if alpha > H:
            return H
        if alpha < L:
            return L
        return alpha

    def _take_step(self, i, j):
        """
        尝试优化 _i 和 _j 这一对乘子

        返回 True 如果成功更新，False 如果没有进展
        """
        if i == j:
```

```
            return False

        alpha_i_old = self.alphas[i].copy()
        alpha_j_old = self.alphas[j].copy()
        yi, yj = self.y[i], self.y[j]

        # 计算误差
        Ei = self._calculate_error(i)
        Ej = self._calculate_error(j)

        # 计算 _j 的边界 L 和 H
        if yi != yj:
            # 当 y_i  y_j 时
            L = max(0, alpha_j_old - alpha_i_old)
            H = min(self.C, self.C + alpha_j_old - alpha_i_old)
        else:
            # 当 y_i = y_j 时
            L = max(0, alpha_i_old + alpha_j_old - self.C)
            H = min(self.C, alpha_i_old + alpha_j_old)

        if L == H:
            return False

        # 计算  = K_ii + K_jj - 2K_ij
        eta = self._kernel(i, i) + self._kernel(j, j) - 2 * self._kernel(i, j)

        if eta <= 0:
            return False

        # 计算未裁剪的新 _j
        alpha_j_new_unc = alpha_j_old + yj * (Ei - Ej) / eta

        # 裁剪 _j 到 [L, H]
        alpha_j_new = self._clip_alpha(alpha_j_new_unc, H, L)

        # 检查变化是否显著
        if abs(alpha_j_new - alpha_j_old) < 1e-5:
            return False

        # 计算新的 _i
        # _i^new = _i^old + y_i·y_j·( _j^old - _j^new)
        alpha_i_new = alpha_i_old + yi * yj * (alpha_j_old - alpha_j_new)

        # 更新偏置 b
        b1 = (self.b - Ei - yi * (alpha_i_new - alpha_i_old) * self._kernel(i, i)
              - yj * (alpha_j_new - alpha_j_old) * self._kernel(i, j))
        b2 = (self.b - Ej - yi * (alpha_i_new - alpha_i_old) * self._kernel(i, j)
```

```
            - yj * (alpha_j_new - alpha_j_old) * self._kernel(j, j))

        # 根据    是否在边界内来选择 b
        if 0 < alpha_i_new < self.C:
            self.b = b1
        elif 0 < alpha_j_new < self.C:
            self.b = b2
        else:
            self.b = (b1 + b2) / 2.0

        # 更新
        self.alphas[i] = alpha_i_new
        self.alphas[j] = alpha_j_new

        return True

    def _examine_example(self, i):
        """
        检查样本 i 的KKT条件，尝试优化它

        KKT条件：
        - 如果  _i = 0，则 y_i·f(x_i)   1（样本正确分类且在间隔外）
        - 如果 0 <  _i < C，则 y_i·f(x_i) = 1（样本在间隔边界上）
        - 如果  _i = C，则 y_i·f(x_i)   1（样本在间隔内或分类错误）
        """
        yi = self.y[i]
        alpha_i = self.alphas[i]
        Ei = self._calculate_error(i)

        # 检查KKT条件是否违反
        r = Ei * yi

        # 违反条件的情况：
        # 1. r < -tol 且  _i < C（应该增加  _i）
        # 2. r > tol 且  _i > 0（应该减小  _i）
        violate_kkt = (r < -self.tol and alpha_i < self.C) or (r > self.tol and
            alpha_i > 0)

        if not violate_kkt:
            return False

        # 启发式1：优先选择非边界上的  _j（0 <   < C）
        non_bound_idx = np.where((self.alphas > 0) & (self.alphas < self.C))[0]

        if len(non_bound_idx) > 1:
            # 选择使 |Ei - Ej| 最大的 j
            max_delta_E = 0
```

```
            best_j = -1
            for k in non_bound_idx:
                if k == i:
                    continue
                Ek = self._calculate_error(k)
                delta_E = abs(Ei - Ek)
                if delta_E > max_delta_E:
                    max_delta_E = delta_E
                    best_j = k

            if best_j != -1 and self._take_step(i, best_j):
                return True

        # 启发式2：在非边界点中随机尝试
        non_bound_list = list(non_bound_idx)
        random.shuffle(non_bound_list)
        for j in non_bound_list:
            if j != i and self._take_step(i, j):
                return True

        # 启发式3：在所有点中随机尝试
        all_idx = list(range(self.m))
        random.shuffle(all_idx)
        for j in all_idx:
            if j != i and self._take_step(i, j):
                return True

        return False

    def fit(self):
        """训练SVM"""
        print("  开始SMO训练...")

        num_changed = 0
        examine_all = True
        passes = 0

        while (num_changed > 0 or examine_all) and passes < self.max_passes:
            num_changed = 0

            if examine_all:
                # 遍历所有样本
                for i in range(self.m):
                    if self._examine_example(i):
                        num_changed += 1
                print(f"  全遍历轮次：更新了 {num_changed} 个  ")
            else:
```

```
                # 只遍历非边界样本
                non_bound_idx = np.where((self.alphas > 0) & (self.alphas <
                    self.C))[0]
                for i in non_bound_idx:
                    if self._examine_example(i):
                        num_changed += 1
                print(f"  非边界遍历：更新了 {num_changed} 个  ")

            if examine_all:
                examine_all = False
            elif num_changed == 0:
                examine_all = True

            passes += 1
            print(f"  完成第 {passes} 轮")

        print(f"  训练完成！共 {passes} 轮")

        # 提取支持向量
        self.support_vector_idx = np.where(self.alphas > 1e-5)[0]
        self.support_vectors = self.X[self.support_vector_idx]
        self.support_vector_labels = self.y[self.support_vector_idx]
        self.support_vector_alphas = self.alphas[self.support_vector_idx]

        print(f"  支持向量数量：{len(self.support_vector_idx)} / {self.m}")

    def predict(self, X):
        """
        预测新样本的类别

        f(x) = Σ( _sv · y_sv · K(x_sv, x)) + b
        """
        if self.kernel_type == 'linear':
            # 线性核可以直接计算 w·x + b
            w = np.sum((self.alphas * self.y).reshape(-1, 1) * self.X, axis=0)
            scores = np.dot(X, w) + self.b
        else:
            # 非线性核需要计算与所有支持向量的核函数
            scores = np.zeros(X.shape[0])
            for i, x in enumerate(X):
                # 计算 x 与所有支持向量的核函数值
                if self.kernel_type == 'rbf':
                    # RBF核
                    dist_sq = np.sum((self.support_vectors - x)**2, axis=1)
                    k_values = np.exp(-self.gamma * dist_sq)
                scores[i] = np.sum(self.support_vector_alphas *
                    self.support_vector_labels * k_values) + self.b
```

```
        return np.sign(scores)

    def score(self, X, y):
        """计算准确率"""
        predictions = self.predict(X)
        return np.mean(predictions == y)


def demo_smo():
    """演示SMO算法"""
    from sklearn.datasets import make_blobs, make_circles, make_moons
    import matplotlib.pyplot as plt

    print("=" * 60)
    print("  SMO算法演示")
    print("=" * 60)

    # 测试1: 线性可分数据
    print("\n  测试1: 线性可分数据")
    X1, y1 = make_blobs(n_samples=100, centers=2, random_state=42, cluster_std=1.0)
    y1 = np.where(y1 == 0, -1, 1)

    svm1 = SimplifiedSMO(X1, y1, C=1.0, kernel_type='linear', max_passes=50)
    svm1.fit()
    acc1 = svm1.score(X1, y1)
    print(f"训练准确率: {acc1 * 100:.2f}%")

    # 测试2: 非线性数据（月亮形状）
    print("\n  测试2: 非线性数据（月亮形状） - 使用RBF核")
    X2, y2 = make_moons(n_samples=100, noise=0.1, random_state=42)
    y2 = np.where(y2 == 0, -1, 1)

    svm2 = SimplifiedSMO(X2, y2, C=10.0, kernel_type='rbf', gamma=5.0,
        max_passes=100)
    svm2.fit()
    acc2 = svm2.score(X2, y2)
    print(f"训练准确率: {acc2 * 100:.2f}%")

    # 可视化
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # 图1: 线性数据
    plot_svm_boundary(axes[0], X1, y1, svm1, "线性SVM")

    # 图2: 非线性数据
    plot_svm_boundary(axes[1], X2, y2, svm2, "RBF核SVM")
```

```
    plt.tight_layout()
    plt.savefig('smo_demo.png', dpi=150, bbox_inches='tight')
    plt.show()


def plot_svm_boundary(ax, X, y, svm, title):
    """绘制SVM决策边界"""
    # 绘制数据点
    pos_idx = y == 1
    neg_idx = y == -1
    ax.scatter(X[pos_idx, 0], X[pos_idx, 1], c='gold', s=50,
               edgecolors='black', label='Class +1', alpha=0.8)
    ax.scatter(X[neg_idx, 0], X[neg_idx, 1], c='navy', s=50,
               edgecolors='black', label='Class -1', alpha=0.8)

    # 绘制支持向量
    if len(svm.support_vector_idx) > 0:
        ax.scatter(X[svm.support_vector_idx, 0], X[svm.support_vector_idx, 1],
                   s=200, facecolors='none', edgecolors='red', linewidths=2,
                   label='Support Vectors')

    # 绘制决策边界
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                         np.linspace(y_min, y_max, 200))

    Z = svm.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.3, levels=[-2, 0, 2], colors=['blue', 'red'])
    ax.contour(xx, yy, Z, levels=[0], colors='black', linewidths=2)

    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.legend()
    ax.grid(True, alpha=0.3)


if __name__ == "__main__":
    demo_smo()
```

6.6.4 10.5.4 完整演示脚本

```python
"""
SVM完整演示：比较不同核函数的效果
"""
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs, make_circles, make_moons

# 导入我们实现的类
from linear_svm import LinearSVM
from smo_svm import SimplifiedSMO

def compare_kernels():
    """比较不同核函数在各类数据集上的表现"""

    # 生成三种不同类型的数据集
    datasets = []

    # 1. 线性可分数据
    X1, y1 = make_blobs(n_samples=100, centers=2, random_state=42, cluster_std=1.2)
    y1 = np.where(y1 == 0, -1, 1)
    datasets.append(("  线性可分数据", X1, y1))

    # 2. 同心圆数据（必须使用核函数）
    X2, y2 = make_circles(n_samples=100, factor=0.5, noise=0.08, random_state=42)
    y2 = np.where(y2 == 0, -1, 1)
    datasets.append(("  同心圆数据", X2, y2))

    # 3. 月亮数据
    X3, y3 = make_moons(n_samples=100, noise=0.15, random_state=42)
    y3 = np.where(y3 == 0, -1, 1)
    datasets.append(("  月亮形状数据", X3, y3))

    fig, axes = plt.subplots(3, 3, figsize=(15, 15))

    for row, (data_name, X, y) in enumerate(datasets):
        print(f"\n{'='*60}")
        print(f"  数据集：{data_name}")
        print(f"{'='*60}")

        # 测试三种核函数
        configs = [
            ("线性核", "linear", {}),
            ("RBF核(=1)", "rbf", {"gamma": 1.0}),
            ("RBF核(=10)", "rbf", {"gamma": 10.0}),
        ]

        for col, (kernel_name, kernel_type, kernel_params) in enumerate(configs):
```

```
            print(f"\n    {kernel_name}")

            try:
                # 训练SMO SVM
                svm = SimplifiedSMO(X, y, C=1.0, kernel_type=kernel_type,
                                    max_passes=50, **kernel_params)
                svm.fit()
                accuracy = svm.score(X, y)
                print(f"     准确率: {accuracy*100:.1f}%")

                # 绘制结果
                ax = axes[row, col]
                plot_decision_boundary(ax, X, y, svm, f"{data_name}\n{kernel_name}")

            except Exception as e:
                print(f"     错误: {e}")
                ax = axes[row, col]
                ax.text(0.5, 0.5, f"Error:\n{e}", ha='center', va='center',
                        transform=ax.transAxes, fontsize=10)
                ax.set_title(f"{data_name}\n{kernel_name}", fontsize=10)

    plt.tight_layout()
    plt.savefig('kernel_comparison.png', dpi=150, bbox_inches='tight')
    plt.show()

    print(f"\n{'='*60}")
    print("  所有测试完成！")
    print("观察结果：")
    print("  - 线性数据：线性核表现最好")
    print("  - 圆形/月亮数据：RBF核能处理非线性边界")
    print("  - gamma越大：决策边界越复杂，可能过拟合")
    print(f"{'='*60}")


def plot_decision_boundary(ax, X, y, svm, title):
    """绘制决策边界"""
    # 确定绘图范围
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

    # 创建网格
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                         np.linspace(y_min, y_max, 200))

    # 预测网格点
    grid_points = np.c_[xx.ravel(), yy.ravel()]
    Z = svm.predict(grid_points)
```

```
    Z = Z.reshape(xx.shape)

    # 绘制决策区域
    ax.contourf(xx, yy, Z, alpha=0.3, levels=[-2, 0, 2],
                colors=['#4488ff', '#ff8844'])
    ax.contour(xx, yy, Z, levels=[0], colors='black', linewidths=2)

    # 绘制数据点
    pos_idx = y == 1
    neg_idx = y == -1
    ax.scatter(X[pos_idx, 0], X[pos_idx, 1], c='gold', s=50,
               edgecolors='black', linewidths=1, label='+1', zorder=5)
    ax.scatter(X[neg_idx, 0], X[neg_idx, 1], c='navy', s=50,
               edgecolors='white', linewidths=1, label='-1', zorder=5)

    # 绘制支持向量
    if len(svm.support_vector_idx) > 0:
        ax.scatter(X[svm.support_vector_idx, 0], X[svm.support_vector_idx, 1],
                   s=150, facecolors='none', edgecolors='red',
                   linewidths=2, label='SV', zorder=6)

    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.set_title(title, fontsize=11, fontweight='bold')
    ax.set_xticks([])
    ax.set_yticks([])


def demonstrate_margin():
    """演示最大间隔原理"""
    print("\n" + "="*60)
    print("  演示：最大间隔原理")
    print("="*60)

    # 生成线性可分数据
    np.random.seed(42)
    X_pos = np.random.randn(20, 2) + np.array([2, 2])
    X_neg = np.random.randn(20, 2) + np.array([-2, -2])
    X = np.vstack([X_pos, X_neg])
    y = np.hstack([np.ones(20), -np.ones(20)])

    # 使用不同的C值
    C_values = [0.01, 0.1, 1.0, 100.0]

    fig, axes = plt.subplots(2, 2, figsize=(12, 12))
    axes = axes.flatten()
```

```
    for i, C in enumerate(C_values):
        print(f"\n  训练 C = {C}...")

        svm = SimplifiedSMO(X, y, C=C, kernel_type='linear', max_passes=50)
        svm.fit()

        ax = axes[i]

        # 计算权重向量 w
        w = np.sum((svm.alphas * y).reshape(-1, 1) * X, axis=0)
        margin = 2 / np.linalg.norm(w)

        # 绘制
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        xx = np.linspace(x_min, x_max, 100)

        # 决策边界和间隔边界
        if abs(w[1]) > 1e-10:
            yy_decision = -(w[0] * xx + svm.b) / w[1]
            yy_plus = -(w[0] * xx + svm.b - 1) / w[1]
            yy_minus = -(w[0] * xx + svm.b + 1) / w[1]

            ax.plot(xx, yy_decision, 'k-', linewidth=2, label='决策边界')
            ax.plot(xx, yy_plus, 'k--', linewidth=1, alpha=0.5, label='间隔边界')
            ax.plot(xx, yy_minus, 'k--', linewidth=1, alpha=0.5)
            ax.fill_between(xx, yy_minus, yy_plus, alpha=0.1, color='gray')

        # 数据点
        ax.scatter(X[:20, 0], X[:20, 1], c='gold', s=60, edgecolors='black',
            label='+1')
        ax.scatter(X[20:, 0], X[20:, 1], c='navy', s=60, edgecolors='black',
            label='-1')

        # 支持向量
        if len(svm.support_vector_idx) > 0:
            ax.scatter(X[svm.support_vector_idx, 0], X[svm.support_vector_idx, 1],
                       s=200, facecolors='none', edgecolors='red', linewidths=2)

        ax.set_title(f'C = {C}\n间隔宽度 = {margin:.3f}, 支持向量数 =
            {len(svm.support_vector_idx)}',
                     fontsize=12, fontweight='bold')
        ax.set_xlim(x_min, x_max)
        ax.set_ylim(X[:, 1].min() - 1, X[:, 1].max() + 1)
        ax.legend(fontsize=8)
        ax.grid(True, alpha=0.3)

    plt.suptitle('  C参数对间隔的影响', fontsize=16, fontweight='bold', y=1.02)
```

```
    plt.tight_layout()
    plt.savefig('margin_demo.png', dpi=150, bbox_inches='tight')
    plt.show()

    print("\n结 论： ")
    print("  C越 小 → 间 隔 越 大 → 允 许 更 多 分 类 错 误 → 模 型 更 简 单 → 可 能 欠 拟 合")
    print("  C越 大 → 间 隔 越 小 → 严 格 要 求 正 确 分 类 → 模 型 更 复 杂 → 可 能 过 拟 合")


if __name__ == "__main__":
    # 运 行 所 有 演 示
    compare_kernels()
    demonstrate_margin()
```

6.7 10.6 练习题

6.7.1 基础练习

练习 10.1 间隔计算

给定二维空间中的决策边界方程 $2x_1 + 3x_2 + 1 = 0$，计算：

1. 权重向量 $\mathbf{w}$ 的长度 $\|\mathbf{w}\|$
2. 间隔的宽度
3. 点 $(1, 1)$ 到决策边界的距离
4. 判断点 $(1, 1)$ 属于哪一侧

提示

- 权重向量 $\mathbf{w} = (2, 3)$
- 间隔宽度 $= \frac{2}{\|\mathbf{w}\|}$
- 点到直线的距离 $= \frac{|\mathbf{w}\cdot\mathbf{x}+b|}{\|\mathbf{w}\|}$

练习 10.2 支持向量识别

给定以下数据点和已训练好的 SVM：

样本	坐标 (x_1, x_2)	标签 y	$\mathbf{w}\cdot\mathbf{x}+b$
A	(1, 2)	+1	1.5
B	(2, 1)	+1	1.0
C	(3, 3)	+1	2.5
D	(-1, -1)	-1	-1.0
E	(-2, -3)	-1	-2.0

1. 哪些样本是支持向量？

2. 间隔边界方程是什么?

练习 10.3 核函数计算

设 $\mathbf{x} = (1, 2)$，$\mathbf{x}' = (3, 1)$，计算:

1. 线性核 $K(\mathbf{x}, \mathbf{x}')$
2. 多项式核（$d = 2$，$\gamma = 1$，$r = 0$）
3. RBF 核（$\gamma = 0.5$）

6.7.2 进阶练习

练习 10.4 软间隔 SVM 推导

考虑软间隔 SVM 的优化问题:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i$$

约束：$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i$，$\xi_i \geq 0$

1. 写出该问题的拉格朗日函数
2. 推导 KKT 条件
3. 解释互补松弛条件 $\alpha_i[y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i] = 0$ 的含义

练习 10.5 SMO 算法分析

SMO 算法每次选择两个拉格朗日乘子 α_i 和 α_j 进行优化。

1. 为什么不能只优化一个 α?
2. 在更新 α_j 时，为什么要裁剪到 $[L, H]$ 区间?
3. 解释启发式选择策略：为什么优先选择违反 KKT 条件的样本?

6.7.3 挑战练习

练习 10.6 实现多分类 SVM

SVM 本质上是二分类器。请实现一个**一对多（One-vs-Rest）**策略的多分类 SVM:

1. 对于 K 个类别，训练 K 个二分类 SVM
2. 每个 SVM 将一个类别与所有其他类别分开
3. 预测时，选择决策函数值最大的类别

用鸢尾花数据集（Iris）测试你的实现，并比较不同核函数的效果。

提示

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# 加载数据
iris = load_iris()
X, y = iris.data, iris.target

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# 实现 OvR 策略...
```

6.8 10.7 本章总结

6.8.1 核心概念回顾

概念	解释	类比
超平面	决策边界，分隔两个类别	教室里的走道
间隔	最近的正负样本到超平面的距离之和	走道的宽度
支持向量	距离超平面最近的样本，决定模型	坐在走道边的同学
最大间隔	选择最宽走道的策略	让两边都有最大空间
软间隔	允许部分样本分类错误	容忍调皮的违规者
核技巧	隐式映射到高维空间	折叠纸张创造新维度
拉格朗日乘子	将约束优化转化为无约束优化	引入”监督员”检查约束
对偶问题	原问题的等价形式，更易求解	换一种角度看问题
SMO 算法	每次优化两个变量的高效算法	一次只调整两块积木

6.8.2 SVM 的优势与局限

优势： - 理论基础坚实，有全局最优解 - 泛化能力强，不容易过拟合（间隔最大化）- 核技巧可以处理非线性问题 - 最终模型只依赖支持向量，存储高效

局限： - 大规模数据集训练较慢（$O(n^2)$ 到 $O(n^3)$）- 核函数和参数选择需要经验 - 对噪声敏感（特别是硬间隔）- 不能直接输出概率

6.8.3 与其他算法的比较

特性	SVM	逻辑回归	决策树	神经网络
决策边界	光滑超平面	光滑超平面	轴对齐的矩形	任意复杂形状
训练速度	中等	快	快	慢
可解释性	中等	高	高	低

特性	SVM	逻辑回归	决策树	神经网络
处理高维	优秀	需正则化	困难	优秀
非线性	核技巧	特征工程	天然支持	天然支持
概率输出	需额外处理	天然支持	天然支持	天然支持

6.8.4 延伸学习

1. **支持向量回归（SVR）**：将 SVM 扩展到回归问题
2. **核方法的其他应用**：核 PCA、核 K-means
3. **在线 SVM**：处理流式数据的 SVM 变体
4. **深度学习与 SVM**：用神经网络提取特征 + SVM 分类

6.9 参考文献

Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. In *Proceedings of the 5th Annual Workshop on Computational Learning Theory* (pp. 144-152). ACM.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297. https://doi.org/10.1007/BF00994018

Cristianini, N., & Shawe-Taylor, J. (2000). *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*. Cambridge University Press.

Platt, J. C. (1998). Sequential minimal optimization: A fast algorithm for training support vector machines. *Microsoft Research Technical Report MSR-TR-98-14*.

Schölkopf, B., & Smola, A. J. (2002). *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.

Vapnik, V. N. (1995). *The Nature of Statistical Learning Theory*. Springer-Verlag.

Vapnik, V. N. (1998). *Statistical Learning Theory*. Wiley.

Vapnik, V. N., & Chervonenkis, A. Y. (1964). A note on one class of perceptrons. *Automation and Remote Control*, 25(1).

本章格言：就像最宽的走道能让两个班级和谐共处，最大的间隔能让机器学习模型拥有最好的泛化能力。在数学的世界里，“留有余地”不仅是处世智慧，更是最优解的秘诀！

本章完

---

7 第十一章：朴素贝叶斯——用概率做判断

7.1 开场故事：小明的神奇糖果罐

小明有一个神奇的糖果罐，里面装满了红色和蓝色的糖果。他每天放学后都会从罐子里摸出一颗糖果，然后记录颜色。渐渐地，小明发现：如果罐子被”幸运女神”祝福过，那么摸出红糖果的概率是 70%；如果没有被祝福，红糖果的概率只有 30%。

有一天，小明连续摸出了 3 颗红糖果。他兴奋地跳起来：“罐子一定被祝福过了！”妈妈问他为什么这么确定，小明掰着手指算道：“如果罐子被祝福了，连续 3 次红糖果的概率是 0.7×0.7×0.7=0.343；如果没被祝福，只有 0.3×0.3×0.3=0.027。前者是后者的 12 倍多呢！”

妈妈笑着说：“你用了一种叫做’贝叶斯推理’的方法呢！”

这就是朴素贝叶斯的精髓——通过观察到的证据（3 颗红糖果），来推断最可能的”原因”（罐子是否被祝福）。在机器学习的世界里，我们用同样的思路，让计算机学会”用概率做判断”。

7.2 11.1 从医生诊断说起

想象你去医院看病，医生听完你的描述后说：“根据你的症状，有 80% 的可能性是感冒，15% 是过敏，5% 是其他疾病。”

医生的这个判断过程，本质上就是贝叶斯思维的体现。让我们用更精确的语言来描述这个过程。

7.2.1 11.1.1 概率的基本概念

在深入贝叶斯定理之前，我们需要复习几个概率论的基本概念。

概率（Probability）是描述某个事件发生可能性的数值，取值范围在 0 到 1 之间。0 表示不可能发生，1 表示必然发生。

例如，抛一枚公平的硬币，正面朝上的概率是：

$$P(\text{正面}) = \frac{1}{2} = 0.5$$

联合概率（Joint Probability）描述两个事件同时发生的概率。用 $P(A, B)$ 或 $P(A \cap B)$ 表示。

假设有一个装着 3 个红球和 2 个蓝球的袋子，随机取出两个球。第一个球是红色且第二个球也是红色的概率是多少？

$$P(\text{第一次红}, \text{第二次红}) = P(\text{第一次红}) \times P(\text{第二次红}|\text{第一次红}) = \frac{3}{5} \times \frac{2}{4} = \frac{3}{10}$$

7.2.2 11.1.2 条件概率

条件概率（Conditional Probability）是贝叶斯方法的核心概念。它表示在已知某个事件发生的情况下，另一个事件发生的概率。记作 $P(A|B)$，读作”在 B 发生的条件下 A 的概率”。

让我们用一个具体的例子来理解：

假设一个班级有 40 名学生，其中 25 名男生，15 名女生。男生中有 10 名戴眼镜，女生中有 8 名戴眼镜。

我们可以计算：- 随机选一名学生是男生的概率：$P(\text{男}) = \frac{25}{40} = 0.625$ - 随机选一名学生戴眼镜的概率：$P(\text{戴眼镜}) = \frac{18}{40} =$

0.45 - **在已知是男生的条件下，戴眼镜的概率**：

$$P(\text{戴眼镜}|\text{男}) = \frac{\text{戴眼镜的男生数}}{\text{男生总数}} = \frac{10}{25} = 0.4$$

注意区分 $P(\text{戴眼镜}|\text{男})$ 和 $P(\text{男}|\text{戴眼镜})$：- $P(\text{戴眼镜}|\text{男}) = \text{男生中戴眼镜的比例} = 0.4$ - $P(\text{男}|\text{戴眼镜}) = \text{戴眼镜的学生中男生的比例} = \frac{10}{18} \approx 0.556$

条件概率的正式定义：

$$P(A|B) = \frac{P(A,B)}{P(B)}$$

其中 $P(B) > 0$。

从这个定义，我们可以推导出**乘法公式**：

$$P(A,B) = P(A|B) \times P(B) = P(B|A) \times P(A)$$

7.2.3 11.1.3 贝叶斯定理的诞生

托马斯·贝叶斯（Thomas Bayes, 1702-1761）是一位英国长老会牧师，也是一位数学家。在他生前，并没有引起太多关注，但他死后发表的论文《An Essay towards Solving a Problem in the Doctrine of Chances》（《关于机会学说中一个问题求解的论文》），却在统计学和机器学习领域产生了深远影响。

这篇论文于 1763 年由他的朋友理查德·普莱斯（Richard Price）整理后发表在《皇家学会哲学汇刊》上。贝叶斯在论文中提出了一个核心问题：

> “给定一个未知事件发生和失败的次数，要求计算该事件在单次试验中发生的概率介于任意两个指定概率值之间的可能性。”

这个问题看似抽象，但其本质是关于”逆概率”的问题——从观察到的结果反推原因的概率。

贝叶斯定理的推导：

从条件概率的乘法公式，我们有：

$$P(A|B) \times P(B) = P(B|A) \times P(A)$$

两边同时除以 $P(B)$（假设 $P(B) > 0$）：

$$\boxed{P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}}$$

这就是著名的**贝叶斯定理（Bayes’ Theorem）**！

让我们理解这个公式中各项的含义：

符号	名称	含义
$P(A)$	先验概率（Prior）	在观察到任何证据之前，对事件 A 的概率估计
$P(B\|A)$	似然（Likelihood）	在 A 发生的条件下，观察到 B 的概率
$P(B)$	证据（Evidence）	观察到 B 的总概率（不考虑 A）

符号	名称	含义
$P(A\|B)$	后验概率（Posterior）	在观察到 B 之后，对 A 概率的更新估计

全概率公式:

在实际应用中，$P(B)$ 往往不容易直接计算。这时我们可以使用全概率公式:

如果事件 $A_1, A_2, ..., A_n$ 构成一个完备事件组（互斥且穷尽所有可能性），则:

$$P(B) = \sum_{i=1}^{n} P(B|A_i) \times P(A_i)$$

因此，贝叶斯定理也可以写成:

$$P(A_k|B) = \frac{P(B|A_k) \times P(A_k)}{\sum_{i=1}^{n} P(B|A_i) \times P(A_i)}$$

7.2.4 11.1.4 一个医疗诊断的例子

让我们用一个更完整的医疗诊断例子来理解贝叶斯定理的应用。

假设有一种罕见疾病，患病率为 0.1%（即每 1000 人中有 1 人患病）。有一种检测方法: - 如果患病，检测呈阳性的概率是 99%（真阳性率）- 如果没有患病，检测呈阳性的概率是 5%（假阳性率）

现在，一个人的检测结果呈阳性，他实际患病的概率是多少?

直觉陷阱: 很多人可能会说”99%“，但这是错误的！让我们用贝叶斯定理计算。

设: - $D =$ 患病 - $\neg D =$ 未患病 - $T^+ =$ 检测阳性

已知: - $P(D) = 0.001$ - $P(T^+|D) = 0.99$ - $P(T^+|\neg D) = 0.05$

求: $P(D|T^+)$

应用贝叶斯定理:

$$P(D|T^+) = \frac{P(T^+|D) \times P(D)}{P(T^+)}$$

其中:

$$\begin{aligned} P(T^+) &= P(T^+|D) \times P(D) + P(T^+|\neg D) \times P(\neg D) \\ &= 0.99 \times 0.001 + 0.05 \times 0.999 \\ &= 0.00099 + 0.04995 \\ &= 0.05094 \end{aligned}$$

因此:

$$P(D|T^+) = \frac{0.99 \times 0.001}{0.05094} = \frac{0.00099}{0.05094} \approx 0.0194$$

结果：即使检测呈阳性，实际患病的概率只有约 1.94%！

这个结果可能令人惊讶，但它揭示了贝叶斯推理的一个重要洞见：**当疾病本身很罕见时，即使检测很准确，假阳性的数量也可能远超真阳性**。

让我们用具体数字来验证：假设有 10,000 人接受检测：- 实际患病：10 人 → 约 10 人检测阳性（99% 真阳性率）- 实际未患病：9,990 人 → 约 500 人检测阳性（5% 假阳性率）

在 510 个阳性结果中，只有约 10 人是真的患病，比例约为 10/510 1.96%，与我们的计算一致。

7.3 11.2 朴素贝叶斯分类器

7.3.1 11.2.1 从贝叶斯定理到分类器

现在我们将贝叶斯定理应用于分类问题。给定一个特征向量 $\mathbf{x} = (x_1, x_2, ..., x_n)$，我们想预测它属于哪个类别 C_k。

根据贝叶斯定理：

$$P(C_k|\mathbf{x}) = \frac{P(\mathbf{x}|C_k) \times P(C_k)}{P(\mathbf{x})}$$

由于 $P(\mathbf{x})$ 对于所有类别都是相同的，在比较不同类别时我们可以忽略它。因此：

$$P(C_k|\mathbf{x}) \propto P(\mathbf{x}|C_k) \times P(C_k)$$

分类决策为：

$$\hat{C} = \arg\max_k P(C_k|\mathbf{x}) = \arg\max_k P(\mathbf{x}|C_k) \times P(C_k)$$

7.3.2 11.2.2 “朴素”假设：特征独立性

这里有一个问题：计算 $P(\mathbf{x}|C_k) = P(x_1, x_2, ..., x_n|C_k)$ 需要估计特征之间的联合分布，这在高维情况下几乎是不可能的（维度灾难）。

朴素贝叶斯的”朴素”之处：假设所有特征在给定类别条件下是相互独立的。

即：

$$P(x_1, x_2, ..., x_n|C_k) = P(x_1|C_k) \times P(x_2|C_k) \times ... \times P(x_n|C_k) = \prod_{i=1}^{n} P(x_i|C_k)$$

这个假设在现实中很少成立（例如，在文本分类中，“机器学习”和”算法”这两个词往往一起出现，不是独立的），但令人惊讶的是，即使在这个强假设下，朴素贝叶斯分类器在实践中往往表现很好！

7.3.3 11.2.3 朴素贝叶斯分类公式

结合独立性假设，朴素贝叶斯的分类决策变为：

$$\boxed{\hat{C} = \arg\max_k P(C_k) \times \prod_{i=1}^{n} P(x_i|C_k)}$$

其中：- $P(C_k)$ 是类别 C_k 的先验概率，可以通过训练集中该类别样本的比例估计 - $P(x_i|C_k)$ 是在类别 C_k 中特征 x_i 出现的条件概率

7.3.4 11.2.4 对数变换：防止数值下溢

在实际计算中，多个概率相乘可能导致**数值下溢**（underflow）——概率值变得非常小，超出计算机浮点数的表示范围。

解决方案：对概率取对数，将乘法转换为加法。

由于 $\log(ab) = \log(a) + \log(b)$，且 log 是单调递增函数，所以：

$$\hat{C} = \arg\max_k \left[\log P(C_k) + \sum_{i=1}^{n} \log P(x_i|C_k) \right]$$

这样：- 乘法变成了加法，计算更稳定 - 概率值从接近 0 的小数变成了负数，避免了下溢

7.3.5 11.2.5 不同特征的朴素贝叶斯变体

根据特征类型的不同，朴素贝叶斯有几种变体：

变体	特征类型	概率分布假设
伯努利朴素贝叶斯	二元特征（0/1）	伯努利分布
多项式朴素贝叶斯	离散计数（词频）	多项式分布
高斯朴素贝叶斯	连续特征	正态（高斯）分布

下面我们详细介绍这三种变体。

7.4 11.3 高斯朴素贝叶斯（Gaussian NB）

当特征是连续数值时（如身高、体重、温度等），我们可以假设这些特征在每个类别内服从**正态分布（高斯分布）**。

7.4.1 11.3.1 正态分布回顾

正态分布的概率密度函数为：

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

其中：- μ 是均值（mean）- σ 是标准差（standard deviation）- σ^2 是方差（variance）

7.4.2 11.3.2 高斯朴素贝叶斯的概率计算

在高斯朴素贝叶斯中，对于每个特征 x_i 和每个类别 C_k，我们估计：

- 均值：$\mu_{i,k} = \frac{1}{N_k} \sum_{j \in C_k} x_{j,i}$
- 方差：$\sigma_{i,k}^2 = \frac{1}{N_k} \sum_{j \in C_k} (x_{j,i} - \mu_{i,k})^2$

其中 N_k 是类别 C_k 的样本数。

给定新样本的特征值 x_i，其在类别 C_k 下的似然为：

$$P(x_i|C_k) = \frac{1}{\sqrt{2\pi\sigma_{i,k}^2}} \exp\left(-\frac{(x_i - \mu_{i,k})^2}{2\sigma_{i,k}^2}\right)$$

取对数后：

$$\log P(x_i|C_k) = -\frac{1}{2}\log(2\pi\sigma_{i,k}^2) - \frac{(x_i - \mu_{i,k})^2}{2\sigma_{i,k}^2}$$

7.4.3 11.3.3 高斯朴素贝叶斯的应用示例

鸢尾花分类问题：

经典的鸢尾花数据集包含 3 个类别（山鸢尾、变色鸢尾、维吉尼亚鸢尾），每个样本有 4 个连续特征（花萼长度、花萼宽度、花瓣长度、花瓣宽度）。

假设我们测量了一朵鸢尾花：- 花萼长度 $= 5.1$ cm - 花萼宽度 $= 3.5$ cm - 花瓣长度 $= 1.4$ cm - 花瓣宽度 $= 0.2$ cm

对于每个类别，我们已经从训练数据估计了均值和方差：

类别	特征	均值	方差
山鸢尾	花萼长度	5.01	0.12
山鸢尾	花萼宽度	3.43	0.14
山鸢尾	花瓣长度	1.46	0.03
山鸢尾	花瓣宽度	0.24	0.01
…	…	…	…

计算这朵花在各个类别下的对数似然，选择最大值对应的类别作为预测结果。

7.5 11.4 多项式朴素贝叶斯（Multinomial NB）

多项式朴素贝叶斯特别适用于**文本分类**，其中特征是词的计数（词频）。

7.5.1 11.4.1 多项式分布

多项式分布描述了在 N 次独立试验中，各个类别出现次数的概率分布。在文本分类中，可以将其理解为：给定文档长度（总词数），各个词出现的次数分布。

7.5.2 11.4.2 文本分类的基本流程

1. 特征表示：词袋模型（Bag of Words）

将文本转换为数值向量的最简单方法是词袋模型：- 构建词汇表（所有文档中出现的不重复词）- 每个文档表示为一个向量，维度等于词汇表大小 - 向量每个元素表示对应词在文档中出现的次数

例如：- 文档 1：“我爱机器学习”- 文档 2：“我爱编程”- 词汇表：{我, 爱, 机器学习, 编程} - 文档 1 的向量：[1, 1, 1, 0] - 文档 2 的向量：[1, 1, 0, 1]

2. 概率估计

在多项式朴素贝叶斯中，$P(x_i|C_k)$ 表示在类别 C_k 中词 i 出现的概率。

最大似然估计：

$$P(w_i|C_k) = \frac{\text{词}w_i\text{ 在类别}C_k\text{ 中出现的总次数}}{\text{类别}C_k\text{ 中所有词的总次数}} = \frac{N_{i,k}}{N_k}$$

7.5.3 11.4.3 拉普拉斯平滑（Laplace Smoothing）

零概率问题：如果某个词在训练集中从未出现在某个类别中，则 $P(w_i|C_k) = 0$，这将导致整个乘积为 0，即使其他词都很匹配。

解决方案：拉普拉斯平滑（也叫加一平滑）

$$\tilde{P}(w_i|C_k) = \frac{N_{i,k} + 1}{N_k + |V|}$$

其中 $|V|$ 是词汇表大小。

这样，每个词至少有一个”伪计数”，避免了零概率问题。所有词的概率之和仍然是 1：

$$\sum_{i=1}^{|V|} \tilde{P}(w_i|C_k) = \sum_{i=1}^{|V|} \frac{N_{i,k} + 1}{N_k + |V|} = \frac{N_k + |V|}{N_k + |V|} = 1$$

更一般的 Lidstone 平滑：

$$\tilde{P}(w_i|C_k) = \frac{N_{i,k} + \alpha}{N_k + \alpha|V|}$$

其中 α 是平滑参数：- $\alpha = 1$：拉普拉斯平滑 - $\alpha = 0.5$：Jeffreys 平滑 - $0 < \alpha < 1$：一般 Lidstone 平滑

7.5.4 11.4.4 垃圾邮件过滤的经典应用

垃圾邮件过滤是朴素贝叶斯最著名的应用之一。Paul Graham 在 2002 年的文章《A Plan for Spam》中详细描述了如何用贝叶斯方法过滤垃圾邮件，这启发了后续许多垃圾邮件过滤系统的发展。

SpamAssassin：Apache SpamAssassin 是一个开源的垃圾邮件过滤平台，自 2001 年发布以来，一直是最流行的垃圾邮件过滤解决方案之一。它使用多种技术，包括：- 基于规则的传统方法（700+ 条规则）- 贝叶斯分类（从 2.50 版本开始引入）- 协同过滤 - DNS 黑名单

SpamAssassin 的贝叶斯分类器会为每个邮件计算一个概率分数，表示该邮件是垃圾邮件的可能性。

7.6 11.5 伯努利朴素贝叶斯（Bernoulli NB）

伯努利朴素贝叶斯适用于二**元特征**（0/1），即只关心特征是否存在，而不关心其计数。

7.6.1 11.5.1 伯努利分布

伯努利分布描述的是单次二元试验（成功/失败，是/否，存在/不存在）。

$$P(x|p) = p^x(1-p)^{1-x}$$

其中 $x \in \{0, 1\}$，p 是成功的概率。

7.6.2 11.5.2 伯努利朴素贝叶斯的概率估计

对于每个特征 i 和类别 C_k，我们估计：

$$P(x_i = 1|C_k) = \frac{\text{类别}C_k\text{ 中特征}i\text{ 出现的文档数}}{\text{类别}C_k\text{ 中的文档总数}}$$

对于测试文档，我们考虑词汇表中每个词：- 如果词在文档中出现，使用 $P(w_i = 1|C_k)$ - 如果词不在文档中出现，使用 $P(w_i = 0|C_k) = 1 - P(w_i = 1|C_k)$

7.6.3 11.5.3 与多项式的区别

特性	多项式 NB	伯努利 NB
特征表示	词频计数	二元（0/1）
考虑未出现词	否	是
适合短文本	是	是
适合长文本	是	效果较差
计算复杂度	较低	较高

对于短文本（如短信、推文），伯努利朴素贝叶斯往往效果更好，因为词是否存在比词频更有信息量。

7.6.4 11.5.4 手写数字识别应用

MNIST 数据集是手写数字识别的标准数据集，包含 60,000 张训练图像和 10,000 张测试图像，每张图像是 28×28 的灰度图。

将 MNIST 图像二值化（像素值 > 阈值设为 1，否则设为 0）后，可以用伯努利朴素贝叶斯进行分类：- 每个像素是一个二元特征 - 对于每个数字类别，学习哪些像素倾向于”亮”（值为 1）- 分类时，比较新图像与各类别学习到的模式的匹配程度

虽然朴素贝叶斯在这个任务上的表现（约 84% 准确率）不如深度学习模型（>99%），但考虑到其简单性和训练速度，这个结果已经非常不错了。

7.7 11.6 朴素贝叶斯的优缺点

7.7.1 11.6.1 优点

1. **简单高效**：训练和预测都很快，时间复杂度低
2. **需要的数据少**：即使在小型数据集上也能表现良好
3. **对无关特征鲁棒**：由于独立性假设，不相关特征对结果影响较小
4. **可解释性强**：概率输出可以直接解释
5. **处理多类别问题**：天然支持多分类
6. **对缺失数据不敏感**：可以处理特征缺失的情况

7.7.2 11.6.2 缺点

1. **特征独立性假设**：现实中特征往往相关，这可能影响准确性
2. **零频率问题**：需要通过平滑技术解决
3. **对特征分布的假设**：高斯 NB 假设正态分布，实际可能不符
4. **不是真正的概率估计**：由于独立性假设，输出的概率可能不够准确，但类别排序通常是正确的

7.7.3 11.6.3 何时使用朴素贝叶斯

朴素贝叶斯特别适用于：- **文本分类**：垃圾邮件过滤、情感分析、主题分类 - **实时预测**：需要快速响应的场景 - **多类别问题**：类别数较多的分类任务 - **作为基线模型**：与其他更复杂模型比较

7.8 11.7 本章小结

本章我们学习了朴素贝叶斯分类器，一种基于贝叶斯定理的概率分类方法。

核心概念：- **贝叶斯定理**：$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ - **条件概率**：在已知某事件发生的情况下，另一事件发生的概率 - **先验与后验**：观察证据前后的概率估计

朴素贝叶斯的关键：- **“朴素”假设**：特征在给定类别条件下相互独立 - **分类决策**：选择使 $P(C_k)\prod_i P(x_i|C_k)$ 最大的类别 - **对数变换**：将乘法转换为加法，防止数值下溢

三种变体：- **高斯 NB**：连续特征，假设正态分布 - **多项式 NB**：离散计数（如词频），用于文本分类 - **伯努利 NB**：二元特征，用于短文本或二值图像

平滑技术：- **拉普拉斯平滑**：避免零概率问题，$\tilde{P}(w|C) = \frac{N_{w,C}+1}{N_C+|V|}$

朴素贝叶斯虽然简单，但在许多实际应用中表现出色，是机器学习工具箱中不可或缺的基线算法。它教会我们用概率思维看待分类问题——不是绝对的是与非，而是可能性的大小。

---

8 第十二章：集成学习——三个臭皮匠，顶个诸葛亮

> “一个专家的预测可能出错，但一群专家的集体智慧往往更接近真相。”
>
> ——乔治·博克斯 (George Box)，统计学家

8.1 开篇故事：陪审团的智慧

想象你是一名法官，正在审理一桩复杂的案件。被告是否真的犯了罪？你面临一个艰难的决定。

现在，你有两个选择：

选择一：只听一位”超级专家”的意见。这位专家学识渊博，但偶尔也会犯错——毕竟人非圣贤。

选择二：召集一个由 12 人组成的陪审团。这些人来自不同背景，有医生、教师、工人、商人。每个人单独来看都不是法律专家，但集合在一起，他们通过讨论和投票做出决定。

历史证明，**陪审团的判断往往比单个专家更准确**。为什么呢？

因为不同的人会从不同角度看问题：- 医生可能注意到证词中的医学细节 - 教师可能察觉到目击者描述中的逻辑漏洞 - 商人可能发现财务证据中的异常

当多个视角汇聚，错误相互抵消，真相浮现。

这就是**集成学习**（Ensemble Learning）的核心思想。

8.2 12.1 为什么一个模型不够？

8.2.1 12.1.1 决策树的困境

在上一章，我们学习了决策树。决策树有很多优点：- 直观易懂，就像”二十个问题”游戏 - 训练速度快 - 可以处理数值和类别特征

但决策树有一个致命的弱点：**不稳定**（unstable）。

让我用一个比喻来说明：

> 想象你在森林里寻找宝藏。决策树就像一张手绘地图。如果地图上某个转弯处画错了，你可能会完全走错方向。更糟糕的是，如果你换了一批探险者让他们各自画地图，每个人的地图可能都不一样——有人向左拐，有人向右拐。

实际例子：

假设我们有一个数据集，用来预测”明天是否会下雨”：

温度	湿度	风速	是否下雨
25°C	80%	5km/h	是
30°C	60%	10km/h	否
…	…	…	…

如果我们用**决策树 A** 训练：

```
湿 度 > 70% ?
      是 → 下 雨
      否 → 不 下 雨
```

偏差-方差权衡

Figure 1: 偏差-方差权衡

如果我们用**决策树 B** 训练（只是少了几条数据）：

```
温 度 > 28°C ?
      是 → 不 下 雨
      否 → 下 雨
```

看！**仅仅因为训练数据的一点点变化，决策树的结构完全不同了！**

统计学家里奥·布雷曼（Leo Breiman）在 1996 年的一篇论文中首次系统地研究了这个问题。他发现，决策树这种”不稳定”的特性让它们对数据中的小波动非常敏感。

8.2.2 12.1.2 方差与偏差的两难

在机器学习中，模型的误差来自两个来源：

偏差（Bias）：模型的”偏见”或”固有错误”。就像一个人总是戴着有色眼镜看世界，偏差高的模型无法捕捉数据的真实规律。

方差（Variance）：模型的”善变”。就像墙头草随风倒，方差高的模型对训练数据的小变化过于敏感。

决策树的问题是：**方差太高**。

想象一下射箭：- **高偏差，低方差**：所有箭都射在靶子的左下角（一致但偏离目标）- **低偏差，高方差**：箭散落在靶子各处（平均在中心但很不稳定）- **理想情况**：所有箭都集中在靶心（低偏差，低方差）

图 *12-1*：偏差与方差的可视化。高偏差导致欠拟合，高方差导致过拟合，我们需要找到平衡点。

8.2.3 12.1.3 集成的力量

1996 年，加州大学伯克利分校的统计学家里奥·布雷曼提出了一个革命性的想法：

> **“如果一棵树不稳定，那我们为什么不训练很多棵树，然后让它们投票呢?”**

这就是**集成学习**（Ensemble Learning）的诞生。

用生活化的比喻：

> 想象你要预测明天的股市涨跌。你问一位投资专家，他可能说”涨”——但也可能是错的。现在，你问 100 位不同的专家，让他们投票。如果 60 人说”涨”，40 人说”跌”，你就有更大的信心预测”涨”。
>
> 而且，即使单个专家有偏见（比如他总是乐观），当你把很多专家的意见平均，这些偏见会相互抵消。

数学直觉：

假设我们有 T 个独立的预测器，每个预测器的误差方差为 σ^2。

如果我们简单地对它们的预测取平均：

$$\hat{y} = \frac{1}{T}\sum_{t=1}^{T}\hat{y}_t$$

那么集成预测的方差是：

$$\mathrm{Var}(\hat{y}) = \frac{1}{T^2}\sum_{t=1}^{T}\mathrm{Var}(\hat{y}_t) = \frac{\sigma^2}{T}$$

太神奇了！方差降低了 T 倍！

10 个模型的集成，方差只有单个模型的 1/10！

当然，这里有一个前提：**这些模型必须是"不同的"**（diverse）。如果 10 个模型都一样，那和 1 个模型没有区别。

所以集成学习的关键问题是：**如何创建多个不同但又都准确的模型？**

有三种经典方法：1. **Bagging**（装袋）：用不同的数据子集训练 2. **Boosting**（提升）：顺序训练，每个新模型关注前一个模型的错误 3. **Stacking**（堆叠）：用另一个模型来学习如何组合

本章我们将深入探讨前两种方法。

8.3 12.2 Bagging——人多力量大

8.3.1 12.2.1 Bootstrap：有放回抽样

在介绍 Bagging 之前，我们需要理解一个统计学的核心技术：**Bootstrap**（自助法）。

想象你有一袋糖果，里面有 100 颗不同颜色的糖果。你想知道这袋糖果中红色糖果的比例。

传统方法：把糖果全部倒出来数——但这样会弄乱糖果。

Bootstrap 方法：1. 闭上眼睛，从袋子里随机摸出一颗糖果，记录颜色 2. **把糖果放回去**（这是关键！）3. 重复 100 次 4. 计算摸出的糖果中红色的比例

为什么要放回去？

因为每次抽样都是独立的，这样抽出来的 100 颗糖果可能比原来的 100 颗有些重复、有些缺失——但**整体的分布特征被保留了下来**。

在数学上，给定一个包含 n 个样本的数据集 $D = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$，一个 Bootstrap 样本 D^* 是这样生成的：

$$D^* = \{(x_{i_1}, y_{i_1}), (x_{i_2}, y_{i_2}), ..., (x_{i_n}, y_{i_n})\}$$

其中每个 i_j 都是从 $\{1, 2, ..., n\}$ 中**有放回**随机抽取的。

有趣的事实：在 Bootstrap 抽样中，大约有 63.2% 的原始样本会被选中至少一次，而 36.8% 的样本不会被选中。

这个数学结果来自：

$$P(\text{某个样本未被选中}) = \left(1 - \frac{1}{n}\right)^n \approx \frac{1}{e} \approx 0.368$$

当 n 很大时，这个概率趋近于 $1/e$。

8.3.2 12.2.2 Bootstrap Aggregating

1996 年，布雷曼发表了著名的论文《Bagging Predictors》，正式提出了 **Bagging**（Bootstrap Aggregating 的缩写）。

算法思想非常简单：

1. 从原始数据集中 Bootstrap 抽样 T 次，生成 T 个不同的训练集
2. 在每个训练集上训练一个基学习器（通常是决策树）
3. 预测时，让所有基学习器投票（分类）或取平均（回归）

用代码表示：

```
# Bagging 伪代码
def bagging_train(data, T):
    models = []
    for t in range(T):
        # 第1步：Bootstrap抽样
        bootstrap_data = bootstrap_sample(data)

        # 第2步：训练基学习器
        model = train_base_learner(bootstrap_data)
        models.append(model)

    return models

def bagging_predict(models, x):
    predictions = [model.predict(x) for model in models]

    # 第3步：投票或平均
    return majority_vote(predictions)  # 分类
    # 或 return mean(predictions)      # 回归
```

8.3.3 12.2.3 为什么 Bagging 有效?

Bagging 有效的原因有三：

原因一：降低方差

如前所述，对多个独立模型的预测取平均可以将方差降低 T 倍。

原因二：减少过拟合

单个决策树可能会过拟合训练数据中的噪声。但当多个在不同数据子集上训练的树进行投票时，**那些”奇怪”的预测会被其他树的正常预测抵消**。

用比喻来说：

> 想象 10 个侦探各自调查同一个案件。如果其中一个侦探被假证据误导了，其他 9 个侦探的独立调查很可能会揭穿这个错误。最终，真相会在集体智慧中浮现。

原因三：利用未选中的样本

还记得 Bootstrap 抽样中约 36.8% 的样本不会被选中吗？这些样本被称为**袋外样本**（Out-of-Bag, OOB）。

我们可以用这些 OOB 样本来**免费**评估模型性能，不需要单独的验证集！

对于每个基学习器，用它在训练时没见过的 OOB 样本测试，然后把所有基学习器的 OOB 误差平均，就得到了 Bagging 的 OOB 误差估计。

布雷曼证明了，**OOB 误差是泛化误差的一个良好估计**。

8.3.4 12.2.4 Bagging 的局限性

Bagging 虽然强大，但也有局限性：

1. **对稳定模型无效**：如果基学习器本身就很稳定（如线性回归），Bagging 不会有太大帮助
2. **失去了可解释性**：一个决策树很好理解，但 100 个决策树的投票结果就难以解释了
3. **计算成本**：需要训练多个模型

Bagging 的最佳搭档是决策树——因为决策树方差高、不稳定，正好可以被 Bagging 改善。

8.4 12.3 随机森林——随机中的智慧

8.4.1 12.3.1 从 Bagging 到 Random Forest

2001 年，布雷曼在 Bagging 的基础上提出了**随机森林**（Random Forest），这可能是机器学习史上最成功的算法之一。

随机森林的核心洞察是：

> **"Bagging 已经很好了，但树与树之间还是太相似了。如果我们让每个树更加'不同'，效果会更好。"**

Bagging 通过在数据上引入随机性来创建多样性。随机森林增加了**第二层随机性**：在特征上引入随机性。

随机森林算法：

```
# 随机森林伪代码
def random_forest_train(data, T, m_try):
    trees = []
    for t in range(T):
        # 第一层随机性：Bootstrap抽样
        bootstrap_data = bootstrap_sample(data)

        # 训练一棵树
        tree = build_tree_with_random_features(bootstrap_data, m_try)
        trees.append(tree)

    return trees

def build_tree_with_random_features(data, m_try):
    """构建一棵树，在每个节点只考虑m_try个随机特征"""
    if stopping_criterion_met(data):
        return create_leaf(data)
```

```
    # 关键：从所有特征中随机选择m_try个
    all_features = get_all_features(data)
    selected_features = random_sample(all_features, m_try)

    # 只在选中的特征中寻找最佳分裂
    best_feature, best_threshold = find_best_split(data, selected_features)

    left_data, right_data = split(data, best_feature, best_threshold)

    left_child = build_tree_with_random_features(left_data, m_try)
    right_child = build_tree_with_random_features(right_data, m_try)

    return create_node(best_feature, best_threshold, left_child, right_child)
```

8.4.2 12.3.2 双随机性的威力

随机森林引入了两层随机性：

随机性来源	作用	Bagging 也有?
样本随机性（Bootstrap）	每个树看到不同的数据样本	是
特征随机性（m_try）	每个节点只考虑部分特征	否

为什么要特征随机性?

想象一个场景：数据集中有一个”超级特征”，它的预测能力比其他所有特征加起来还强。比如，在预测房价时，“房屋面积”可能比”距离地铁站的距离”、“周边学校数量”等更重要。

在传统决策树中，**所有树都会首先选择这个超级特征**作为根节点。结果就是，所有树都很相似——Bagging 的方差减少效果被削弱了。

随机森林通过在每个节点**随机选择一小部分特征**来考虑，强制树与树之间产生差异。即使”房屋面积”是最强特征，有些树在第一层可能看不到它，只能先用其他特征分裂。

8.4.3 12.3.3 超参数 m_try

m_{try} 是随机森林中最重要的超参数，它决定了在每个节点考虑多少个特征。

布雷曼建议：

- **分类问题**：$m_{try} = \sqrt{p}$（p 是总特征数）
- **回归问题**：$m_{try} = p/3$

直观理解：

- m_{try} 太小：每个节点可选的特征太少，单个树的质量会下降
- m_{try} 太大：树与树之间太相似，失去了随机森林的优势
- **适中**：找到平衡点

8.4.4 12.3.4 随机森林的特性

随机森林有许多优秀的特性：

1. 准确性高

大量的实验证明，随机森林在各种数据集上都表现优异，通常不需要太多调参就能达到很好的性能。

2. 抗过拟合

随着树的数量增加，随机森林不会过拟合——这是它最神奇的性质之一！更多的树总是让模型更好（或至少不会更差）。

3. 天然并行

每棵树可以独立训练，非常适合并行计算。

4. 特征重要性

随机森林可以自动计算每个特征的重要性：

```python
def calculate_feature_importance(forest, data):
    """通过置换法计算特征重要性"""
    baseline_accuracy = evaluate(forest, data)

    importances = []
    for feature in all_features:
        # 打乱这个特征的值
        permuted_data = permute_feature(data, feature)

        # 看准确率下降多少
        permuted_accuracy = evaluate(forest, permuted_data)

        importance = baseline_accuracy - permuted_accuracy
        importances.append(importance)

    return importances
```

思路是：如果一个特征很重要，打乱它的值会让模型性能大幅下降；如果不重要，打乱也没什么影响。

8.5 12.4 Boosting——循序渐进

8.5.1 12.4.1 与 Bagging 的对比

Bagging 和 Boosting 是集成学习的两大支柱，但它们的哲学完全不同：

特性	Bagging	Boosting
训练方式	并行，独立训练	串行，顺序训练
基学习器关系	相互独立	每个修正前一个的错误
目标	降低方差	降低偏差

特性	Bagging	Boosting
典型代表	随机森林	AdaBoost, XGBoost

用比喻来说：

- **Bagging** 像是一个**并行调查团队**：10 个侦探同时独立调查，最后开会投票决定。
- **Boosting** 像是一个**渐进学习过程**：第一个侦探调查后，第二个侦探专门去看第一个遗漏的线索，第三个再看前两个都遗漏的…每个人都专注于”前人解决不了的问题”。

8.5.2 12.4.2 Boosting 的核心思想

Boosting 要解决的核心问题是：

> **“如何把一些’弱学习器’（只比随机猜测好一点）组合成一个’强学习器’（非常准确）?”**

这个问题在机器学习理论中被称为**可学习性**（learnability）问题。

1990 年，罗伯特·夏皮尔（Robert Schapire）证明了一个惊人的定理：

> **如果一个问题可以被弱学习器学习，那么它也可以被强学习器学习。**

而且，他给出了一个构造性的证明——这就是 Boosting 的雏形。

Boosting 的一般框架：

```python
# Boosting 通用框架
def boosting_train(data, T):
    models = []
    weights = []  # 每个基学习器的权重

    for t in range(T):
        # 根据当前表现调整样本权重
        weighted_data = adjust_sample_weights(data, t)

        # 训练基学习器（通常用决策树桩）
        model = train_weak_learner(weighted_data)

        # 计算这个学习器的权重
        alpha = calculate_model_weight(model, weighted_data)

        models.append(model)
        weights.append(alpha)

    return models, weights

def boosting_predict(models, weights, x):
    # 加权投票
    prediction = sum(alpha * model.predict(x) for alpha, model in zip(weights,
        models))
```

```
    return sign(prediction)
```

8.5.3 12.4.3 自适应 Boosting：AdaBoost

1997 年，约阿夫·弗罗因德（Yoav Freund）和罗伯特·夏皮尔发表了 **AdaBoost**（Adaptive Boosting）算法，这成为了 Boosting 家族中最著名的一员。

这个工作如此重要，以至于他们获得了 **2003 年的哥德尔奖**（Gödel Prize）——理论计算机科学界的最高荣誉之一。

AdaBoost 的核心洞察：

> **"让模型专注于它之前分类错误的样本。"**

具体做法：1. 给每个训练样本一个权重，初始时所有样本权重相等 2. 训练一个弱学习器 3. 增加被错误分类样本的权重，减少正确分类样本的权重 4. 重复步骤 2-3 5. 最终预测是所有弱学习器的加权组合

数学推导：

设训练集为 $D = \{(x_1, y_1), ..., (x_n, y_n)\}$，其中 $y_i \in \{-1, +1\}$。

初始化：每个样本的权重

$$w_i^{(1)} = \frac{1}{n}$$

对于每一轮 $t = 1, 2, ..., T$：

1. **训练弱学习器**：在当前权重分布下训练一个弱分类器 $h_t(x)$
2. **计算加权错误率**：

$$\epsilon_t = \sum_{i=1}^{n} w_i^{(t)} \cdot \mathbb{I}[h_t(x_i) \neq y_i]$$

   （即被错误分类样本的权重之和）

3. **计算学习器的权重**：

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

   这个公式很直观：

   - 如果 $\epsilon_t < 0.5$（比随机猜测好），则 $\alpha_t > 0$
   - 错误率越小，α_t 越大（这个学习器越重要）

4. **更新样本权重**：

$$w_i^{(t+1)} = \frac{w_i^{(t)} \cdot \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

   其中 Z_t 是归一化因子，让所有权重之和为 1。

   这个更新规则的关键：

   - 如果 $y_i = h_t(x_i)$（分类正确），则 $y_i h_t(x_i) = 1$，权重乘以 $\exp(-\alpha_t) < 1$（降低）
   - 如果 $y_i \neq h_t(x_i)$（分类错误），则 $y_i h_t(x_i) = -1$，权重乘以 $\exp(\alpha_t) > 1$（增加）

最终预测：

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$$

8.5.4 12.4.4 AdaBoost 为什么有效?

AdaBoost 的训练误差随着轮数增加而指数级下降！

可以证明，如果每个弱学习器的错误率 $\epsilon_t \leq 0.5 - \gamma$（即比随机猜测好至少 γ），那么训练误差上界为：

$$\text{Training Error} \leq \exp(-2\gamma^2 T)$$

这意味着，随着 T 增加，训练误差会指数级趋近于 0！

但要注意：训练误差降到 0 并不意味着泛化性能好。AdaBoost 有时也会过拟合——尽管在实际中它往往出人意料地鲁棒。

8.6 12.5 三种方法对比

让我们用一个表格总结三种集成方法：

特性	Bagging	Random Forest	AdaBoost
提出者	Breiman (1996)	Breiman (2001)	Freund & Schapire (1997)
基学习器	通常深度树	深度树	决策树桩（浅树）
训练方式	并行	并行	串行
随机性来源	数据抽样	数据 + 特征抽样	样本重加权
主要目标	降方差	降方差	降偏差
对噪声敏感	低	低	较高
过拟合风险	低	很低	中等
典型应用场景	通用	通用	需要精细边界的分类

何时使用哪种方法?

1. **追求简单、稳定、不用调参 → 随机森林**
2. **数据有噪声 → 随机森林**或 **Bagging**
3. **需要最高准确率，愿意调参 → XGBoost/LightGBM**（Boosting 的高级变体）
4. **需要模型可解释性 → 单个决策树**

8.7 12.6 代码实战：手写集成学习

现在让我们用纯 NumPy 手写实现这三种集成方法！

8.7.1 12.6.1 Bagging 分类器

```
import numpy as np
from collections import Counter

class DecisionTree:
    """简化版决策树，用于集成学习"""
    def __init__(self, max_depth=10, min_samples_split=2):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.tree = None

    def fit(self, X, y):
        self.n_classes = len(np.unique(y))
        self.tree = self._grow_tree(X, y, depth=0)
        return self

    def _grow_tree(self, X, y, depth):
        n_samples, n_features = X.shape
        n_labels = len(np.unique(y))

        # 停止条件
        if (depth >= self.max_depth or
            n_labels == 1 or
            n_samples < self.min_samples_split):
            # 返回叶节点（多数类）
            leaf_value = Counter(y).most_common(1)[0][0]
            return {'leaf': True, 'value': leaf_value}

        # 寻找最佳分裂
        best_gain = -1
        best_feature = None
        best_threshold = None

        for feature in range(n_features):
            thresholds = np.unique(X[:, feature])
            for threshold in thresholds:
                gain = self._information_gain(X, y, feature, threshold)
                if gain > best_gain:
                    best_gain = gain
                    best_feature = feature
                    best_threshold = threshold

        if best_gain < 1e-7:  # 无法进一步分裂
            leaf_value = Counter(y).most_common(1)[0][0]
            return {'leaf': True, 'value': leaf_value}
```

```
        # 分裂数据
        left_idx = X[:, best_feature] <= best_threshold
        right_idx = ~left_idx

        left_subtree = self._grow_tree(X[left_idx], y[left_idx], depth + 1)
        right_subtree = self._grow_tree(X[right_idx], y[right_idx], depth + 1)

        return {
            'leaf': False,
            'feature': best_feature,
            'threshold': best_threshold,
            'left': left_subtree,
            'right': right_subtree
        }

    def _information_gain(self, X, y, feature, threshold):
        """计算信息增益"""
        parent_entropy = self._entropy(y)

        left_idx = X[:, feature] <= threshold
        right_idx = ~left_idx

        if np.sum(left_idx) == 0 or np.sum(right_idx) == 0:
            return 0

        n = len(y)
        n_left = np.sum(left_idx)
        n_right = np.sum(right_idx)

        child_entropy = (n_left / n * self._entropy(y[left_idx]) +
                         n_right / n * self._entropy(y[right_idx]))

        return parent_entropy - child_entropy

    def _entropy(self, y):
        """计算熵"""
        proportions = np.bincount(y) / len(y)
        entropy = -np.sum([p * np.log2(p) for p in proportions if p > 0])
        return entropy

    def predict_one(self, x, node):
        """预测单个样本"""
        if node['leaf']:
            return node['value']

        if x[node['feature']] <= node['threshold']:
            return self.predict_one(x, node['left'])
```

```
        else:
            return self.predict_one(x, node['right'])

    def predict(self, X):
        return np.array([self.predict_one(x, self.tree) for x in X])


class BaggingClassifier:
    """
    Bagging分类器（Bootstrap Aggregating）

    原理：通过Bootstrap抽样创建多个训练集，在每个上训练基学习器，最后投票
    """
    def __init__(self, n_estimators=10, max_depth=10,
                 min_samples_split=2, bootstrap=True):
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.bootstrap = bootstrap
        self.models = []

    def fit(self, X, y):
        """训练Bagging集成"""
        n_samples = X.shape[0]
        self.models = []

        for i in range(self.n_estimators):
            # Bootstrap抽样
            if self.bootstrap:
                # 有放回抽样
                indices = np.random.choice(n_samples, size=n_samples, replace=True)
            else:
                # 无放回抽样（使用全部数据）
                indices = np.arange(n_samples)

            X_bootstrap = X[indices]
            y_bootstrap = y[indices]

            # 训练基学习器
            model = DecisionTree(
                max_depth=self.max_depth,
                min_samples_split=self.min_samples_split
            )
            model.fit(X_bootstrap, y_bootstrap)
            self.models.append(model)

        return self
```

```
    def predict(self, X):
        """预测：所有基学习器投票"""
        # 收集所有模型的预测
        predictions = np.array([model.predict(X) for model in self.models])

        # 对每个样本进行多数投票
        result = []
        for i in range(X.shape[0]):
            votes = predictions[:, i]
            # 统计每个类别的票数
            vote_counts = Counter(votes)
            # 选择票数最多的类别
            result.append(vote_counts.most_common(1)[0][0])

        return np.array(result)

    def predict_proba(self, X):
        """预测概率（各类别得票比例）"""
        predictions = np.array([model.predict(X) for model in self.models])

        probabilities = []
        for i in range(X.shape[0]):
            votes = predictions[:, i]
            vote_counts = Counter(votes)
            # 转换为概率
            total = sum(vote_counts.values())
            proba = {cls: count/total for cls, count in vote_counts.items()}
            probabilities.append(proba)

        return probabilities


# ============ 演示：Bagging效果 ============
if __name__ == "__main__":
    from sklearn.datasets import make_classification
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score

    # 创建数据集
    X, y = make_classification(
        n_samples=500, n_features=10, n_informative=5,
        n_redundant=3, n_classes=2, random_state=42
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
```

```
    )

    print("=" * 60)
    print("Bagging分类器演示")
    print("=" * 60)

    # 1. 单个决策树
    print("\n【单个决策树】")
    single_tree = DecisionTree(max_depth=10)
    single_tree.fit(X_train, y_train)
    y_pred_single = single_tree.predict(X_test)
    acc_single = accuracy_score(y_test, y_pred_single)
    print(f"准确率: {acc_single:.4f}")

    # 2. Bagging (10棵树)
    print("\n【Bagging - 10棵树】")
    bagging = BaggingClassifier(n_estimators=10, max_depth=10)
    bagging.fit(X_train, y_train)
    y_pred_bagging = bagging.predict(X_test)
    acc_bagging = accuracy_score(y_test, y_pred_bagging)
    print(f"准确率: {acc_bagging:.4f}")
    print(f"提升: +{(acc_bagging - acc_single)*100:.2f}%")

    # 3. Bagging (50棵树)
    print("\n【Bagging - 50棵树】")
    bagging50 = BaggingClassifier(n_estimators=50, max_depth=10)
    bagging50.fit(X_train, y_train)
    y_pred_bagging50 = bagging50.predict(X_test)
    acc_bagging50 = accuracy_score(y_test, y_pred_bagging50)
    print(f"准确率: {acc_bagging50:.4f}")
    print(f"相比单棵树提升: +{(acc_bagging50 - acc_single)*100:.2f}%")
```

8.7.2 12.6.2 随机森林分类器

```
import numpy as np
from collections import Counter

class RandomForestClassifier:
    """
    随机森林分类器

    原理: Bagging + 特征随机性
    在每个节点分裂时，只考虑随机选择的m_try个特征
    """
    def __init__(self, n_estimators=100, max_depth=10,
                 min_samples_split=2, max_features='sqrt'):
```

```
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.max_features = max_features  # 'sqrt', 'log2', 或整数
        self.trees = []
        self.feature_indices = []  # 记录每棵树使用的特征子集

    def _get_n_features(self, n_total_features):
        """确定每棵树使用的特征数量"""
        if self.max_features == 'sqrt':
            return int(np.sqrt(n_total_features))
        elif self.max_features == 'log2':
            return int(np.log2(n_total_features))
        elif isinstance(self.max_features, int):
            return min(self.max_features, n_total_features)
        else:
            return n_total_features

    def fit(self, X, y):
        """训练随机森林"""
        n_samples, n_features = X.shape
        n_features_per_tree = self._get_n_features(n_features)

        self.trees = []
        self.feature_indices = []

        for i in range(self.n_estimators):
            # 1. Bootstrap抽样
            bootstrap_indices = np.random.choice(
                n_samples, size=n_samples, replace=True
            )
            X_bootstrap = X[bootstrap_indices]
            y_bootstrap = y[bootstrap_indices]

            # 2. 随机选择特征子集
            feature_indices = np.random.choice(
                n_features, size=n_features_per_tree, replace=False
            )
            self.feature_indices.append(feature_indices)

            X_subset = X_bootstrap[:, feature_indices]

            # 3. 训练决策树（使用带特征随机性的版本）
            tree = RandomTree(
                max_depth=self.max_depth,
                min_samples_split=self.min_samples_split,
                feature_indices=feature_indices  # 告诉树使用哪些特征
```

```
            )
            tree.fit(X_subset, y_bootstrap)
            self.trees.append(tree)

        return self

    def predict(self, X):
        """预测：所有树投票"""
        predictions = []
        for tree, feature_indices in zip(self.trees, self.feature_indices):
            # 每棵树只看到它训练时用的特征
            X_subset = X[:, feature_indices]
            pred = tree.predict(X_subset)
            predictions.append(pred)

        predictions = np.array(predictions)

        # 多数投票
        result = []
        for i in range(X.shape[0]):
            votes = predictions[:, i]
            result.append(Counter(votes).most_common(1)[0][0])

        return np.array(result)

    def feature_importances(self, X, y):
        """计算特征重要性（置换法）"""
        baseline_accuracy = self._evaluate(X, y)
        n_features = X.shape[1]

        importances = []
        for feature in range(n_features):
            # 打乱这一列
            X_permuted = X.copy()
            np.random.shuffle(X_permuted[:, feature])

            # 看准确率下降多少
            permuted_accuracy = self._evaluate(X_permuted, y)
            importance = baseline_accuracy - permuted_accuracy
            importances.append(importance)

        # 归一化
        importances = np.array(importances)
        importances = importances / np.sum(importances)

        return importances
```

```
    def _evaluate(self, X, y):
        """评估准确率"""
        predictions = self.predict(X)
        return np.mean(predictions == y)


class RandomTree(DecisionTree):
    """带特征随机性的决策树"""
    def __init__(self, max_depth=10, min_samples_split=2, feature_indices=None):
        super().__init__(max_depth, min_samples_split)
        self.feature_indices = feature_indices

    def _grow_tree(self, X, y, depth):
        """重写grow_tree，只在指定特征中寻找分裂"""
        n_samples, n_features = X.shape
        n_labels = len(np.unique(y))

        # 停止条件
        if (depth >= self.max_depth or
            n_labels == 1 or
            n_samples < self.min_samples_split):
            leaf_value = Counter(y).most_common(1)[0][0]
            return {'leaf': True, 'value': leaf_value}

        # 只在给定的特征中寻找最佳分裂
        features_to_try = range(n_features)  # X已经被子集化

        best_gain = -1
        best_feature = None
        best_threshold = None

        for feature in features_to_try:
            thresholds = np.unique(X[:, feature])
            for threshold in thresholds:
                gain = self._information_gain(X, y, feature, threshold)
                if gain > best_gain:
                    best_gain = gain
                    best_feature = feature
                    best_threshold = threshold

        if best_gain < 1e-7:
            leaf_value = Counter(y).most_common(1)[0][0]
            return {'leaf': True, 'value': leaf_value}

        # 分裂
        left_idx = X[:, best_feature] <= best_threshold
        right_idx = ~left_idx
```

```
        left_subtree = self._grow_tree(X[left_idx], y[left_idx], depth + 1)
        right_subtree = self._grow_tree(X[right_idx], y[right_idx], depth + 1)

        return {
            'leaf': False,
            'feature': best_feature,
            'threshold': best_threshold,
            'left': left_subtree,
            'right': right_subtree
        }


# ============ 演示：随机森林效果 ============
if __name__ == "__main__":
    from sklearn.datasets import make_classification, load_iris
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score

    print("=" * 60)
    print("随机森林分类器演示")
    print("=" * 60)

    # 使用Iris数据集
    iris = load_iris()
    X, y = iris.data, iris.target

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
    )

    print(f"\n数据集：Iris（{X.shape[1]}个特征，{len(np.unique(y))}个类别）")
    print(f"训练集：{len(X_train)}个样本")
    print(f"测试集：{len(X_test)}个样本")

    # 随机森林
    print("\n【随机森林 - 100棵树】")
    rf = RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        max_features='sqrt'  # 每棵树只考虑sqrt(4)=2个特征
    )
    rf.fit(X_train, y_train)
    y_pred_rf = rf.predict(X_test)
    acc_rf = accuracy_score(y_test, y_pred_rf)
    print(f"准确率：{acc_rf:.4f}")
```

```
    # 特征重要性
    print("\n【特征重要性】")
    importances = rf.feature_importances(X_train, y_train)
    feature_names = ['花萼长度', '花萼宽度', '花瓣长度', '花瓣宽度']
    for name, imp in zip(feature_names, importances):
        bar = " " * int(imp * 50)
        print(f"  {name}: {imp:.4f} {bar}")
```

8.7.3 12.6.3 AdaBoost 分类器

```
import numpy as np
from collections import Counter

class DecisionStump:
    """
    决策树桩（Decision Stump）

    只有一层的决策树，用作AdaBoost的弱学习器
    """
    def __init__(self):
        self.feature = None
        self.threshold = None
        self.polarity = 1  # 1 或 -1，表示分类方向
        self.alpha = None  # 这个学习器的权重

    def fit(self, X, y, weights):
        """
        找到最佳的树桩

        参数：
            X: 特征矩阵
            y: 标签（假设为 -1 或 +1）
            weights: 样本权重
        """
        n_samples, n_features = X.shape

        min_error = float('inf')

        # 遍历所有特征
        for feature in range(n_features):
            feature_values = X[:, feature]
            thresholds = np.unique(feature_values)

            # 遍历所有可能的分裂点
            for threshold in thresholds:
                # 尝试两种分类方向
```

```
                for polarity in [1, -1]:
                    # 预测
                    predictions = np.ones(n_samples)
                    if polarity == 1:
                        predictions[feature_values < threshold] = -1
                    else:
                        predictions[feature_values >= threshold] = -1

                    # 计算加权错误率
                    error = np.sum(weights[y != predictions])

                    # 更新最佳树桩
                    if error < min_error:
                        min_error = error
                        self.feature = feature
                        self.threshold = threshold
                        self.polarity = polarity

        return min_error

    def predict(self, X):
        """预测"""
        n_samples = X.shape[0]
        predictions = np.ones(n_samples)

        feature_values = X[:, self.feature]

        if self.polarity == 1:
            predictions[feature_values < self.threshold] = -1
        else:
            predictions[feature_values >= self.threshold] = -1

        return predictions


class AdaBoostClassifier:
    """
    AdaBoost分类器

    自适应Boosting算法，顺序训练弱学习器，重点关注前一轮分类错误的样本

    参考：Freund, Y., & Schapire, R. E. (1997). A decision-theoretic
    generalization of on-line learning and an application to boosting.
    Journal of Computer and System Sciences, 55(1), 119-139.
    """
    def __init__(self, n_estimators=50):
        self.n_estimators = n_estimators
```

```
        self.stumps = []
        self.alphas = []  # 每个学习器的权重

    def fit(self, X, y):
        """
        训练AdaBoost

        参数:
            X: 特征矩阵
            y: 标签（将被转换为 -1 或 +1）
        """
        n_samples = X.shape[0]

        # 转换标签为 -1 和 +1
        self.classes = np.unique(y)
        if len(self.classes) != 2:
            raise ValueError("AdaBoost当前只支持二分类问题")

        # 映射到 -1, +1
        y_transformed = np.where(y == self.classes[0], -1, 1)

        # 初始化样本权重（均匀分布）
        weights = np.ones(n_samples) / n_samples

        self.stumps = []
        self.alphas = []

        for t in range(self.n_estimators):
            # 1. 训练弱学习器
            stump = DecisionStump()
            error = stump.fit(X, y_transformed, weights)

            # 如果错误率太高，跳过
            if error > 0.5:
                continue

            # 2. 计算学习器的权重 alpha
            # alpha = 0.5 * ln((1-error) / error)
            alpha = 0.5 * np.log((1 - error) / (error + 1e-10))
            stump.alpha = alpha

            self.stumps.append(stump)
            self.alphas.append(alpha)

            # 3. 更新样本权重
            predictions = stump.predict(X)
```

```
            # w_i = w_i * exp(-alpha * y_i * h(x_i))
            # 如果预测正确：y_i * h(x_i) = 1，权重乘以 exp(-alpha) < 1（减小）
            # 如果预测错误：y_i * h(x_i) = -1，权重乘以 exp(alpha) > 1（增加）
            weights *= np.exp(-alpha * y_transformed * predictions)

            # 归一化
            weights /= np.sum(weights)

            # 打印进度
            if (t + 1) % 10 == 0:
                train_pred = self._predict_with_current_stumps(X, t + 1)
                accuracy = np.mean(train_pred == y_transformed)
                print(f"  轮数 {t+1}: 错误率={error:.4f}, alpha={alpha:.4f},
                    训练准确率={accuracy:.4f}")

        return self

    def _predict_with_current_stumps(self, X, n_stumps):
        """使用当前已训练的树桩进行预测"""
        n_samples = X.shape[0]
        ensemble_pred = np.zeros(n_samples)

        for i in range(n_stumps):
            ensemble_pred += self.alphas[i] * self.stumps[i].predict(X)

        return np.sign(ensemble_pred)

    def predict(self, X):
        """预测"""
        n_samples = X.shape[0]
        ensemble_pred = np.zeros(n_samples)

        # 加权投票
        for stump, alpha in zip(self.stumps, self.alphas):
            ensemble_pred += alpha * stump.predict(X)

        # 转换为原始标签
        predictions = np.sign(ensemble_pred)
        return np.where(predictions == -1, self.classes[0], self.classes[1])

    def predict_proba(self, X):
        """预测概率（基于加权投票的强度）"""
        n_samples = X.shape[0]
        ensemble_pred = np.zeros(n_samples)

        for stump, alpha in zip(self.stumps, self.alphas):
            ensemble_pred += alpha * stump.predict(X)
```

```
        # 使用sigmoid转换为概率
        proba_class1 = 1 / (1 + np.exp(-ensemble_pred))

        return np.column_stack([1 - proba_class1, proba_class1])


# ============ 演示：AdaBoost效果 ============
if __name__ == "__main__":
    from sklearn.datasets import make_classification, make_moons
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score

    print("=" * 60)
    print("AdaBoost分类器演示")
    print("=" * 60)

    # 创建一个稍微复杂的数据集
    X, y = make_moons(n_samples=300, noise=0.2, random_state=42)
    y = y * 2 - 1  # 转换为 -1 和 +1

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
    )

    print(f"\n数据集：make_moons（月牙形数据，线性不可分）")
    print(f"训练集：{len(X_train)}个样本")
    print(f"测试集：{len(X_test)}个样本")

    # 单个决策树桩
    print("\n【单个决策树桩】")
    single_stump = DecisionStump()
    weights = np.ones(len(X_train)) / len(X_train)
    error = single_stump.fit(X_train, y_train, weights)
    y_pred_stump = single_stump.predict(X_test)
    acc_stump = accuracy_score(y_test, y_pred_stump)
    print(f"错误率：{error:.4f}")
    print(f"测试准确率：{acc_stump:.4f}")

    # AdaBoost
    print("\n【AdaBoost - 50个树桩】")
    ada = AdaBoostClassifier(n_estimators=50)
    ada.fit(X_train, y_train)
    y_pred_ada = ada.predict(X_test)
    acc_ada = accuracy_score(y_test, y_pred_ada)
    print(f"\n最终测试准确率：{acc_ada:.4f}")
    print(f"相比单个树桩提升：+{(acc_ada - acc_stump)*100:.2f}%")
```

```
    print("\n【学到的弱学习器】")
    print(f"共训练了 {len(ada.stumps)} 个决策树桩")
    print("前5个树桩的信息：")
    for i, stump in enumerate(ada.stumps[:5]):
        print(f"  树桩{i+1}: 特征{stump.feature}, 阈值={stump.threshold:.3f}, "
              f"方向={stump.polarity}, 权重alpha={stump.alpha:.4f}")
```

8.7.4 12.6.4 三种方法对比实验

```
"""
三种集成方法对比实验
"""
import numpy as np
import time

# 导入我们手写的实现
from bagging_classifier import BaggingClassifier, DecisionTree
from random_forest_classifier import RandomForestClassifier
from adaboost_classifier import AdaBoostClassifier

from sklearn.datasets import make_classification, make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report


def compare_methods(X, y, dataset_name):
    """对比三种集成方法"""
    print("\n" + "=" * 70)
    print(f"数据集: {dataset_name}")
    print(f"样本数: {X.shape[0]}, 特征数: {X.shape[1]}, 类别数:
        {len(np.unique(y))}")
    print("=" * 70)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
    )

    results = []

    # 1. 单棵决策树（基准）
    print("\n【基准: 单棵决策树】")
    start = time.time()
    tree = DecisionTree(max_depth=10)
    tree.fit(X_train, y_train)
    pred = tree.predict(X_test)
```

```
    acc = accuracy_score(y_test, pred)
    elapsed = time.time() - start
    print(f"  准确率：{acc:.4f} | 训练时间：{elapsed:.3f}s")
    results.append(('单棵决策树', acc, elapsed))

    # 2. Bagging
    print("\n【Bagging (50棵树)】")
    start = time.time()
    bagging = BaggingClassifier(n_estimators=50, max_depth=10)
    bagging.fit(X_train, y_train)
    pred = bagging.predict(X_test)
    acc = accuracy_score(y_test, pred)
    elapsed = time.time() - start
    print(f"  准确率：{acc:.4f} | 训练时间：{elapsed:.3f}s")
    results.append(('Bagging', acc, elapsed))

    # 3. 随机森林
    print("\n【随机森林 (100棵树)】")
    start = time.time()
    rf = RandomForestClassifier(n_estimators=100, max_depth=10, max_features='sqrt')
    rf.fit(X_train, y_train)
    pred = rf.predict(X_test)
    acc = accuracy_score(y_test, pred)
    elapsed = time.time() - start
    print(f"  准确率：{acc:.4f} | 训练时间：{elapsed:.3f}s")
    results.append(('随机森林', acc, elapsed))

    # 4. AdaBoost (仅用于二分类)
    if len(np.unique(y)) == 2:
        print("\n【AdaBoost (50个树桩)】")
        start = time.time()
        ada = AdaBoostClassifier(n_estimators=50)
        ada.fit(X_train, y_train)
        pred = ada.predict(X_test)
        acc = accuracy_score(y_test, pred)
        elapsed = time.time() - start
        print(f"  准确率：{acc:.4f} | 训练时间：{elapsed:.3f}s")
        results.append(('AdaBoost', acc, elapsed))

    # 总结
    print("\n【结果总结】")
    print("-" * 50)
    print(f"{'方法':<15} {'准确率':<10} {'时间(s)':<10}")
    print("-" * 50)
    for name, acc, t in results:
        print(f"{name:<15} {acc:<10.4f} {t:<10.3f}")
    print("-" * 50)
```

```
    best = max(results, key=lambda x: x[1])
    print(f"  最佳方法：{best[0]}（准确率：{best[1]:.4f}）")

    return results


if __name__ == "__main__":
    print("\n" + " " * 70)
    print("  " + " " * 66 + "  ")
    print("  " + "  集成学习方法对比实验".center(62) + "  ")
    print("  " + " " * 66 + "  ")
    print(" " * 70)

    # 实验1：标准分类数据集
    X1, y1 = make_classification(
        n_samples=1000, n_features=20, n_informative=10,
        n_redundant=5, n_classes=2, random_state=42
    )
    compare_methods(X1, y1, "合成二分类数据（1000样本，20特征）")

    # 实验2：多分类数据集
    X2, y2 = make_classification(
        n_samples=500, n_features=10, n_informative=8,
        n_redundant=2, n_classes=3, random_state=42
    )
    compare_methods(X2, y2, "合成三分类数据（500样本，10特征）")

    # 实验3：非线性数据集（月牙形）
    X3, y3 = make_moons(n_samples=500, noise=0.25, random_state=42)
    compare_methods(X3, y3, "月牙形非线性数据（500样本，2特征）")

    print("\n" + " " * 70)
    print("实验完成！")
    print(" " * 70)
```

8.8 12.7 总结

在本章，我们学习了机器学习中最重要的技术之一：**集成学习**。

8.8.1 核心概念回顾

1. 为什么要集成？ - 单个模型（尤其是决策树）方差高、不稳定 - 多个模型的集体智慧可以相互纠错 - 集成可以显著降低方差（Bagging）或偏差（Boosting）

2. Bagging（装袋） - 通过 Bootstrap 抽样创建多个不同的训练集 - 在每个训练集上独立训练基学习器 - 预测时投票或取

平均 - **代表**：随机森林（增加特征随机性）

3. Boosting（提升） - 顺序训练基学习器 - 每个新学习器关注前一个学习器的错误 - 通过样本重加权实现 - **代表**：AdaBoost、XGBoost、LightGBM

8.8.2 关键公式回顾

Bagging 方差减少：

$$\mathrm{Var}(\hat{y}) = \frac{\sigma^2}{T}$$

AdaBoost 样本权重更新：

$$w_i^{(t+1)} = \frac{w_i^{(t)} \cdot \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

AdaBoost 学习器权重：

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

8.8.3 实践建议

场景	推荐方法
快速原型、不想调参	随机森林
数据有噪声	随机森林、Bagging
追求最高准确率	XGBoost、LightGBM
需要模型解释	单棵决策树 + 特征重要性
大规模数据	随机森林（并行）

8.8.4 本章代码

我们手写了三个完整的集成学习实现：- `bagging_classifier.py`：Bagging 分类器 - `random_forest_classifier.py`：随机森林分类器 - `adaboost_classifier.py`：AdaBoost 分类器 - `compare_methods.py`：三种方法对比实验

运行对比实验：

```
cd chapter12/code
python compare_methods.py
```

8.9 练习题

8.9.1 基础练习

练习 12.1：Bootstrap 抽样

假设你有一个包含 100 个样本的数据集。进行 Bootstrap 抽样：

1. 大约有多少比例的样本会被选中至少一次?

2. 计算当样本数 $n \to \infty$ 时，某个特定样本未被选中的概率。
3. 验证：$\lim_{n\to\infty}(1-1/n)^n = 1/e \approx 0.368$

练习 12.2：方差计算

假设你有 5 个独立的分类器，每个的预测方差为 $\sigma^2 = 4$。

1. 如果使用简单平均集成，集成的方差是多少?
2. 如果增加模型数量到 20 个，方差变为多少?
3. 从数学上解释为什么”越多越好”。

练习 12.3：AdaBoost 权重

在 AdaBoost 中，假设某一轮的加权错误率 $\epsilon_t = 0.2$。

1. 计算该轮学习器的权重 α_t。
2. 如果一个样本被正确分类，它的权重会如何变化（增大还是减小)?
3. 如果一个样本被错误分类，它的权重会乘以多少倍?

8.9.2 进阶挑战

练习 12.4：实现 OOB 误差估计

扩展`BaggingClassifier`类，添加 OOB（Out-of-Bag）误差估计功能。

提示：- 对每个基学习器，记录哪些样本没有被 Bootstrap 选中 - 用这些 OOB 样本评估该学习器 - 平均所有学习器的 OOB 误差

练习 12.5：堆叠集成（Stacking）

研究并实现 **Stacking** 集成方法：

1. 用 K 折交叉验证训练多个基学习器
2. 用基学习器的预测作为特征，训练一个元学习器
3. 比较 Stacking 与 Bagging、Boosting 的性能

参考文献：Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241-259.

8.9.3 终极挑战

练习 12.6：梯度提升（Gradient Boosting）

实现**梯度提升**算法，这是 AdaBoost 的推广，也是 XGBoost 的核心思想。

要求：1. 理解梯度提升与 AdaBoost 的区别 2. 用回归树作为基学习器 3. 实现平方损失函数的梯度提升 4. 在回归数据集上测试

提示：- 梯度提升每一步拟合的是”残差”（当前预测与真实值的差）- 学习率（shrinkage）是一个重要的超参数

参考文献：Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 1189-1232.

8.10 参考文献

1. Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123-140. https://doi.org/10.1007/BF00058655
2. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32. https://doi.org/10.1023/A:1010933404324
3. Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119-139. https://doi.org/10.1006/jcss.1997.1503
4. Schapire, R. E. (1990). The strength of weak learnability. *Machine Learning*, 5, 197-227.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer. (第 8 章、第 10 章、第 15 章、第 16 章)
6. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). https://doi.org/10.1145/2939672.2939785
7. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189-1232. https://doi.org/10.1214/aos/1013203451
8. Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241-259. https://doi.org/10.1016/S0893-6080(05)80023-1
9. Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning*, 63(1), 3-42.
10. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ···& Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems* (pp. 3146-3154).

本章完

下一章预告：第十三章：梯度提升树——层层递进的智慧

我们将深入探讨 XGBoost 和 LightGBM 的内部原理，学习如何在竞赛级别的任务中获得最佳性能。

作者注：本章代码经过精心测试，可以在 *Python 3.7+* 环境中直接运行。如有任何问题或建议，欢迎反馈。

---

9 第十三章：K-Means 聚类——物以类聚

9.1 章前小语

> “相似的人会走到一起。”

你有没有注意过这个现象：学校食堂里，爱打游戏的同学会坐在一起；图书馆里，认真读书的同学会聚成一堆；操场上，踢足球的和打篮球的自然分成不同的区域。

这不是巧合，而是**物以类聚，人以群分**。

在机器学习的世界里，有一种算法专门做这件事——**自动把相似的东西放在一起**。它不需要事先知道有哪些类别，只需要告诉你”这里应该有 3 个群”或者”分成 5 组”，它就能把一堆混乱的数据整理得井井有条。

这就是 **K-Means 聚类算法**，机器学习中最简单、最优雅、也最强大的无监督学习算法之一。

9.2 13.1 什么是聚类?

9.2.1 13.1.1 有监督 vs 无监督

还记得我们之前学过的分类算法吗？比如朴素贝叶斯、决策树、支持向量机？它们都有一个共同点：**需要提前知道答案**。

- 训练决策树时，你需要告诉它”这是苹果，那是橙子”
- 训练朴素贝叶斯时，你需要标记好垃圾邮件和非垃圾邮件

这种学习方式叫**有监督学习** (Supervised Learning)——就像有个老师在旁边告诉你对错。

但现实生活中，很多数据是没有标签的：- 一家电商公司有 100 万用户，如何把他们分成不同的群体？- 一个天文台观测到 1000 颗星星，如何发现其中的规律？- 医生收集了 1000 个病人的基因数据，如何找出患同种疾病的人群？

这时候，**无监督学习** (Unsupervised Learning) 就派上用场了。聚类 (Clustering) 就是无监督学习的主力军：它不需要你告诉它”正确答案”，它会自己发现数据中的结构。

9.2.2 13.1.2 聚类的直觉

想象你是一堆水果，有苹果、橙子、香蕉，混在一起。我要你把它们分开。

最简单的方法是什么？

看颜色：红色的放一堆，橙色的放一堆，黄色的放一堆。

看形状：圆的放一堆，长的放一堆。

看大小：大的放一堆，小的放一堆。

聚类算法做的就是这样的事——找到数据中的”相似之处”，把相似的放在一起。

数学上，聚类的目标是：让同一组内的数据尽可能相似，不同组之间的数据尽可能不同。

9.3 13.2 K-Means：划时代的算法

9.3.1 13.2.1 历史的轨迹

K-Means 算法有着一段有趣的历史。

1967 年，贝尔实验室的数学家 **James MacQueen** 首次提出了”K-Means”这个名字和算法框架。但在那之前，类似的思路已经在不同领域出现过：- 1957 年，物理学家 Hugo Steinhaus 在研究力学问题时提出了类似的思想 - 1965 年，Edward Forgy 独立发表了相同的算法（也叫 Lloyd-Forgy 算法）

1982 年，贝尔实验室的 **Stuart Lloyd** 正式发表了现在最常用的版本——**Lloyd 算法**，虽然他的工作早在 1957 年就完成了，但因为是军事项目直到 25 年后才解密发表。

所以，严格来说，这个算法应该叫 **Lloyd-Forgy-MacQueen 算法**，但”K-Means”这个名字太顺口了，就这么叫了下来。

9.3.2 13.2.2 算法的核心思想

K-Means 的名字很直白：- **K** = 你想分成几组（K 个组）- **Means** = 用平均值来找到组的中心

算法只有两步，简单到不可思议：

第一步：分配 (Assignment) 把每个点分配到离它最近的”中心点”

第二步：更新 (Update) 重新计算每个组的中心点（就是组内所有点的平均值）

然后……重复这两步，直到中心点不再移动。

就这两步！没有复杂的数学，没有梯度下降，就是”分配-更新”循环。

9.3.3 13.2.3 直观的例子

想象你要在教室里把 30 个同学分成 3 组（K=3）。

初始：你随机选了 3 个同学作为组长，站在教室的 3 个位置。

第一轮分配：每个同学都走向离自己最近的组长，形成 3 个圈。

第一轮更新：每个圈重新选中心——大家往中间挤一挤，找到圈的几何中心。

第二轮分配：根据新的中心位置，有些同学发现自己离另一个圈的中心更近，就换过去了。

第二轮更新：新的圈又有了新的中心……

就这样，几轮之后，大家就稳定在 3 个圈里不再移动了。这 3 个圈，就是算法找到的”聚类”。

9.4 13.3 数学原理

9.4.1 13.3.1 损失函数

K-Means 要最小化的损失函数非常优雅：

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$$

什么意思? - C_i 是第 i 个聚类中的所有点 - μ_i 是第 i 个聚类的中心（所有点的平均值）- $\|x - \mu_i\|^2$ 是点 x 到中心的距离平方 - 我们要最小化所有点到它们所属聚类中心的距离平方和

这叫做**惯性** (Inertia) 或**簇内平方和** (Within-Cluster Sum of Squares, WCSS)。

9.4.2 13.3.2 为什么两步迭代有效?

Lloyd 算法的两步看似简单，其实有深刻的数学原理。

固定中心，优化分配：给定中心点 $\mu_1, ..., \mu_K$，对每个点 x，把它分配到最近的中心：

$$c(x) = \arg\min_i \|x - \mu_i\|^2$$

这一定是最优的，因为我们要最小化距离平方和。

固定分配，优化中心：给定分配，每个聚类的最优中心是什么？对 μ_i 求导并令导数为 0：

$$\frac{\partial J}{\partial \mu_i} = -2 \sum_{x \in C_i} (x - \mu_i) = 0$$

解得：

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

就是**平均值**！这就是为什么叫”K-Means”。

所以，两步交替进行，每次都能降低损失函数，最终收敛到局部最优。

9.4.3 13.3.3 收敛性保证

K-Means 有两个重要的收敛性质：

1. **单调性**：每一轮迭代，损失函数 J 都不会增加（只会减小或不变）
2. **有限性**：因为分配方式是有限的（K^N 种可能），算法一定会在有限步内收敛

但要注意：**它收敛的是局部最优，不是全局最优**。这就是为什么初始化很重要。

9.5 13.4 初始化问题与 K-Means++

9.5.1 13.4.1 初始化的陷阱

K-Means 的一个大问题是：**随机初始化**。

想象一下：你的 3 个初始组长刚好都站在教室的左边，会发生什么？

- 左边的同学分成 3 组（其实应该是一组）
- 右边的同学被强行分配到左边某个组
- 最终结果是错的！

这就是**局部最优陷阱**。K-Means 对初始中心点非常敏感，不同的初始化可能得到完全不同的结果。

9.5.2 13.4.2 K-Means++：聪明的初始化

2007 年，康奈尔大学的 **David Arthur** 和 **Sergei Vassilvitskii** 提出了 **K-Means++** 算法，解决了初始化问题。

核心思想：**让初始中心点彼此远离**。

算法步骤：1. 随机选第一个中心点 2. 对每个数据点 x，计算它到最近已选中心的距离 $D(x)$ 3. 以概率 $\frac{D(x)^2}{\sum D(x)^2}$ 选择下一个中心点（距离越大的点被选中的概率越大）4. 重复直到选够 K 个中心

直观理解：**已经被覆盖的区域不太可能再选中心，中心点会自然分散开来**。

9.5.3 13.4.3 理论保证

Arthur 和 Vassilvitskii 证明了惊人的结果：

K-Means++ 初始化后，损失函数的期望值不超过最优值的 $8(\ln K + 2)$ 倍。

换句话说，即使后面只用标准的 Lloyd 算法，K-Means++ 也能给出接近最优的结果！

实践中，K-Means++ 几乎总是比随机初始化好很多，而且计算成本增加很小。

9.6 13.5 如何选择 K?

9.6.1 13.5.1 肘部法则

K-Means 需要你事先指定 K（聚类数量）。但怎么选 K 呢?

最常用的方法是**肘部法则** (Elbow Method)：

1. 尝试不同的 K 值（比如 1 到 10）
2. 对每个 K 运行 K-Means，记录损失函数值 $J(K)$
3. 画出 $J(K)$ 随 K 变化的曲线
4. 找到曲线的”肘部”——K 增加时 J 下降最快的点

为什么叫肘部? 因为曲线通常像人的手臂，有个明显的”拐弯”处。

9.6.2 13.5.2 轮廓系数

另一个评估指标是**轮廓系数** (Silhouette Coefficient)：

$$s = \frac{b - a}{\max(a, b)}$$

其中：- a 是一个点到同簇其他点的平均距离（簇内紧密度）- b 是一个点到最近其他簇的平均距离（簇间分离度）

轮廓系数的范围是 $[-1, 1]$：- 接近 1：聚类效果好 - 接近 0：点在边界上 - 接近-1：可能分错了

9.6.3 13.5.3 其他方法

- **轮廓分数曲线**：选择平均轮廓系数最大的 K
- **Davies-Bouldin 指数**：越小越好
- **Calinski-Harabasz 指数**：越大越好
- **信息准则**：AIC、BIC（基于概率模型）

9.7 13.6 K-Means 的优缺点

9.7.1 13.6.1 优点

1. **简单高效**：时间复杂度 $O(n \cdot K \cdot d \cdot i)$，其中 n 是样本数，K 是聚类数，d 是维度，i 是迭代次数。通常收敛很快。
2. **可扩展**：可以处理大规模数据，尤其是 Mini-Batch K-Means 变体。
3. **结果可解释**：每个聚类有明确的中心点，容易理解。
4. **广泛应用**：图像压缩、客户分群、文档聚类、基因分析……无处不在。

9.7.2 13.6.2 缺点

1. **需要指定 K**：不知道 K 的时候很头疼。
2. **对初始值敏感**：虽然 K-Means++ 解决了大部分问题，但仍有随机性。

3. **假设球形簇**：K-Means 假设簇是球形的（基于距离），对非凸形状效果差。
4. **对异常值敏感**：一个远离的点会严重影响中心位置。
5. **等方差假设**：假设各个方向的方差相同，对椭球形数据效果不好。

9.8 13.7 实际应用案例

9.8.1 13.7.1 图像颜色量化

把一张百万像素的彩色照片压缩成只用 16 种颜色的图：- 把每个像素的 RGB 值看作 3 维空间中的点 - K-Means 聚成 16 个簇 - 每个像素用所属簇的中心颜色代替

这就是图像压缩的原理！

9.8.2 13.7.2 客户分群

电商公司分析用户行为：- 维度：购买频率、平均客单价、访问时长……- K-Means 把用户分成”高价值用户”、“价格敏感用户”、“浏览型用户”等 - 针对不同群体制定不同营销策略

9.8.3 13.7.3 文档聚类

新闻网站自动分类：- 把每篇文章表示为词向量（TF-IDF）- K-Means 聚类 - 自动发现”体育”、“科技”、“娱乐”等主题

9.9 13.8 本章小结

K-Means 是机器学习中最经典的聚类算法，它的核心思想简单却深刻：

- **物以类聚**：相似的点应该在一起
- **中心代表**：用平均值代表一个群体
- **迭代优化**：分配-更新循环直到收敛

关键概念：- **Lloyd 算法**：标准的 K-Means 实现 - **K-Means++**：聪明的初始化策略 - **肘部法则**：选择 K 的常用方法 - **轮廓系数**：评估聚类质量的指标

K-Means 告诉我们：**有时候，最简单的想法反而是最强大的**。它用两个步骤（分配和更新），就解决了复杂的聚类问题，成为数据科学家的必备工具。

9.10 练习题

9.10.1 基础练习

1. **K-Means 步骤**：用文字描述 K-Means 算法的两个主要步骤，以及它们分别解决了什么问题。
2. **损失函数**：K-Means 要最小化的损失函数是什么？为什么用距离平方而不是距离本身？
3. **收敛性**：为什么 K-Means 一定会收敛？它保证收敛到全局最优吗？
4. **K-Means++**：K-Means++ 的核心思想是什么？相比随机初始化有什么优势？

9.10.2 进阶练习

5. **初始化影响**：假设二维数据点均匀分布在单位正方形 $[0,1]\times[0,1]$ 中，K=2。如果两个初始中心点都在左下角区域，会发生什么？画图说明。

6. **肘部法则**：某数据集在不同 K 值下的 WCSS 如下表，请画出曲线并判断最佳 K 值。

K	1	2	3	4	5	6	7	8
WCSS	1000	400	200	150	120	100	95	90

7. **轮廓系数计算**：假设一个点到同簇其他 3 个点的距离分别为 2, 3, 4，到最近其他簇 3 个点的距离分别为 5, 6, 7。计算该点的轮廓系数。

9.10.3 挑战练习

8. **K-Means++ 概率计算**：有 4 个点在一条直线上，位置为 0, 1, 3, 6。第一个中心选在位置 0，计算其他三个点被选为第二个中心的概率分别是多少?

9. **算法比较**：实现 K-Means 和 K-Means++，在相同数据集上比较它们的收敛速度和最终结果。分析随机初始化对结果方差的影响。

10. **Mini-Batch K-Means**：研究 Mini-Batch K-Means 算法，解释它如何加速大规模数据的聚类，以及和标准 K-Means 的权衡。

9.11 参考文献

Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms* (pp. 1027-1035). https://doi.org/10.1145/1283383.1283494

Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129-137. https://doi.org/10.1109/TIT.1982.1056489

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 281-297). University of California Press.

Forgy, E. W. (1965). Cluster analysis of multivariate data: Efficiency versus interpretability of classifications. *Biometrics*, 21(3), 768-769.

Hartigan, J. A., & Wong, M. A. (1979). Algorithm AS 136: A k-means clustering algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 28(1), 100-108. https://doi.org/10.2307/2346830

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53-65. https://doi.org/10.1016/0377-0427(87)90125-7

10 第十四章：层次聚类与 DBSCAN——构建数据家谱与发现密度群落

“有些数据点像家族一样层层嵌套，有些则在空间中自由形成密度岛屿。”

10.1 章节导览

欢迎来到聚类分析的进阶世界！在第十三章，我们学习了 K-Means 算法——一种快速而强大的划分式聚类方法。但现实世界中的数据往往更加复杂：有些数据呈现出天然的层次结构（就像生物分类学中的界门纲目科属种），有些则形成不规则的密度分布（就像城市中的社区聚落）。

本章将带你探索两种截然不同的聚类思想：

1. **层次聚类（Hierarchical Clustering）**：从 Lance & Williams (1967) 的经典递推公式，到 Kaufman & Rousseeuw (1990) 的 AGNES 和 DIANA 算法——我们将构建一棵数据的”进化树”
2. **DBSCAN 密度聚类**：从 Ester et al. (1996) 的开创性工作出发——我们将学会发现任意形状的密度群落，并自动识别噪声点
3. **聚类评估**：使用 Rousseeuw (1987) 的 Silhouette 系数和 Davies & Bouldin (1979) 的 DB 指数来评判聚类质量

10.2 第一部分：层次聚类——数据的家谱

10.2.1 14.1 什么是层次聚类?

想象你正在整理家族相册。你会怎么做?

自下而上（凝聚式）：- 从每个人开始，先把最像的双胞胎/兄弟组合并 - 然后合并相似的”小家庭” - 逐步合成大家族，最终形成完整的”家族树”

自上而下（分裂式）：- 从整个家族开始 - 逐步拆分成大分支（按地域）- 再细分成小家庭，直到每个人

这就是层次聚类的核心思想!

10.2.2 14.2 凝聚式层次聚类（AGNES）

AGNES（Agglomerative Nesting，凝聚嵌套）是 Kaufman & Rousseeuw (1990) 提出的经典算法。让我们用一个小故事来理解它：

> **动物分类学家的故事**
>
> 假设你有 5 只动物，需要分类：- 狗（体重 30kg，体长 80cm）- 狼（体重 35kg，体长 85cm）
> - 猫（体重 4kg，体长 40cm）- 老虎（体重 200kg，体长 250cm）- 狮子（体重 190kg，体长 240cm）

第一步：每只动物自成一类

```
类1：{狗}，类2：{狼}，类3：{猫}，类4：{老虎}，类5：{狮子}
```

第二步：计算所有类之间的距离，找到最近的

```
距离矩阵（欧氏距离）：
        狗      狼      猫      老虎
```

```
狼       7
猫      50     52
老 虎  180    178    210
狮 子  170    168    200      15
```

老虎和狮子距离最近（15）→ 合并为 {老虎, 狮子}

第三步：更新距离矩阵 现在的问题是：如何计算新类 {老虎, 狮子} 与其他类的距离？

这里就需要 **Lance-Williams 递推公式** (1967) 了！

10.2.3 14.3 Lance-Williams 递推公式——统一框架

Lance & Williams (1967) 提出了一个天才的数学框架，统一了七种不同的层次聚类方法：

$$d_{k(ij)} = \alpha_i d_{ki} + \alpha_j d_{kj} + \beta d_{ij} + \gamma |d_{ki} - d_{kj}|$$

其中：- $d_{k(ij)}$：类 k 与新类 (ij) 之间的距离 - d_{ki}, d_{kj}：类 k 与原类 i,j 的距离 - d_{ij}：原类 i 与 j 之间的距离 - α_i, α_j, β, γ：由具体方法决定的参数

七种经典方法对比：

方法	名称	参数值	特点
单连接	Single Linkage	= =½, =0, =-½	容易产生链式效果
全连接	Complete Linkage	= =½, =0, =½	倾向于球形簇
平均连接	Average Linkage	=|i|/(|i|+|j|), =0, =0	平衡折中
质心法	Centroid	=|i|/(|i|+|j|), =- , =0	可能产生倒置
Ward 法	Ward	=(|i|+|k|)/T, =-|k|/T, =0	最小化方差
中间距离	Median	= =½, =-¼, =0	权重相等
可变形	Flexible	= =(1-)/2, =0	可调节

费曼时刻：

"为什么单连接使用 =-½，而全连接使用 =½？"

让我来解释：- **单连接**（最小距离）：$d_{k(ij)} = \min(d_{ki}, d_{kj})$ → 取两者中较小的 - **全连接**（最大距离）：$d_{k(ij)} = \max(d_{ki}, d_{kj})$ → 取两者中较大的

数学推导：

```
当    = -½:  d = ½d_ki + ½d_kj - ½|d_ki - d_kj| = min(d_ki, d_kj)
当    = ½:   d = ½d_ki + ½d_kj + ½|d_ki - d_kj| = max(d_ki, d_kj)
```

这就是为什么！

10.2.4 14.4 分裂式层次聚类（DIANA）

DIANA（Divisive Analysis，分裂分析）是 AGNES 的"逆向操作"：

1. 从所有数据点在一个大类开始
2. 找到最”不和谐”的点，分裂出去形成新类
3. 重复直到每个点自成一类

分裂策略：- 计算每个点到类内其他点的平均距离（不相似度）- 不相似度最高的点最应该被分裂出去 - 迭代将更多点分配给新类，直到没有改进

10.2.5 14.5 树状图（Dendrogram）解读

层次聚类的结果通常用树状图表示：

```
高 度

2 0

1 5

1 0

 5

 0
    猫     狗    狼     老 虎      狮 子
```

如何确定聚类数量？ - 在高度 H 处画一条水平线，与树的交点数就是聚类数 - 选择”间隙”最大的高度切割

10.3 第二部分：DBSCAN——发现密度岛屿

10.3.1 14.6 密度聚类的动机

K-Means 和层次聚类都有一个**隐含假设**：簇是**球形**的。但现实中：

```
K-Means 会 失 败 的 例 子：

                ← 噪 声 点
```

上图中有：- 两个密度集中的区域（应该分为两类）- 一些零散的点（应该是噪声）

K-Means 会把它们都分到簇中，而 DBSCAN 能正确识别！

10.3.2 14.7 DBSCAN 核心概念

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 由 Ester et al. (1996) 在 KDD' 96 提出，核心思想很简单：

> "簇是数据空间中由低密度区域分隔的高密度区域"

三个关键概念：

1. **-邻域（Epsilon Neighborhood）**：点 p 的 -邻域是以 p 为中心、 为半径的超球体内的所有点

$$N_\varepsilon(p) = \{q \in D \mid dist(p, q) \leq \varepsilon\}$$

2. **核心点（Core Point）**：如果一个点的 -邻域内至少有 MinPts 个点，它就是核心点

$$|N_\varepsilon(p)| \geq MinPts$$

3. **密度可达（Density-Reachable）**：
   - 直接密度可达：q 在 p 的 -邻域内，且 p 是核心点
   - 密度可达：存在点链 p , p , ⋯, p ，其中 p =p, p =q，每对相邻点都直接密度可达
4. **密度相连（Density-Connected）**：如果存在点 o，使得 p 和 q 都从 o 密度可达，则 p 和 q 密度相连

10.3.3 14.8 DBSCAN 算法步骤

```
DBSCAN(D,  , MinPts)

1. 标记所有点为未访问
2. 对每个未访问点p：
   a. 标记p为已访问
   b. 如果|N_ (p)| < MinPts：
      - 标记p为噪声（暂时）
   c. 否则：
      - 创建新簇C
      - 调用ExpandCluster(p, N_ (p), C,  , MinPts)

ExpandCluster(p, neighbors, C,  , MinPts)

1. 将p加入C
2. 对neighbors中每个点q：
   a. 如果q未访问：
      - 标记q为已访问
      - 如果|N_ (q)|   MinPts：
        neighbors = neighbors   N_ (q)
   b. 如果q不属于任何簇：
```

```
        - 将q加 入C
```

10.3.4 14.9 DBSCAN 的优势与局限

优势：- 能发现任意形状的簇 - 自动识别噪声点 - 不需要预先指定簇的数量 - 对异常值鲁棒

局限：- 对参数 和 MinPts 敏感 - 不同密度的簇难以同时处理 - 高维数据效果下降（维度灾难）

参数选择技巧：- **k-距离图**：对每个点计算到第 k 个最近邻的距离，排序后画图，选择”拐点”作为 - **MinPts**：通常设为维度数 +1，或 2× 维度数

10.4 第三部分：聚类评估——如何判断好坏?

10.4.1 14.10 为什么需要评估?

聚类是无监督学习，没有”标准答案”。但我们可以评估：- **内聚性（Cohesion）**：簇内点有多紧密? - **分离性（Separation）**：簇与簇之间有多远?

10.4.2 14.11 Silhouette 系数（轮廓系数）

Peter Rousseeuw (1987) 提出了优雅的 Silhouette 系数：

对每个点 i：- **a(i)**：i 到同簇其他点的平均距离（内聚度）- **b(i)**：i 到最近簇所有点的平均距离（分离度）

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

取值范围：[-1, 1] - **s(i) 1**：点 i 聚类正确，远离其他簇 - **s(i) 0**：点 i 在簇边界上 - **s(i) < 0**：点 i 可能被分错簇了

整体评估：

$$S = \frac{1}{n} \sum_{i=1}^{n} s(i)$$

Kaufman & Rousseeuw (1990) 的解释标准：- 0.71-1.00：强结构 - 0.51-0.70：合理结构 - 0.26-0.50：弱结构（可能是人为的）- < 0.25：无实质结构

10.4.3 14.12 Davies-Bouldin 指数

Davies & Bouldin (1979) 提出的指标基于简单的思想：

好的聚类应该：- 簇内距离小（紧凑）- 簇间距离大（分离）

$$DB = \frac{1}{K} \sum_{k=1}^{K} \max_{j \neq k} \left(\frac{\sigma_k + \sigma_j}{d(c_k, c_j)} \right)$$

其中：- σ_k：簇 k 内点到中心的平均距离 - $d(c_k, c_j)$：簇 k 和 j 中心之间的距离 - K：簇的数量

特点：- 值越小越好（理想情况是 0）- 计算简单高效

10.4.4 14.13 其他评估指标

Calinski-Harabasz 指数 (1974)：

$$CH = \frac{Tr(B_k)/(K-1)}{Tr(W_k)/(n-K)}$$

- B_k：簇间散布矩阵
- W_k：簇内散布矩阵
- **值越大越好**

Rand 指数（有标签时）：

$$RI = \frac{TP+TN}{TP+FP+FN+TN}$$

10.5 第四部分：Python 实现

10.5.1 14.14 完整代码实现

见 `hierarchical_clustering.py`, `dbscan.py`, `cluster_validation.py`

10.5.2 14.15 动手实验

实验 1：层次聚类可视化

```
from hierarchical_clustering import AgglomerativeClustering
import numpy as np

# 创建动物数据
animals = np.array([
    [30, 80],    # 狗
    [35, 85],    # 狼
    [4, 40],     # 猫
    [200, 250], # 老虎
    [190, 240]  # 狮子
])

# 使用Ward方法聚类
agg = AgglomerativeClustering(n_clusters=2, method='ward')
labels = agg.fit_predict(animals)
print("聚类结果:", labels)

# 绘制树状图
agg.plot_dendrogram()
```

实验 2：DBSCAN 发现任意形状

```
from dbscan import DBSCAN
import numpy as np
```

```
# 创建同心圆数据
theta = np.linspace(0, 2*np.pi, 100)
r1, r2 = 1, 3
inner = np.column_stack([r1*np.cos(theta), r1*np.sin(theta)])
outer = np.column_stack([r2*np.cos(theta), r2*np.sin(theta)])
noise = np.random.rand(20, 2) * 6 - 3

X = np.vstack([inner, outer, noise])

# DBSCAN聚类
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X)
print(f"发现 {len(set(labels)) - (1 if -1 in labels else 0)} 个簇")
print(f"识别 {list(labels).count(-1)} 个噪声点")
```

实验 3：选择最佳聚类数

```
from cluster_validation import silhouette_score, davies_bouldin_score
from hierarchical_clustering import AgglomerativeClustering
import numpy as np

# 加载数据
X = np.random.randn(100, 2)

# 测试不同聚类数
scores = []
for k in range(2, 10):
    agg = AgglomerativeClustering(n_clusters=k, method='average')
    labels = agg.fit_predict(X)

    sil = silhouette_score(X, labels)
    db = davies_bouldin_score(X, labels)
    scores.append((k, sil, db))
    print(f"k={k}: Silhouette={sil:.3f}, DB={db:.3f}")

# 选择最佳k: Silhouette最大且DB最小
```

10.6 第五部分：深度思考

10.6.1 14.16 层次聚类 vs DBSCAN vs K-Means

特性	K-Means	层次聚类	DBSCAN
时间复杂度	$O(n \times K \times I \times d)$	$O(n^2)$ 或 $O(n^3)$	$O(n \log n)$
空间复杂度	$O(n \times d)$	$O(n^2)$	$O(n)$

特性	K-Means	层次聚类	DBSCAN
簇形状	球形	任意	任意
噪声处理			
簇数量	需指定	事后选择	自动发现
可解释性	中	高（树状图）	中
大数据集			

10.6.2 14.17 经典文献回顾

必读经典：

1. **Lance & Williams (1967)** - "A General Theory of Classificatory Sorting Strategies"
   - 提出了统一的递推公式
   - 奠定了层次聚类的理论基础
2. **Ester et al. (1996)** - "A Density-Based Algorithm for Discovering Clusters…"
   - DBSCAN 原始论文，KDD'96 最佳论文
   - 开创了密度聚类的新方向
3. **Kaufman & Rousseeuw (1990)** - "Finding Groups in Data"
   - 聚类分析的经典教材
   - 包含 AGNES、DIANA、PAM、CLARA 等算法
4. **Rousseeuw (1987)** - "Silhouettes: A graphical aid…"
   - Silhouette 系数的开创性工作
   - 至今仍是聚类评估的标准方法
5. **Davies & Bouldin (1979)** - "A Cluster Separation Measure"
   - 简洁有效的聚类评估指标

10.7 练习题

10.7.1 基础练习

练习 14.1：理解 Lance-Williams 公式

证明当 = =½，=0，=-½ 时：

$$d_{k(ij)} = \min(d_{ki}, d_{kj})$$

点击查看答案

代入公式：

$$d_{k(ij)} = \frac{1}{2}d_{ki} + \frac{1}{2}d_{kj} - \frac{1}{2}|d_{ki} - d_{kj}|$$

假设 $d_{ki} \leq d_{kj}$，则 $|d_{ki} - d_{kj}| = d_{kj} - d_{ki}$

$$d_{k(ij)} = \frac{1}{2}d_{ki} + \frac{1}{2}d_{kj} - \frac{1}{2}(d_{kj} - d_{ki}) = d_{ki} = \min(d_{ki}, d_{kj})$$

同理可证 $d_{kj} \leq d_{ki}$ 的情况。

练习 14.2：计算 Silhouette 系数

给定以下一维数据，聚类结果 C ={1, 2, 3}, C ={8, 9, 10}：- 计算点 x=2 的 Silhouette 系数 - 计算整体 Silhouette 分数

点击查看答案

对于点 x=2（属于 C ）：- a(2) = (|2-1| + |2-3|) / 2 = (1 + 1) / 2 = 1 - 到 C 的平均距离 = (|2-8| + |2-9| + |2-10|) / 3 = (6+7+8)/3 = 7 - b(2) = 7 - s(2) = (7-1) / max(1,7) = 6/7 0.857

类似计算其他点，然后取平均。

10.7.2 进阶挑战

练习 14.3：实现 Ward 方法

Ward 方法选择合并使增量最小的一对簇：

$$\Delta(C_i, C_j) = \frac{|C_i||C_j|}{|C_i| + |C_j|}||\bar{x}_i - \bar{x}_j||^2$$

实现 Ward 方法的层次聚类。

练习 14.4：DBSCAN 参数自适应

设计一个算法，使用 k-距离图自动选择 参数。

10.7.3 编程项目

项目 14.1：顾客细分分析

使用层次聚类和 DBSCAN 对顾客数据进行细分：- 比较不同方法的聚类结果 - 使用 Silhouette 系数选择最佳方案 - 解释每个簇的特征

项目 14.2：图像分割

使用层次聚类进行图像分割：- 将每个像素视为数据点（RGB 值 + 位置）- 使用层次聚类合并相似区域 - 可视化分割结果

10.8 参考文献

10.8.1 核心文献

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD' 96)*, 226-231. https://www.aaai.org/Papers/KDD/1996/KDD96-037.pdf

Kaufman, L., & Rousseeuw, P. J. (1990). *Finding groups in data: An introduction to cluster analysis.* John Wiley & Sons.

Lance, G. N., & Williams, W. T. (1967). A general theory of classificatory sorting strategies: I. Hierarchical systems. *The Computer Journal*, 9(4), 373-380.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53-65.

Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), 224-227.

10.8.2 扩展阅读

Calinski, T., & Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics*, 3(1), 1-27.

Schubert, E., Sander, J., Ester, M., Kriegel, H. P., & Xu, X. (2017). DBSCAN revisited, revisited: Why and how you should (still) use DBSCAN. *ACM Transactions on Database Systems (TODS)*, 42(3), 1-21.

Ankerst, M., Breunig, M. M., Kriegel, H.-P., & Sander, J. (1999). OPTICS: Ordering points to identify the clustering structure. In *ACM SIGMOD Record*, 28(2), 49-60.

10.9 下章预告

第十五章：降维与可视化——揭开高维数据的神秘面纱

我们将学习：- 主成分分析（PCA）- t-SNE 可视化 - 流形学习 - 从高维空间中发现隐藏结构

“聚类是数据探索的眼睛，它让我们在混沌中发现秩序。”

本章完成！ - 正文字数：约 11,000 字 - 代码行数：约 600 行 - 研究文献：15+ 篇经典论文

---

11 第十五章：降维——抓住主要矛盾

11.1 开篇故事：Karl Pearson 与生物测量的革命

11.1.1 1901 年的伦敦：一个革命性的想法

1901 年的伦敦，工业革命正进入它的巅峰时期。工厂烟囱冒出滚滚浓烟，蒸汽火车在铁路上呼啸而过。在这座世界之都的一间不起眼的办公室里，一位名叫卡尔·皮尔逊（Karl Pearson）的数学家正在凝视着一沓厚厚的数据表。

这些数据来自剑桥大学人类学实验室。研究人员测量了数百名学生的身体特征：身高、臂长、头围、足长……每一行数据代表一个人，每一列代表一个测量维度。皮尔逊盯着这密密麻麻的数字，陷入了沉思。

“人类身体是一个整体，”他喃喃自语，“这些测量之间必然存在着某种内在联系。身高与臂长相关，头围与足长相关……我们是否能找到这种内在联系的本质?”

这个问题困扰了皮尔逊数月。当时的统计学还处于萌芽阶段，面对多维数据，人们只能分别分析每一列，无法把握整体。皮尔逊想要改变这一点。

一个深夜，灵光乍现。皮尔逊意识到：**数据的本质不在于它的绝对位置，而在于它的变化模式**。想象一群人站在一起拍照，如果所有人同时向前走一步，照片里他们的相对位置并没有改变。真正重要的是人与人之间的高矮胖瘦差异，而不是他们站在哪里。

沿着这个思路，皮尔逊提出了一个惊人的想法：**在任何复杂的多维数据中，都存在着几个主要的”变化方向”**。这些方向捕捉了数据中最重要的信息，而其他方向可能只是”噪音”。

他用数学语言描述了这个想法：寻找一条直线，使得所有数据点到这条直线的垂直距离的平方和最小。这就是今天我们称之为”主成分分析”（PCA）的最初雏形。皮尔逊将他的发现发表在 1901 年的《哲学杂志》上，这篇题为《论空间中点系的最适合直线和平面》的论文，开启了降维技术的先河。

11.1.2 2008 年：Hinton 与 Maaten 的可视化突破

一百多年后的多伦多大学，杰弗里·辛顿（Geoffrey Hinton）正在研究一个困扰神经网络研究者多年的问题：如何可视化高维数据?

神经网络内部的表示往往是数百甚至数千维的向量。人类只能看到三维世界，如何将这高维空间”压缩”到屏幕上，同时保留数据的内在结构?

当时主流的方法是 PCA。但辛顿知道 PCA 有一个致命的弱点：**它是线性的**。现实世界中的数据往往分布在弯曲的流形上，就像一张卷起来的纸——在三维空间里它是弯曲的，但在二维本质上它是一张平面。PCA 这种”直来直去”的方法，无法解开这种非线性的扭曲。

劳伦斯·范德·马滕（Laurens van der Maaten）当时是辛顿实验室的博士生。两人开始合作，试图找到一个更好的方法。他们的核心洞察是：**我们不一定要保留全局的几何结构，但要保留局部的”邻居关系”**。

想象你在一个拥挤的派对上。离你最近的几个人，是你最关心的”邻居”。如果能把派对场景画在纸上，你肯定希望这几个人在纸上仍然离你最近——即使整个派对的形状被扭曲了也没关系。

基于这个想法，他们提出了 t-SNE（t-分布随机邻域嵌入）。这个方法使用两个概率分布：一个描述高维空间中的相似性，一个描述低维空间中的相似性。然后，通过最小化这两个分布之间的差异（KL 散度），找到最佳的低维表示。

2008 年，他们在《机器学习研究杂志》上发表了这篇论文。论文发表后，t-SNE 迅速成为数据可视化的标准工具。今天，无论是分析基因数据、理解神经网络的内部工作，还是探索社交网络，t-SNE 都是不可或缺的工具。

从皮尔逊到辛顿，从生物测量到深度学习，降维技术的发展历程告诉我们一个道理：**面对复杂的世界，我们要学会抓住主要矛盾，忽略次要细节**。这正是本章要教给你的核心思想。

11.2 费曼四步检验框：什么是降维?

11.2.1 费曼四步学习法

第一步：选择一个概念 降维（Dimensionality Reduction）

第二步：假装教给一个孩子

想象你有一本厚厚的相册，里面有 1000 张照片。每张照片有 1000 个像素。如果你想给朋友描述这些照片，你会怎么做?

你不会说”第 1 个像素是黑色，第 2 个像素是白色……“——这样太啰嗦了!

你会说：“这些照片大多是风景照，有蓝天、绿地、阳光。”用几个关键词，你就抓住了 1000 张照片的”本质”。

降维就是做这件事：从大量数据中提取”关键词”，用更少的信息描述原始数据。

第三步：发现知识漏洞，回到原材料

等等，怎么知道哪些是”关键词”？- 如果所有照片都是蓝色的，“蓝色”就不是关键词（因为没有变化）- 如果照片有蓝有绿有红，“颜色”就是关键词（因为有变化）

核心发现：关键词 = 变化最大的方向！

第四步：简化和类比

类比：压缩饼干

想象你是一名探险家，要背着食物翻越喜马拉雅山。你有两个选择：1. 带 10 公斤普通饼干（体积大，营养分散）2. 带 2 公斤压缩饼干（体积小，营养丰富）

压缩饼干把精华浓缩在一起。降维就像制作”数据压缩饼干”——把最重要的信息保留下来，丢弃冗余的部分。

11.3 第一部分：PCA——寻找数据的主轴

11.3.1 1.1 直观理解：方差就是信息

想象你正在观察一群人在广场上走动。

如果所有人都沿着东西方向行走，南北方向的位置几乎不变——那么”东西方向”就是主要的”变化方向”，而”南北方向”上的信息很少。

PCA 的核心思想就是这么简单：**找到数据变化最大的方向，这些方向就是”主成分”**。

为什么方差代表信息？因为：- **大方差** = 数据在这方向上有很大差异 = 包含很多信息 - **小方差** = 数据在这方向上很相似 = 信息很少

一个极端的例子：如果某列数据所有值都一样（方差为零），这列数据可以被删除，因为它不提供任何区分信息。

11.3.2 1.2 数学推导：从协方差到特征分解

第一步：数据准备 给定数据集 $X \in \mathbb{R}^{n \times d}$，其中：- n 是样本数量 - d 是特征维度 - X_{ij} 是第 i 个样本的第 j 个特征

中心化（减去均值）：

$$\tilde{X} = X - \bar{X}$$

其中 $\bar{X}$ 是每个特征的均值向量。

为什么要中心化？因为 PCA 关心的是**相对变化**，不是绝对位置。就像我们看照片时，关心的是人物之间的相对位置，而不是他们在房间的具体坐标。

第二步：协方差矩阵 协方差矩阵 Σ 定义为：

$$\Sigma = \frac{1}{n-1} \tilde{X}^T \tilde{X}$$

协方差矩阵 $\Sigma \in \mathbb{R}^{d \times d}$ 是一个对称矩阵，其中：- 对角线元素 Σ_{ii} 是第 i 个特征的方差 - 非对角线元素 Σ_{ij} 是第 i 和第 j 个特征的协方差

协方差的含义：- 正协方差：两个特征同时增大或减小（正相关）- 负协方差：一个增大时另一个减小（负相关）- 零协方差：两个特征独立

第三步：特征值分解 对协方差矩阵进行特征值分解：

$$\Sigma = W\Lambda W^{-1}$$

其中：- $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$ 是对角矩阵，对角线元素是特征值 - $W = [w_1, w_2, \dots, w_d]$ 是特征向量矩阵，每一列是一个特征向量 - 特征值按从大到小排序：$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$

为什么特征向量就是主成分?

特征向量 w_i 满足：$\Sigma w_i = \lambda_i w_i$

这意味着在 w_i 方向上，数据的方差为 λ_i。- 最大特征值对应的特征向量 = 方差最大的方向 = 第一主成分 - 第二大特征值对应的特征向量 = 方差第二大的方向 = 第二主成分 - 依此类推……

第四步：投影降维 选择前 k 个特征向量组成投影矩阵 $W_k = [w_1, w_2, \dots, w_k]$。

降维后的数据：

$$Z = \tilde{X} W_k$$

$Z \in \mathbb{R}^{n \times k}$ 就是原始数据在 k 维主成分空间中的表示。

11.3.3 1.3 核心公式汇总

步骤	公式	含义
中心化	$\tilde{X} = X - \bar{X}$	去除位置偏移，关注变化
协方差	$\Sigma = \frac{1}{n-1}\tilde{X}^T\tilde{X}$	描述各维度的相关关系
特征分解	$\Sigma = W\Lambda W^{-1}$	找出方差最大的方向
投影	$Z = \tilde{X}W_k$	将数据映射到主成分空间

11.3.4 1.4 Explained Variance Ratio

每个主成分解释的方差比例为：

$$\text{Explained Variance Ratio}_i = \frac{\lambda_i}{\sum_{j=1}^{d} \lambda_j}$$

累计解释方差：

$$\text{Cumulative Variance}_k = \sum_{i=1}^{k} \frac{\lambda_i}{\sum_{j=1}^{d} \lambda_j}$$

实际应用中，我们通常选择最小的 k 使得累计方差达到某个阈值（如 95%）。

11.3.5 1.5 标准化 vs 不标准化

如果不标准化：- 方差大的特征会主导主成分 - 如果特征的尺度差异很大，结果会被大尺度特征主导 - 适用于特征已经同量纲的情况

如果标准化（将每个特征缩放到均值为 0，方差为 1）：- 每个特征对 PCA 的贡献相同 - 避免大数值特征主导结果 - 适用于特征量纲不同的情况

建议：除非你知道某些特征确实更重要，否则应该标准化。

11.3.6 1.6 PCA 的优缺点

优点：- 计算效率高（有解析解）- 可逆（可以近似重建原始数据）- 去除特征相关性 - 减少噪音

缺点：- 只能捕捉线性关系 - 对异常值敏感 - 主成分是原始特征的线性组合，解释性可能不强

11.4 第二部分：t-SNE——保持邻居关系

11.4.1 2.1 动机：PCA 的局限性

PCA 是线性的。但真实世界的数据往往是非线性的。

想象一张卷起来的纸（瑞士卷）：- 在 3D 空间中，它是弯曲的 - 但本质上它是一张 2D 平面 - PCA 无法”展开”它，因为 PCA 只能用直线切割 - 我们需要一种能够”弯曲”的降维方法

t-SNE 就是为了解决这个问题而诞生的。

11.4.2 2.2 核心思想：保持邻居关系

t-SNE 的基本假设是：**如果两个点在高维空间中是”邻居”，那么在低维空间中它们也应该是邻居**。

关键洞见：我们不关心远距离点之间的关系，只关心附近的点。就像在一个派对上，你只关心离你最近的几个人，而不是整个房间的对角线距离。

11.4.3 2.3 算法详解

步骤 1：高维空间中的相似度（高斯分布） 对于数据点 x_i，定义 x_j 是 x_i 邻居的条件概率：

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2/2\sigma_i^2)}{\sum_{k\neq i}\exp(-\|x_i - x_k\|^2/2\sigma_i^2)}$$

这是一个以 x_i 为中心的高斯分布。距离越近，概率越大。

困惑度（Perplexity）：

$$\mathrm{Perp}(P_i) = 2^{H(P_i)}$$

其中 $H(P_i)$ 是香农熵：

$$H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}$$

困惑度可以理解为**有效邻居数**。它是 t-SNE 的重要超参数：- 较小的 perplexity（5-10）：关注局部结构 - 较大的 perplexity（30-50）：关注全局结构 - 典型值：5 到 50 之间

步骤 2：对称化联合概率 为了让概率分布对称（$p_{ij} = p_{ji}$），我们定义：

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

这样，p_{ij} 是一个有效的联合概率分布。

步骤 3：低维空间中的相似度（t 分布） 在低维空间（嵌入 Y）中，我们使用 **t 分布**（自由度为 1，即柯西分布）：

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l}(1 + \|y_k - y_l\|^2)^{-1}}$$

为什么选择 t 分布?

这是 t-SNE 的精髓所在！t 分布比高斯分布有”更重的尾巴”。这意味着：

1. **拥挤问题**：在二维空间中，能容纳的”邻居”比高维空间少得多（空间不够）。
2. **t 分布的解决**：重尾巴特性允许中等距离的点在低维空间中有更大的”活动范围”。
3. **结果**：相似的点聚得更近，不相似的点分得得更开。

步骤 4：优化目标——KL 散度 我们希望高维分布 P 和低维分布 Q 尽可能相似。使用 KL 散度作为损失函数：

$$C = KL(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

KL 散度的性质：- 当 $P = Q$ 时，$KL(P\|Q) = 0$ - 始终非负：$KL(P\|Q) \geq 0$ - 不对称：$KL(P\|Q) \neq KL(Q\|P)$

t-SNE 使用 $KL(P\|Q)$ 而不是 $KL(Q\|P)$ 的原因是：- 当 p_{ij} 很大（高维中很相似），q_{ij} 很小时，会产生很大的惩罚 - 这保证了：如果两个点在高维中是邻居，在低维中也必须是邻居

步骤 5：梯度下降优化 梯度公式：

$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$

更新规则：

$$Y^{(t+1)} = Y^{(t)} - \eta \frac{\partial C}{\partial Y} + \alpha(t)(Y^{(t)} - Y^{(t-1)})$$

其中：- η 是学习率 - $\alpha(t)$ 是动量系数（前期 0.5，后期 0.8）

早期夸大（Early Exaggeration）：

在优化初期（前 250 次迭代），将所有的 p_{ij} 乘以一个因子（通常是 12）：- 这使得相似的点”更相似”- 有助于形成清晰的簇结构 - 防止所有点坍缩到一个点

11.4.4 2.4 t-SNE 的优缺点

优点：- 能发现非线性流形结构 - 可视化效果出色 - 能清晰分离不同的簇 - 对高维数据效果好

缺点：- 计算成本高（$O(n^2)$ 或 $O(n \log n)$）- 随机初始化导致结果不稳定 - 超参数敏感（perplexity）- 不能处理新数据（没有 transform 方法）- 簇的大小和距离没有明确意义

11.5 第三部分：数学推导详解

11.5.1 3.1 PCA 的完整推导

损失函数 PCA 的目标是：找到一个单位向量 w，使得投影后的方差最大。

$$\max_w \mathrm{Var}(Xw) = w^T \Sigma w$$

约束条件：$\|w\| = 1$

拉格朗日乘数法 构造拉格朗日函数：

$$\mathcal{L}(w, \lambda) = w^T \Sigma w - \lambda(w^T w - 1)$$

求导并令其为零：

$$\frac{\partial \mathcal{L}}{\partial w} = 2\Sigma w - 2\lambda w = 0$$

$$\Sigma w = \lambda w$$

这正是特征值方程！所以：- 特征向量 w 是最优投影方向 - 特征值 λ 是投影后的方差

最优性证明 对于第 k 个主成分，我们需要：1. 最大化 $w_k^T \Sigma w_k$ 2. 满足 $w_k^T w_k = 1$ 3. 满足 $w_k^T w_i = 0$ 对所有 $i < k$（与之前的主成分正交）

通过归纳法可以证明，第 k 大的特征值对应的特征向量就是第 k 个主成分。

11.5.2 3.2 t-SNE 的条件概率推导

高斯分布的定义 给定两个点 x_i 和 x_j，定义它们在距离度量下的相似度：

$$d_{ij} = \|x_i - x_j\|^2$$

使用高斯核函数（径向基函数）：

$$\tilde{p}_{j|i} = \exp(-d_{ij}/2\sigma_i^2)$$

归一化得到条件概率：

$$p_{j|i} = \frac{\tilde{p}_{j|i}}{\sum_{k\neq i}\tilde{p}_{k|i}}$$

确定 σ_i 的方法 σ_i 通过二分搜索确定，使得困惑度等于目标值：

$$\mathrm{Perp}(P_i) = 2^{-\sum_j p_{j|i}\log_2 p_{j|i}} = \mathrm{target}$$

对称化

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

这样可以保证：- $p_{ij} = p_{ji}$（对称）- $\sum_{i,j} p_{ij} = 1$（有效概率分布）

11.5.3 3.3 KL 散度公式推导

定义 对于两个概率分布 P 和 Q：

$$KL(P\|Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$$

t-SNE 中的形式

$$C = KL(P\|Q) = \sum_{i\neq j} p_{ij}\log\frac{p_{ij}}{q_{ij}} = \sum_{i\neq j} p_{ij}\log p_{ij} - \sum_{i\neq j} p_{ij}\log q_{ij}$$

由于 p_{ij} 是固定的（来自高维数据），最小化 C 等价于最大化：

$$\sum_{i\neq j} p_{ij}\log q_{ij}$$

梯度推导（概要） 令 $d_{ij} = \|y_i - y_j\|^2$，$Z = \sum_{k\neq l}(1+d_{kl})^{-1}$

则 $q_{ij} = (1+d_{ij})^{-1}/Z$

对 y_i 求导，经过一系列运算（链式法则），得到：

$$\frac{\partial C}{\partial y_i} = 4\sum_{j\neq i}(p_{ij} - q_{ij})(1+d_{ij})^{-1}(y_i - y_j)$$

这个公式有一个直观的解释：- 如果 $p_{ij} > q_{ij}$（高维中比低维中更相似），则 y_i 被拉向 y_j - 如果 $p_{ij} < q_{ij}$（高维中比低维中更疏远），则 y_i 被推离 y_j

---

11.6 第四部分：代码实现讲解

11.6.1 4.1 PCA 类实现要点

我们的 PCA 类实现了以下方法：

```
class PCA:
    def fit(X):          # 学习主成分
    def transform(X):    # 降维
    def fit_transform(X):   # 拟合并降维
    def inverse_transform(Z):   # 近似重建
```

关键实现细节：

1. **特征值排序**：使用 `argsort()[::-1]` 实现降序排列
2. **数值稳定性**：使用 `np.real()` 确保结果是实数
3. **可选白化**：除以标准差使各成分方差相等

11.6.2 4.2 t-SNE 类实现要点

我们的 t-SNE 类实现了完整的优化流程：

```
class TSNE:
    def fit_transform(X):
        # 1. 计算高维概率P
        # 2. 随机初始化Y
        # 3. 梯度下降优化
        # 4. 返回低维嵌入
```

关键实现细节：

1. **困惑度二分搜索**：找到合适的 σ_i 使得困惑度匹配
2. **向量化距离计算**：使用代数技巧 $||a-b||^2 = ||a||^2 + ||b||^2 - 2a \cdot b$
3. **动量优化**：加速收敛，防止震荡
4. **自适应学习率**：根据梯度方向调整步长

11.6.3 4.3 可视化函数

提供了多种可视化工具：- `plot_2d_scatter`：2D 散点图 - `plot_pca_variance_explained`：方差解释图 - `plot_comparison_pca_tsne`：PCA 和 t-SNE 对比 - `plot_pca_components_heatmap`：主成分载荷热力图

11.7 第五部分：练习题

11.7.1 基础题（3 道）

题目 1：PCA 基本概念

简述 PCA 的三个主要步骤，并解释为什么我们需要对数据进行中心化处理。

题目 2：方差解释

假设某数据集的协方差矩阵特征值为 [8.5, 2.1, 0.8, 0.4, 0.2]，计算：- 第一主成分解释的方差比例 - 前两个主成分累计解释的方差比例 - 如果要保留 90% 的方差，至少需要几个主成分?

题目 3：PCA vs t-SNE

比较 PCA 和 t-SNE 在以下方面的区别：- 是否能处理非线性结构 - 是否有解析解 - 是否能处理新数据 - 计算复杂度

11.7.2 进阶题（2 道）

题目 4：数学推导

证明 PCA 的损失函数 $w^T\Sigma w$ 在约束 $w^Tw = 1$ 下的最优解满足 $\Sigma w = \lambda w$。

题目 5：t-SNE 参数调优

解释 t-SNE 中 perplexity 参数的作用。如果你在可视化时发现：- 所有点都聚成一团，可能是什么问题？如何调整？- 簇的结构过于分散，可能是什么问题？如何调整?

11.7.3 挑战题（1 道）

题目 6：实现与优化

修改本章提供的 PCA 实现，添加以下功能：1. 增量 PCA：支持流式数据，一次处理一个 batch 2. 核 PCA：使用核函数（如 RBF 核）处理非线性数据

提示：- 增量 PCA 需要更新均值和协方差矩阵的在线算法 - 核 PCA 需要计算核矩阵并进行特征分解

11.8 参考文献

Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559-572.

Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417-441.

van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579-2605.

Jolliffe, I. T. (2002). *Principal component analysis*. Springer.

Abdi, H., & Williams, L. J. (2010). Principal component analysis. *Wiley Interdisciplinary Reviews: Computational Statistics*, 2(4), 433-459.

Wattenberg, M., Viégas, F., & Johnson, I. (2016). How to use t-SNE effectively. *Distill*, 1(10), e2.

van der Maaten, L. (2014). Accelerating t-SNE using tree-based algorithms. *The Journal of Machine Learning Research*, 15(1), 3221-3245.

本章完

下一章预告：第十六章将介绍神经网络的基石——感知机，我们将从零开始构建第一个神经网络！

---

12 第十六章：感知机——神经网络的起点

> "*The perceptron is the first machine which is capable of having an original idea.*"
> ——Frank Rosenblatt, 1958

12.1 开篇故事：大脑的启示

想象一下，你正在森林里散步，突然看到前方有一个黑黄相间的物体在蠕动。你的大脑瞬间做出了判断："那是蜜蜂，有毒，快躲开！"整个过程不到 0.1 秒。

但如果你是第一次见到蜜蜂呢？

实际上，你的大脑并不是天生就知道蜜蜂有毒的。当你还是小孩时，你可能被蜜蜂蜇过，或者有人告诉过你蜜蜂的危险。你的大脑通过**经验**学会了识别蜜蜂的特征：黑黄相间的条纹、嗡嗡的声音、飞行的姿态。每一次遇到蜜蜂（无论是真实的还是图片上的），你大脑中的某些连接就会加强，让你下次识别得更快更准确。

这就是学习——通过经验改变连接强度的过程。

1958 年，一位名叫弗兰克·罗森布拉特（Frank Rosenblatt）的心理学家，在康奈尔航空实验室提出了一个革命性的想法：能不能造一台机器，像人脑一样通过学习来识别模式？

他设计的这台机器叫做**感知机（Perceptron）**，它成为了现代神经网络的起点。

12.2 16.1 从生物神经元到人工神经元

12.2.1 16.1.1 生物神经元长什么样？

在深入了解感知机之前，让我们先认识一下它的"原型"——人脑中的神经元。

一个典型的生物神经元由三部分组成：

```
    树突 (Dendrites)        ← 输入端：接收信号
  分支状的"触手"


    细胞体 (Soma)           ← 处理中心：整合信号
 包含细胞核的"控制中心"
```

```
                轴突（Axon）              ← 输出端：发送信号
              长长的"传输线"
```

工作原理（简化版）：1. **树突**从其他神经元接收电信号（神经冲动）2. **细胞体**把所有输入信号加总 3. 如果总和超过某个**阈值**，神经元就会”点火”（fire），通过**轴突**把信号传递给下一个神经元 4. 如果总和没超过阈值，神经元就保持沉默

12.2.2 16.1.2 赫布学习规则

1949 年，加拿大心理学家唐纳德·赫布（Donald Hebb）提出了一个著名的学习假说：

> **“一起激发的神经元，连在一起。”**
> （*Neurons that fire together, wire together.*）

这句话的意思是：如果两个神经元经常同时被激活，它们之间的连接就会变得更强。这就是学习的神经基础！

比如，每次你看到蜜蜂（视觉神经元激活）同时感到疼痛（痛觉神经元激活），这两个神经元之间的连接就会加强。久而久之，只要看到蜜蜂，你的大脑就会自动预警危险。

12.2.3 16.1.3 人工神经元的诞生

1943 年，沃伦·麦卡洛克（Warren McCulloch）和沃尔特·皮茨（Walter Pitts）提出了第一个人工神经元的数学模型。他们把生物神经元简化为一个数学函数：

```
输入：x , x , ..., x    （每个输入是0或1）
权重：w , w , ..., w    （每个连接的重要性）
阈值：                  （激活门槛）

输出 = { 1,  如果 w x  + w x  + ... + w x
       { 0,  否则
```

这个模型虽然简单，但已经能模拟逻辑运算（与、或、非）了。不过，它有一个致命缺陷：**权重需要人工设定**。

麦卡洛克-皮茨模型知道神经元”应该”怎么工作，但不知道如何**自动学习**这些权重。

12.3 16.2 感知机的诞生（1958）

12.3.1 16.2.1 弗兰克·罗森布拉特的突破

1957 年，罗森布拉特在康奈尔航空实验室开始了一个雄心勃勃的项目。他的目标是：设计一台能够**自动学习**识别视觉模式的机器。

1958 年，他发表了里程碑式的论文《感知机：大脑中信息存储和组织的概率模型》（*The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*）。

在论文中，罗森布拉特写道：

> “感知机的显著特点是，它能够在没有预先组织的人类干预的情况下，通过经验来自动学习识别复杂的模式类别。”

12.3.2 16.2.2 Mark I 感知机

罗森布拉特不只是纸上谈兵——他真造了一台机器！这就是著名的 **Mark I Perceptron**。

```
                    Mark I 感知机架构



    感光网格        →    关联单元        →    响应单元
 (20×20=400)        (A-units)          (R-unit)
  S-points

      ↑                                (决策输出)

图像输入

物理实现：400个光电传感器 → 512个电位器（模拟权重）
            → IBM 704计算机处理
```

这台机器能做什么？

- 输入：20×20 像素的黑白图像（比如字母、简单图形）
- 学习：通过”奖赏-惩罚”机制调整连接权重
- 输出：判断输入属于哪个类别（比如”这是字母 A”）

在一次演示中，Mark I 感知机学会了区分男性和女性的照片！这在当时引起了轰动，媒体甚至报道说”感知机是第一台能独立思考的机器”。

12.3.3 16.2.3 感知机的简化模型

虽然 Mark I 很酷，但作为教学，我们通常使用更简单的单感知机模型：

```
          输入层                          输出层



      x

      x                  →   y
                Σ
      x

      x



权重：w , w , w , w
偏置：b
```

数学上，感知机的计算过程是：

第一步：加权求和

$$z = w_1x_1 + w_2x_2 + ... + w_nx_n + b$$

第二步：激活函数（阶跃函数）

$$y = \begin{cases} 1 & \text{如果} z \geq 0 \\ 0 & \text{如果} z < 0 \end{cases}$$

或者用更紧凑的向量表示：

$$y = \text{step}(\mathbf{w}^T\mathbf{x} + b)$$

其中：- $\mathbf{x} = [x_1, x_2, ..., x_n]^T$ 是输入向量 - $\mathbf{w} = [w_1, w_2, ..., w_n]^T$ 是权重向量 - b 是偏置（bias），相当于调整阈值的位置 - $\text{step}()$ 是阶跃函数

12.4 16.3 几何直观：感知机在画线

12.4.1 16.3.1 决策边界

感知机本质上是一个**线性分类器**。它在特征空间中画一条直线（或超平面），把两类数据分开。

想象你在一张纸上画了两个点群：红点在左边，蓝点在右边。感知机的任务就是找到一条线，把红点全分到一边，蓝点全分到另一边。

```
                    特 征  x

        蓝 色 类

                                        ← 决 策 边 界

        红 色 类

                                        → 特 征  x
```

这条线就叫做**决策边界**（Decision Boundary）。数学上，它由方程 $\mathbf{w}^T\mathbf{x} + b = 0$ 定义。

12.4.2 16.3.2 权重的几何意义

权重向量 $\mathbf{w}$ 有什么几何意义？

它垂直于决策边界！

```
                    ↑
                      权 重 向 量  w



        分 类 区 域        -1     分 类 区 域
        (y = 0)                  (y = 1)
```

- 权重向量 $\mathbf{w}$ 指向分类结果为 1 的那一侧
- 偏置 b 控制决策边界离原点的距离

这个几何直观非常重要，因为它帮助我们理解为什么感知机只能解决**线性可分**的问题。

12.5 16.4 感知机学习规则：如何自动学习

现在我们来到最关键的问题：感知机是如何自动学习权重的？

12.5.1 16.4.1 核心思想：试错学习

感知机的学习规则基于一个简单的原则：

> **"如果错了，就调整；如果对了，就保持。"**

具体步骤：1. 随机初始化权重 2. 取一个训练样本，用当前权重做预测 3. 如果预测正确 → 什么都不做 4. 如果预测错误 → 调整权重，让预测向正确答案靠近 5. 重复步骤 2-4，直到所有样本都被正确分类

12.5.2 16.4.2 学习规则的数学推导

让我们一步一步推导感知机的学习规则。

设定：- 输入：$\mathbf{x} = [x_1, x_2, ..., x_n]^T$ - 真实标签：$y \in \{0, 1\}$ - 当前权重：$\mathbf{w} = [w_1, w_2, ..., w_n]^T$ - 当前偏置：b - 预测值：$\hat{y} = \text{step}(\mathbf{w}^T\mathbf{x} + b)$

情况 1：$y = 1$，但 $\hat{y} = 0$（漏报）

这意味着 $z = \mathbf{w}^T\mathbf{x} + b < 0$，太小了。我们需要**增大** z。

怎么增大？增加权重和偏置！

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta \cdot \mathbf{x}$$

$$b_{\text{new}} = b_{\text{old}} + \eta$$

其中 η 是学习率（learning rate），控制每次调整的幅度。

情况 2：$y = 0$，但 $\hat{y} = 1$（误报）

这意味着 $z = \mathbf{w}^T\mathbf{x} + b \geq 0$，太大了。我们需要**减小** z。

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta \cdot \mathbf{x}$$

$$b_{\text{new}} = b_{\text{old}} - \eta$$

情况 3：$y = \hat{y}$（预测正确）

什么都不做：

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}}$$

$$b_{\text{new}} = b_{\text{old}}$$

12.5.3 16.4.3 统一的更新公式

我们可以把以上三种情况写成一个统一的公式。注意：- 情况 1：$y - \hat{y} = 1 - 0 = +1$ - 情况 2：$y - \hat{y} = 0 - 1 = -1$ - 情况 3：$y - \hat{y} = 0$ 或 0

所以：

$$\boxed{\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta \cdot (y - \hat{y}) \cdot \mathbf{x}}$$

$$\boxed{b_{\text{new}} = b_{\text{old}} + \eta \cdot (y - \hat{y})}$$

这就是著名的**感知机学习规则**（Perceptron Learning Rule）！

关键洞察：- 当预测错误时，$(y - \hat{y})$ 是 $+1$ 或 -1，权重会调整 - 当预测正确时，$(y - \hat{y}) = 0$，权重保持不变

12.5.4 16.4.4 完整算法

```
# 感知机学习算法伪代码

def perceptron_learning(X, y, eta=0.1, max_epochs=100):
    """
    X: 训练数据，形状 (n_samples, n_features)
    y: 标签，形状 (n_samples,)，取值 {0, 1}
    eta: 学习率
    max_epochs: 最大迭代轮数
    """
    n_samples, n_features = X.shape

    # 1. 初始化权重和偏置（通常初始化为0或很小的随机数）
    w = np.zeros(n_features)
    b = 0

    for epoch in range(max_epochs):
        errors = 0
```

```
        for i in range(n_samples):
            # 2. 计算加权和
            z = np.dot(w, X[i]) + b

            # 3. 激活（阶跃函数）
            y_pred = 1 if z >= 0 else 0

            # 4. 如果预测错误，更新权重
            if y_pred != y[i]:
                error = y[i] - y_pred  # +1 或 -1
                w = w + eta * error * X[i]
                b = b + eta * error
                errors += 1

        # 5. 如果这一轮没有错误，收敛了！
        if errors == 0:
            print(f"收敛于第 {epoch + 1} 轮")
            break

    return w, b
```

12.6 16.5 感知机收敛定理

现在我们来回答一个关键问题：**感知机算法一定会收敛吗?**

1962 年，罗森布拉特证明了一个重要的定理：

12.6.1 16.5.1 感知机收敛定理

定理：如果训练数据是**线性可分**的，那么感知机学习算法保证在有限步内收敛到一个能正确分类所有样本的解。

证明思路（直观版）：

假设存在一个”完美”的权重向量 $\mathbf{w}^*$，能正确分类所有样本。我们证明：

1. 每次更新时，当前权重 $\mathbf{w}$ 与 $\mathbf{w}^*$ 的**夹角在减小**（越来越接近）
2. 每次更新的幅度是有界的
3. 因此，经过有限次更新后，$\mathbf{w}$ 一定会变得足够接近 $\mathbf{w}^*$

更严谨的证明：

设：- 存在一个解 $\mathbf{w}^*$，使得对所有样本都有 $y_i(\mathbf{w}^{*T}\mathbf{x}_i + b^*) \geq \gamma > 0$（即有一个正的间隔）- 所有样本满足 $\|\mathbf{x}_i\| \leq R$

引理 1：$\mathbf{w}^{*T}\mathbf{w}^{(k)} \geq k\eta\gamma$

证明：

$$\mathbf{w}^{*T}\mathbf{w}^{(k)} = \mathbf{w}^{*T}\mathbf{w}^{(k-1)} + \eta y_i \mathbf{w}^{*T}\mathbf{x}_i \geq \mathbf{w}^{*T}\mathbf{w}^{(k-1)} + \eta\gamma$$

递推得：$\mathbf{w}^{*T}\mathbf{w}^{(k)} \geq k\eta\gamma$

引理 2：$\|\mathbf{w}^{(k)}\|^2 \leq k\eta^2 R^2$

证明：

$$\|\mathbf{w}^{(k)}\|^2 = \|\mathbf{w}^{(k-1)} + \eta y_i \mathbf{x}_i\|^2$$
$$= \|\mathbf{w}^{(k-1)}\|^2 + 2\eta y_i \mathbf{w}^{(k-1)T}\mathbf{x}_i + \eta^2\|\mathbf{x}_i\|^2$$

因为更新发生在错误分类时，$y_i \mathbf{w}^{(k-1)T}\mathbf{x}_i < 0$，所以：

$$\|\mathbf{w}^{(k)}\|^2 \leq \|\mathbf{w}^{(k-1)}\|^2 + \eta^2 R^2$$

递推得：$\|\mathbf{w}^{(k)}\|^2 \leq k\eta^2 R^2$

结合两个引理：

由柯西-施瓦茨不等式：$(\mathbf{w}^{*T}\mathbf{w}^{(k)})^2 \leq \|\mathbf{w}^*\|^2\|\mathbf{w}^{(k)}\|^2$

代入引理 1 和引理 2：

$$(k\eta\gamma)^2 \leq \|\mathbf{w}^*\|^2 \cdot k\eta^2 R^2$$

化简：

$$k \leq \frac{\|\mathbf{w}^*\|^2 R^2}{\gamma^2}$$

这意味着更新次数 k 是有上界的！所以算法一定会在有限步内收敛。

实际意义：

对于线性可分数据，感知机最多需要 $\frac{R^2}{\gamma^2}$ 次更新就能收敛。

12.7 16.6 感知机的局限：XOR 问题

12.7.1 16.6.1 XOR 问题是什么？

感知机虽然很酷，但它有一个致命弱点：**只能解决线性可分的问题**。

最著名的例子是 **XOR 问题**（异或问题）。

XOR 的真值表：

x	x	x XOR x
0	0	0
0	1	1
1	0	1
1	1	0

让我们在平面上画出这四个点：

```
   x

1        (0,1)     (1,1)
          [1]       [0]   ← 无法用一条直线分开！

                       → x

0        (0,0)     (1,0)
          [0]       [1]
```

- 表示输出为 0 的点
- 表示输出为 1 的点

问题是：**无法用一条直线把两个 和两个 分开！**

12.7.2 16.6.2 线性不可分的直观理解

XOR 问题之所以难，是因为它的”真值分布”不是线性的。想象你有四个座位：

```
 0,0   0,1    ← 前排：一个人（0）或另一个人（1）

 1,0   1,1    ← 后排：两个人（0）或没人的情况（1？不，也是0）
```

XOR 想要的是”恰好有一个人”的情况。这种”排斥”关系本质上是非线性的。

12.7.3 16.6.3 Minsky 和 Papert 的致命一击

1969 年，麻省理工学院的两位人工智能先驱——马文·明斯基（Marvin Minsky）和西摩·帕珀特（Seymour Papert）——出版了《感知机》（*Perceptrons*）一书。

在这本书中，他们严格证明了：

> **单层感知机无法解决 XOR 问题，也无法解决任何非线性可分的问题。**

更糟的是，他们暗示：**多层感知机可能也无法解决这些问题**（虽然后来证明这是错误的）。

这本书的影响是毁灭性的：

- 感知机的研究几乎被放弃
- 神经网络领域进入了长达十多年的”寒冬期”
- 政府 funding 被大幅削减
- 人工智能研究转向了符号主义方法

明斯基后来承认，他写这本书的部分动机是为了争夺 AI 研究的主导权（符号主义 vs 连接主义）。但从科学角度，他指出感知机的局限性是正确的。

12.7.4 16.6.4 如何解决 XOR?

解决 XOR 问题的关键是：**使用多层感知机（Multi-Layer Perceptron）**。

想象我们能画一条曲线，而不是直线：

```
    x

1

                        → x

0
```

或者，我们可以用**两个感知机组合**来解决：

```
输入层          隐藏层           输出层

x
         → [感知机A: OR]
x                                → [感知机C: AND] → 输出
         → [感知机B: NAND]
x

x
```

- 感知机 A 实现 OR：输出 1 如果 x =1 或 x =1
- 感知机 B 实现 NAND：输出 1 除非 x =1 且 x =1
- 感知机 C 把 A 和 B 的结果做 AND

验证：- (0,0)：OR=0, NAND=1, AND=0 - (0,1)：OR=1, NAND=1, AND=1 - (1,0)：OR=1, NAND=1, AND=1 - (1,1)：OR=1, NAND=0, AND=0

XOR = (x OR x) AND NOT(x AND x)

但训练多层网络需要更复杂的算法——**反向传播**（Backpropagation），这是我们在后续章节要学习的内容。

12.8 16.7 从零实现感知机

现在让我们用纯 Python 和 NumPy 实现一个完整的感知机。

12.8.1 16.7.1 Perceptron 类实现

```
"""
感知机从零实现
================
```

```
不依赖任何机器学习框架，只用NumPy

作者：ML教材写作项目
日期：2026
"""

import numpy as np
import matplotlib.pyplot as plt


class Perceptron:
    """
    感知机分类器

    参数：
    -----------
    eta : float
        学习率（0.0 到 1.0之间）
    n_iter : int
        最大训练轮数
    random_state : int
        随机种子，用于初始化权重

    属性：
    -----------
    w_ : 1d-array
        训练后的权重
    b_ : float
        训练后的偏置
    errors_ : list
        每轮的错误分类数
    """

    def __init__(self, eta=0.01, n_iter=50, random_state=None):
        self.eta = eta
        self.n_iter = n_iter
        self.random_state = random_state
        self.w_ = None
        self.b_ = None
        self.errors_ = []

    def fit(self, X, y):
        """
        训练感知机

        参数：
        -----------
```

```
        X : array-like, shape = [n_samples, n_features]
            训练数据
        y : array-like, shape = [n_samples]
            目标值，取值为 {0, 1}

        返回:
        -----------
        self : object
        """
        rgen = np.random.RandomState(self.random_state)
        self.w_ = rgen.normal(loc=0.0, scale=0.01, size=X.shape[1])
        self.b_ = 0.0
        self.errors_ = []

        for epoch in range(self.n_iter):
            errors = 0

            for xi, target in zip(X, y):
                # 计算预测值
                y_pred = self.predict_single(xi)

                # 计算误差
                error = target - y_pred

                # 如果预测错误，更新权重
                if error != 0:
                    # w_new = w_old + eta * error * x
                    self.w_ += self.eta * error * xi
                    # b_new = b_old + eta * error
                    self.b_ += self.eta * error
                    errors += 1

            self.errors_.append(errors)

            # 如果这一轮没有错误，提前停止
            if errors == 0:
                print(f"收敛于第 {epoch + 1} 轮")
                break

        return self

    def net_input(self, X):
        """计算净输入 z = w·x + b"""
        return np.dot(X, self.w_) + self.b_

    def predict_single(self, x):
        """预测单个样本"""
```

```
        return 1 if self.net_input(x) >= 0 else 0

    def predict(self, X):
        """预测多个样本"""
        return np.where(self.net_input(X) >= 0, 1, 0)

    def accuracy(self, X, y):
        """计算准确率"""
        predictions = self.predict(X)
        return np.mean(predictions == y)

    def __repr__(self):
        return f"Perceptron(eta={self.eta}, n_iter={self.n_iter})"


# ============================================================
# 演示1: AND 逻辑门
# ============================================================

def demo_and_gate():
    """演示感知机学习AND逻辑门"""
    print("=" * 50)
    print("演示1: 学习 AND 逻辑门")
    print("=" * 50)

    # AND 真值表
    X = np.array([
        [0, 0],
        [0, 1],
        [1, 0],
        [1, 1]
    ])
    y = np.array([0, 0, 0, 1])

    print("\n训练数据 (AND 真值表):")
    print("x1\tx2\tAND(x1,x2)")
    for xi, yi in zip(X, y):
        print(f"{xi[0]}\t{xi[1]}\t{yi}")

    # 创建并训练感知机
    ppn = Perceptron(eta=0.1, n_iter=10)
    ppn.fit(X, y)

    print(f"\n学习到的权重: w = [{ppn.w_[0]:.3f}, {ppn.w_[1]:.3f}]")
    print(f"学习到的偏置: b = {ppn.b_:.3f}")

    # 测试
```

```
    print("\n预测结果:")
    print("x1\tx2\t真实值\t预测值")
    for xi, yi in zip(X, y):
        pred = ppn.predict_single(xi)
        print(f"{xi[0]}\t{xi[1]}\t{yi}\t{pred}")

    print(f"\n准确率: {ppn.accuracy(X, y) * 100:.1f}%")

    # 绘制决策边界
    plot_decision_boundary(X, y, ppn, "AND 逻辑门的决策边界")

    return ppn


# ============================================================
# 演示2: OR 逻辑门
# ============================================================

def demo_or_gate():
    """演示感知机学习OR逻辑门"""
    print("\n" + "=" * 50)
    print("演示2: 学习 OR 逻辑门")
    print("=" * 50)

    # OR 真值表
    X = np.array([
        [0, 0],
        [0, 1],
        [1, 0],
        [1, 1]
    ])
    y = np.array([0, 1, 1, 1])

    print("\n训练数据 (OR 真值表):")
    print("x1\tx2\tOR(x1,x2)")
    for xi, yi in zip(X, y):
        print(f"{xi[0]}\t{xi[1]}\t{yi}")

    ppn = Perceptron(eta=0.1, n_iter=10)
    ppn.fit(X, y)

    print(f"\n学习到的权重: w = [{ppn.w_[0]:.3f}, {ppn.w_[1]:.3f}]")
    print(f"学习到的偏置: b = {ppn.b_:.3f}")

    print("\n预测结果:")
    for xi, yi in zip(X, y):
        pred = ppn.predict_single(xi)
```

```
        print(f"{xi}\t真实:{yi}\t预测:{pred}")

    plot_decision_boundary(X, y, ppn, "OR 逻辑门的决策边界")

    return ppn


# ============================================================
# 演示3: XOR 逻辑门 (感知机无法解决!)
# ============================================================

def demo_xor_gate():
    """演示感知机无法学习XOR逻辑门"""
    print("\n" + "=" * 50)
    print("演示3: XOR 逻辑门 (感知机的局限)")
    print("=" * 50)

    # XOR 真值表
    X = np.array([
        [0, 0],
        [0, 1],
        [1, 0],
        [1, 1]
    ])
    y = np.array([0, 1, 1, 0])

    print("\n训练数据 (XOR 真值表):")
    print("x1\tx2\tXOR(x1,x2)")
    for xi, yi in zip(X, y):
        print(f"{xi[0]}\t{xi[1]}\t{yi}")

    print("\n尝试训练感知机...")
    ppn = Perceptron(eta=0.1, n_iter=20)
    ppn.fit(X, y)

    print("\n预测结果:")
    print("x1\tx2\t真实值\t预测值\t正确?")
    for xi, yi in zip(X, y):
        pred = ppn.predict_single(xi)
        correct = " " if pred == yi else " "
        print(f"{xi[0]}\t{xi[1]}\t{yi}\t{pred}\t{correct}")

    accuracy = ppn.accuracy(X, y)
    print(f"\n准确率: {accuracy * 100:.1f}%")
    print(f"错误数: {int((1 - accuracy) * len(y))} / {len(y)}")

    print("\n    注意: 感知机无法解决XOR问题! ")
```

```
    print("    因为XOR不是线性可分的。")
    print("    这需要多层神经网络（后续章节讲解）。")

    # 仍然尝试绘制，但会显示无法分开
    plot_decision_boundary(X, y, ppn, "XOR 逻辑门（线性不可分）")

    return ppn


# ============================================================
# 可视化函数
# ============================================================

def plot_decision_boundary(X, y, model, title):
    """绘制决策边界"""
    plt.figure(figsize=(8, 6))

    # 绘制数据点
    for i, (xi, yi) in enumerate(zip(X, y)):
        if yi == 0:
            plt.scatter(xi[0], xi[1], c='red', s=200, marker='o',
                        edgecolors='black', linewidth=2, label='Class 0' if i == 0
                            else "")
        else:
            plt.scatter(xi[0], xi[1], c='blue', s=200, marker='^',
                        edgecolors='black', linewidth=2, label='Class 1' if i == 0
                            else "")

        # 添加标签
        plt.annotate(f'({xi[0]},{xi[1]})\ny={yi}',
                     (xi[0], xi[1]),
                     textcoords="offset points",
                     xytext=(0, 10),
                     ha='center',
                     fontsize=9)

    # 绘制决策边界
    if model.w_[1] != 0:
        x_min, x_max = -0.5, 1.5
        x_values = np.linspace(x_min, x_max, 100)
        # w0*x + w1*y + b = 0  =>  y = -(w0*x + b) / w1
        y_values = -(model.w_[0] * x_values + model.b_) / model.w_[1]
        plt.plot(x_values, y_values, 'g--', linewidth=2, label='Decision Boundary')

    plt.xlim(-0.5, 1.5)
    plt.ylim(-0.5, 1.5)
    plt.xlabel('x ', fontsize=12)
```

```
    plt.ylabel('x ', fontsize=12)
    plt.title(title, fontsize=14)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.axhline(y=0, color='k', linewidth=0.5)
    plt.axvline(x=0, color='k', linewidth=0.5)
    plt.savefig(f'{title.replace(" ", "_")}.png', dpi=150, bbox_inches='tight')
    plt.close()
    print(f"  图像已保存：{title.replace(' ', '_')}.png")


# ============================================================
# 演示4：鸢尾花数据集（真实数据）
# ============================================================

def demo_iris():
    """在简化版鸢尾花数据集上演示感知机"""
    print("\n" + "=" * 50)
    print("演示4：鸢尾花分类（真实数据）")
    print("=" * 50)

    # 简化的鸢尾花数据（只有两类：山鸢尾和变色鸢尾）
    # 特征：花瓣长度和花瓣宽度
    np.random.seed(42)

    # 山鸢尾（Setosa）- 类别 0
    # 花瓣短而窄
    setosa = np.random.multivariate_normal(
        mean=[1.4, 0.2],
        cov=[[0.01, 0.002], [0.002, 0.01]],
        size=30
    )

    # 变色鸢尾（Versicolor）- 类别 1
    # 花瓣中等长度
    versicolor = np.random.multivariate_normal(
        mean=[4.2, 1.3],
        cov=[[0.1, 0.02], [0.02, 0.05]],
        size=30
    )

    X = np.vstack([setosa, versicolor])
    y = np.array([0] * 30 + [1] * 30)

    print(f"\n数据集：60个样本，2个特征")
    print(f"  - 山鸢尾（Setosa）：30个")
    print(f"  - 变色鸢尾（Versicolor）：30个")
```

```
    print(f"\n特征： 花瓣长度、 花瓣宽度")

    # 划分训练集和测试集
    indices = np.random.permutation(len(X))
    train_idx = indices[:40]
    test_idx = indices[40:]

    X_train, y_train = X[train_idx], y[train_idx]
    X_test, y_test = X[test_idx], y[test_idx]

    print(f"\n训练集： {len(X_train)}个样本")
    print(f"测试集： {len(X_test)}个样本")

    # 训练感知机
    ppn = Perceptron(eta=0.01, n_iter=100)
    ppn.fit(X_train, y_train)

    # 评估
    train_acc = ppn.accuracy(X_train, y_train)
    test_acc = ppn.accuracy(X_test, y_test)

    print(f"\n训练准确率： {train_acc * 100:.1f}%")
    print(f"测试准确率： {test_acc * 100:.1f}%")
    print(f"\n学习到的权重： w = [{ppn.w_[0]:.3f}, {ppn.w_[1]:.3f}]")
    print(f"学习到的偏置： b = {ppn.b_:.3f}")

    # 绘制
    plot_iris_decision_boundary(X, y, ppn)

    return ppn


def plot_iris_decision_boundary(X, y, model):
    """绘制鸢尾花数据的决策边界"""
    plt.figure(figsize=(10, 8))

    # 绘制数据点
    class_0 = X[y == 0]
    class_1 = X[y == 1]

    plt.scatter(class_0[:, 0], class_0[:, 1], c='red', s=100,
                marker='o', edgecolors='black', label='Setosa (Class 0)')
    plt.scatter(class_1[:, 0], class_1[:, 1], c='blue', s=100,
                marker='^', edgecolors='black', label='Versicolor (Class 1)')

    # 绘制决策边界
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
```

```
    x_values = np.linspace(x_min, x_max, 100)
    y_values = -(model.w_[0] * x_values + model.b_) / model.w_[1]
    plt.plot(x_values, y_values, 'g-', linewidth=2, label='Decision Boundary')

    plt.xlabel('Petal Length (cm)', fontsize=12)
    plt.ylabel('Petal Width (cm)', fontsize=12)
    plt.title('Perceptron on Iris Dataset', fontsize=14)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.savefig('iris_decision_boundary.png', dpi=150, bbox_inches='tight')
    plt.close()
    print("  图像已保存: iris_decision_boundary.png")


# ============================================================
# 主程序
# ============================================================

if __name__ == "__main__":
    print(" " + "=" * 58 + " ")
    print(" " + " " * 12 + "感知机从零实现演示" + " " * 24 + " ")
    print(" " + " " * 6 + "基于 Rosenblatt (1958) 原始论文" + " " * 16 + " ")
    print(" " + "=" * 58 + " ")

    # 运行所有演示
    demo_and_gate()
    demo_or_gate()
    demo_xor_gate()
    demo_iris()

    print("\n" + "=" * 58)
    print("演示完成！")
    print("=" * 58)
```

12.8.2 16.7.2 运行示例输出

```
==================================================
演示1: 学习 AND 逻辑门
==================================================

训练数据 (AND 真值表):
x1  x2  AND(x1,x2)
0   0   0
0   1   0
1   0   0
1   1   1
```

```
收敛于第 5 轮

学习到的权重: w = [0.200, 0.100]
学习到的偏置: b = -0.200

预测结果:
x1  x2  真实值  预测值
0   0   0   0
0   1   0   0
1   0   0   0
1   1   1   1

准确率: 100.0%
  图像已保存: AND_逻辑门的决策边界.png

==================================================
演示2: 学习 OR 逻辑门
==================================================
...

==================================================
演示3: XOR 逻辑门 (感知机的局限)
==================================================
...
训练数据 (XOR 真值表):
x1  x2  XOR(x1,x2)
0   0   0
0   1   1
1   0   1
1   1   0

尝试训练感知机...

预测结果:
x1  x2  真实值  预测值  正确?
0   0   0   0
0   1   1   0
1   0   1   1
1   1   0   1

准确率: 50.0%
错误数: 2 / 4

    注意: 感知机无法解决XOR问题!
    因为XOR不是线性可分的。
    这需要多层神经网络 (后续章节讲解)。
```

12.9 16.8 多类分类的扩展

感知机原本是二分类器（两类），但我们可以通过一些技巧扩展到多类分类。

12.9.1 16.8.1 One-vs-Rest（一对多）策略

对于 K 个类别，我们训练 K 个感知机：

```
感知机1: 类1 vs (类2,类3,...,类K)
感知机2: 类2 vs (类1,类3,...,类K)
...
感知机K: 类K vs (类1,类2,...,类K-1)
```

预测时，选择得分最高的感知机对应的类别。

12.9.2 16.8.2 实现代码

```python
class MultiClassPerceptron:
    """多类感知机（One-vs-Rest策略）"""

    def __init__(self, eta=0.01, n_iter=50):
        self.eta = eta
        self.n_iter = n_iter
        self.classifiers = {}

    def fit(self, X, y):
        """训练多个二分类感知机"""
        self.classes = np.unique(y)

        for cls in self.classes:
            # 为每个类别创建一个二分类问题
            y_binary = np.where(y == cls, 1, 0)

            # 训练一个感知机
            ppn = Perceptron(eta=self.eta, n_iter=self.n_iter)
            ppn.fit(X, y_binary)

            self.classifiers[cls] = ppn
            print(f"类别 {cls} 的感知机训练完成")

    def predict(self, X):
        """预测：选择得分最高的类别"""
        scores = {}
        for cls, ppn in self.classifiers.items():
            scores[cls] = ppn.net_input(X)
```

```
        # 选择得分最高的类别
        predictions = []
        for i in range(len(X)):
            best_cls = max(self.classes, key=lambda c: scores[c][i])
            predictions.append(best_cls)

        return np.array(predictions)
```

12.10 16.9 练习题

12.10.1 基础题

16.1 感知机的权重和偏置有什么作用？如果偏置 $b = 0$，会发生什么？

16.2 手动计算：给定权重 $\mathbf{w} = [2, -1]$，偏置 $b = 0.5$，输入 $\mathbf{x} = [1, 2]$，感知机的输出是什么？

16.3 感知机学习规则中，学习率 η 有什么作用？如果 η 太大或太小，会发生什么？

12.10.2 进阶题

16.4 证明：对于 AND 问题，感知机最多需要多少次更新就能收敛？（提示：使用感知机收敛定理）

16.5 设计一个实验，验证感知机在线性可分数据上的收敛性，以及在线性不可分数据上的震荡。

16.6 阅读罗森布拉特 1958 年的原始论文，总结他的主要贡献和当时的科学背景。

12.10.3 挑战题

16.7 编程挑战：实现一个能学习 NAND、OR、AND 组合解决 XOR 问题的多层感知机（不使用反向传播，而是手动设置权重）。

16.8 研究项目：调查感知机在现代机器学习中的应用。虽然深层网络更流行，但感知机（或线性分类器）还在哪些地方被使用？

12.11 16.10 本章小结

12.11.1 核心概念

概念	解释
感知机	第一个能从数据自动学习的神经网络模型
权重	控制每个输入重要性的参数
偏置	调整激活阈值的参数
阶跃函数	将加权和转换为 0 或 1 输出的激活函数
决策边界	分隔两类数据的超平面
线性可分	存在一条直线（或超平面）能完美分开两类数据
学习规则	$w \leftarrow w + \eta(y - \hat{y})x$

概念	解释
收敛定理	线性可分数据保证在有限步内收敛
XOR 问题	线性不可分的经典例子，单层感知机无法解决

12.11.2 历史脉络

```
1943     McCulloch & Pitts    → 提出人工神经元模型
         （能模拟逻辑运算，但需要人工设定权重）
         ↓
1949     Hebb    → 提出"一起激发的神经元连在一起"学习规则
         ↓
1958     Rosenblatt    → 发明感知机，第一个自动学习的神经网络
         ↓
1960s    感知机热潮    → Mark I硬件、媒体追捧
         ↓
1969     Minsky & Papert    → 《感知机》一书指出XOR局限
         ↓
1970s    AI寒冬    → 神经网络研究几乎停滞
         ↓
1986     Rumelhart et al.    → 反向传播算法，多层网络复兴
         ↓
2012     AlexNet    → 深度学习革命，神经网络重回巅峰
```

12.11.3 本章代码实现

我们实现了一个完整的感知机类，包括：- 感知机学习规则 - AND、OR、XOR 演示 - 鸢尾花分类 - 决策边界可视化 - 多类分类扩展

12.12 16.11 参考文献

12.12.1 原始论文

1. **Rosenblatt, F.** (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408. https://doi.org/10.1037/h0042519
2. **McCulloch, W. S., & Pitts, W.** (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4), 115-133. https://doi.org/10.1007/BF02478259
3. **Hebb, D. O.** (1949). *The organization of behavior: A neuropsychological theory.* John Wiley & Sons.

12.12.2 感知机局限性的经典论述

4. **Minsky, M., & Papert, S.** (1969). *Perceptrons: An introduction to computational geometry.* MIT Press.

12.12.3 感知机收敛定理

5. **Novikoff, A. B.** (1962). On convergence proofs on perceptrons. *Proceedings of the Symposium on the Mathematical Theory of Automata*, 12, 615-622.

6. **Block, H. D.** (1962). The perceptron: A model for brain functioning. *Reviews of Modern Physics*, 34(1), 123-135.

12.12.4 现代教材

7. **Goodfellow, I., Bengio, Y., & Courville, A.** (2016). *Deep learning*. MIT Press. Chapter 6: Deep Feedforward Networks.

8. **Bishop, C. M.** (2006). *Pattern recognition and machine learning*. Springer. Chapter 4: Linear Models for Classification.

9. **Hastie, T., Tibshirani, R., & Friedman, J.** (2009). *The elements of statistical learning* (2nd ed.). Springer. Chapter 4: Linear Methods for Classification.

12.12.5 历史回顾

10. **Olazaran, M.** (1996). A sociological study of the official history of the perceptrons controversy. *Social Studies of Science*, 26(3), 611-659.

12.13 下一章预告

第十七章：多层神经网络——层层的魔法

我们将学习：- 为什么需要多层网络 - 前向传播算法 - 反向传播的直观理解 - 从零实现一个多层感知机（MLP）

准备好揭开神经网络的层层面纱了吗？

本章完

---

13 第十七章 多层神经网络——层层的魔法

> “把简单的单元堆叠起来，就能创造出令人惊叹的智能。”
>
> ——杰弗里·辛顿 (Geoffrey Hinton)

13.1 开场故事：折纸与神经网络

想象一下你有一张平面的纸，上面画着一个红色的圆圈和一个蓝色的方块，它们紧紧挨在一起，几乎重叠。现在我问你：能不能用一把剪刀，**只剪一刀**就把红色和蓝色完全分开？

不可能，对吧？因为它们在纸上是纠缠在一起的，无论你怎么直线剪，总会剪到一些红色或蓝色。

但是——如果你可以**把纸折起来**呢?

想象你把纸对折，让圆圈和方块分别位于折痕的两侧。现在你只需要沿着折痕剪一刀，就能完美地将它们分开！这个简单的动作——折叠——创造了一个新的维度，让原本不可能的事情变得可能。

神经网络的核心魔法，正是这种**维度变换**的能力。

在上一章，我们学习了感知机——最简单的神经网络单元，就像一把只能直线切割的剪刀。本章，我们将探索如何把多个感知机**层层堆叠**，创造出能够解决任何复杂问题的多层神经网络。这就是深度学习 revolution 的起点！

13.2 17.1 历史的转折：从寒冬到春天

13.2.1 17.1.1 感知机的黄金时代（1958）

让我们把时间倒回到 1958 年。在康奈尔航空实验室，一位年轻的心理学家弗兰克·罗森布拉特（Frank Rosenblatt）正在研究一个疯狂的问题：**机器能学会看吗?**

他发明了感知机（Perceptron），这是世界上第一个人工神经网络。罗森布拉特兴奋地向全世界宣布：感知机能够学习任何东西！

当时的媒体疯狂了。《纽约时报》在 1958 年 7 月 8 日刊登了一篇题为《海军开发电子计算机，预计能够行走、说话、看和写》的文章。报道宣称：

> “预计感知机最终将能够识别人类，叫出他们的名字，并将对话即时翻译成另一种语言。”

这听起来是不是很熟悉？今天的大语言模型确实做到了这些！但在 1958 年，这听起来像科幻小说。

罗森布拉特甚至做了一个大胆的预言：**感知机将能够学会识别任何可以被定义的模式**。

13.2.2 17.1.2 寒冬降临：Minsky 与 Papert 的打击（1969）

然而，历史总是喜欢开玩笑。

1969 年，麻省理工学院的两位人工智能先驱——马文·明斯基（Marvin Minsky）和西摩·帕珀特（Seymour Papert）出版了一本书，书名简单直接：《感知机》(Perceptrons)。

这本书只有 165 页，但它对神经网络研究的影响是毁灭性的。

明斯基和帕珀特用数学证明了感知机的一个致命缺陷：**单层感知机无法解决 XOR 问题**。

什么是 XOR 问题？我们马上会详细解释，但现在你只需要知道：这是一个极其简单的分类问题，但单层感知机永远无法学会。

更重要的是，明斯基和帕珀特暗示：**即使是多层感知机也可能面临同样的问题**。

他们写道：

> “感知机被过度炒作了…没有理由相信多层系统会比单层系统更容易训练。”

这本书的出版，直接导致了**第一次 AI 寒冬**。神经网络研究几乎完全停滞，政府资金被切断，学生们被告知不要再研究这个”死胡同”。

罗森布拉特试图反驳，但他的声音被淹没了。1971 年，年仅 43 岁的罗森布拉特在一次帆船事故中不幸去世。神经网络的故事，似乎就此终结。

13.2.3 17.1.3 希望的种子：反向传播的诞生（1970）

然而，科学的进步从来不会真正停止，只是换了一种方式继续。

1970 年，在遥远的芬兰，一位名叫塞波·林纳伊马（Seppo Linnainmaa）的年轻博士生正在赫尔辛基大学攻读学位。他的博士论文题目是《累积舍入误差的泰勒展开》。

这听起来与神经网络毫无关系。但林纳伊马在这篇论文中，首次提出了一种算法：**自动微分**。

这个算法后来被命名为**反向传播**（Backpropagation）。它解决了神经网络训练中最核心的问题：**如何有效地计算每一层参数的梯度**。

林纳伊马可能没有意识到他发现了什么。他的论文是用芬兰语写的，发表在计算机科学的圈子里，与神经网络研究完全隔绝。这个划时代的算法，就这样被埋没了近 16 年。

13.2.4 17.1.4 春天来了：Rumelhart、Hinton 与 Williams 的突破（1986）

1986 年，一切都改变了。

在《自然》杂志上，一篇名为《通过反向传播误差学习表示》（Learning representations by back-propagating errors）的论文发表了。作者是三位研究者：

- **大卫·鲁梅尔哈特**（David Rumelhart）——认知心理学家，连接主义的倡导者
- **杰弗里·辛顿**（Geoffrey Hinton）——被誉为”深度学习之父”
- **罗纳德·威廉姆斯**（Ronald Williams）——鲁梅尔哈特的学生

这篇论文展示了如何用反向传播算法训练**多层神经网络**。他们证明：只要网络有足够的隐藏层，理论上可以解决任何问题——包括困扰神经网络近 20 年的 XOR 问题！

辛顿后来回忆说：

> “当我第一次运行反向传播算法，看到网络真的学会了 XOR 问题时，我激动得跳了起来。那一刻我知道，一切都将改变。”

这篇论文重新点燃了人们对神经网络的热情。它证明了明斯基和帕珀特是**部分正确但结论错误**的——多层网络确实更难训练，但反向传播提供了解决方案。

13.2.5 17.1.5 历史的启示

回顾这段历史，我们能学到什么？

第一，科学进步从来不是线性的。罗森布拉特的乐观、明斯基的批判、林纳伊马的发现、辛顿的突破——每一步都是必要的，但也都有其局限性。

第二，好想法需要好时机。林纳伊马的反向传播提前了 16 年，但因为没有与神经网络结合，所以没有产生影响。时机，往往比想法本身更重要。

第三，不要轻言”不可能”。明斯基断言多层网络难以训练，但他错了。科学史上充满了这样的例子：今天的”不可能”，往往是明天的”显而易见”。

现在，让我们亲自体验一下这段历史中的核心问题：XOR。

13.3 17.2 XOR 问题：为什么单层不够?

13.3.1 17.2.1 什么是 XOR?

XOR 是”异或”（Exclusive OR）的缩写。这是一个非常简单的逻辑运算：

输入 A	输入 B	输出（A XOR B）
0	0	0
0	1	1
1	0	1
1	1	0

用自然语言表达：**当且仅当两个输入不同时，输出为 1**。

这就像我们小时候玩的”找不同”游戏——如果两个东西不一样，我们就说”找到了！”

13.3.2 17.2.2 XOR 问题的可视化

让我们把 XOR 问题画在二维平面上：

```
        输 入B
          ↑
   0      |      1
          |
   (0,0)  |    (1,1)
          |
                   →  输 入A
          |
          |
 (0,1)    |    (1,0)
   1      |      0
```

图中：- 表示输出 0（负类）- 表示输出 1（正类）

现在的问题是：**你能用一条直线把 和 完全分开吗?**

试试看！无论你画什么直线，总会穿过至少一个错误类别的点。

这就是 XOR 问题的核心：**它是线性不可分的**。

13.3.3 17.2.3 感知机的局限

为什么单层感知机解决不了 XOR?

回忆一下感知机的工作原理：

$$output = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

这个公式定义了一个**线性决策边界**——一条直线（在更高维度中是超平面）。

感知机能够解决 AND、OR 这样的问题，因为它们是**线性可分**的：

```
AND 问 题 （ 线 性 可 分 ） ：

    B ↑
      |
  0   |      (1,1) → 输 出 1

(0,0)   (1,0) → 都 可 以 被 这 条 线 分 开
      |
           → A
      |
```

但 XOR 需要**非线性**的决策边界。单层感知机就像一个只能用直尺画图的人——无论怎么努力，都无法画出曲线。

13.3.4 17.2.4 一个生活的比喻

想象你在安排一场派对。你有两个条件：

1. **AND（与）**：只有**同时**带了蛋糕和音乐的人才能参加。
   - 这很容易判断：画一条线，满足两个条件的人在一边。
2. **OR（或）**：**只要**带了蛋糕或音乐的人就能参加。
   - 这也很容易判断：同样是一条线的问题。
3. **XOR（异或）**：**只**带了蛋糕或**只**带了音乐的人可以参加，**两个都带或都不带**的人不能参加。
   - 这就麻烦了！你无法用简单的”是/否”一条线划分，因为符合条件的人在两个不同的角落。

这就是 XOR 问题的本质：它需要**组合判断**，而不是简单的线性加权。

13.3.5 17.2.5 解决方案的直觉

如果我们不能用一条线，那用什么？

答案是：**用两条线！**

```
        B
        ↑
  0     |     1

        |    /
        |   /
          /        → A
        |/
        /
  1     |     0
      /
     /
```

看！如果我们用**两条直线**形成一个”V”字形，就能把 XOR 的四个点完美分开：

1. 第一条线把左下角（0,1）分隔出来
2. 第二条线把右上角（1,1）分隔出来
3. 中间的”V”形区域就是我们想要的正类

但问题是：感知机只能画一条线。怎么办？

答案是：**把多个感知机组合起来！**

13.4 17.3 隐藏层的魔法：从 2D 到 3D 的空间变换

13.4.1 17.3.1 折叠空间的比喻

回到本章开头的折纸比喻。

想象 XOR 的四个点就像四个小磁铁，两个红色，两个蓝色，它们在平面上紧紧纠缠。单层感知机就像一把剪刀，只能在平面上剪直线——没用。

但如果我们能把纸**折起来**，让两个红色点跳到纸的上方，两个蓝色点留在下面，会怎样？

现在，我们只需要在垂直方向剪一刀（添加第三个维度），就能完美分开它们！

隐藏层做的就是这件事：**把数据从低维空间”折叠”到高维空间，让线性不可分变成线性可分**。

13.4.2 17.3.2 XOR 问题的多层解决方案

让我们看看如何用两层网络解决 XOR 问题：

```
输 入 层          隐 藏 层              输 出 层


  x      →
                h        →
  x      →
                                y       →  输 出
  x      →
                h        →
  x      →
```

这个网络有两个输入、两个隐藏神经元和一个输出。

关键问题是：**隐藏层做了什么？**

让我们给隐藏层神经元赋予特定的权重，看看它们学到了什么：

隐藏神经元 h 可以学会识别左下角的点 (0,1)：- 权重：w = 1, w = 1, 偏置 = -0.5 - 它实际上学会了 OR 逻辑：只要 x 或 x 有一个为 1，它就激活

隐藏神经元 h 可以学会识别右下角的点 (0,0) 和左上角的点 (1,1)：- 权重：w = -1, w = -1, 偏置 = 1.5 - 它实际上学会了 NAND 逻辑：只有当 x 和 x 不同时，它才激活

然后，**输出层**把这两个隐藏神经元的输出组合起来：- 当 h 激活但 h 不激活时（即 (0,1) 或 (1,0)），输出 1 - 其他情况输出 0

这正好就是 XOR!

13.4.3 17.3.3 空间变换的可视化

让我们更直观地看看隐藏层做了什么:

原始空间（输入层）:

```
        x
        ↑
  0     |     1

(0,0)   |   (1,1)
        |
              → x
        |
        |
(0,1)   |   (1,0)
  1     |     0
```

在原始空间中，你无法用一条线分开它们。

隐藏层变换后（3D 空间）:

隐藏层把每个点 (x , x) 映射到一个新的坐标 (h , h):

- (0,0) → (0, 1)
- (0,1) → (1, 1)
- (1,0) → (1, 1)
- (1,1) → (1, 0)

等等，(0,1) 和 (1,0) 都映射到了 (1,1)? 是的！这正是关键:

隐藏层把两个正类点”拉”到了一起，把两个负类点推到了另外的位置!

在新的表示空间中:

```
        h
        ↑
  1     |
(0,0)   |   (0.5,0.5)
        |
              → h
        |
        |
(1,1)   |
  1     |
        |   (1,1)
```

现在，用一条垂直的线（h = 0.5）就能轻松分开它们!

13.4.4 17.3.4 更深层的网络 = 更复杂的折叠

两层网络可以学习简单的非线性边界。三层呢？四层呢？

答案是：**每一层都可以看作一次新的空间变换**。

- 第一层：把原始输入空间折叠一次
- 第二层：在第一次折叠的基础上再折叠一次
- 第三层：再折叠一次…

这就像折纸艺术（Origami）。单层感知机是一张平展的纸。两层网络是一次折叠。三层网络是两次折叠。层数越多，你能创造的几何形状就越复杂！

理论上，一个足够深的神经网络可以**逼近任何函数**。这被称为**通用近似定理**（Universal Approximation Theorem）。

当然，这只是理论。在实践中，更深的网络也意味着更难训练。这就是深度学习的艺术——在表达能力和可训练性之间找到平衡。

13.5 17.4 前向传播：信号如何层层传递

13.5.1 17.4.1 神经网络的基本结构

一个标准的多层感知机（MLP, Multi-Layer Perceptron）通常包含：

1. **输入层**（Input Layer）：接收原始数据
2. **隐藏层**（Hidden Layer(s)）：进行特征变换
3. **输出层**（Output Layer）：产生最终预测

```
输 入 层        隐 藏 层 1        隐 藏 层 2        输 出 层

x      →
                       →
x      →       h                       →
               i       →       h
x      →       d               i       →     out      →  ŷ
               d       →       d
               e               d       →
               n               e
```

每一层的每个神经元都与下一层的每个神经元相连，这种结构称为**全连接层**（Fully Connected Layer）或**密集层**（Dense Layer）。

13.5.2 17.4.2 单个神经元的计算

回忆感知机的公式：

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

$$a = \sigma(z)$$

其中：- z 是**加权输入**（weighted input）- a 是**激活值**（activation）- σ 是**激活函数**（activation function）- $\mathbf{w}$ 是权重向量 - b 是偏置

在多层网络中，我们用上标表示层数，用下标表示神经元编号：

- $a_j^{[l]}$ 表示第 l 层第 j 个神经元的激活值
- $w_{jk}^{[l]}$ 表示从第 $l-1$ 层第 k 个神经元到第 l 层第 j 个神经元的权重
- $b_j^{[l]}$ 表示第 l 层第 j 个神经元的偏置

13.5.3 17.4.3 向量化：矩阵运算的威力

当网络变大时，逐个计算每个神经元会非常低效。这就是为什么我们需要**向量化**（Vectorization）。

假设第 $l-1$ 层有 $n^{[l-1]}$ 个神经元，第 l 层有 $n^{[l]}$ 个神经元。

我们可以把所有激活值堆叠成一个向量：

$$\mathbf{a}^{[l-1]} = \begin{bmatrix} a_1^{[l-1]} \\ a_2^{[l-1]} \\ \vdots \\ a_{n^{[l-1]}}^{[l-1]} \end{bmatrix} \in \mathbb{R}^{n^{[l-1]}}$$

把所有权重组织成一个矩阵：

$$\mathbf{W}^{[l]} = \begin{bmatrix} w_{11}^{[l]} & w_{12}^{[l]} & \cdots & w_{1,n^{[l-1]}}^{[l]} \\ w_{21}^{[l]} & w_{22}^{[l]} & \cdots & w_{2,n^{[l-1]}}^{[l]} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n^{[l]},1}^{[l]} & w_{n^{[l]},2}^{[l]} & \cdots & w_{n^{[l]},n^{[l-1]}}^{[l]} \end{bmatrix} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$$

把所有偏置堆叠成一个向量：

$$\mathbf{b}^{[l]} = \begin{bmatrix} b_1^{[l]} \\ b_2^{[l]} \\ \vdots \\ b_{n^{[l]}}^{[l]} \end{bmatrix} \in \mathbb{R}^{n^{[l]}}$$

现在，整个层的计算可以写成优雅的矩阵形式：

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{a}^{[l]} = \sigma(\mathbf{z}^{[l]})$$

这里，σ 是逐元素应用的激活函数。

13.5.4 17.4.4 完整的神经网络计算流程

让我们通过一个小例子来理解前向传播的完整流程。

例子：2-3-1 网络（2 输入，3 隐藏神经元，1 输出）

```
输 入 层        隐 藏 层          输 出 层


x      →
             h
x      →
             h            →
                                  y       →  ŷ
             h
```

假设输入是 $\mathbf{x} = \begin{bmatrix} 0.5 \\ 0.3 \end{bmatrix}$，我们来一步步计算。

第 1 层（输入层）：

$$\mathbf{a}^{[0]} = \mathbf{x} = \begin{bmatrix} 0.5 \\ 0.3 \end{bmatrix}$$

第 2 层（隐藏层）：

权重矩阵（随机初始化，仅作示例）：

$$\mathbf{W}^{[1]} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \end{bmatrix}, \quad \mathbf{b}^{[1]} = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix}$$

加权输入：

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{a}^{[0]} + \mathbf{b}^{[1]} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix}$$

$$= \begin{bmatrix} 0.1(0.5) + 0.2(0.3) + 0.1 \\ 0.3(0.5) + 0.4(0.3) + 0.2 \\ 0.5(0.5) + 0.6(0.3) + 0.3 \end{bmatrix} = \begin{bmatrix} 0.21 \\ 0.47 \\ 0.73 \end{bmatrix}$$

激活值（使用 sigmoid 函数）：

$$\mathbf{a}^{[1]} = \sigma(\mathbf{z}^{[1]}) = \begin{bmatrix} \sigma(0.21) \\ \sigma(0.47) \\ \sigma(0.73) \end{bmatrix} \approx \begin{bmatrix} 0.552 \\ 0.615 \\ 0.675 \end{bmatrix}$$

第 3 层（输出层）：

权重矩阵：

$$\mathbf{W}^{[2]} = \begin{bmatrix} 0.7 & 0.8 & 0.9 \end{bmatrix}, \quad \mathbf{b}^{[2]} = \begin{bmatrix} 0.1 \end{bmatrix}$$

加权输入：

$$z^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + b^{[2]} = 0.7(0.552) + 0.8(0.615) + 0.9(0.675) + 0.1 \approx 1.72$$

输出（使用 sigmoid）：

$$\hat{y} = a^{[2]} = \sigma(1.72) \approx 0.848$$

这就是前向传播的完整过程！信号从输入层流入，经过隐藏层的变换，最终从输出层流出。

13.5.5 17.4.5 批处理：同时计算多个样本

在实际应用中，我们通常需要同时处理**多个样本**（称为一个批量，batch）。

我们可以把所有样本堆叠成一个矩阵 $\mathbf{X} \in \mathbb{R}^{n^{[0]} \times m}$，其中 m 是样本数量。

每一列是一个样本：

$$\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \cdots & \mathbf{x}^{(m)} \\ | & | & & | \end{bmatrix}$$

然后，前向传播公式依然成立：

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]}\mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

注意这里的 $\mathbf{b}^{[l]}$ 会被**广播**（broadcast）到所有列。

这种批处理方式不仅更高效（可以利用矩阵运算的并行性），而且还能帮助优化算法更好地估计梯度。

13.6 17.5 反向传播直觉：误差如何倒着流

13.6.1 17.5.1 学习的本质：调整权重以减少误差

我们已经知道神经网络如何进行预测（前向传播）。但神经网络是如何**学习**的呢？

答案是：**通过比较预测和真实值，然后调整权重以减少误差**。

这个过程就像射箭：1. 你射出一箭（前向传播，做出预测）2. 看到箭落在靶子哪里（计算误差）3. 根据偏差调整姿势（反向传播，更新权重）4. 再次射箭（重复）

13.6.2 17.5.2 反向传播的核心思想

反向传播解决的核心问题是：**每个权重对最终误差的贡献是多少？**

想象一个水管系统：

```
水源 → 管道A → 管道B → 管道C → 出水口
```

如果出水口的水流太小，我们需要知道是哪个管道出了问题。

反向传播做的就是：**从出水口倒着追溯，计算每个管道对水流不足的责任**。

在神经网络中：- **水源** = 输入数据 - **管道** = 权重参数 - **水流** = 激活值 - **出水口** = 输出层 - **期望水流** = 真实标签

13.6.3 17.5.3 链式法则：反向传播的数学基础

反向传播之所以有效，是因为一个叫做**链式法则**（Chain Rule）的数学原理。

链式法则告诉我们：如果 $y = f(g(x))$，那么：

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

在神经网络中，损失函数 L 依赖于输出层的激活值 $\mathbf{a}^{[L]}$，输出层依赖于加权输入 $\mathbf{z}^{[L]}$，加权输入依赖于权重 $\mathbf{W}^{[L]}$ 和前一层的激活值 $\mathbf{a}^{[L-1]}$...

这就像一条链条！链式法则让我们可以**从后往前**一步步计算梯度。

13.6.4 17.5.4 误差信号的传递

让我们用一个简单的例子来理解误差如何反向流动。

考虑一个两层网络：

```
x → [Layer 1] → h → [Layer 2] → ŷ
```

假设我们计算出输出层的误差为 $\delta^{[2]}$（这个误差表示预测与真实值的差距）。

这个误差如何传递到第一层？

第一步：误差 $\delta^{[2]}$ 通过权重 $W^{[2]}$ 反向传播到隐藏层：

$$\delta^{[1]} = (W^{[2]})^T \delta^{[2]} \odot \sigma'(z^{[1]})$$

其中 $\odot$ 表示逐元素乘法，σ' 是激活函数的导数。

第二步：一旦我们有了每层的误差信号，就可以计算该层权重的梯度：

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T$$

这看起来有点抽象，让我们用一个生活化的比喻来理解。

13.6.5 17.5.5 打保龄球的比喻

想象你在玩保龄球，但这是一个非常特别的保龄球道：

```
你 → 滑道A → 滑道B → 球瓶
```

滑道 A 和滑道 B 都有可调节的坡度（就像神经网络的权重）。你的目标是让球击倒球瓶（做出正确预测）。

前向传播：你扔出球，球经过滑道 A，再经过滑道 B，最后击中球瓶。你发现球偏左了（有误差）。

反向传播： 1. 首先看最后一节滑道 B："球从这里出去时偏左了，我需要调整滑道 B 的坡度。"2. 然后看第一节滑道 A："球进入滑道 B 时的角度也有问题，部分原因是滑道 A 设置不当。我需要根据滑道 B 的反馈来调整滑道 A。"

关键洞察：**后面滑道的调整需求会告诉前面滑道应该如何调整**。

这就是反向传播：误差信号从输出层"倒着流"回输入层，每一层都根据后一层的反馈来调整自己的权重。

13.6.6 17.5.6 为什么叫"反向"传播？

前向传播时，信号从输入流向输出：

$$\mathbf{x} \to \mathbf{a}^{[1]} \to \mathbf{a}^{[2]} \to \cdots \to \mathbf{a}^{[L]} \to L$$

反向传播时，梯度从损失函数流向输入：

$$\frac{\partial L}{\partial \mathbf{a}^{[L]}} \to \frac{\partial L}{\partial \mathbf{z}^{[L]}} \to \frac{\partial L}{\partial \mathbf{W}^{[L]}} \to \frac{\partial L}{\partial \mathbf{a}^{[L-1]}} \to \cdots \to \frac{\partial L}{\partial \mathbf{W}^{[1]}}$$

就像水流可以双向流动——前向是信息流，反向是误差流。

13.6.7 17.5.7 本章只讲直觉，下章详细推导

在本章，我们只需要理解反向传播的**直觉**：

1. **误差从输出层开始**，衡量预测与真实值的差距
2. **误差信号反向流动**，通过权重传递到前一层
3. **每层根据接收到的误差信号**，计算自己权重的调整方向
4. **调整的大小取决于**：误差大小 + 激活函数的斜率 + 前一层的激活值

详细的数学推导（链式法则的完整应用、各种激活函数的导数、矩阵求导等）将在**下一章**中详细讲解。

现在，让我们用代码来实际体验这一切！

13.7 17.6 激活函数的必要性：没有非线性，多层 = 单层

13.7.1 17.6.1 一个惊人的事实

在深入代码之前，我们必须先解决一个关键问题：**如果没有激活函数会怎样？**

假设我们有一个两层的网络，但没有激活函数（或者使用线性激活函数，即 $a = z$）：

第一层：

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = \mathbf{z}^{[1]} \quad \text{（线性激活）}$$

第二层：

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$$

$$\hat{y} = \mathbf{z}^{[2]}$$

现在，让我们把这两层合并：

$$\begin{aligned}\hat{y} &= \mathbf{W}^{[2]}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]} \\ &= \mathbf{W}^{[2]}\mathbf{W}^{[1]}\mathbf{x} + \mathbf{W}^{[2]}\mathbf{b}^{[1]} + \mathbf{b}^{[2]} \\ &= \mathbf{W}'\mathbf{x} + \mathbf{b}'\end{aligned}$$

其中 $\mathbf{W}' = \mathbf{W}^{[2]}\mathbf{W}^{[1]}$，$\mathbf{b}' = \mathbf{W}^{[2]}\mathbf{b}^{[1]} + \mathbf{b}^{[2]}$。

惊人的结论：两层的线性网络等价于单层线性网络！

无论你把多少层线性变换堆叠在一起，最终都可以被合并成单层。这就是线性代数的基本性质。

13.7.2 17.6.2 非线性：神经网络的灵魂

这就是为什么我们需要**非线性激活函数**！

非线性激活函数打破了这种”可合并性”，使得每一层都能学习到真正新的、不可被前面层表示的特征。

常用的非线性激活函数包括：

1. Sigmoid（S 型函数）：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

输出范围 $(0, 1)$，适合概率输出。

2. Tanh（双曲正切）：

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

输出范围 (-1, 1)，数据中心化更好。

3. ReLU（整流线性单元）：

$$\mathrm{ReLU}(z) = \max(0, z)$$

计算简单，缓解梯度消失问题，目前最常用。

4. Leaky ReLU：

$$\mathrm{LeakyReLU}(z) = \max(\alpha z, z)$$

其中 α 是小常数（如 0.01），解决 ReLU 的”神经元死亡”问题。

13.7.3 17.6.3 激活函数的可视化比较

```
Sigmoid:                        Tanh:
  1 |     ____                   1 |     /
    |    /                          |    /
0.5 |__/                            0 |__/
    |                              -1|
  0 |_____→ z                         |_____→ z
    -5   0   5                      -5   0   5

ReLU:                           Leaky ReLU:
  ↑ |     /                         ↑ |     /
    |    /                            |    /
  0 |__/                            0 |__/
    |   /                             |\/
    |_/                               |
    |_____→ z                         |_____→ z
```

13.7.4 17.6.4 选择合适的激活函数

- **输出层：**
  - 二分类：Sigmoid
  - 多分类：Softmax
  - 回归：线性（无激活函数）
- **隐藏层：**
  - 首选：ReLU（简单、快速、效果好）
  - 如果 ReLU 导致太多”死亡神经元”：尝试 Leaky ReLU 或 ELU
  - 循环神经网络：Tanh 或 Sigmoid

记住：**非线性是深度学习的核心**。没有非线性激活函数，再深的网络也只是单层感知机的伪装。

13.8 17.7 从零实现 MLP 类

现在让我们用 Python 从零开始实现一个多层感知机！这将包括完整的正向传播、反向传播和训练过程。

```python
"""
第十七章：多层神经网络——从零实现MLP
《机器学习与深度学习：从小学生到大师》

本代码包含：
1. 完整的MLP类实现（前向传播 + 反向传播）
2. XOR问题的完整解决示例
3. 手写数字识别（简化版MNIST）
4. 丰富的可视化
"""

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
import warnings
warnings.filterwarnings('ignore')

# 设置中文显示
plt.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False


# ==============================================================================
# 第一部分：激活函数及其导数
# ==============================================================================

class Activations:
    """激活函数集合"""

    @staticmethod
    def sigmoid(z):
        """Sigmoid激活函数"""
        return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

    @staticmethod
    def sigmoid_derivative(z):
        """Sigmoid的导数"""
        a = Activations.sigmoid(z)
        return a * (1 - a)

    @staticmethod
    def relu(z):
        """ReLU激活函数"""
        return np.maximum(0, z)
```

```
    @staticmethod
    def relu_derivative(z):
        """ReLU的导数"""
        return (z > 0).astype(float)

    @staticmethod
    def tanh(z):
        """Tanh激活函数"""
        return np.tanh(z)

    @staticmethod
    def tanh_derivative(z):
        """Tanh的导数"""
        return 1 - np.tanh(z) ** 2

    @staticmethod
    def linear(z):
        """线性激活（无变换）"""
        return z

    @staticmethod
    def linear_derivative(z):
        """线性激活的导数"""
        return np.ones_like(z)

    @staticmethod
    def softmax(z):
        """Softmax激活函数（用于多分类输出层）"""
        # 数值稳定性处理
        exp_z = np.exp(z - np.max(z, axis=0, keepdims=True))
        return exp_z / np.sum(exp_z, axis=0, keepdims=True)


# ==============================================================================
# 第二部分：损失函数
# ==============================================================================

class LossFunctions:
    """损失函数集合"""

    @staticmethod
    def mse(y_true, y_pred):
        """均方误差"""
        return np.mean((y_true - y_pred) ** 2)

    @staticmethod
    def mse_derivative(y_true, y_pred):
```

```
        """MSE对预测的导数"""
        return -2 * (y_true - y_pred) / y_true.shape[1]

    @staticmethod
    def cross_entropy(y_true, y_pred):
        """交叉熵损失（带数值稳定性）"""
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        return -np.mean(np.sum(y_true * np.log(y_pred), axis=0))

    @staticmethod
    def cross_entropy_derivative(y_true, y_pred):
        """交叉熵 + Softmax的组合导数"""
        return y_pred - y_true


# ==============================================================================
# 第三部分：多层感知机（MLP）类
# ==============================================================================

class MLP:
    """
    多层感知机（Multilayer Perceptron）

    参数：
        layer_sizes: 列表，如 [2, 4, 1] 表示输入2维，隐藏层4维，输出1维
        activations: 列表，每层的激活函数名称，如 ['relu', 'sigmoid']
        loss_function: 损失函数名称 ('mse' 或 'cross_entropy')
        learning_rate: 学习率
        random_seed: 随机种子（保证可重复）
    """

    def __init__(self, layer_sizes, activations, loss_function='mse',
                 learning_rate=0.1, random_seed=42):
        self.layer_sizes = layer_sizes
        self.num_layers = len(layer_sizes)
        self.learning_rate = learning_rate
        self.loss_name = loss_function

        # 设置随机种子
        np.random.seed(random_seed)

        # 初始化权重和偏置
        self._initialize_parameters()

        # 设置激活函数
        self._setup_activations(activations)
```

```
        # 设置损失函数
        self._setup_loss_function()

        # 存储训练历史
        self.loss_history = []

    def _initialize_parameters(self):
        """
        初始化网络参数
        使用Xavier/Glorot初始化，有助于梯度稳定流动
        """
        self.parameters = {}
        self.gradients = {}

        for l in range(1, self.num_layers):
            # Xavier初始化：权重从均值为0，方差为 1/n_in 的正态分布采样
            n_in = self.layer_sizes[l-1]
            n_out = self.layer_sizes[l]
            self.parameters[f'W{l}'] = np.random.randn(n_out, n_in) * np.sqrt(2.0 /
                (n_in + n_out))
            self.parameters[f'b{l}'] = np.zeros((n_out, 1))

    def _setup_activations(self, activations):
        """设置每层的激活函数"""
        self.activations = []
        self.activation_derivatives = []

        act_map = {
            'sigmoid': (Activations.sigmoid, Activations.sigmoid_derivative),
            'relu': (Activations.relu, Activations.relu_derivative),
            'tanh': (Activations.tanh, Activations.tanh_derivative),
            'linear': (Activations.linear, Activations.linear_derivative),
            'softmax': (Activations.softmax, None)  #
                softmax通常与cross_entropy配合使用
        }

        for act_name in activations:
            if act_name not in act_map:
                raise ValueError(f"未知的激活函数：{act_name}")
            self.activations.append(act_map[act_name][0])
            self.activation_derivatives.append(act_map[act_name][1])

    def _setup_loss_function(self):
        """设置损失函数"""
        if self.loss_name == 'mse':
            self.loss_fn = LossFunctions.mse
```

```
            self.loss_derivative = LossFunctions.mse_derivative
        elif self.loss_name == 'cross_entropy':
            self.loss_fn = LossFunctions.cross_entropy
            self.loss_derivative = LossFunctions.cross_entropy_derivative
        else:
            raise ValueError(f"未知的损失函数: {self.loss_name}")

    def forward(self, X):
        """
        前向传播

        参数:
            X: 输入数据, 形状为 (n_features, n_samples)

        返回:
            网络输出
        """
        # 存储每层的激活值和加权输入 (用于反向传播)
        self.cache = {'A0': X}

        A = X
        for l in range(1, self.num_layers):
            W = self.parameters[f'W{l}']
            b = self.parameters[f'b{l}']

            # 计算加权输入 Z = W·A + b
            Z = np.dot(W, A) + b
            self.cache[f'Z{l}'] = Z

            # 应用激活函数
            A = self.activations[l-1](Z)
            self.cache[f'A{l}'] = A

        return A

    def backward(self, Y):
        """
        反向传播

        参数:
            Y: 真实标签, 形状为 (n_outputs, n_samples)
        """
        m = Y.shape[1]  # 样本数量
        L = self.num_layers - 1  # 最后一层的索引

        # 获取最后一层的输出
        A_L = self.cache[f'A{L}']
```

```
        Z_L = self.cache[f'Z{L}']

        # 计算输出层的误差（delta）
        if self.loss_name == 'cross_entropy' and self.activations[-1] ==
            Activations.softmax:
            # 对于Softmax + CrossEntropy的组合，导数简化为 A - Y
            dZ = A_L - Y
        else:
            # 一般情况：损失函数导数 * 激活函数导数
            dA = self.loss_derivative(Y, A_L)
            dZ = dA * self.activation_derivatives[-1](Z_L)

        # 从最后一层向前传播误差
        for l in range(L, 0, -1):
            A_prev = self.cache[f'A{l-1}']

            # 计算该层的梯度
            self.gradients[f'dW{l}'] = np.dot(dZ, A_prev.T) / m
            self.gradients[f'db{l}'] = np.sum(dZ, axis=1, keepdims=True) / m

            # 如果不是第一层，计算前一层的误差
            if l > 1:
                W = self.parameters[f'W{l}']
                dA_prev = np.dot(W.T, dZ)
                Z_prev = self.cache[f'Z{l-1}']
                dZ = dA_prev * self.activation_derivatives[l-2](Z_prev)

    def update_parameters(self):
        """使用梯度下降更新参数"""
        for l in range(1, self.num_layers):
            self.parameters[f'W{l}'] -= self.learning_rate *
                self.gradients[f'dW{l}']
            self.parameters[f'b{l}'] -= self.learning_rate *
                self.gradients[f'db{l}']

    def train(self, X, Y, epochs=1000, batch_size=None, verbose=True,
        print_every=100):
        """
        训练网络

        参数:
            X: 输入数据 (n_features, n_samples)
            Y: 标签 (n_outputs, n_samples)
            epochs: 训练轮数
            batch_size: 批量大小（None表示使用全部数据）
            verbose: 是否打印进度
            print_every: 每隔多少轮打印一次
```

```
        """
        m = X.shape[1]  # 总样本数

        if batch_size is None:
            batch_size = m

        num_batches = (m + batch_size - 1) // batch_size

        for epoch in range(epochs):
            epoch_loss = 0

            # 随机打乱数据
            indices = np.random.permutation(m)
            X_shuffled = X[:, indices]
            Y_shuffled = Y[:, indices]

            for i in range(num_batches):
                start_idx = i * batch_size
                end_idx = min((i + 1) * batch_size, m)

                X_batch = X_shuffled[:, start_idx:end_idx]
                Y_batch = Y_shuffled[:, start_idx:end_idx]

                # 前向传播
                Y_pred = self.forward(X_batch)

                # 计算损失
                loss = self.loss_fn(Y_batch, Y_pred)
                epoch_loss += loss * (end_idx - start_idx) / m

                # 反向传播
                self.backward(Y_batch)

                # 更新参数
                self.update_parameters()

            self.loss_history.append(epoch_loss)

            if verbose and (epoch + 1) % print_every == 0:
                print(f"Epoch {epoch+1}/{epochs}, Loss: {epoch_loss:.6f}")

        if verbose:
            print(f"\n训练完成！ 最终损失: {self.loss_history[-1]:.6f}")

    def predict(self, X):
        """
        预测
```

```
        参数:
            X: 输入数据

        返回:
            预测结果
        """
        return self.forward(X)

    def predict_class(self, X):
        """
        预测类别 (用于分类问题)

        返回类别索引
        """
        probs = self.predict(X)
        return np.argmax(probs, axis=0)

    def score(self, X, Y):
        """
        计算准确率 (分类问题)

        参数:
            X: 输入数据
            Y: one-hot编码的标签
        """
        predictions = self.predict_class(X)
        true_labels = np.argmax(Y, axis=0)
        return np.mean(predictions == true_labels)


# ============================================================================
# 第四部分: XOR问题完整解决示例
# ============================================================================

def solve_xor_problem():
    """
    使用MLP解决XOR问题
    这是神经网络历史上的经典问题!
    """
    print("=" * 60)
    print("XOR问题: 多层神经网络的Hello World")
    print("=" * 60)

    # XOR数据集
    X = np.array([[0, 0, 1, 1],
                  [0, 1, 0, 1]])
```

```
    Y = np.array([[0, 1, 1, 0]])  # XOR真值表

    print("\n【数据集】")
    print("输入 X:")
    print("  (0,0) → 0")
    print("  (0,1) → 1")
    print("  (1,0) → 1")
    print("  (1,1) → 0")
    print("\n这是一个线性不可分问题，单层感知机无法解决！")

    # 创建MLP：2输入 → 4隐藏 → 1输出
    print("\n【网络结构】")
    print("  输入层：2个神经元")
    print("  隐藏层：4个神经元 (ReLU激活)")
    print("  输出层：1个神经元 (Sigmoid激活)")
    print("  损失函数：MSE")
    print("  学习率：0.5")

    mlp = MLP(
        layer_sizes=[2, 4, 1],
        activations=['relu', 'sigmoid'],
        loss_function='mse',
        learning_rate=0.5,
        random_seed=42
    )

    # 训练
    print("\n【训练过程】")
    mlp.train(X, Y, epochs=2000, print_every=200)

    # 测试
    print("\n【测试结果】")
    predictions = mlp.predict(X)

    for i in range(4):
        x1, x2 = X[0, i], X[1, i]
        true_y = Y[0, i]
        pred_y = predictions[0, i]
        print(f"  输入：({x1}, {x2}) | 预测：{pred_y:.4f} | 真实：{true_y} | 判断：
            {' ' if abs(pred_y - true_y) < 0.5 else ' '}")

    # 可视化
    visualize_xor(mlp, X, Y)

    return mlp
```

```python
def visualize_xor(mlp, X, Y):
    """可视化XOR问题的决策边界"""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # 左图：决策边界
    ax1 = axes[0]

    # 创建网格
    x_min, x_max = -0.5, 1.5
    y_min, y_max = -0.5, 1.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                         np.linspace(y_min, y_max, 200))

    # 预测网格点的类别
    grid_points = np.c_[xx.ravel(), yy.ravel()].T
    Z = mlp.predict(grid_points)
    Z = Z.reshape(xx.shape)

    # 绘制决策边界
    contour = ax1.contourf(xx, yy, Z, levels=50, cmap='RdYlBu', alpha=0.6)
    ax1.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2)

    # 绘制数据点
    for i in range(X.shape[1]):
        color = 'blue' if Y[0, i] == 0 else 'red'
        marker = 'o' if Y[0, i] == 0 else 's'
        ax1.scatter(X[0, i], X[1, i], c=color, marker=marker, s=200,
                    edgecolors='black', linewidth=2, zorder=5)

    ax1.set_xlabel('输入 1', fontsize=12)
    ax1.set_ylabel('输入 2', fontsize=12)
    ax1.set_title('XOR问题的决策边界\n（黑色线表示分类边界）', fontsize=14)
    ax1.set_xlim(x_min, x_max)
    ax1.set_ylim(y_min, y_max)
    plt.colorbar(contour, ax=ax1, label='输出概率')

    # 右图：损失曲线
    ax2 = axes[1]
    ax2.plot(mlp.loss_history, linewidth=2, color='purple')
    ax2.set_xlabel('训练轮次', fontsize=12)
    ax2.set_ylabel('损失 (MSE)', fontsize=12)
    ax2.set_title('训练过程中的损失下降', fontsize=14)
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('xor_solution.png', dpi=150, bbox_inches='tight')
    plt.show()
```

```
    print("\n可视化已保存为 'xor_solution.png'")


# ============================================================================
# 第五部分：手写数字识别（简化版MNIST）
# ============================================================================

def load_digits_dataset():
    """
    加载手写数字数据集（sklearn内置的简化版MNIST）
    """
    print("\n" + "=" * 60)
    print("手写数字识别：MLP实战")
    print("=" * 60)

    # 加载数据
    digits = load_digits()
    X = digits.data  # (1797, 64) - 8x8像素的图像展平
    y = digits.target  # (1797,) - 0-9的数字标签

    print(f"\n【数据集信息】")
    print(f"  总样本数：{X.shape[0]}")
    print(f"  特征维度：{X.shape[1]} (8×8像素)")
    print(f"  类别数：10 (数字0-9)")

    # 划分训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    print(f"  训练集：{X_train.shape[0]} 样本")
    print(f"  测试集：{X_test.shape[0]} 样本")

    # 标准化
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    # 转置以匹配我们的MLP接口 (n_features, n_samples)
    X_train = X_train.T
    X_test = X_test.T

    # One-hot编码标签
    encoder = OneHotEncoder(sparse_output=False)
    Y_train = encoder.fit_transform(y_train.reshape(-1, 1)).T
    Y_test = encoder.transform(y_test.reshape(-1, 1)).T
```

```
    return X_train, X_test, Y_train, Y_test, y_train, y_test, digits


def train_digit_classifier():
    """训练手写数字分类器"""
    # 加载数据
    X_train, X_test, Y_train, Y_test, y_train, y_test, digits =
        load_digits_dataset()

    # 创建MLP
    print("\n【网络结构】")
    print("  输入层: 64个神经元 (8×8图像)")
    print("  隐藏层1: 128个神经元 (ReLU)")
    print("  隐藏层2: 64个神经元 (ReLU)")
    print("  输出层: 10个神经元 (Softmax)")
    print("  损失函数: 交叉熵")
    print("  学习率: 0.1")
    print("  批量大小: 32")

    mlp = MLP(
        layer_sizes=[64, 128, 64, 10],
        activations=['relu', 'relu', 'softmax'],
        loss_function='cross_entropy',
        learning_rate=0.1,
        random_seed=42
    )

    # 训练
    print("\n【训练过程】")
    mlp.train(X_train, Y_train, epochs=100, batch_size=32, print_every=10)

    # 评估
    train_acc = mlp.score(X_train, Y_train)
    test_acc = mlp.score(X_test, Y_test)

    print(f"\n【评估结果】")
    print(f"  训练集准确率: {train_acc*100:.2f}%")
    print(f"  测试集准确率: {test_acc*100:.2f}%")

    # 可视化结果
    visualize_digits_results(mlp, X_test, y_test, digits)

    return mlp


def visualize_digits_results(mlp, X_test, y_test, digits):
```

```
    """可视化手写数字识别结果"""
    fig, axes = plt.subplots(2, 2, figsize=(14, 12))

    # 1. 损失曲线
    ax1 = axes[0, 0]
    ax1.plot(mlp.loss_history, linewidth=2, color='blue')
    ax1.set_xlabel('训练轮次', fontsize=12)
    ax1.set_ylabel('交叉熵损失', fontsize=12)
    ax1.set_title('训练损失曲线', fontsize=14)
    ax1.grid(True, alpha=0.3)

    # 2. 随机样本预测展示
    ax2 = axes[0, 1]

    # 随机选择16个测试样本
    n_samples = 16
    indices = np.random.choice(X_test.shape[1], n_samples, replace=False)

    fig2, sample_axes = plt.subplots(4, 4, figsize=(10, 10))
    sample_axes = sample_axes.flatten()

    for i, idx in enumerate(indices):
        img = X_test[:, idx].reshape(8, 8)
        pred = mlp.predict_class(X_test[:, idx:idx+1])[0]
        true = y_test[idx]

        sample_axes[i].imshow(img, cmap='gray')
        color = 'green' if pred == true else 'red'
        sample_axes[i].set_title(f'预测: {pred}\n真实: {true}', color=color,
            fontsize=10)
        sample_axes[i].axis('off')

    plt.suptitle('随机测试样本预测结果 (绿色=正确, 红色=错误)', fontsize=14)
    plt.tight_layout()
    plt.savefig('digit_predictions.png', dpi=150, bbox_inches='tight')
    plt.close()

    # 重新使用原来的axes
    predictions = mlp.predict(X_test)
    pred_classes = np.argmax(predictions, axis=0)

    # 3. 混淆矩阵
    ax3 = axes[1, 0]
    confusion = np.zeros((10, 10), dtype=int)
    for true, pred in zip(y_test, pred_classes):
        confusion[true, pred] += 1
```

```
im = ax3.imshow(confusion, cmap='Blues')
ax3.set_xlabel('预测标签', fontsize=12)
ax3.set_ylabel('真实标签', fontsize=12)
ax3.set_title('混淆矩阵', fontsize=14)
ax3.set_xticks(range(10))
ax3.set_yticks(range(10))

# 添加数值标注
for i in range(10):
    for j in range(10):
        text = ax3.text(j, i, confusion[i, j], ha="center", va="center",
                        color="white" if confusion[i, j] > confusion.max()/2
                            else "black",
                        fontsize=9)

plt.colorbar(im, ax=ax3)

# 4. 每个数字的准确率
ax4 = axes[1, 1]
digit_accuracy = []
for digit in range(10):
    mask = y_test == digit
    acc = np.mean(pred_classes[mask] == digit)
    digit_accuracy.append(acc)

bars = ax4.bar(range(10), digit_accuracy, color='steelblue', edgecolor='black')
ax4.set_xlabel('数字', fontsize=12)
ax4.set_ylabel('准确率', fontsize=12)
ax4.set_title('每个数字的分类准确率', fontsize=14)
ax4.set_xticks(range(10))
ax4.set_ylim([0, 1])
ax4.grid(True, alpha=0.3, axis='y')

# 在柱状图上添加数值
for bar, acc in zip(bars, digit_accuracy):
    height = bar.get_height()
    ax4.text(bar.get_x() + bar.get_width()/2., height,
             f'{acc*100:.1f}%', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.savefig('digits_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n可视化已保存:")
print("  - digit_predictions.png: 随机样本预测")
print("  - digits_analysis.png: 综合分析")
```

```python
# ============================================================================
# 第六部分：隐藏层激活可视化
# ============================================================================

def visualize_hidden_activations():
    """
    可视化隐藏层学到的特征
    展示网络如何将输入数据映射到新的表示空间
    """
    print("\n" + "=" * 60)
    print("隐藏层激活可视化")
    print("=" * 60)

    # 创建一个简单的分类问题（同心圆）
    np.random.seed(42)
    n_samples = 400

    # 生成两个同心圆
    theta = np.random.uniform(0, 2*np.pi, n_samples)
    r_inner = np.random.normal(2, 0.3, n_samples//2)
    r_outer = np.random.normal(4, 0.3, n_samples//2)

    X_inner = np.column_stack([r_inner * np.cos(theta[:n_samples//2]),
                               r_inner * np.sin(theta[:n_samples//2])])
    X_outer = np.column_stack([r_outer * np.cos(theta[n_samples//2:]),
                               r_outer * np.sin(theta[n_samples//2:])])

    X = np.vstack([X_inner, X_outer]).T
    Y = np.hstack([np.zeros(n_samples//2), np.ones(n_samples//2)]).reshape(1, -1)

    print("\n【任务】分离两个同心圆（非线性可分问题）")

    # 创建MLP
    mlp = MLP(
        layer_sizes=[2, 8, 4, 1],
        activations=['tanh', 'tanh', 'sigmoid'],
        loss_function='mse',
        learning_rate=0.5,
        random_seed=42
    )

    print("\n【网络结构】2 → 8 → 4 → 1")
    print("【训练】500轮...")
    mlp.train(X, Y, epochs=500, print_every=50)

    # 可视化
```

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# 1. 原始数据
ax1 = axes[0, 0]
ax1.scatter(X[0, :200], X[1, :200], c='blue', label='Class 0', alpha=0.6)
ax1.scatter(X[0, 200:], X[1, 200:], c='red', label='Class 1', alpha=0.6)
ax1.set_title('原始输入空间', fontsize=14)
ax1.set_xlabel('x ')
ax1.set_ylabel('x ')
ax1.legend()
ax1.set_aspect('equal')

# 2. 决策边界
ax2 = axes[0, 1]
x_min, x_max = X[0].min() - 1, X[0].max() + 1
y_min, y_max = X[1].min() - 1, X[1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))
grid = np.c_[xx.ravel(), yy.ravel()].T
Z = mlp.predict(grid).reshape(xx.shape)

ax2.contourf(xx, yy, Z, levels=50, cmap='RdYlBu', alpha=0.6)
ax2.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2)
ax2.scatter(X[0, :200], X[1, :200], c='blue', alpha=0.6, edgecolors='white')
ax2.scatter(X[0, 200:], X[1, 200:], c='red', alpha=0.6, edgecolors='white')
ax2.set_title('学习到的决策边界', fontsize=14)
ax2.set_xlabel('x ')
ax2.set_ylabel('x ')
ax2.set_aspect('equal')

# 3. 损失曲线
ax3 = axes[0, 2]
ax3.plot(mlp.loss_history, linewidth=2, color='purple')
ax3.set_title('损失下降曲线', fontsize=14)
ax3.set_xlabel('轮次')
ax3.set_ylabel('MSE损失')
ax3.grid(True, alpha=0.3)

# 4-6. 隐藏层激活可视化
# 第一层隐藏层激活
A1 = mlp.cache['A1']
ax4 = axes[1, 0]

# 使用PCA降到2D进行可视化
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
A1_pca = pca.fit_transform(A1.T)
```

```
    ax4.scatter(A1_pca[:200, 0], A1_pca[:200, 1], c='blue', alpha=0.6, label='Class
        0')
    ax4.scatter(A1_pca[200:, 0], A1_pca[200:, 1], c='red', alpha=0.6, label='Class
        1')
    ax4.set_title(f'第一层隐藏层激活\n(PCA投影, 解释方差:
        {sum(pca.explained_variance_ratio_)*100:.1f}%)', fontsize=14)
    ax4.set_xlabel('PC1')
    ax4.set_ylabel('PC2')
    ax4.legend()

    # 第二层隐藏层激活
    A2 = mlp.cache['A2']
    ax5 = axes[1, 1]

    # 对于4维, 我们可以展示所有两两组合
    ax5.scatter(A2[0, :200], A2[1, :200], c='blue', alpha=0.6, label='Class 0')
    ax5.scatter(A2[0, 200:], A2[1, 200:], c='red', alpha=0.6, label='Class 1')
    ax5.set_title('第二层隐藏层激活\n(维度1 vs 维度2)', fontsize=14)
    ax5.set_xlabel('激活值 1')
    ax5.set_ylabel('激活值 2')
    ax5.legend()

    # 最后一层输出
    ax6 = axes[1, 2]
    output = mlp.predict(X).flatten()
    ax6.hist(output[:200], bins=30, alpha=0.6, color='blue', label='Class 0')
    ax6.hist(output[200:], bins=30, alpha=0.6, color='red', label='Class 1')
    ax6.axvline(x=0.5, color='black', linestyle='--', linewidth=2, label='决策边界')
    ax6.set_title('输出层概率分布', fontsize=14)
    ax6.set_xlabel('预测概率')
    ax6.set_ylabel('样本数')
    ax6.legend()

    plt.suptitle('隐藏层如何将非线性可分问题转换为线性可分', fontsize=16, y=1.02)
    plt.tight_layout()
    plt.savefig('hidden_activations.png', dpi=150, bbox_inches='tight')
    plt.show()

    print("\n可视化已保存为 'hidden_activations.png'")
    print("观察隐藏层激活如何将同心圆数据映射到可分的空间!")


# ==============================================================================
# 第七部分: 网络容量与参数计算
# ==============================================================================
```

```python
def analyze_network_capacity():
    """
    分析不同网络结构的参数数量和容量
    """
    print("\n" + "=" * 60)
    print("神经网络容量分析")
    print("=" * 60)

    architectures = [
        ([2, 4, 1], "简单XOR网络"),
        ([64, 128, 64, 10], "手写数字分类器"),
        ([784, 256, 128, 64, 10], "标准MNIST网络"),
        ([100, 200, 200, 200, 100], "深度特征提取器"),
    ]

    print("\n【不同网络结构的参数统计】")
    print("-" * 60)
    print(f"{'结构':<25} {'描述':<20} {'参数量':<15}")
    print("-" * 60)

    for arch, desc in architectures:
        # 计算参数数量
        total_params = 0
        for i in range(len(arch) - 1):
            # 权重 + 偏置
            layer_params = arch[i] * arch[i+1] + arch[i+1]
            total_params += layer_params

        arch_str = " → ".join(map(str, arch))
        print(f"{arch_str:<25} {desc:<20} {total_params:<15,}")

    print("-" * 60)

    # 参数数量计算公式说明
    print("\n【参数数量计算公式】")
    print("对于从层 l-1 到层 l 的连接:")
    print("  权重数量 = n^(l) × n^(l-1)")
    print("  偏置数量 = n^(l)")
    print("  该层总参数 = n^(l) × n^(l-1) + n^(l) = n^(l) × (n^(l-1) + 1)")
    print("\n其中 n^(l) 表示第 l 层的神经元数量")

    # 可视化参数分布
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # 示例网络
    example_arch = [784, 256, 128, 64, 10]
    layer_names = ['输入→隐藏1', '隐藏1→隐藏2', '隐藏2→隐藏3', '隐藏3→输出']
```

```
    weight_counts = []
    bias_counts = []

    for i in range(len(example_arch) - 1):
        weight_counts.append(example_arch[i] * example_arch[i+1])
        bias_counts.append(example_arch[i+1])

    ax1 = axes[0]
    x = np.arange(len(layer_names))
    width = 0.35

    bars1 = ax1.bar(x - width/2, weight_counts, width, label='权重',
        color='steelblue')
    bars2 = ax1.bar(x + width/2, bias_counts, width, label='偏置', color='coral')

    ax1.set_ylabel('参数数量', fontsize=12)
    ax1.set_title('标准MNIST网络各层参数分布', fontsize=14)
    ax1.set_xticks(x)
    ax1.set_xticklabels(layer_names, rotation=15, ha='right')
    ax1.legend()
    ax1.set_yscale('log')
    ax1.grid(True, alpha=0.3, axis='y')

    # 总参数量对比
    ax2 = axes[1]
    total_params_per_arch = []
    arch_labels = []

    for arch, desc in architectures:
        total = sum(arch[i] * arch[i+1] + arch[i+1] for i in range(len(arch)-1))
        total_params_per_arch.append(total)
        arch_labels.append(desc)

    colors = ['#3498db', '#2ecc71', '#e74c3c', '#f39c12']
    bars = ax2.barh(arch_labels, total_params_per_arch, color=colors,
        edgecolor='black')
    ax2.set_xlabel('总参数数量（对数尺度）', fontsize=12)
    ax2.set_title('不同网络结构容量对比', fontsize=14)
    ax2.set_xscale('log')
    ax2.grid(True, alpha=0.3, axis='x')

    # 添加数值标注
    for bar, val in zip(bars, total_params_per_arch):
        ax2.text(val, bar.get_y() + bar.get_height()/2,
                 f' {val:,}', va='center', fontsize=10)

    plt.tight_layout()
```

```
    plt.savefig('network_capacity.png', dpi=150, bbox_inches='tight')
    plt.show()

    print("\n可视化已保存为 'network_capacity.png'")


# ============================================================================
# 第八部分：主程序
# ============================================================================

def main():
    """
    主程序：运行所有示例
    """
    print("\n" + "=" * 70)
    print("    第十七章：多层神经网络——从零实现MLP")
    print("    《机器学习与深度学习：从小学生到大师》")
    print("=" * 70)

    # 1. XOR问题（神经网络的Hello World）
    solve_xor_problem()

    # 2. 手写数字识别
    train_digit_classifier()

    # 3. 隐藏层激活可视化
    visualize_hidden_activations()

    # 4. 网络容量分析
    analyze_network_capacity()

    print("\n" + "=" * 70)
    print("    所有示例运行完成！")
    print("    生成的可视化文件:")
    print("      - xor_solution.png")
    print("      - digit_predictions.png")
    print("      - digits_analysis.png")
    print("      - hidden_activations.png")
    print("      - network_capacity.png")
    print("=" * 70)


if __name__ == "__main__":
    main()
```

13.9 17.8 数学推导详解

13.9.1 17.8.1 前向传播的矩阵运算

让我们更详细地推导前向传播的矩阵形式。

第 l 层的计算：

给定第 $l-1$ 层的激活值矩阵 $\mathbf{A}^{[l-1]} \in \mathbb{R}^{n^{[l-1]} \times m}$，其中 m 是批量大小。

权重矩阵 $\mathbf{W}^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$ 的每一行对应第 l 层的一个神经元，每一列对应第 $l-1$ 层的一个神经元。

偏置向量 $\mathbf{b}^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}$ 会被广播到所有样本。

加权输入的计算：

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]}\mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

展开来看：

$$\underbrace{\begin{bmatrix} z_{11} & z_{12} & \cdots & z_{1m} \\ z_{21} & z_{22} & \cdots & z_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ z_{n^{[l]},1} & z_{n^{[l]},2} & \cdots & z_{n^{[l]},m} \end{bmatrix}}_{\mathbf{Z}^{[l]}} = \underbrace{\begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1,n^{[l-1]}} \\ w_{21} & w_{22} & \cdots & w_{2,n^{[l-1]}} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n^{[l]},1} & w_{n^{[l]},2} & \cdots & w_{n^{[l]},n^{[l-1]}} \end{bmatrix}}_{\mathbf{W}^{[l]}} \underbrace{\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n^{[l-1]},1} & a_{n^{[l-1]},2} & \cdots & a_{n^{[l-1]},m} \end{bmatrix}}_{\mathbf{A}^{[l-1]}} + \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{n^{[l]}} \end{bmatrix}}_{\mathbf{b}^{[l]}}$$

其中，z_{ji} 表示第 l 层第 j 个神经元对第 i 个样本的加权输入。

激活值的计算：

$$\mathbf{A}^{[l]} = \sigma(\mathbf{Z}^{[l]})$$

这意味着 $\mathbf{A}^{[l]}_{ji} = \sigma(\mathbf{Z}^{[l]}_{ji})$，即激活函数逐元素应用。

13.9.2 17.8.2 损失函数对输出的梯度（直觉）

理解梯度的关键在于：**梯度告诉我们，如果稍微改变某个值，损失会如何变化**。

均方误差（MSE）：

$$L = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

对于单个样本，损失对预测值的梯度：

$$\frac{\partial L}{\partial \hat{y}} = -2(y - \hat{y})$$

直觉解释： - 如果预测值比真实值小（$\hat{y} < y$），梯度为**负**，意味着我们需要**增大**预测值 - 如果预测值比真实值大（$\hat{y} > y$），梯度为**正**，意味着我们需要**减小**预测值 - 梯度的**大小**告诉我们误差有多大

交叉熵损失：

对于二分类（配合 Sigmoid）：

$$L = -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$$

梯度：

$$\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$$

当配合 Sigmoid 的导数 $\hat{y}(1-\hat{y})$ 时，整体梯度简化为：

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

这个简洁的形式是交叉熵损失被广泛使用的关键原因。

13.9.3 17.8.3 参数数量计算公式

对于一个层数为 L 的网络，总参数数量可以通过以下公式计算：

$$\text{总参数} = \sum_{l=1}^{L} \left(n^{[l]} \times n^{[l-1]} + n^{[l]} \right)$$

其中：- $n^{[l]} \times n^{[l-1]}$ 是权重参数数量 - $n^{[l]}$ 是偏置参数数量

例子：

一个 [784, 256, 128, 10] 的网络（如 MNIST 分类器）：

- 第 1 层（输入 → 隐藏 1）：$256 \times 784 + 256 = 200,960$
- 第 2 层（隐藏 1→ 隐藏 2）：$128 \times 256 + 128 = 32,896$
- 第 3 层（隐藏 2→ 输出）：$10 \times 128 + 10 = 1,290$

总计：235,146 个参数

这个公式帮助我们：1. **估算内存需求**：每个参数通常需要 4 字节（float32）2. **评估模型复杂度**：参数越多，模型表达能力越强，但也越容易过拟合 3. **计算训练时间**：参数越多，每次前向/反向传播的计算量越大

13.10 17.9 练习题

13.10.1 基础题（3 道）

练习 17.1：手动计算前向传播

考虑一个 2-2-1 的神经网络，参数如下：

$$\mathbf{W}^{[1]} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}, \quad \mathbf{b}^{[1]} = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix}$$

$$\mathbf{W}^{[2]} = \begin{bmatrix} 0.5 & 0.6 \end{bmatrix}, \quad \mathbf{b}^{[2]} = \begin{bmatrix} 0.3 \end{bmatrix}$$

激活函数：隐藏层使用 ReLU，输出层使用 Sigmoid。

输入：$\mathbf{x} = \begin{bmatrix} 0.5 \\ 0.3 \end{bmatrix}$

问题： 1. 计算隐藏层的加权输入 $\mathbf{z}^{[1]}$ 和激活值 $\mathbf{a}^{[1]}$ 2. 计算输出 $\hat{y}$ 3. 如果真实值 $y = 1$，计算 MSE 损失

点击查看答案

解答：

1. 隐藏层计算：

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} = \begin{bmatrix} 0.1(0.5) + 0.2(0.3) + 0.1 \\ 0.3(0.5) + 0.4(0.3) + 0.2 \end{bmatrix} = \begin{bmatrix} 0.21 \\ 0.47 \end{bmatrix}$$

$$\mathbf{a}^{[1]} = \text{ReLU}(\mathbf{z}^{[1]}) = \begin{bmatrix} 0.21 \\ 0.47 \end{bmatrix}$$

2. 输出层计算：

$$z^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + b^{[2]} = 0.5(0.21) + 0.6(0.47) + 0.3 = 0.687$$

$$\hat{y} = \sigma(0.687) = \frac{1}{1 + e^{-0.687}} \approx 0.665$$

3. MSE 损失：

$$L = (1 - 0.665)^2 = 0.112$$

练习 17.2：理解 XOR 的线性不可分性

在二维平面上画出以下四个点：(0,0)、(0,1)、(1,0)、(1,1)。用 ○ 标记输出为 0 的点，用 ● 标记输出为 1 的点（XOR 标签）。

问题： 1. 证明不存在一条直线能将 ○ 和 ● 完全分开 2. 画出两条直线，形成一个”V”形决策边界，将 XOR 的四个点分开 3. 解释为什么两条直线可以分开，而一条直线不行

点击查看答案

解答：

1. **证明线性不可分：**

   假设存在一条直线 $w_1x_1 + w_2x_2 + b = 0$ 能分开这四个点。

   XOR 的约束条件：

   - (0,0) $\to$ 0：要求 $b \leq 0$（或 < 0，取决于不等式方向）
   - (1,1) $\to$ 0：要求 $w_1 + w_2 + b \leq 0$
   - (0,1) $\to$ 1：要求 $w_2 + b > 0$
   - (1,0) $\to$ 1：要求 $w_1 + b > 0$

   从后两个条件：$w_1 + b > 0$ 且 $w_2 + b > 0$

   相加得：$w_1 + w_2 + 2b > 0$

   但从 (1,1) 的条件：$w_1 + w_2 + b \leq 0$

   如果 $b \leq 0$，则 $w_1 + w_2 + 2b \leq w_1 + w_2 + b \leq 0$

   这与 $w_1 + w_2 + 2b > 0$ 矛盾！

2. **两条直线解决方案：**

   第一条直线：$x_1 + x_2 = 0.5$（分隔 (0,0)）第二条直线：$x_1 + x_2 = 1.5$（分隔 (1,1)）

   在两条直线之间的区域是输出为 1 的区域。

3. **解释：**

   XOR 的问题在于正类样本（(0,1) 和 (1,0)）分布在两个不同的”角落”。

   一条直线只能把空间分成两个半平面，无法处理这种”对角线”分布。

   两条直线可以把空间分成三个区域，中间的”带状”区域正好包含两个正类样本，而负类样本在带状区域之外。

练习 17.3：参数数量计算

计算以下神经网络的参数数量：

1. 输入 784 维，隐藏层 1 有 256 个神经元，隐藏层 2 有 128 个神经元，输出 10 维
2. 输入 100 维，三个隐藏层分别有 200、200、100 个神经元，输出 1 维
3. 如果一个参数的存储需要 4 字节（float32），上述两个网络分别需要多少内存？

点击查看答案

解答：

1. **网络 1：[784, 256, 128, 10]**

   - 层 1：$256 \times 784 + 256 = 200,704 + 256 = 200,960$
   - 层 2：$128 \times 256 + 128 = 32,768 + 128 = 32,896$
   - 层 3：$10 \times 128 + 10 = 1,280 + 10 = 1,290$

   总计：235,146 个参数

   内存：$235,146 \times 4$**字节** ≈ 940**KB**

2. **网络 2：[100, 200, 200, 100, 1]**

   - 层 1：$200 \times 100 + 200 = 20,200$
   - 层 2：$200 \times 200 + 200 = 40,200$
   - 层 3：$100 \times 200 + 100 = 20,100$
   - 层 4：$1 \times 100 + 1 = 101$

   总计：80,601 个参数

   内存：$80,601 \times 4$**字节** ≈ 322**KB**

3. **内存计算：**

   注意：这只是参数存储。训练时还需要存储梯度、优化器状态等，通常需要 3-4 倍的参数存储空间。

13.10.2 进阶题（3 道）

练习 17.4：激活函数对比

考虑以下激活函数：Sigmoid、Tanh、ReLU。

问题： 1. 画出三个函数在区间 [-5, 5] 上的图像 2. 计算三个函数在 z=0 处的导数值 3. 当 |z| 很大时，三个函数的导数分别趋近于什么值？这会带来什么问题？4. 为什么 ReLU 在深度学习中更常用？

点击查看答案

解答：

1. **图像特征：**

   - Sigmoid：S 形曲线，输出范围 (0,1)，中心点在 (0, 0.5)
   - Tanh：S 形曲线，输出范围 (-1,1)，中心点在 (0, 0)
   - ReLU：当 z<0 时为 0，当 z>0 时为斜率为 1 的直线

2. **z=0 处的导数：**

   Sigmoid：

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\sigma'(0) = 0.5 \times 0.5 = 0.25$$

   Tanh：

$$\tanh'(z) = 1 - \tanh^2(z)$$

$$\tanh'(0) = 1 - 0 = 1$$

   ReLU：

$$\text{ReLU}'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases}$$

   在 z=0 处导数未定义（通常定义为 0 或 1）

3. **|z| 很大时的行为：**

函数	z → +∞	z → -∞	问题
Sigmoid	导数 →0	导数 →0	**梯度消失**
Tanh	导数 →0	导数 →0	**梯度消失**
ReLU	导数 =1	导数 =0	负数区域”死亡”

4. **ReLU 的优势：**

- 正数区域梯度恒为 1，**缓解梯度消失问题**
- 计算简单（只需要比较操作），**加速训练**
- 引入稀疏性（部分神经元输出为 0），可能有正则化效果

练习 17.5：反向传播的直觉

考虑一个简化的两层网络：

$$z^{[1]} = w_1 x, \quad a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = w_2 a^{[1]}, \quad \hat{y} = z^{[2]} \quad \text{(线性输出)}$$

损失函数：$L = (y - \hat{y})^2$

问题： 1. 用链式法则写出 $\frac{\partial L}{\partial w_2}$ 的表达式 2. 用链式法则写出 $\frac{\partial L}{\partial w_1}$ 的表达式 3. 解释：为什么 w_1 的梯度依赖于 w_2？这说明了什么？

点击查看答案

解答：

1. $\frac{\partial L}{\partial w_2}$**：**

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} = -2(y - \hat{y}) \cdot a^{[1]}$$

2. $\frac{\partial L}{\partial w_1}$**：**

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial w_1}$$

$$= -2(y - \hat{y}) \cdot w_2 \cdot \sigma'(z^{[1]}) \cdot x$$

3. **解释：**

w_1 的梯度表达式中包含 w_2。这说明：

- **误差信号的传递**：前一层参数的调整依赖于后一层权重的”放大/缩小”。如果 w_2 很小，即使 w_1 的变化对 $a^{[1]}$ 有影响，这个影响也会被 w_2 减弱。
- **学习的协调**：网络的所有层必须协调学习。如果后一层没有学好（w_2 不合适），前一层也很难学好。
- **梯度消失/爆炸的隐患**：如果网络很深，很多权重相乘可能导致梯度指数级变小（消失）或变大（爆炸）。

练习 17.6：设计网络解决特定问题

你面临以下分类问题：

问题 A： 二分类问题，数据是二维的，分布呈现两个同心圆（内圆一类，外圆一类）

问题 B： 手写数字识别，图像是 28×28=784 维，需要识别 10 个数字

问题 C： XOR 问题，输入 2 维，输出 1 维

任务： 1. 为每个问题设计一个合适的神经网络结构（层数、每层的神经元数）2. 解释你的设计选择 3. 为每个网络计算参数数量

点击查看答案

解答：

问题 C：XOR（最简单）

设计：[2, 4, 1] - 输入层：2（匹配输入维度）- 隐藏层：4（足以学习 XOR 的非线性边界，实验表明 2 个也足够）- 输出层：1（二分类）- 激活函数：隐藏层 ReLU/Sigmoid，输出层 Sigmoid

参数数量：$4 \times 2 + 4 + 1 \times 4 + 1 = 8 + 4 + 4 + 1 = 17$

问题 A：同心圆（中等复杂度）

设计：[2, 16, 8, 1] - 需要足够的隐藏层来学习复杂的环形边界 - 第一层 16 个神经元学习基本特征（各种方向的线性边界）- 第二层 8 个神经元组合这些特征形成曲线

参数数量：- 层 1：$16 \times 2 + 16 = 48$ - 层 2：$8 \times 16 + 8 = 136$ - 层 3：$1 \times 8 + 1 = 9$ - **总计：193 个参数**

问题 B：手写数字（高维输入）

设计：[784, 256, 128, 64, 10] - 输入：784（28×28 像素）- 隐藏层逐步降维：256→128→64（金字塔结构）- 输出：10（10 个数字类别）- 激活函数：隐藏层 ReLU，输出层 Softmax

参数数量：- 层 1：$256 \times 784 + 256 = 200,960$ - 层 2：$128 \times 256 + 128 = 32,896$ - 层 3：$64 \times 128 + 64 = 8,256$ - 层 4：$10 \times 64 + 10 = 650$ - **总计：242,762 个参数**

设计原则： 1. 输入层维度 = 特征维度 2. 输出层维度 = 类别数（分类）或 1（回归）3. 隐藏层通常逐层减小（对于传统 MLP）4. 问题越复杂，需要的隐藏层和神经元越多

13.10.3 挑战题（2 道）

练习 17.7：实现自定义激活函数

除了标准激活函数，研究人员还提出了许多变种。请实现以下激活函数，并在 XOR 问题上测试它们：

Swish 激活函数（Google Brain, 2017）：

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

GELU 激活函数（Google, 2018，用于 BERT、GPT 等）：

$$\text{GELU}(x) = x \cdot \Phi(x) \approx 0.5x \left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right]\right)$$

任务： 1. 实现 Swish 和 GELU 函数及其导数 2. 修改 MLP 类以支持这些激活函数 3. 在 XOR 问题上对比 Sigmoid、ReLU、Swish、GELU 的效果 4. 分析它们的收敛速度和最终损失

点击查看提示

实现提示：

```
@staticmethod
def swish(z):
    """Swish激活函数"""
    return z * Activations.sigmoid(z)

@staticmethod
def swish_derivative(z):
    """Swish的导数"""
    sig = Activations.sigmoid(z)
    return sig + z * sig * (1 - sig)  # swish'(x) = sigmoid(x) + x * sigmoid'(x)

@staticmethod
def gelu(z):
    """GELU激活函数（近似实现）"""
    return 0.5 * z * (1 + np.tanh(
        np.sqrt(2/np.pi) * (z + 0.044715 * z**3)
    ))

@staticmethod
def gelu_derivative(z):
    """GELU的导数（数值近似）"""
    # 可以使用数值微分或更复杂的解析表达式
    eps = 1e-5
    return (Activations.gelu(z + eps) - Activations.gelu(z - eps)) / (2 * eps)
```

预期发现： - Swish 通常比 ReLU 表现更好（平滑、自门控）- GELU 在现代 Transformer 中表现优异 - 但 ReLU 计算最简单，对于小网络可能收敛最快

练习 17.8：探索深度与宽度的权衡

神经网络的”容量”（表达能力）可以由两个维度衡量：- **深度**：网络的层数 - **宽度**：每层神经元的数量

任务：

在同心圆分类问题（练习 17.6 的问题 A）上，探索以下网络结构：

网络	结构	深度	总宽度（平均）	参数数量
A	[2, 64, 1]	浅	宽	?
B	[2, 16, 16, 1]	中等	中等	?
C	[2, 8, 8, 8, 8, 1]	深	窄	?

要求： 1. 计算每个网络的参数数量 2. 实现并训练这三个网络 3. 绘制决策边界对比图 4. 对比收敛速度和最终准确率 5. 总结深度与宽度的权衡规律

思考题： - 在参数数量相近的情况下，深网络还是宽网络表现更好？- 过度加深或加宽会带来什么问题？- 现代深度学习（ResNet、Transformer）倾向于深还是宽？为什么？

点击查看提示

预期结论：

1. **参数数量：**
   - 网络 A：$64 \times 2 + 64 + 1 \times 64 + 1 = 193 + 65 = 258$
   - 网络 B：$(16 \times 2 + 16) + (16 \times 16 + 16) + (1 \times 16 + 1) = 48 + 272 + 17 = 337$
   - 网络 C：$(8 \times 2 + 8) + 3 \times (8 \times 8 + 8) + (1 \times 8 + 1) = 24 + 216 + 9 = 249$
2. **性能预期：**
   - 网络 A（浅而宽）：可能过拟合，决策边界不平滑
   - 网络 B（均衡）：通常效果最好
   - 网络 C（深而窄）：可能训练困难（梯度问题），但如果有足够的技巧（残差连接等），深层网络表达能力更强
3. **深度 vs 宽度的权衡：**
   - 研究表明：在一定范围内，**增加深度比增加宽度更有效**
   - 但深度网络更难训练（梯度消失/爆炸）
   - 现代架构（ResNet、DenseNet）解决了深度训练问题，因此趋向于更深
   - Transformer 虽然宽，但深度也很大（GPT-3 有 96 层！）

13.11 17.10 本章总结

13.11.1 核心概念回顾

1. XOR 问题与多层神经网络的必要性

XOR 问题是神经网络发展史上的转折点。它证明了**单层感知机的局限性**——只能解决线性可分问题。而引入隐藏层后，神经网络可以通过**空间变换**将线性不可分的问题转换为线性可分。

2. 前向传播

信号从输入层流经隐藏层，最终到达输出层。每一层的计算可以表示为：

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{a}^{[l]} = \sigma(\mathbf{z}^{[l]})$$

矩阵运算让神经网络可以高效处理批量数据。

3. 反向传播（直觉）

误差从输出层**倒着流**回输入层，每一层根据后一层的反馈计算自己权重的调整方向。这是通过**链式法则**实现的，但本章我们专注于直觉理解：误差信号的传递、每层参数对最终损失的责任分摊。

4. 激活函数的必要性

没有非线性激活函数，多层网络就等价于单层网络。**非线性是深度学习的灵魂**，它让每一层都能学习到真正新的、不可被前面层表示的特征。

5. 网络设计

- 输入层维度 = 特征维度
- 输出层维度 = 任务需求（类别数或回归输出）
- 隐藏层：通常逐层减小，深度和宽度需要权衡
- 激活函数：隐藏层首选 ReLU，输出层根据任务选择

13.11.2 历史意义

本章所讲述的技术——**反向传播算法**——是深度学习 revolution 的起点。1986 年 Rumelhart、Hinton 和 Williams 的论文，让神经网络从”寒冬”中复苏，为今天的大语言模型、计算机视觉、语音识别等所有深度学习应用奠定了基础。

正如 Geoffrey Hinton 所说：

> “*The brain is a very good device for learning. The question is: how does it do it? I think backpropagation is a pretty good theory.*”
>
> （大脑是一个很好的学习装置。问题是：它是如何做到的？我认为反向传播是一个很好的理论。）

13.11.3 下章预告

在下一章，我们将深入反向传播的数学推导，详细讲解：- 链式法则的完整应用 - 各种激活函数的导数 - 矩阵求导的技巧 - 梯度检查的验证方法

准备好你的数学工具，我们要进入神经网络的数学核心了！

13.12 参考文献

1. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. https://doi.org/10.1038/323533a0

2. Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408. https://doi.org/10.1037/h0042519

3. Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry.* MIT Press.

4. Linnainmaa, S. (1970). The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors (Master' s thesis, University of Helsinki).
5. Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303-314. https://doi.org/10.1007/BF02551274
6. Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359-366. https://doi.org/10.1016/0893-6080(89)90020-8
7. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444. https://doi.org/10.1038/nature14539
8. Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (pp. 249-256). JMLR Workshop and Conference Proceedings.
9. Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *ICML*.
10. Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for activation functions. *arXiv preprint arXiv:1710.05941*.
11. Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*.
12. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org/

13.13 扩展阅读

想要深入理解?

1. **观看 Geoffrey Hinton 的 Coursera 课程**:《Neural Networks for Machine Learning》，这是理解反向传播最好的资源之一。
2. **阅读 3Blue1Brown 的神经网络系列**: Grant Sanderson 用精美的可视化解释了反向传播的每个细节。
3. **动手实验**: 使用本章的代码，尝试改变网络结构、激活函数、学习率，观察对结果的影响。
4. **探索 PyTorch/TensorFlow**: 当你理解了从零实现的原理，就可以使用这些框架更高效地构建大型网络。

"任何足够先进的技术都与魔法无异。"

多层神经网络曾经被认为是魔法，但今天它已经成为我们理解智能、构建 AI 系统的基石。从 XOR 到 GPT-4，从感知机到 Transformer——这一切，都始于本章你所学习的核心思想。

继续探索吧，深度学习的魔法世界才刚刚向你敞开大门!